

The Backend Engineer's Library

Distributed Systems

Volume 6

Contents

Foundations: System Models, Failures, and Assumptions

Time, Clocks, and the Ordering of Events

Replication and Consistency Models

CAP, PACELC, and the Real Trade-Offs

Consensus I: Paxos

Consensus II: Raft

Quorum Systems and Dynamo-Style Replication

Coordination Services: ZooKeeper and etcd

Idempotency, Deduplication, and Exactly-Once

Failure Detection and Membership: Gossip and SWIM

CRDTs and Eventual Consistency

Testing Distributed Systems: Jepsen, Chaos, and Simulation

Chapter 1 — Foundations: System Models, Failures, and Assumptions

What this chapter covers. Volume 4 opened with the claim that a distributed system is concurrency writ large — the same hazards, replayed across machines, with weaker guarantees and no shared memory to fall back on. This volume takes that claim seriously, and this chapter lays the foundation everything else stands on. Before Paxos or Raft, before consistency models or CRDTs, you need three things: a precise account of what makes a system "distributed" and why it is qualitatively harder than concurrent-but-local (the one-word answer is *partial failure*); a working command of **system models** — the timing, failure, and network assumptions that sit, usually unstated, under every protocol and every proof; and an honest understanding of the canonical impossibility results — Two Generals, FLP, and the crashed-versus-slow ambiguity — with their exact preconditions, because knowing precisely what is impossible is what tells you what the possible must cost. The chapter closes by setting the engineering stance for the whole volume: protocols are machines for converting assumptions into guarantees, and when the assumptions break, the guarantees break with them. We also introduce the volume's running example — a replicated key-value store that each subsequent chapter upgrades — in its Chapter 1 form: a single node and a prayer.

Learning goals — after this chapter you should be able to:

- Define a distributed system by its essential properties — partial failure, no shared memory, no shared clock, communication only by fallible messages — and explain why partial failure, not physical distribution, is the property that changes everything.
- Recite the eight fallacies of distributed computing and identify each one in a modern architecture review.
- State the three timing models (synchronous, asynchronous, partially synchronous) precisely, and explain why real systems are designed for partial synchrony: safety always, liveness when the network behaves.

- Place crash-stop, crash-recovery, omission, and Byzantine failures in a hierarchy, know which model real infrastructure assumes, and know when Byzantine tolerance is and is not worth $3f+1$ replicas.
- State the Two Generals result and the FLP impossibility theorem with their exact preconditions, and explain what each does *not* say.
- Explain why a crashed node and a slow node are indistinguishable in an asynchronous system, and trace that single ambiguity to split brain, fencing tokens, and leases.
- Describe how at-least-once delivery is built from fair-loss links, why "exactly-once delivery" is really at-least-once plus deduplication, and distinguish delivery from processing.

What "distributed" actually means

Leslie Lamport, in a May 1987 email to his colleagues at DEC's Systems Research Center, gave the definition that the field has never improved on:

A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

The joke is precise. Lamport does not say a distributed system is one whose parts are far apart, or one with many nodes, or one that uses a network. He identifies the property that actually changes the engineering: **your fate is coupled to components you do not control, cannot observe directly, and may not even know about — and those components fail independently of you.**

It is worth dwelling on why *partial failure* is the defining property, because the contrast with everything you learned in Volume 4 is stark. In a concurrent-but-local program, failure is all-or-nothing. If a thread dereferences a null pointer or the process runs out of memory, the whole process dies — every thread, every lock, every half-updated data structure, all at once. This sounds bad, and locally it is, but it makes *recovery* almost trivially simple: the operating system reclaims everything, a supervisor restarts the process, and the process begins again from a clean state (plus whatever it persisted). There is never a moment when thread 3 is dead but thread 7 is still running and holding a mutex that thread 3 will never release. Crash semantics inside one process are clean precisely because they are total.

Distribution breaks that totality. In a system of fifty nodes, one node crashing is not an event that stops the system — it is Tuesday. The other forty-nine keep running, keep holding state, keep making decisions, and must somehow cope with the fact that a participant vanished mid-protocol: mid-transaction, mid-replication, holding a lease, having acknowledged some messages and not others. Worse, as we will see, they cannot even be sure it vanished. Partial failure means the system is *permanently* in a state where some components are up, some are down, some are slow, and nobody has an authoritative list of which is which. Every protocol in this volume is, at bottom, a way of making progress anyway.

Three further properties complete the definition, and each removes a tool you leaned on in Volume 4:

- **No shared memory.** There is no address space in common, so there is nothing like a mutex, an atomic compare-and-swap, or a shared data structure. The only way node A can affect node B is to send it a message and hope. Every synchronization primitive you rely on in-process must be rebuilt out of messages, and the rebuilt versions are weaker, slower, and can fail partway.
- **No shared clock.** Each node has its own oscillator, its own drift, its own opinion of what time it is. There is no global "now," and — more subtly — no global ordering of events that all nodes agree on for free. Chapter 2 is devoted to what can be salvaged; the short version is *ordering* can be reconstructed (Lamport clocks, vector clocks) but *simultaneity* cannot.
- **Messages are the only channel, and messages are fallible.** A message may be lost, delayed arbitrarily, delivered more than once, or delivered out of order relative to other messages. Any of these, at any time, in any combination. Volume 3 explained the physical reasons — queue drops, retransmission, route changes, buffer bloat; here we take them as axioms and build on them.

A useful mental exercise: take any piece of concurrent code you trust and ask what happens if a mutex acquisition could *silently fail to return*, if a write to a shared variable could be *applied twice*, or if a thread could observe writes in an order no other thread observes. In a single process these are absurdities. Across a network they are the baseline. That is the qualitative gap this volume exists to cross.

The eight fallacies

In 1994 L. Peter Deutsch, then at Sun Microsystems, codified seven assumptions that "essentially everyone" makes when first building distributed applications, all false; James Gosling added the eighth in 1997. (The list has roots in earlier observations by Bill Joy and Tom Lyon at Sun.) Three decades later, every one of them still ships to production weekly. The list is worth memorizing not as trivia but as a review checklist — each fallacy names a default assumption your design must either avoid or explicitly pay to remove.

1. **The network is reliable.** It is not; packets and whole partitions of the network vanish. Modern instance: a service calls another service with no retry, no timeout, and no fallback, and a single dropped TCP connection during a top-of-rack switch reboot turns into a user-visible error. The entire discipline of retries, idempotency (Chapter 9), and circuit breaking exists because of this fallacy.
2. **Latency is zero.** A call across an availability zone costs on the order of a millisecond; across continents, tens to a couple hundred milliseconds — and unlike bandwidth, latency is bounded below by the speed of light in fiber and is not improving. Modern instance: an ORM issuing $N+1$ queries is annoying against a local database and catastrophic against one 60 ms away; a microservice decomposition that turns one request into a chain of twelve sequential RPCs has bought a $12\times$ latency floor.
3. **Bandwidth is infinite.** Modern instance: a chatty gRPC service streaming full objects when deltas would do, or a Kafka consumer group re-reading a topic from offset zero, saturating the very links its neighbors need. Bandwidth has grown enormously, which makes this fallacy feel safe right up until an antique 1-Gb link between data centers becomes the bottleneck for a replication stream.
4. **The network is secure.** Modern instance: a service mesh rolled out with mTLS in permissive mode "temporarily," internal APIs that trust any caller inside the VPC, and then a single compromised pod can read everything. Volume 9 treats zero-trust properly; the fallacy here is assuming the network boundary does security work it cannot do.
5. **Topology doesn't change.** In the Kubernetes era this fallacy is almost charmingly obsolete in its original form — pods are rescheduled constantly, IPs are ephemeral, autoscalers add and

remove nodes by the minute. Yet it survives in subtler forms: DNS results cached forever, connection pools that never re-resolve, clients pinned to a load balancer IP that moved.

6. **There is one administrator.** Modern instance: your service depends on a managed database, a third-party auth provider, a CDN, and a payments API — four organizations' change calendars, none of which consult yours. Even in-house, the team that owns the message broker will upgrade it on their schedule. Design implication: you cannot coordinate maintenance globally, so you must tolerate dependencies degrading without notice.
7. **Transport cost is zero.** Serialization burns CPU (Volume 8 quantifies this for JSON versus protobuf), and cloud providers bill for cross-AZ and egress traffic in real money. Modern instance: a data platform whose inter-AZ replication traffic quietly becomes one of the largest line items on the cloud bill.
8. **The network is homogeneous.** Different links have wildly different latency, loss, and MTU; different stacks speak subtly different dialects. Modern instance: a protocol tuned in a single-AZ test environment melting down over a VPN link with 2% loss, or path-MTU blackholes that only affect the one customer behind a misconfigured firewall.

The fallacies are the informal statement of this chapter's thesis. The formal statement is the system model, to which we now turn.

System models: the assumptions under every proof

Every claim of the form "this protocol guarantees X" is incomplete. The honest form is: "this protocol guarantees X *in system model M*" — under stated assumptions about timing, about how components fail, and about what the network does to messages. Change the model and the guarantee can evaporate. This is not pedantry; it is the difference between knowing that Raft is safe and knowing *under what circumstances* Raft is safe (and what happens to it when a GC pause violates them — see below). A system model has three axes: timing, failure, and network.

Timing models

The timing model states what may be assumed about how long things take — both message delivery and processing steps.

Synchronous. There is a known upper bound Δ on message delay and a known bound on the relative speed of processes (no step takes longer than some known time). In this model, distributed computing is almost easy: if you send a request and hear nothing for Δ plus the processing bound, you *know* the peer has failed. Timeouts are perfect failure detectors. Lock-step round-based protocols work. The problem is that no real network is synchronous: any bound you pick will eventually be violated by a congested link, a rebooting switch, or a stop-the-world GC pause — and a protocol whose *safety* depends on the bound does something wrong at exactly that moment.

Asynchronous. No bounds at all. Messages take arbitrarily long (though, in the usual formulation, messages sent are eventually delivered); processes run arbitrarily slowly; there are no useful clocks. Nothing whatsoever may be assumed about timing, so nothing can be concluded from the passage of time — a timeout tells you nothing, because "a long time" is not evidence of anything in a model where delivery may take longer than any time you name. This model is not a description of a real network; it is an *adversarial worst case*, and its value is exactly that: anything that works in the asynchronous model works everywhere, and anything impossible in it (FLP, below) marks a boundary that no amount of engineering can cross without adding assumptions.

Partially synchronous. The model of Dwork, Lynch, and Stockmeyer (1988), and the one real systems actually live in. Two equivalent flavors: bounds Δ on message delay exist but are *unknown*; or the bounds are known but hold only *eventually* — after some unknown Global Stabilization Time (GST), the network behaves synchronously. Either way the operational content is the same: **the network behaves well most of the time, misbehaves in bursts of unknown duration, and you never know which regime you are currently in.** That is a recognizable description of a production network: normally sub-millisecond within a rack, occasionally deranged for seconds or minutes by congestion, failover, or a maintenance event.

Partial synchrony matters because it dictates the shape of every practical protocol in this volume, via a split you should internalize now:

Safety must hold unconditionally — in all executions, even fully asynchronous ones. Liveness is only promised when the network behaves — after GST, during the good periods.

A correct consensus protocol never, under any timing behavior, allows two nodes to decide different values (safety). What it sacrifices during network chaos is *progress*: elections churn, commits stall, clients time out — but nothing wrong is ever decided, and when the network calms down, progress resumes. Raft and Paxos (Chapters 5 and 6) are built exactly this way, and when you evaluate any distributed component, the first question to ask is which of its guarantees are safety (unconditional) and which are liveness (conditional on timing) — vendors are chronically vague about the difference.

```
flowchart LR
    SYNC["SYNC: Synchronous  
known bounds on delay  
and processing"] --> PART["PART: Partially synchronous  
bounds exist but unknown,  
or hold only after GST"]
    PART --> ASYNC["ASYNC: Asynchronous  
no bounds at all –  
time implies nothing"]
    ASYNC --> S1["S1: Timeouts are perfect  
failure detectors.  
But no real network  
keeps the bound."]
    S1 --> P1["P1: The model real systems  
design for: safety always,  
liveness when the  
network behaves."]
    P1 --> A1["A1: FLP applies: no deterministic  
consensus with guaranteed  
termination. Worst-case  
model for proofs."]
```

Failure models

The failure model states *how* components are allowed to fail. The models form a strict hierarchy — each admits everything below it plus new misbehavior — and tolerating a stronger model costs more.

Crash-stop (fail-stop). A process executes its protocol faithfully and then, at an arbitrary moment, halts forever. It never sends another message, never comes back. This is the cleanest model and the one most textbook protocols are first stated in. Its unrealism is the "forever": real processes restart.

Crash-recovery. A process may crash, losing everything in memory, and later recover, resuming with whatever it had written to *stable storage*

(fsync'd disk — Volume 5, Chapter 2, covers what "stable" actually takes). This is the model real services live in, and the gap between it and crash-stop is where a great deal of real engineering happens: a recovering node must rejoin the protocol knowing only what it persisted, which is why consensus implementations are obsessive about what gets written to the log *before* which message is sent. A useful reflex: whenever you read a protocol described in crash-stop terms, ask what happens when a participant comes back from the dead with stale in-memory state and a valid identity. (Half of the subtle bugs in Chapter 8's coordination recipes live in that question.)

Omission failures. A process may fail to send messages it should have sent, or fail to receive messages that arrived (full buffers, drops in the process's own stack). Crash-stop is the special case of omitting *everything* after some point. In practice, omission on the network side is usually folded into the link model (below), and process-side omission is folded into crash-recovery; you will rarely design for it separately, but it appears in the literature and in proofs.

Byzantine (arbitrary) failures. A faulty process may do *anything*: send contradictory messages to different peers, lie about its state, collude with other faulty processes, actively attempt to violate the protocol. The name comes from Lamport, Shostak, and Pease's 1982 framing of the problem as Byzantine generals coordinating an attack with traitors among them. Tolerating f Byzantine failures in a consensus protocol requires $3f+1$ replicas (versus $2f+1$ for crash failures) plus substantially more communication and, typically, cryptographic authentication of every message.

Honest guidance, and this volume's position: **nearly all infrastructure you will build or operate assumes crash-recovery, and that is the right call.** Within a single organization's trust boundary, machines do not lie strategically; they crash, they slow down, and occasionally they corrupt data — and the corruption cases are better handled by checksums and end-to-end validation than by Byzantine agreement. Byzantine fault tolerance earns its $3f+1$ cost in genuinely adversarial, multi-party settings: public blockchains, cross-organization consortia, systems where some participants profit from cheating. There, protocols like PBFT and its descendants apply. This volume goes no deeper into BFT than this paragraph; when you see it advertised for single-

organization infrastructure, the right response is skepticism about whether the threat model justifies tripling the replica count.

```
flowchart TD
    CS["Crash-stop  
halts forever, silently"] --> CR["Crash-recovery  
halts, may return with only  
stable storage intact –  
the model real services live in"]
    CR --> OM["Omission  
drops some sends or receives  
but otherwise follows protocol"]
    OM --> BYZ["Byzantine  
arbitrary behavior: lies,  
equivocation, collusion"]
    CS --> N1["Each level includes all  
failures of the levels above it"]
    BYZ --> N2["Tolerating f failures:  
crash models need 2f+1 replicas,  
Byzantine needs 3f+1"]
```

Network models

The third axis states what the links do to messages.

- **Reliable links:** every message sent is eventually delivered, exactly once. No real physical link is reliable, but reliable links can be *constructed* over unreliable ones — that construction is the subject of the communication-abstractions section below, and assuming reliable links in a protocol description is shorthand for "run this over such a construction."
- **Fair-loss links:** messages may be lost, duplicated, or reordered, but with a crucial non-triviality guarantee: if you send a message infinitely often, it is delivered infinitely often. The network is lossy but not a perfect censor — it cannot eat *every* retry forever. This is the standard honest model of a real IP network, and it is all you need: everything else is built on top.
- **FIFO versus unordered channels:** a FIFO channel delivers messages from a given sender in the order sent; an unordered channel makes no promise. Some protocols need FIFO and say so; others are explicitly designed to survive reordering.

Where does TCP sit? Volume 3, Chapters 4 and 5 covered the mechanics; the model-level summary is: **TCP gives you a per-connection FIFO**,

reliable channel — until the connection breaks, at which point you know nothing about in-flight data. Within one healthy connection, bytes arrive in order, without gaps or duplicates, and that is a genuinely strong guarantee worth building on. But when the connection resets or times out, TCP tells you only that *some prefix* of what you sent was delivered — not which prefix. The last write you made before the error may have been fully received and processed, or never have left your kernel's send buffer, and nothing in the API can tell you which. Every application-level retry-after-reconnect therefore risks duplication, which is why the fair-loss model (loss *and* duplication) remains the right mental model even in an all-TCP world, and why idempotency (Chapter 9) is not optional. Note also that TCP's ordering guarantee is per-connection only: two messages sent on different connections, or before and after a reconnect, may be observed in any order.

The model is a load-bearing declaration

To restate the section's point as method: when you design or evaluate a distributed component, write the model down. *Timing*: partially synchronous. *Failures*: crash-recovery with stable storage, up to f of $2f+1$ nodes. *Network*: fair-loss, unordered across connections. Every guarantee the component claims should be traceable to those assumptions, and every assumption is a thing that can be violated in production and should be monitored (the closing sections return to this). A protocol whose model you cannot state is a protocol whose failure modes you have not found yet.

The canonical impossibility results

Three results bound what any distributed protocol can achieve. Each is frequently misquoted in both directions — cited to claim things are impossible that are merely expensive, and waved away as "theoretical" when they in fact explain production outages. We state each precisely, sketch why it is true, and draw the engineering consequence.

Two Generals: no certainty over a lossy link

The setup (posed by Akkoyunlu, Ekanadham, and Huber in 1975; named and popularized by Jim Gray in 1978): two generals on separate hills must attack a city simultaneously — attacking alone loses. They communicate only by messengers who may be captured (a fair-loss link).

Can they agree on an attack time such that *both know the agreement is in force*?

The result: no finite protocol can achieve this. The proof is a short and beautiful contradiction. Suppose a correct protocol exists; among all correct protocols take one using the fewest messages, and consider its final message. Either that message matters or it does not. If it does not, delete it — contradicting minimality. If it does matter, then the sender must attack having *not observed* whether it arrived (nothing follows it to tell them), while the receiver's behavior *depends* on it arriving; since it can be lost, there is an execution where one attacks and the other does not — contradicting correctness. So no such protocol exists. Intuitively, every acknowledgment needs its own acknowledgment: A cannot act until sure B knows, B cannot be sure A knows B knows, ad infinitum. Common knowledge — I know, you know, I know you know, forever — is unattainable over a link that can lose even one message.

```
sequenceDiagram
    participant A as General A
    participant B as General B
    A->>B: "Attack at dawn" – but did it arrive?
    B->>A: "ACK – I got it" – but did the ACK arrive?
    A->>B: "ACK your ACK" – but did THAT arrive?
    B--xA: "ACK the ACK-ACK" – lost. Now what?
    Note over A,B: Every message needs its own confirmation.
    The regress never terminates: common knowledge
    is unreachable over a fair-loss link.
```

What it means in practice — and what it does not. It does *not* mean reliable communication is hopeless: retransmission gets a message through with probability approaching 1, and TCP works fine. What it kills, permanently, is **certainty**: the sender of a message can never be certain it was received, and no handshake of any finite length fixes this. Every "did my write commit?" timeout you have ever handled is Two Generals wearing work clothes: the client that times out on a payment API cannot know whether the charge happened. The engineering answer is not to seek the impossible certainty but to make the uncertainty harmless: **retry until acknowledged, and make the operation idempotent so that retries are safe**. That pairing — at-least-once delivery plus idempotent processing — is the standard resolution, and Chapter 9 is devoted to doing it properly. (Distributed transactions inherit the problem in sharper

form: 2PC's coordinator crash window, Volume 5, Chapter 10, is Two Generals with money on the table.)

FLP: consensus cannot guarantee termination in an asynchronous world

The 1985 result of Fischer, Lynch, and Paterson is the most famous theorem in distributed computing, and the most misquoted. Here is the precise statement:

In an **asynchronous** system (no bounds on message delay or processing speed), with a **reliable** network (every message is eventually delivered), no **deterministic** protocol can solve consensus with **guaranteed termination** if even **one** process may **crash**.

Every qualifier is load-bearing. The network is reliable — the result is not about lost messages. Only one crash, of the mildest kind — not Byzantine. And the impossibility targets *termination*: for any deterministic protocol that never violates agreement, there exists at least one admissible execution — one diabolical schedule of message deliveries — in which the protocol runs forever without deciding. The intuition connects to the timeout dilemma below: in an asynchronous system a protocol cannot distinguish "that process crashed, decide without it" from "that process is slow, its vote is still coming," and the proof shows an adversarial scheduler can exploit that ambiguity indefinitely, forever keeping the system in a state where the decision still hangs on a message that has not yet arrived.

Now, what FLP does **not** say — this list matters more in practice than the theorem:

- It does **not** say consensus is impossible. Agreement and validity (the safety properties) are achievable, and achieved, unconditionally. Only guaranteed-in-all-executions termination is not.
- It does **not** say consensus is impractical. The non-terminating executions require an adversarial scheduler contriving infinite bad luck; real networks are not adversarial in that way, and real consensus systems decide in milliseconds essentially always.
- It does **not** apply once you strengthen the model. That is the escape hatch, and every practical system uses one of three:

- **Partial synchrony** — assume DLS instead of full asynchrony, use timeouts, and accept that liveness is conditional on the network eventually behaving. This is Raft's and (deployed) Paxos's choice; Chapters 5 and 6.
- **Randomization** — a coin flip breaks the adversary's schedule; randomized protocols (Ben-Or onward) terminate with probability 1, which FLP's determinism requirement does not forbid.
- **Failure detectors** — encapsulate the timing assumption in an oracle that suspects crashed processes (Chandra-Toueg, 1996); consensus becomes solvable given a sufficiently good detector, and the detector is where the impossibility is quarantined. Chapter 10.

So the honest reading of FLP: it is not a stop sign but a price list. It says *guaranteed* termination is not on the menu in the asynchronous model, and therefore every real consensus system has, somewhere inside it, a timing assumption — usually a timeout — on which its liveness rests. Find that timeout and you have found the knob that determines how the system behaves during network chaos, and the assumption whose violation explains its stalls. When your etcd cluster churns through leader elections during a network incident and refuses writes, it is not broken — it is being exactly as live as FLP permits a safe protocol to be.

The timeout dilemma: crashed is indistinguishable from slow

The third result is not a named theorem but a direct corollary of asynchrony, and it does more damage in production than the other two combined: **in an asynchronous or partially synchronous system, no observation can distinguish a crashed process from a slow one.** Silence is the only symptom of both. A node that has not responded for ten seconds might be dead, or partitioned, or sitting in a stop-the-world GC pause, or descheduled by an oversubscribed hypervisor, or waiting on a disk that is timing out internally — and from the outside these are one and the same observation: no message.

Every practical system resolves the ambiguity the only way it can — a timeout — and a timeout is a *decision*, not a *discovery*. Declaring a node dead after T seconds of silence is a bet, and both ways of losing it hurt: declare too eagerly and you evict healthy-but-slow nodes (and if they held leadership, you get churn, or worse, two nodes acting as leader); declare too lazily and the system stalls behind a corpse. This single ambiguity is the root of a remarkable fraction of distributed-systems machinery: it is

why leadership is granted as a *lease* (a term that expires) rather than a permanent title, why leases alone are insufficient, and why **fencing tokens** exist — Volume 4, Chapter 2 closed on exactly this design, and here is the failure it prevents:

```
sequenceDiagram
    participant N1 as Node 1 – leaseholder
    participant LS as Lock service
    participant N2 as Node 2
    participant ST as Storage
    N1->>LS: acquire lease – granted, expires t plus 10s
    Note over N1: Stop-the-world GC pause begins.
    Note over N1: Node 1 is alive but silent – indistinguishable
    from crashed.
    LS->>LS: t plus 10s – lease expires, holder presumed dead
    N2->>LS: acquire lease – granted, token 34
    N2->>ST: write with token 34 – accepted
    Note over N1: Note over N1: GC ends. Node 1 resumes mid-instruction,
    still believing it holds the lease.
    N1->>ST: write with token 33
    ST--xN1: rejected – token 33 is older than 34
    Note over N1,ST: Note over N1,ST: Without fencing tokens, storage accepts both
    writes:
    split brain. The pause exceeded the lease –
    a timing assumption broke, so the guarantee broke.
```

Read the diagram as a parable of the whole chapter. The lease protocol is safe *under the assumption* that a live leaseholder renews before expiry — a timing assumption. A GC pause longer than the lease violates the assumption; the guarantee ("at most one writer") evaporates at that instant; and the fix is not a longer lease (any bound can be exceeded) but a mechanism — the monotonically increasing fencing token, checked at the resource — that restores safety *without* any timing assumption at all. This pattern, "make safety independent of timing, let timing govern only liveness," is the partial-synchrony split made concrete, and you will see it in every well-designed system in this volume. These are not hypothetical failures: multi-second GC pauses, VM live-migrations, and laptop-lid-style suspensions of cloud instances all really occur, and each one is a "crashed" node coming back to life convinced it is still the leader.

Chapter 10 formalizes the timeout side of this as **failure detectors**: modules that output a list of suspected processes, characterized by their completeness (crashed processes are eventually suspected) and accuracy (correct processes are not wrongly suspected — the hard part). The

practically important class is the **eventually perfect detector**, $\diamond P$: it may make mistakes for an arbitrary while — suspecting slow-but-alive nodes — but eventually it stops wrongly suspecting correct processes and permanently suspects all crashed ones. $\diamond P$ is exactly what an adaptive-timeout implementation converges to in a partially synchronous network after GST, which is the theory's way of saying: your timeouts will be wrong sometimes, design so that wrongness costs availability, never correctness.

Building upward: communication abstractions

The impossibility results bound what cannot be done; this section shows the first constructive steps — the standard ladder from fair-loss links up to the primitives that Chapters 5 through 7 assume. The treatment follows the layered style of Cachin, Guerraoui, and Rodrigues' textbook, which this volume recommends as its formal companion.

From fair-loss to at-least-once. Given a fair-loss link, build a *stubborn* sender: retransmit the message, on a timer, until an acknowledgment arrives. Fair loss guarantees that infinite retries produce infinite deliveries, so the message eventually gets through and the ack eventually gets back: every message sent is eventually delivered — possibly many times. This is **at-least-once delivery**, and it is the honest native guarantee of every retrying system you operate: HTTP clients with retry policies, message queues redelivering unacked messages, Kafka producers with `retries` set. Note what was traded: duplication is now *designed in*. The retry that makes delivery reliable is precisely the mechanism that makes delivery non-unique — you cannot have the first without risking the second (Two Generals again: the ack for the original may be the message that was lost).

From at-least-once to the exactly-once illusion. Layer deduplication on top: give every message a unique identifier, have the receiver remember identifiers it has processed, and discard repeats. The result behaves like **exactly-once delivery** — and it is important to say plainly that this is at-least-once plus dedup wearing a trench coat, with real costs: the receiver must persist the dedup state (crash-recovery model — forget the set on crash and duplicates walk right in), must bound it somehow (time windows, sequence numbers per sender), and the guarantee is only as strong as that state's durability. More important still is the distinction the marketing term blurs: **delivery is not processing**.

Exactly-once *delivery* to the process's doorstep says nothing about what happens if the process crashes after performing the side effect but before recording the message ID — on recovery, the redelivered message performs the side effect again. End-to-end exactly-once *processing* requires the dedup record and the side effect to be committed atomically (or the side effect to be idempotent), which is an application-level contract, not a transport feature. Chapter 9 dissects this properly, including what Kafka's "exactly-once semantics" actually promises and where its edges are.

Broadcast. Consensus and replication protocols are built not on point-to-point sends but on broadcast primitives, of ascending strength:

- **Best-effort broadcast:** if the *sender* stays up, all correct processes receive the message. A crash mid-broadcast may leave some receivers with the message and others without — usually unacceptable, since it creates permanent disagreement about what was even said.
- **Reliable broadcast:** all *correct* processes agree on the set of delivered messages — if any correct process delivers *m*, every correct process eventually delivers *m*, even if the sender crashed mid-send. The standard construction: every receiver re-broadcasts what it delivers, so a message that reaches anyone correct reaches everyone correct.
- **Uniform reliable broadcast:** strengthens "any correct process" to "any process at all" — if *any* process delivers *m*, even one that crashes immediately after, all correct processes eventually deliver *m*. The distinction sounds fussy and is not: a non-uniform protocol allows a process to deliver a message, act on it (respond to a client, apply a write), and crash, leaving a system in which that message otherwise never happened. Uniformity is what you need when delivery triggers externally visible effects — which is to say, almost always in this volume; the commit rules of Paxos and Raft (Chapters 5 and 6) are, in this vocabulary, machinery for a uniform primitive, and Chapter 3's replication anomalies are what non-uniformity looks like from the client's seat.

None of these order messages; adding ordering yields FIFO, causal, and — the crown jewel — *total order* broadcast, which is equivalent to consensus itself. That equivalence, and the whole ordering story, is Chapter 2's business.

The engineering stance, and the running example

This volume takes a definite stance, assembled from the pieces above; stated once here, it will be implicit everywhere after.

Distributed systems design is choosing which guarantees to buy, from whom, at what price. Guarantees are not free and not default; each one — at-most-one leader, reads see latest write, message processed once — is purchased from some protocol at a price paid in latency (coordination rounds before answering), availability (refusing to answer when the guarantee cannot be upheld), or both. Chapters 3 and 4 price this out systematically; the stance here is simply that "what guarantees does this component actually give, and what do they cost" is *the* engineering question, and vague answers to it are how systems end up simultaneously slow and wrong.

Protocols are machines for converting assumptions into guarantees. Raft converts "crash-recovery nodes, stable storage that honors fsync, partial synchrony, fair-loss links" into "a replicated log with at most one leader per term and no committed entry ever lost, live whenever a majority is up and the network is calm." Every protocol in this volume has this shape, and both halves of the conversion deserve scrutiny: the guarantee you are buying, and the assumptions you are being asked to supply.

When assumptions break, guarantees break. The lease diagram above is the canonical instance: GC pause exceeds lease, at-most-one-writer evaporates. Others recur throughout the volume: a clock that jumps backward under NTP step correction breaks anything that trusted monotonic timestamps (Chapter 2); a disk that acknowledges writes it has not durably stored breaks the crash-recovery model out from under a consensus log; a network that duplicates "impossible" messages breaks a dedup window sized to assumptions about maximum delay. Two consequences follow. First, assumption violations are *monitoring targets*: GC pause duration versus lease length, clock offset versus sync bound, fsync latency — these belong on dashboards precisely because the correctness argument cites them. Second, testing distributed systems means *attacking assumptions directly* — inject partitions, pause processes, skew clocks, kill nodes mid-commit — which is why Chapter 12 is structured as an assault on every assumption this chapter has named.

The running example. Each chapter of this volume upgrades the same system: a key-value store with `GET` and `PUT`. Its Chapter 1 form is deliberately primitive: **one node, an in-memory hash map, a write-ahead log fsync'd on every `PUT`** — the crash-recovery model made concrete: crash and restart, replay the log, and no acknowledged write is lost. Its deficiencies define the rest of the volume. It has no replicas, so a dead disk loses everything and a dead node is an outage: Chapter 3 adds replication and immediately collides with consistency. Its clients already face Two Generals — a timed-out `PUT` may or may not have committed — which Chapter 9 fixes with idempotent request IDs. Once replicated, its replicas will disagree about event order (Chapter 2), split brain under partition (Chapters 4 and 5), need leader election and log replication done right (Chapter 6), or renounce leaders for quorums (Chapter 7) or for CRDTs (Chapter 11). It will need to detect failed peers (Chapter 10), coordinate configuration (Chapter 8), and prove any of this works (Chapter 12). Keep the little store in mind throughout: every abstraction in this volume earns its place by fixing a concrete way this system loses data, serves lies, or goes down.

The distributed-systems lens

Elsewhere in this suite, this recurring section connects a local topic outward to distributed reality. In this volume the lens points reflexively — at the models themselves, and back down at the single machine.

The models are idealizations of Volume 3's realities. Fair-loss links are the model-theoretic summary of everything Volume 3 taught about IP: queue drops, corruption discards, route flaps, retransmission-induced duplication. Partitions — the model's dramatic "network splits in two" — are, in the flesh, a misconfigured BGP announcement, a spanning-tree convergence, an overloaded top-of-rack switch, a fat-fingered ACL. Partial synchrony's GST is the model's way of saying "failovers finish and congestion clears, but you don't get to know when." The models earn their abstraction: a proof against fair-loss links covers every one of those concrete failure modes at once, which is exactly why we bother with models instead of enumerating incidents.

A single machine is a distributed system at small scale — with one saving grace. Look closely at the "local" machine Volume 1 and Volume 2 dissected and the distributed structure is plainly there: NUMA nodes with distinctly non-uniform access latencies exchanging cache-coherence

messages over an interconnect; PCIe devices with their own processors, firmware, and failure modes doing DMA on their own schedule; the kernel and a device negotiating over rings and doorbells like any two networked peers. Cache-coherence protocols are consensus-flavored message protocols; an NVMe timeout is a failure detector. The saving grace is *bounded timing*: on one board, worst-case latencies are nanoseconds-to-microseconds, engineered, and effectively never violated — the hardware really is close to the synchronous model, which is why these systems can be hidden behind the abstraction of a single coherent machine and why Volume 4 could take shared memory as a primitive rather than a protocol. Distribution is what happens when timing bounds leave the realm of engineering tolerance and enter the realm of weather.

The reasoning discipline generalizes, and Volume 11 operationalizes it. The method this chapter has been teaching — state your model, prove (or at least argue) within it, then monitor for assumption violations in production — is not specific to consensus protocols. It is the same discipline as an SLO: an explicit statement of assumed behavior ("99.9% of requests under 200 ms"), consequences reasoned from it (error budgets, alerting thresholds), and instrumentation watching for the moment reality departs from the assumption. Volume 11 builds that machinery in full; the connection to make now is that an SLO on your dependency is precisely a *system model you have negotiated* — a bound on someone else's timing and failure behavior, with a paper trail — and an error budget burn is GST failing to arrive. Engineers who internalize this chapter's habit of asking "what am I assuming, and how would I know it stopped being true?" find that the reliability practices of Volume 11 are the same habit with dashboards attached.

Key takeaways

- **Partial failure is the defining property of distribution.** In-process failure is total, so recovery is all-or-nothing and simple; across machines, components fail independently and the system must make progress amid permanent uncertainty about who is up. Lamport's definition — a system where a computer you didn't know existed can render yours unusable — names the coupling precisely.
- **You lose shared memory, a shared clock, and reliable communication all at once.** Messages may be lost, delayed,

duplicated, and reordered; every Volume 4 primitive must be rebuilt from such messages, weaker and able to fail partway.

- **The eight fallacies are a design-review checklist.** Reliable network, zero latency, infinite bandwidth, secure network, static topology, one administrator, free transport, homogeneous network — each still ships to production regularly.
- **Every guarantee lives inside a system model** — timing (synchronous, asynchronous, partially synchronous), failures (crash-stop $\subset$ crash-recovery $\subset$ omission $\subset$ Byzantine), and network (fair-loss versus reliable, FIFO versus unordered). Real systems design for partial synchrony and crash-recovery over fair-loss links; TCP gives per-connection FIFO reliability until the connection breaks, after which you know nothing about in-flight data.
- **Safety must hold unconditionally; liveness only when the network behaves.** This split is the practical content of partial synchrony and the shape of every correct protocol in this volume.
- **Byzantine tolerance is for adversarial, multi-party settings.** It costs $3f+1$ replicas versus $2f+1$ and is rarely justified inside one organization's trust boundary.
- **Two Generals:** no finite protocol achieves common knowledge over a fair-loss link — a sender can never be certain a message arrived. Engineering answer: retry to at-least-once, make processing idempotent (Chapter 9).
- **FLP, precisely:** no *deterministic* consensus protocol *guarantees termination* in an *asynchronous* system with even *one crash* failure — a statement about guaranteed termination, not practical impossibility. Real systems escape via partial synchrony, randomization, or failure detectors, so every real consensus system hides a timeout on which its liveness rests.
- **Crashed and slow are indistinguishable;** timeouts are decisions, not discoveries. This ambiguity begets leases, fencing tokens, and failure detectors ($\diamond P$: eventually accurate, wrong for a while) — and the rule that timeout mistakes may cost availability, never correctness.
- **"Exactly-once delivery" is at-least-once plus deduplication,** only as durable as the dedup state — and delivery is not processing; the end-to-end contract is the application's job.

- **Protocols convert assumptions into guarantees; broken assumptions break guarantees.** GC pause exceeding a lease, a backward-stepping clock, a lying fsync — monitor the assumptions your correctness cites, and test by attacking them (Chapter 12).

Further reading

- Fischer, M. J., Lynch, N. A., and Paterson, M. S., "Impossibility of Distributed Consensus with One Faulty Process," *Journal of the ACM* 32(2), April 1985 — the FLP result; short, readable, and worth the effort. <https://dl.acm.org/doi/10.1145/3149.214121>
- Dwork, C., Lynch, N., and Stockmeyer, L., "Consensus in the Presence of Partial Synchrony," *Journal of the ACM* 35(2), 1988 — the partial-synchrony model and consensus protocols within it. <https://dl.acm.org/doi/10.1145/42282.42283>
- Chandra, T. D. and Toueg, S., "Unreliable Failure Detectors for Reliable Distributed Systems," *Journal of the ACM* 43(2), 1996 — failure detector classes including $\diamond P$, and consensus built on them. <https://dl.acm.org/doi/10.1145/226643.226647>
- Lamport, L., email of 28 May 1987, distributed at DEC SRC — source of the "computer you didn't even know existed" definition; reproduced in the "distribution" entry of Lamport's *My Writings* page. <https://lamport.azurewebsites.net/pubs/distributed-system.txt>
- Deutsch, L. P. (with Gosling's eighth), "The Eight Fallacies of Distributed Computing," Sun Microsystems, 1994-1997 — discussed at length in Rotem-Gal-Oz, A., "Fallacies of Distributed Computing Explained."
- Akkoyunlu, E. A., Ekanadham, K., and Huber, R. V., "Some Constraints and Tradeoffs in the Design of Network Communications," *SOSP*, 1975 — origin of the Two Generals problem; named and popularized in Gray, J., "Notes on Data Base Operating Systems," 1978.
- Lamport, L., Shostak, R., and Pease, M., "The Byzantine Generals Problem," *ACM TOPLAS* 4(3), 1982 — the Byzantine failure model and the $3f+1$ bound.
- Cachin, C., Guerraoui, R., and Rodrigues, L., *Introduction to Reliable and Secure Distributed Programming*, 2nd ed. (Springer, 2011) — the

layered treatment of links, broadcast, and consensus this chapter's abstractions section follows; this volume's formal companion.

- Kleppmann, M., *Designing Data-Intensive Applications* (O'Reilly, 2017), Chapter 8 — "The Trouble with Distributed Systems": the best practitioner-level survey of partial failure, unreliable clocks, and process pauses, including real GC-pause and clock-skew war stories.
- Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *CACM* 21(7), 1978 — the bridge to Chapter 2, and the same happens-before relation Volume 4, Chapter 3 used in-process.
- Volume 4, Chapter 1 — Models of Concurrency — the claim this volume opens by taking seriously; Volume 4, Chapter 2 for the fencing-token design revisited here.
- Volume 3 — the physical realities (TCP, routing, partitions) that this chapter's network models idealize; Volume 5, Chapter 10 — distributed transactions, where Two Generals meets two-phase commit.

Chapter 2 — Time, Clocks, and the Ordering of Events

What this chapter covers. Chapter 1 established that a distributed system has no shared clock, and promised that this chapter would explain what can be salvaged. The answer is precise and a little unsettling: *ordering* can be reconstructed, but *simultaneity* cannot, and physical timestamps — the thing every engineer reaches for first — are the wrong tool for both. We begin with the physics and the plumbing: why quartz oscillators drift, what NTP actually guarantees (far less than assumed), why leap seconds and clock steps make wall time non-monotonic, and why the distinction between `CLOCK_REALTIME` and `CLOCK_MONOTONIC` is not pedantry but the difference between a working system and Cloudflare's 2017 New Year's Day outage. We then rebuild ordering from first principles using Lamport's 1978 happens-before relation — the same relation, literally, that Volume 4, Chapter 3 used for memory models — and the machinery erected on it: Lamport clocks, vector clocks, and hybrid logical clocks, each with its exact guarantee and its exact failure to guarantee. TrueTime appears briefly as the engineered endpoint of the

spectrum. We close with the practical patterns — leases, timeouts, fencing tokens, log ordering, causal metadata — and a synthesis section mapping how every later chapter of this volume consumes what this one builds.

Learning goals — after this chapter you should be able to:

- Do the ppm arithmetic for quartz drift, state what NTP does and does not guarantee, and explain stepping versus slewing, asymmetric-path error, and leap-second smearing accurately.
- State the cardinal rule — never measure a duration with a wall clock — and name the bug classes (negative durations, mis-expired leases) that violating it produces, with the Cloudflare leap-second incident as the canonical case.
- Explain why timestamps cannot order events across nodes, and why last-write-wins conflict resolution silently loses data under skew.
- Define happens-before rigorously — program order, send-to-receive, transitivity — and explain why it is a partial order and what "concurrent" means, connecting it to the identical relation in the Java Memory Model.
- Implement Lamport clocks and state their one-way guarantee and its converse failure; implement vector clocks and state their two-way guarantee; distinguish version vectors from vector clocks.
- Describe the hybrid logical clock construction, what it provides, and what it cannot provide without TrueTime-class hardware.
- Apply the practical patterns: monotonic-clock timeouts, drift-aware leases backstopped by fencing tokens, sequence numbers over timestamps in logs, and causal metadata propagation.

Physical time: what your clock actually is

Every server keeps time with a quartz crystal oscillator: a sliver of quartz that resonates at a nominal frequency when voltage is applied, feeding a counter. The crystal is cheap, and its actual frequency differs from nominal by an amount specified in **parts per million**. Commodity server crystals are typically specified in the range of tens of ppm, and the error moves with temperature, age, and manufacturing variance.

The arithmetic is worth internalizing because it converts a vague "clocks drift" into numbers you can design against. One ppm is one microsecond

of error per second, which is 86.4 milliseconds per day. A 50 ppm crystal therefore drifts up to **4.3 seconds per day**, or roughly two minutes per month, if left undisciplined. Two nodes drifting in opposite directions at 50 ppm diverge from *each other* at 100 ppm — 8.6 seconds per day. An undisciplined fleet does not have "roughly the same time"; it has fifty opinions diverging at meters-per-second scale, and any code that compares timestamps across nodes is comparing those opinions.

Drift rate itself is not constant. Temperature is the dominant factor — a server under load runs hotter than an idle one, and its clock drifts differently — which is why you cannot measure a node's drift once and correct for it forever. This matters later: lease safety arguments assume a *bound* on drift rate, and the bound must hold across the operating envelope, not just at the temperature you measured.

NTP: what it guarantees, and what it merely attempts

The Network Time Protocol disciplines these oscillators against reference clocks. A client exchanges timestamped packets with a server, measures the round-trip, and estimates its offset under one crucial assumption: **that the network path is symmetric** — that the outbound delay equals the return delay. NTP has no way to verify this. The theoretical error bound of a single measurement is half the round-trip time; the *undetectable* error is half the path asymmetry. If the outbound path takes 10 ms and the return takes 30 ms — entirely plausible with asymmetric routing, one congested direction, or asymmetric last-mile links — the offset estimate is wrong by 10 ms and NTP cannot know it. Filtering across many samples and servers reduces noise, not systematic asymmetry.

The practical numbers: a well-run deployment syncing against nearby stratum-1/2 servers on a LAN typically holds offsets under a millisecond; over the public internet, single-digit to tens of milliseconds is normal; a misconfigured or partitioned node can be off by seconds or worse, silently. NTP is a best-effort discipline loop, not a bounded-error service. Nothing in the protocol tells an application "your clock is currently within ϵ of true time" with a guaranteed ϵ — that absence is exactly the gap TrueTime was engineered to fill, and we return to it.

How the correction is applied matters as much as its size:

- **Slewing.** For small offsets, the daemon adjusts the clock's *rate* — running it slightly fast or slow until the error is absorbed. Time remains monotonic; it just flows at up to a few hundred ppm off nominal. Classic `ntpd` slews at most 500 ppm, so absorbing a single second of error takes about 33 minutes.
- **Stepping.** For larger offsets — `ntpd`'s default threshold is 128 ms — the daemon *sets* the clock, discontinuously. The clock jumps, and **it can jump backward**. Every wall-clock reading before and after a backward step is a trap for any code computing differences. (`chrony` behaves similarly under its `makestep` directive; both daemons will also refuse to act at all above a panic threshold — around 1000 s for `ntpd` — on the theory that something is badly wrong.)

So the honest model of a production wall clock is: usually within tens of milliseconds of true time, with an unknown and unknowable error bound, subject to occasional discontinuous jumps in either direction. Design accordingly.

Leap seconds and smearing

UTC is kept within 0.9 s of the Earth's rotation angle by occasionally inserting a **leap second**: a 61st second, labeled 23:59:60, at the end of June 30 or December 31. POSIX time cannot represent 23:59:60 — Unix timestamps pretend leap seconds do not exist — so at insertion, systems must either repeat a second (the kernel steps the clock back, and wall time visits the same timestamps twice) or otherwise fudge. This is the one scheduled event where wall clocks on correctly configured machines go backward by design.

The mitigation most large operators adopted is **leap smearing**: instead of inserting the second discontinuously, the time service lies smoothly. Google's public NTP service (time.google.com) applies a linear smear over the 24 hours centered on the leap second — noon to noon UTC — making each second about 11.6 ppm longer, so the extra second is absorbed with no discontinuity and no 23:59:60. (Google's first smear in 2011 used a cosine-shaped adjustment over a window before the leap; the 24-hour linear smear became their standard afterward.) AWS's Time Sync Service smears similarly. Smearing trades a discontinuity for a deliberate error of up to 0.5 s against true UTC during the window — and creates a new hazard: **smearred and non-smearred sources must never be**

mixed in one fleet, or nodes will disagree by up to a second while each is faithfully following its upstream. Note also that the CGPM resolved in 2022 to discontinue leap seconds by or before 2035; until then, they remain a live operational event.

The net of this section: physical time on a real node is a disciplined-but-drifting oscillator, corrected by a protocol with unverifiable assumptions, subject to steps in both directions, with a scheduled backward jump every few years unless your time source smears. **Clock skew between nodes is not an anomaly; it is the steady state.** Everything else in this chapter follows from taking that seriously.

Two clocks, one rule

Operating systems expose (at least) two distinct clocks, and conflating them is the single most common time bug in backend code. On Linux (Volume 2 covers the timekeeping machinery itself):

- **CLOCK_REALTIME** — the wall clock. Represents UTC as best the system knows it. Settable by the administrator, stepped and slewed by NTP, subject to leap-second handling. Meaningful for *timestamps*: "this happened at 2026-08-14T09:00:00Z."
- **CLOCK_MONOTONIC** — a clock that only moves forward, measuring time since an arbitrary origin (typically boot). It is never stepped. Its *rate* is still gently adjusted by NTP slewing — so it is not a perfect frequency standard — but it cannot jump, and two readings can always be subtracted safely. Meaningful for *durations*: "12.3 ms elapsed." (Linux also offers **CLOCK_BOOTTIME**, which additionally advances across suspend; for servers the distinction rarely matters.)

The cardinal rule, worth stating as an absolute because the exceptions are negligible and the violations are catastrophic:

Never measure a duration with the wall clock. Timeouts, leases, retries, backoff, cache TTLs measured locally, rate limiters, latency measurements, profiling — all of these subtract two clock readings, and all of them must use the monotonic clock. The wall clock is for timestamping events for humans and for cross-referencing with the outside world, nothing else.

The bug class this rule prevents is easy to state. Subtract two wall-clock readings across an NTP backward step and you get a **negative duration**;

across a forward step, a spuriously huge one. A negative duration fed into a timeout makes it fire instantly or never; fed into a lease-expiry check, it makes a lease look expired while valid or valid while expired — a node continues acting as leader after its lease is gone, which is precisely the split-brain scenario leases exist to prevent (Chapter 1's crashed-versus-slow ambiguity, now self-inflicted); fed into a metrics pipeline, it produces the negative latencies that ornament many a post-incident graph.

The language APIs make the distinction easy to honor once you know it exists:

```
// Go – correct: time.Time carries a hidden monotonic reading (since Go 1.9),
// and Sub/Since use it when both operands have one.
start := time.Now()
handleRequest()
elapsed := time.Since(start) // monotonic arithmetic: immune to clock steps

// WRONG: extracting wall-clock values discards the monotonic reading.
t1 := time.Now().UnixMilli()
handleRequest()
d := time.Now().UnixMilli() - t1 // can be negative across an NTP step
_ = d
```

```
// Java – correct: System.nanoTime is the monotonic clock.
long t0 = System.nanoTime();
handleRequest();
long elapsedNanos = System.nanoTime() - t0; // safe: monotonic

// WRONG: currentTimeMillis is the wall clock.
long w0 = System.currentTimeMillis();
handleRequest();
long elapsedMs = System.currentTimeMillis() - w0; // can be negative
```

`System.nanoTime()` values are meaningful only as differences within one JVM — the origin is arbitrary — which is exactly the point: a monotonic reading is not a timestamp and must never be stored, shipped, or compared across processes.

The canonical incident: Cloudflare, 1 January 2017

The leap second inserted at the end of 31 December 2016 gave this bug class its textbook example. At midnight UTC, Cloudflare's authoritative DNS servers began failing for a subset of domains. The server, RRDNS, is

written in Go, and one code path selected among upstream servers using weights derived from recent performance — computed by subtracting wall-clock readings taken with `time.Now()`. At the leap second, the machines' wall clocks went backward, the subtraction produced a **negative duration**, the negative value propagated into the weighting logic, and was eventually passed to `rand.Int63n`, which panics when its argument is not positive. The panic killed the resolution path; the visible symptom, per Cloudflare's postmortem, was failed lookups for a fraction of queries — notably ones requiring CNAME resolution against upstreams — until a fix was deployed in the following hours.

Two details make this canonical rather than merely embarrassing. First, the arithmetic was innocent-looking: `now - then` on values from the standard clock API. Nothing in the code *looked* like it assumed clocks never go backward, yet the assumption was load-bearing. Second, the language fixed it structurally: Go 1.9 (August 2017) embedded a monotonic reading inside `time.Time` precisely so that the natural idiom — `time.Since(start)` — became step-immune by default. That is the correct lesson: durations from wall clocks are a latent bug that fires on the rare backward step, and the fix belongs in the API layer where it cannot be forgotten, not in each call site's discipline.

Why timestamps cannot order events

Now the deeper problem — not that clocks misbehave occasionally, but that even well-behaved clocks cannot do the job people assign them: establishing *which of two events on different nodes happened first*.

The argument is a two-line inequality. Ordering two events by timestamp is valid only if the inter-node clock skew is smaller than the interval between the events. Real skew under healthy NTP is milliseconds to tens of milliseconds. Real systems generate causally related events — request and response, write and dependent write — microseconds to a few milliseconds apart. The skew exceeds the spacing, routinely, by orders of magnitude. So for exactly the events you most need to order — close-together ones, the ones contending for the same key — timestamp order is noise. Node A stamps a write at 10:00:00.005, node B stamps a *later* (causally later — it read A's write first) write at 10:00:00.003 because B's clock runs 4 ms behind. Sorted by timestamp, the effect precedes its cause.

This stops being philosophical the moment a system resolves conflicts by timestamp. **Last-write-wins** replication — Cassandra's cell-level conflict resolution is the prominent example, covered in Volume 5, Chapter 11 — keeps, of two concurrent writes to the same key, whichever bears the larger timestamp, and silently discards the other. Under skew, "larger timestamp" and "later" part company: a client's genuinely newer write loses to an older one stamped by a fast clock. The update is not rejected, not logged, not conflicted — it is simply gone, and the anti-entropy machinery will helpfully propagate the wrong survivor everywhere. This is our running key-value store's first real design decision: when two replicas accept writes to the same key, *something* must decide, and deciding by wall clock means deciding by whose oscillator runs fast. The honest alternatives are to detect concurrency explicitly and keep both (version vectors, below, and Chapter 7's Dynamo lineage) or to make concurrency harmless by construction (Chapter 11's CRDTs).

If physical time cannot order events, what can? Lamport's 1978 answer: stop asking clocks, and read the order off the communication structure of the system itself.

Happens-before: causality without clocks

Lamport's observation is that in a distributed system there are exactly two ways one event can influence another: they occur in the same process, one after the other; or one is the sending of a message and the other is (or follows) its receipt. Everything else is chaining. Formally, the **happens-before** relation $\rightarrow$ is the smallest relation on events such that:

1. **Process order.** If a and b occur in the same process and a precedes b , then $a \rightarrow b$.
2. **Message order.** If a is the sending of a message and b is the receipt of that same message, then $a \rightarrow b$.
3. **Transitivity.** If $a \rightarrow b$ and $b \rightarrow c$, then $a \rightarrow c$.

If neither $a \rightarrow b$ nor $b \rightarrow a$, then a and b are **concurrent**, written $a \parallel b$. Concurrency here is not about wall-clock overlap; it means *no causal path* connects the events — no chain of process steps and messages by which one could have influenced the other. Two events years apart on nodes that never communicated are concurrent in this sense, and that is the right call: for consistency purposes, unrelated is unordered.

This makes $\rightarrow$ a **partial order** — irreflexive, transitive, and crucially *not total*. Most pairs of events in a large system are simply incomparable, and the entire discipline of this chapter is refusing to invent an order where none exists. If you worked through Volume 4, Chapter 3, you have seen this relation before — not an analogue of it, the relation itself. The Java Memory Model's happens-before is Lamport's definition with "message send/receive" replaced by "volatile write/read" and "unlock/lock," and JSR-133 says so. Learned once, it applies twice: a volatile write publishing prior plain writes and a message send carrying prior state are the same edge in the same partial order, one across threads, one across machines.

```
sequenceDiagram
    participant P1 as Process P1
    participant P2 as Process P2
    participant P3 as Process P3
    Note over P1: event a - local
    P1->>P2: message m1 - sent at event b, received at event c
    Note over P3: event e - local, before m2 arrives
    P2->>P3: message m2 - sent at event d, received at event f
    Note over P1,P3: Ordered by chaining: a to b by process order,
    b to c by message m1, c to d by process order,
    d to f by message m2 - so a happens-before f
    Note over P1,P3: CONCURRENT pair: e and a - also e and b, c, d.
    No path of process steps and messages connects them.
    e is ordered only after f arrives on P3.
```

The relation answers the ordering question exactly where it can be answered and refuses where it cannot. What it does not yet give us is anything a program can *compute with*: $\rightarrow$ is defined over the global execution, which no node sees. The rest of the chapter is about mechanisms that let nodes carry enough local state to answer questions about $\rightarrow$ — each mechanism trading size and complexity for how much of the relation it captures.

Lamport clocks: timestamps consistent with causality

The first mechanism is almost embarrassingly small. Each process keeps a single integer counter L , and:

- Before each local event, increment L ; the event's timestamp is the new value.
- On sending a message, increment L and attach it to the message.

- On receiving a message carrying timestamp t , set $L = \max(L, t) + 1$; that is the receive event's timestamp.

```
class LamportClock:
    def __init__(self):
        self.t = 0

    def local_event(self):
        self.t += 1
        return self.t

    def send(self):
        self.t += 1
        return self.t          # attach this value to the outgoing
message

    def receive(self, msg_t):
        self.t = max(self.t, msg_t) + 1
        return self.t
```

The `max` on receive is the whole trick: it drags a lagging receiver's counter forward past the sender's, so every message edge — and by induction every happens-before chain — strictly increases the timestamp. That yields the **clock condition**:

If $a \rightarrow b$, then $L(a) < L(b)$.

And now the part that generates production bugs when missed: **the converse is false**. $L(a) < L(b)$ does *not* imply $a \rightarrow b$. Two concurrent events on nodes that have never exchanged messages get whatever counter values their local histories produced; one will be numerically smaller, and the comparison means nothing. A Lamport timestamp compresses a partial order into a single integer, and the compression is lossy in exactly one direction: it can *confirm* an ordering it was told about, but it **cannot detect concurrency** — given two timestamps, you cannot distinguish "a caused b" from "unrelated." Any design that reads causality out of Lamport timestamp comparisons is broken by construction.

```

flowchart LR
    subgraph P1["Process P1 - exchanges no messages with P2"]
        a1["event a  
L = 1"] --> a2["event  
L = 2"]
    end
    subgraph P2["Process P2"]
        b1["event  
L = 1"] --> b2["event  
L = 2"]
        b2 --> b3["event b  
L = 3"]
    end
    a1 -->| "L of a = 1, L of b = 3, so L a is less than L b  
yet a and b are CONCURRENT:  
no happens-before path exists between them.  
The converse of the clock condition FAILS." | b3

```

What Lamport clocks *are* good for is manufacturing a **total order consistent with causality**. Break ties between equal timestamps by process ID — order events by the pair (L, pid) — and every node that sees the same set of events sorts them identically, with the sort never contradicting $\rightarrow$. The order is partly arbitrary (it orders concurrent events too, by fiat), but it is *agreed*, and an agreed total order is the raw material of state machine replication: deliver the same operations in the same order everywhere and replicas stay identical. Lamport's own paper demonstrates this with a distributed mutual-exclusion algorithm — requests served in (L, pid) order, every node maintaining the same request queue with no central lock server. Total-order broadcast, and the term and ballot numbers of Chapters 5 and 6, are this idea industrialized; hold that thought for the synthesis section.

Vector clocks: capturing the whole relation

To detect concurrency, one counter is provably insufficient — the mechanism must record *whose* history an event has absorbed, per process. That is a **vector clock**, developed independently by Fidge and by Mattern in 1988–89. In a system of N processes, each process i keeps a vector V of N counters, where $V[j]$ means "the number of events of process j that this process has heard of":

- On a local event or send, increment your own slot $V[i]$; a send attaches a copy of the whole vector.

- On receive, merge: take the **element-wise max** of your vector and the message's, then increment your own slot (the receipt is itself an event).

```
class VectorClock:
    def __init__(self, n, i):
        self.v = [0] * n          # one slot per process
        self.i = i                # this process's own index

    def local_event(self):
        self.v[self.i] += 1

    def send(self):
        self.v[self.i] += 1
        return list(self.v)       # attach a copy to the message

    def receive(self, msg_v):
        self.v = [max(a, b) for a, b in zip(self.v, msg_v)]
        self.v[self.i] += 1

def leq(a, b):
    # every component of a is <= that of b
    return all(x <= y for x, y in zip(a, b))

def happened_before(a, b):      # a -> b
    return leq(a, b) and a != b

def concurrent(a, b):
    return not leq(a, b) and not leq(b, a)
```

Compare vectors component-wise: $VC(a) < VC(b)$ iff every component of $VC(a)$ is $\leq$ the corresponding component of $VC(b)$ and at least one is strictly less. The guarantee is now an **if and only if**:

$VC(a) < VC(b) \iff a \rightarrow b$, and the vectors are incomparable — each strictly greater than the other in some slot — $\iff a \parallel b$.

The vector clock *characterizes* happens-before: the partial order on events is exactly mirrored by the partial order on vectors. Given two timestamps and nothing else, you can answer "did one cause the other, or are they concurrent?" — the question Lamport clocks cannot.

A worked example with three nodes:

```

sequenceDiagram
    participant A as Node A
    participant B as Node B
    participant C as Node C
    Note over A: a1 local event
    VA = [1,0,0]
    Note over B: b1 local event
    VB = [0,1,0]
    A->>B: a2 send, attaches VA = [2,0,0]
    Note over B: b2 receive - merge:
    max of [0,1,0] and [2,0,0] is [2,1,0],
    then own slot +1: VB = [2,2,0]
    Note over C: c1 local event
    VC = [0,0,1]
    C->>B: c2 send, attaches VC = [0,0,2]
    Note over B: b3 receive - merge:
    max of [2,2,0] and [0,0,2] is [2,2,2],
    then own slot +1: VB = [2,3,2]
    Note over A,C: c1 = [0,0,1] vs b2 = [2,2,0]: each is greater
    in some slot - incomparable - CONCURRENT.
    a1 = [1,0,0] vs b3 = [2,3,2]: [1,0,0] is smaller
    in every slot - a1 happened-before b3.

```

Walk the two comparisons. $c1 = [0,0,1]$ versus $b2 = [2,2,0]$: $c1$ is greater in slot C, $b2$ greater in slots A and B — incomparable, therefore concurrent, which is right: no message connected C to B before $b2$. After $b3$ merges C's vector, $b3 = [2,3,2]$ dominates $c1$ — and indeed $c1 \rightarrow c2 \rightarrow b3$ via the message. The mechanism detected exactly what the communication structure created, nothing more.

Version vectors: the same mathematics, a different question

A near-identical structure appears in replicated storage under the name **version vector**, and the two are worth distinguishing because the literature (and more than one codebase) confuses them. A vector clock timestamps *every event* in a computation, one slot per process. A version vector tracks the state of *one replicated object*, one slot per replica, incremented only when that replica *writes* the object. The comparison rule is the same element-wise partial order, but the question answered is "does this copy of the object subsume that one, or did they diverge?" — per-key causality, not whole-system causality.

This is the Dynamo lineage (Chapter 7 covers the full design): each object carries a version vector; a replica accepting a write increments its slot; when a read finds copies with incomparable vectors, the store has

detected **concurrent writes** and surfaces both as **siblings** for the application (or a merge function) to reconcile, rather than silently picking one. Contrast that directly with last-write-wins: same situation, but LWW consults the wall clock and destroys a branch; version vectors consult causality and preserve both. Chapter 11's CRDTs complete the arc by making the merge automatic and principled.

The costs

The two-way guarantee is bought with real costs, and they bound where vector clocks are usable:

- **O(N) size.** Every message and every stored version carries a vector with one slot per participant. Tolerable for five replicas; painful for thousands of clients.
- **Actor churn.** Slots are per-actor, and actors come and go. If *clients* get slots — as early Dynamo-style designs did, to correctly attribute writes — the vector grows with every client that ever touched the key. Riak's history documents this pressure and the move to server-side IDs plus **dotted version vectors**, which pin down precisely which write each slot's counter refers to and keep vectors sized by replica count.
- **Pruning hazards.** The tempting fix — cap the vector and evict old entries, as Dynamo did at ten entries — is not free: discarding a slot discards causal history, and two versions that were genuinely ordered can afterward appear concurrent, resurfacing conflicts the system had already resolved. Pruning trades bounded metadata for false concurrency; do it knowingly or not at all.

Hybrid logical clocks: causality at wall-clock scale

Lamport and vector clocks share a practical annoyance: their timestamps mean nothing to humans or to anything outside the system. You cannot look at Lamport time 4,182,331 and know it was last Tuesday, cannot correlate it with logs, cannot use it in a `WHERE` clause a DBA would recognize. Physical timestamps have meaning but violate causality; logical timestamps honor causality but have no meaning. The **hybrid logical clock** (Kulkarni, Demirbas, et al., 2014) is the construction that gets both: timestamps that respect happens-before *and* stay provably close to physical time.

An HLC timestamp is a pair `(l, c)`: `l` is the largest *physical* clock reading the node has encountered — its own or any message sender's — and `c` is a logical counter that breaks ties when causally related events would otherwise share an `l`. The update rules are a Lamport clock wearing physical time:

```
type HLC struct {
    mu sync.Mutex
    l  int64 // max physical time seen so far (e.g., nanoseconds)
    c  int32 // logical counter, tie-break within one value of l
}

// Now: timestamp a local or send event. pt is the wall clock, e.g.
// time.Now() truncated to the chosen resolution.
func (h *HLC) Now(pt int64) (int64, int32) {
    h.mu.Lock()
    defer h.mu.Unlock()
    if pt > h.l {
        h.l, h.c = pt, 0 // physical time moved past us: adopt it,
        reset c
    } else {
        h.c++ // physical time stalled or behind: advance logically
    }
    return h.l, h.c
}

// Update: timestamp a receive event carrying the sender's (ml, mc).
func (h *HLC) Update(pt, ml int64, mc int32) (int64, int32) {
    h.mu.Lock()
    defer h.mu.Unlock()
    switch {
    case pt > h.l && pt > ml: // wall clock beats everything
        h.l, h.c = pt, 0
    case ml > h.l: // sender's l is the max: adopt it, go past its c
        h.l, h.c = ml, mc+1
    case h.l > ml: // our l is the max: keep it, bump c
        h.c++
    default: // h.l == ml: merge the counters
        if mc > h.c {
            h.c = mc
        }
        h.c++
    }
    return h.l, h.c
}
```

Compare timestamps lexicographically: (l, c) pairs ordered by l first, then c . The properties, from the paper:

- **Causality, one-way.** If $e \rightarrow f$ then $hlc(e) < hlc(f)$ — the Lamport clock condition, inherited by the same argument. (And with the same converse failure: HLCs, like Lamport clocks, cannot detect concurrency. They order; they do not diagnose.)
- **Bounded divergence from physical time.** $l \geq pt$ always, and $l - pt$ is bounded by the maximum clock offset among communicating nodes: l can only run ahead of the local wall clock by adopting some other node's wall clock reading. The counter c is bounded too. So an HLC reads as "wall time, possibly a hair fast, plus a small tie-break" — close enough to real time for TTLs, log correlation, and human forensics.
- **One machine word.** Because c is small, the pair packs into 64 bits — the paper's scheme rounds the low-order bits of a 64-bit timestamp and stores c there — so an HLC drops into any schema, API, or protocol field designed for a plain timestamp. Monotone, causal, and wire-compatible with `int64` time: that is the entire sales pitch.

```

flowchart TB
    PT["local wall clock reading pt  
CLOCK_REALTIME, NTP-disciplined"]
    MSG["incoming message timestamp  
ml and mc from the sender"]
    MAX["new l = max of old l, ml, pt"]
    DEC["which source  
supplied the max?"]
    C0["pt alone:  
c = 0"]
    C1["sender's ml:  
c = mc + 1"]
    C2["old l:  
c = c + 1"]
    merge["merge counters on a tie"]
    OUT["timestamp l dot c - fits in 64 bits  
l stays within max clock offset of pt  
c carries causality when l ties or stalls"]
    PT --> MAX
    MSG --> MAX
    MAX --> DEC
    DEC --> C0
    DEC --> C1
    DEC --> C2
    C0 --> OUT
    C1 --> OUT
    C2 --> OUT

```

This is why HLCs are the workhorse of modern distributed databases — CockroachDB's transaction timestamps and MongoDB's cluster time are HLCs, and Volume 5, Chapter 12 covered how CockroachDB builds MVCC and its uncertainty-interval reads on top of them; the database mechanics stay there. What matters here is the foundational boundary: **an HLC guarantees that causally related events are correctly ordered, but says nothing about causally *unrelated* ones.** If client 1 commits at node A, phones client 2 out of band, and client 2 then writes at node B — no message between A and B — the HLCs may order the second write before the first, because out-of-band causality never touched the clock. Closing that hole means bounding real clock error and *waiting out the bound*, which is not an algorithm but a hardware-and-operations commitment.

TrueTime: buying the bound

That commitment is Google's **TrueTime**, the other endpoint of the spectrum, presented with Spanner in 2012 (and applied to transactions in Volume 5, Chapter 12 — foundations only here). TrueTime changes the API: instead of a scalar, `TT.now()` returns an **interval** `[earliest, latest]` guaranteed to contain true time. The guarantee is manufactured, not assumed: GPS receivers and atomic clocks in every datacenter, redundant time masters cross-checking one another, and per-machine daemons that compute the uncertainty ε from measured round-trips and worst-case drift since last sync. The paper reports ε typically ranging up to about 7 ms, sawtoothing as clocks are re-disciplined.

The interval enables **commit wait**: a transaction takes its commit timestamp `s` at the top of its interval, then deliberately waits until `TT.now().earliest > s` before making the commit visible. After the wait, `s` is unambiguously in the past on every node's clock, so any transaction that starts afterward — anywhere, related by messages or not — gets a strictly larger timestamp. That is **external consistency**: timestamp order matches real-time order even for the out-of-band case that defeats HLCs. The price is explicit — commit latency of roughly ε on every write, plus the clock infrastructure — and the design lesson is symmetrical: Spanner pays milliseconds and hardware to make wall time trustworthy; HLC systems pay uncertainty handling and weaker guarantees to avoid the hardware. There is no third option in which ordinary NTP-grade clocks provide external consistency; a bound you did not engineer is a bound you do not have.

Practical patterns

The theory compresses into a short list of habits that distinguish systems that survive clock trouble from systems that discover it in production.

Leases are clock-rate bets; fence them. A lease — "you are the leader for the next 10 seconds" — is the workhorse liveness tool from Chapter 1, and it is built entirely on clock assumptions: the grantor and holder measure the same 10 seconds only if both clocks run at approximately the correct *rate*. Bounded drift makes the rate bet reasonable (100 ppm across a 10 s lease is 1 ms of error — absorbable with a small safety margin subtracted from the holder's view). But the bet is only sound if both sides measure with monotonic clocks — a wall-clock step can stretch

or collapse the holder's view of its own lease arbitrarily — and even then it can be lost outright: a GC pause, a VM migration, or an I/O stall can suspend the holder past expiry with the holder none the wiser, exactly the crashed-versus-slow ambiguity again. So the lease must be backstopped by causality rather than time: a **fencing token**, a counter that increases with each lease grant, carried on every operation and checked by the resource, so a stale holder's writes are rejected no matter what its clock believes (the pattern Volume 4, Chapter 2 introduced alongside locks, and Chapter 8 shows implemented with ZooKeeper's `zxid` and etcd's revision numbers — both of which are, notice, Lamport-style logical counters). Time grants the lease; causality revokes it.

Timeout hygiene. Every timeout, deadline, retry budget, and backoff computation uses the monotonic clock — which in practice means using your language's duration-native APIs (`time.Since`, `context.WithTimeout`, `System.nanoTime`, deadline objects) and never storing "the wall time at which this expires" for local arithmetic. Audit for calls that serialize a wall-clock expiry into a struct and compare it against `now` later; each one is a Cloudflare waiting for a step.

Timestamp columns, honestly. `created_at` columns and audit logs should carry server-assigned wall-clock timestamps — client clocks are not evidence — and consumers must treat them as approximate and skew-bearing: fine for humans, for retention policies, and for correlating with the outside world; not valid as an ordering key between rows written by different nodes, and never as a uniqueness or concurrency-control mechanism. If ordering matters, store an explicit sequence.

In logs and streams, sequence numbers beat timestamps. A partitioned log's real ordering contract is the per-partition sequence — Kafka's offsets — assigned by the single broker that owns the partition: a genuine total order within the partition, no clocks involved, and deliberately no order across partitions. Event-time timestamps ride along as payload for windowing and analytics (Volume 10 treats watermarks and lateness), but consumers that need "what order did these happen" must read offsets, not timestamps. The same rule generalizes: whenever a single writer exists, a plain sequence number is the cheapest and strongest ordering primitive available — which is much of why systems go to such lengths (Chapters 5 and 6) to elect one.

Propagate causal metadata explicitly. Causality only orders what the metadata path records, so real systems thread tokens through their calls:

session tokens that encode "reads must observe at least this state" (the session guarantees of Chapter 3 — MongoDB's causal-consistency sessions carry an HLC value in exactly this role), and distributed-tracing context — the W3C `traceparent` header with its trace and parent-span IDs — which is happens-before plumbing in production dress: each span records its causal parent, and the assembled trace is a fragment of the happens-before DAG rendered as a flame graph (Volume 11, Chapter 4). If a causal dependency crosses a channel that carries no metadata — a phone call, a human swivel-chairing between two UIs — no clock in this chapter sees it; only TrueTime-class bounds cover out-of-band causality.

Synthesis: how the rest of the volume consumes this chapter

This chapter is infrastructure for every one that follows; it is worth being explicit about the load each puts on it.

Chapter 3 — Replication and Consistency Models needs a definition of "afterness" before it can define anything. Linearizability's whole content is that operations respect *real-time* order — if operation A completes before operation B begins, B must observe A — which quietly assumes an observer for whom "completes before" is meaningful; the formal definitions use an idealized global time precisely because, as this chapter showed, no node has one. Causal consistency, one rung down, is this chapter's `→` promoted to a consistency model: replicas must agree on the order of causally related writes and are free to disagree on concurrent ones. The ladder of consistency models is largely a ladder of how much ordering you demand.

Chapters 5 and 6 — Paxos and Raft — replace clocks with counters, and the counters are Lamport clocks. A Paxos ballot number and a Raft term are logical values that only move forward, carried on every message, with every receiver updating to the max it has seen and rejecting messages from the past — reread the Lamport clock rules and you will recognize every clause. Consensus is what you use when you need the *agreed total order* that Lamport timestamps sketch but cannot make fault-tolerant on their own; where the protocols do touch physical time (Raft's randomized election timeouts, leader leases for local reads), they inherit exactly the drift and monotonicity caveats of this chapter.

Chapter 7 — Quorums and Dynamo-style replication puts version vectors to work: sibling detection on read, read-repair deciding which copies subsume which, and the pruning trade-offs above become operational decisions with data-loss consequences.

Chapter 10 — Failure detection is built on timeouts, and a timeout is a clock assumption wearing a trench coat: "no heartbeat for T seconds" presumes the detector's T seconds bear some relation to the peer's. Every accuracy/completeness trade-off in failure detectors is the crashed-versus-slow ambiguity of Chapter 1 measured with the imperfect clocks of Chapter 2.

Chapter 11 — CRDTs takes the most radical position: abandon ordering questions wherever possible by making the data types commutative, so concurrent updates merge to the same state in any order. The mathematics is the partial-order toolkit of this chapter — CRDT state forms a lattice whose join is a generalized "element-wise max," and vector-clock merge is itself the canonical example of one.

And the tie back inward: Volume 4, Chapter 3's table mapped store buffers to replication lag and volatile edges to message edges. You can now read that mapping with full precision — a volatile write/read pair and a message send/receive pair are the same happens-before edge; a replica that has not yet applied a write is a store buffer that has not yet drained; and DRF-SC's bargain ("order everything through proper edges and you may think sequentially") reappears here as the choice among consistency models. One relation, learned once, load-bearing in both worlds.

Key takeaways

- **Clock skew is the steady state.** Quartz drifts at tens of ppm — 50 ppm is 4.3 s/day — and NTP disciplines it to maybe milliseconds-to-tens-of-milliseconds with *no guaranteed bound*: path asymmetry is undetectable, and corrections arrive as rate slews or as discontinuous steps that can move the clock backward. Leap seconds add a scheduled backward step unless your time source smears; never mix smeared and non-smeared sources.
- **Never measure durations with the wall clock.** Timeouts, leases, backoff, profiling — all use `CLOCK_MONOTONIC ( time.Since , System.nanoTime )`. The failure class is negative or wildly wrong

durations at clock steps; Cloudflare's 2017 outage — a leap-second backward step producing a negative Go duration that panicked `rand.Int63n` — is the canonical instance, and Go 1.9's `monotonic time.Time` is the structural fix.

- **Timestamps cannot order events across nodes** when skew exceeds event spacing, which it does for exactly the events that contend. Last-write-wins resolution converts that into silent data loss: the causally newer write can lose to the faster clock.
- **Happens-before** — process order plus send-to-receive plus transitivity — is a *partial* order; unordered pairs are *concurrent*, meaning no causal path exists. It is literally the JMM's relation from Volume 4, Chapter 3, one layer up.
- **Lamport clocks** guarantee $a \rightarrow b \Rightarrow L(a) < L(b)$ in one integer — but the converse fails: they cannot detect concurrency. With a node-ID tie-break they yield an agreed total order consistent with causality — the seed of total-order broadcast and of consensus terms/ballots.
- **Vector clocks** capture the relation exactly — $VC(a) < VC(b) \Leftrightarrow a \rightarrow b$, incomparable $\Leftrightarrow$ concurrent — at $O(N)$ cost per message, with actor churn and pruning as the operational hazards. **Version vectors** apply the same math per replicated object for sibling detection.
- **HLCs** fuse a Lamport clock with physical time: causally monotone, provably close to the wall clock, one 64-bit word — but one-way like Lamport clocks, and no external consistency for out-of-band causality. That requires **TrueTime**: an engineered uncertainty interval plus commit-wait, paying ϵ of latency and real hardware for the bound.
- **Practical defaults**: leases assume bounded drift and monotonic measurement, and must be fenced with tokens (causality) anyway; server-assigned timestamps are for humans, sequence numbers are for ordering; per-partition sequence is the real contract of a log; propagate causal tokens (sessions, `traceparent`) because clocks cannot see causality they were never told about.

Further reading

- Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *CACM* 21(7), 1978 — the happens-before relation, logical clocks, and the mutual-exclusion example; arguably

the most influential paper in distributed systems. <https://dl.acm.org/doi/10.1145/359545.359563>

- Fidge, C., "Timestamps in Message-Passing Systems That Preserve the Partial Ordering," *Proc. 11th Australian Computer Science Conference*, 1988 — vector clocks, one of the two independent inventions.
- Mattern, F., "Virtual Time and Global States of Distributed Systems," *Parallel and Distributed Algorithms*, 1989 — the other, with the global-snapshot connection.
- Kulkarni, S., Demirbas, M., Madappa, D., Avva, B., and Leone, M., "Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases" (Hybrid Logical Clocks), *OPODIS*, 2014 — the HLC construction, bounds, and 64-bit encoding. <https://cse.buffalo.edu/tech-reports/2014-04.pdf>
- Corbett, J. et al., "Spanner: Google's Globally-Distributed Database," *OSDI*, 2012 — TrueTime, commit wait, and external consistency. <https://research.google/pubs/spanner-googles-globally-distributed-database/>
- Graham-Cumming, J., "How and why the leap second affected Cloudflare DNS," Cloudflare blog, January 2017 — the canonical wall-clock-duration postmortem. <https://blog.cloudflare.com/how-and-why-the-leap-second-affected-cloudflare-dns/>
- Google, "Leap Smear" — <https://developers.google.com/time/smeat> — the 24-hour linear smear used by time.google.com, with the history of the earlier cosine smear.
- Kleppmann, M., "How to do distributed locking," 2016 — <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html> — the fencing-token argument in full, including why clock assumptions alone cannot make locks safe.
- Preguiça, N., Baquero, C., et al., "Dotted Version Vectors: Logical Clocks for Optimistic Replication," 2010 — the fix for actor-churn growth in version vectors.
- *The Go Time Package* — <https://pkg.go.dev/time> — the documented wall/monotonic dual representation added in Go 1.9.
- Volume 4, Chapter 3 — Memory Models and Happens-Before — the same relation inside one process.

- Volume 5, Chapters 11 and 12 — Cassandra's LWW mechanics, and Spanner/CockroachDB's use of TrueTime and HLCs in transaction processing.

Chapter 3 — Replication and Consistency Models

What this chapter covers. Volume 5, Chapter 8 — Replication covered the machinery: leaders and followers, replication logs, synchronous versus asynchronous propagation, failover. This chapter is about what that machinery *promises*. A consistency model is a contract between a replicated data system and its clients, stated as a constraint on the histories of operations the system may produce. Every replicated store implements some model, whether its documentation names it or not, and every anomaly your users report — the comment that appears before the post it replies to, the profile edit that vanishes on refresh, the counter that goes backwards — is the visible shape of a contract weaker than the one your code silently assumed.

This chapter also closes a promise made in Volume 4, Chapter 3 — Memory Models and Happens-Before. That chapter ended by claiming that memory consistency and distributed consistency are the same design space, one layer apart. Here we cash that claim in full: sequential consistency reappears verbatim, happens-before returns as causal consistency, the store buffer returns as replica lag, and the DRF-SC bargain returns as the linearizable-spine architecture. If you worked through that chapter, most of the intuitions here will feel like recognition rather than learning.

Learning goals — after this chapter you should be able to:

- Define a consistency model precisely, as a predicate over operation histories, and judge concrete read/write histories valid or invalid under a given model.
- State linearizability exactly (Herlihy & Wing), explain why composability makes it the gold standard, name what provides it, and price its coordination costs.
- Distinguish linearizability from serializability without hand-waving, and define strict serializability as their conjunction.

- Define sequential consistency, causal consistency, and the four session guarantees of Terry et al., each with the specific anomaly it forbids.
- Explain why causal consistency is the strongest model available under partition, and how vector clocks and dependency tracking implement it.
- Describe eventual consistency honestly — as a liveness property, not an ordering guarantee — and enumerate what applications must then handle themselves.
- Assemble the model lattice, place the transaction-isolation axis orthogonally to it, and read the Jepsen consistency map.
- Audit a real product's consistency claim: find the precise statement, identify its scope, and know how to test it.
- Choose models invariant-first, and design linearizable-spine / eventual-periphery systems.

A consistency model is a contract over histories

Strip away the implementation and a replicated data system is an object — a key-value map, a register, a queue — that many clients invoke operations on. Each operation is not a point in time but an **interval**: it has an *invocation* (the client sends the request) and a *response* (the client receives the result), with real time passing between them. A **history** is the record of all invocations and responses across all clients. Two operations are **concurrent** if their intervals overlap; otherwise one *precedes* the other in real time.

A **consistency model is a predicate on histories**: it partitions all conceivable histories into those the system is permitted to produce and those it is not. "Strong" models admit few histories and therefore forbid many anomalies; "weak" models admit many histories, including surprising ones. This framing is exactly the one Volume 4, Chapter 3 used for memory models — there the operations were loads and stores by threads against shared memory, here they are reads and writes by clients against replicated storage — and it is the right framing because it separates the *contract* from the *mechanism*. Volume 5, Chapter 8 showed that the same contract can be implemented by very different machinery, and the same machinery, misconfigured, can silently deliver a much weaker contract than intended.

Judging histories

Return to the picture above. Suppose B returns 1 and C returns 1:

```
A: |----- write(x, 1) -----|
B:      |-- read(x) → 1 --|
C:                                |-- read(x) → 1 --|
```

Linearizable. Place A's linearization point early in its interval, before B's; B reads 1; C reads 1. Suppose instead B returns 0 and C returns 1: also linearizable — place A's point *after* B's but before its own response. Concurrency buys the system freedom, and both outcomes for B are legal precisely because B overlaps the write.

Now the history that is *not* linearizable, and it is the one that matters in practice:

```
A: |----- write(x, 1) -----|
B:      |-- read(x) → 1 --|
C:                                |-- read(x) → 0 --|      ←
INVALID
```

B returned 1, so A's linearization point must precede B's response. C began after B's response, so C's linearization point comes later still — yet C read 0, the value from before the write. There is no placement of points that explains this history. The value went *backwards* in real time. This is exactly what a lagging read replica produces: B's read hit the leader, C's hit a follower that had not yet applied the write. The machinery from Volume 5, Chapter 8 was working as designed; the contract it delivered was not linearizability.

```
sequenceDiagram
    participant A as Client A
    participant R as Register x
    participant B as Client B
    participant C as Client C

    Note over R: x = 0
    A->>R: invoke write x = 1
    B->>R: invoke read x
    R-->>B: response 1
    Note over R: write must have taken effect
    before B's response
    A-->>A: write response OK
    C->>R: invoke read x
    R-->>C: response 0
    Note over C: INVALID under linearizability
    C began after B finished
    yet read an older value –
    no commit points explain this
```

Why linearizability is the gold standard: locality

Herlihy and Wing proved a property they called **locality**, and it is the deepest reason linearizability occupies the position it does: **a history over multiple objects is linearizable if and only if each object's sub-history is linearizable**. Linearizable objects *compose*. Build your lock from a linearizable register, your queue from a linearizable log, your leader election from a linearizable compare-and-swap, and the combined system is linearizable with no further reasoning. Sequential consistency, which we meet next, does not have this property — two individually sequentially-consistent objects can combine into a system that is not sequentially consistent — and that non-composability is a large part of why linearizability, not SC, became the reference model for distributed systems even though SC came first.

Locality is also why linearizability is what you need for the classic coordination invariants: mutual exclusion, uniqueness constraints, "exactly one leader." Each reduces to operations on one linearizable object, and composition takes care of the rest.

What provides it

Three families of mechanisms deliver linearizability, all of them recognizably "make it act like one machine":

- **Consensus-backed replication.** A Paxos or Raft group (Chapters 5 and 6) serializes all writes through a replicated log and serves reads either through the log or via leader leases. This is what etcd, ZooKeeper's write path, Spanner, and CockroachDB do. It is the mainstream answer.
- **Single-leader synchronous replication** with reads at the leader — the Volume 5, Chapter 8 configuration where followers exist for durability and failover, not for serving reads. Correct until failover, at which point you need consensus (or fencing) anyway to prevent two leaders.
- **Strict quorums over an unreplicated-log substrate.** The ABD algorithm (Attiya, Bar-Noy, and Dolev, 1995) shows that a linearizable read/write register — reads and writes only, no compare-and-swap — is achievable in an asynchronous message-passing system with a majority of nodes up, *without* consensus. The essential trick is that a read must complete a **write-back phase**: after collecting values from a quorum, the reader writes the freshest value back to a quorum before returning it, so a later reader cannot observe an older value. This is the caveat that bites Dynamo-style systems: overlapping quorums ($R + W > N$) alone do **not** give linearizability if reads return without the write-back, because two readers can straddle a partially-completed write and observe it in opposite orders. Cassandra's `QUORUM` reads with asynchronous read repair are in exactly this position, which is why linearizable operations there require the separate Paxos-based `SERIAL` path (Chapter 7 works through this in detail).

What it costs

Linearizability's price is coordination on **every operation**, and the bill has three lines:

- **Latency.** An operation must be acknowledged by, or observed at, a set of nodes sufficient to order it against all concurrent operations — in practice at least one round trip to a quorum or to a leader that

holds a lease. Cross-region, that is tens of milliseconds *per operation*, an irreducible floor set by geography, not software.

- **Throughput.** Serializing through a leader or a quorum concentrates load; the object's operation rate is bounded by what one coordination group can order.
- **Availability.** During a network partition, the minority side cannot know whether the majority is processing writes, so it must refuse to serve linearizable reads or writes. This is not an implementation weakness; it is the content of the CAP theorem, and Chapter 4 states it precisely. For now: a linearizable register cannot be available on both sides of a partition, full stop.

These costs are why the rest of this chapter exists. If linearizability were free, no other model would be worth naming.

Linearizability versus serializability — disambiguated

These two words are confused constantly, including in vendor documentation, and the confusion is worth killing carefully because they constrain different things.

Linearizability is about *single operations on single objects*, with a *real-time* constraint. It says nothing about transactions — there is no notion of grouping operations.

Serializability (Volume 5, Chapter 6 — Isolation and MVCC) is about *multi-operation transactions over many objects*. It requires the outcome to equal *some* serial execution of the transactions — but **any** serial order will do, with no obligation to respect real time. A database may commit transaction T2 "before" T1 in its serial order even though T1 completed in real time before T2 began. Serializability without a real-time constraint permits stale reads: a serializable read-only transaction may legally observe a days-old snapshot, because ordering it into the past is *some* serial order.

The conjunction — serializable transaction ordering *and* the real-time constraint — is **strict serializability**: transactions appear to execute atomically, in an order consistent with real time. This is what Spanner's TrueTime machinery buys (Volume 5, Chapter 12 called it *external consistency*, Google's term for the same property), and it sits at the top of the lattice we assemble below. A useful mnemonic: **linearizability = strict serializability restricted to one-operation transactions**;

serializability = strict serializability minus real time. The two axes — how operations group into transactions, and whether real time is respected — are independent, and the lattice section returns to this.

Sequential consistency: the same total order, without the clock

Lamport defined sequential consistency in 1979 for multiprocessors, and Volume 4, Chapter 3 quoted the definition in full: the result of any execution equals *some* single total order of all operations, in which each process's operations appear in its program order. Transplanted to distributed systems word-for-word — replace "processor" with "client session" — it is exactly the same model, and this is the transplant that chapter promised.

Compare it to linearizability: the total order survives, the per-client program order survives, but the **real-time constraint is gone**. The system must behave as if there were one global sequence of operations that everyone agrees on — but that sequence may disagree with wall-clock order for operations from *different* clients. Concretely:

```
A: |-- write(x, 1) --|
                                     (much later, in real time)
B:                                     |-- read(x) → 0 --| ← valid
under SC
B:                                     |-- read(x) → 0
--| ← still valid
B:                                     |--
read(x) → 1 --|
```

B reading 0 long after A's write completed is **legal**: order B's reads before A's write in the total order. What SC forbids is *inconsistency in the story*: once B has observed the write, B may never observe its absence again (that would violate B's program order against the total order), and no two clients may observe writes to the system in contradictory orders. The intuition is **a stale but internally consistent prefix**: every client watches the same movie, in the same scene order; some clients are simply further behind, and no one ever sees scenes out of order or un-happen.

The store-buffer litmus test from Volume 4, Chapter 3 — `r1 == 0 && r2 == 0` — is exactly a sequential-consistency violation,

which is why that outcome felt impossible: your intuition *is* SC. Distributed systems that provide SC-but-not-linearizability are not exotic. ZooKeeper is the canonical example: writes are totally ordered through the ZAB protocol, but a client reads from the follower it happens to be connected to, which may serve a stale prefix of that order. Every client sees the same order of updates; not every client is equally caught up. ZooKeeper's `sync()` operation is the escape hatch — it forces the client's view up to at least the point of the call, buying linearizable-read behavior per use — and Chapter 8 examines why that design point is exactly right for a coordination service's read-heavy workload.

SC's cost profile explains its niche: it still requires a global total order (so writes still need consensus or a leader), but reads escape coordination — any replica can serve its prefix. You pay for ordering writes; reads are fast and stale.

Causal consistency: happens-before, across machines

Drop the requirement of a single total order and keep only what actually matters to humans: **cause precedes effect**. Causal consistency requires that writes related by **happens-before** be observed in that order by every client, while writes that are *concurrent* — neither happens-before the other — may be observed in different orders by different clients.

The happens-before relation here is literally Lamport's 1978 relation, which Chapter 2 developed and which Volume 4, Chapter 3 showed is also the JMM's core: write W_1 happens-before write W_2 if the same session issued W_1 then W_2 ; or if some session *read* W_1 's value and then issued W_2 (reading establishes potential causality — W_2 may depend on what was read); plus transitivity.

The canonical anomaly, and the reason this model earns its keep:

```
Alice:  write(post, "We're getting married!")
Bob:    read(post) → "We're getting married!"
Bob:    write(comment, "Congratulations!")
Carol:  read(comment) → "Congratulations!"
Carol:  read(post) → ∅                                ← forbidden by causal
consistency
```

Bob's comment causally depends on Alice's post — he read it before replying. An eventually consistent store with independent per-key replication can deliver the comment to Carol's replica before the post,

and Carol sees a congratulation for nothing, or worse, for the previous thing Alice posted. Causal consistency forbids exactly this while still allowing genuinely concurrent writes — two unrelated posts by strangers — to arrive in either order, which no user will ever notice or care about.

Why causal is special: the strongest model under partition

Causal consistency's significance is not aesthetic; it is an impossibility frontier. Linearizable and sequential systems must refuse service on the minority side of a partition, because a total order cannot be maintained without communication. Causal consistency requires no total order — each replica need only apply a write after the writes it depends on, all of which the writer had already seen — so **a causally consistent store can keep accepting reads and writes on both sides of a partition and converge afterwards**. Mahajan, Alvisi, and Dahlin proved the sharp version of this in 2011: real-time causal consistency is the strongest model achievable by a system that is always available and convergent; nothing strictly stronger is possible. The COPS paper (Lloyd et al., SOSP 2011) made the result concrete at scale, defining **causal+** — causal consistency plus convergent conflict handling for concurrent writes — and showing it implementable across datacenters with local-latency reads and writes. Causal is, in a precise sense, the best you can do without giving up availability, which is why it anchors the middle of the lattice.

Implementation: dependency tracking

The mechanisms are Chapter 2's. Each write carries metadata identifying the writes it depends on — a **vector clock**, or an explicit dependency list of (key, version) pairs as in COPS. A replica receiving a write buffers it until every dependency has been applied locally, then applies it. Reads are always local and never block. The costs are metadata (dependency information grows with causal history and must be pruned), delayed visibility (a write waits behind its dependencies), and — the important one — **no conflict resolution for free**: concurrent writes to the same key are still concurrent, and the system must merge them (causal+'s convergent handler), which is the door through which last-writer-wins and CRDTs enter below.

Session guarantees: the contract most applications actually need

Terry et al. (1994), working on the Bayou system, isolated four properties defined **per session** — one client's sequence of operations — rather than globally. They are the vocabulary for the most common real-world requirement: *my own actions should look consistent to me*, even if I see other people's actions with delay.

Guarantee	Contract (per session)	Anomaly it prevents
Read-your-writes	A read observes all earlier writes from the same session	You update your profile photo, refresh, and see the old photo
Monotonic reads	Successive reads observe a non-decreasing set of writes	A comment is visible on one refresh and vanishes on the next
Monotonic writes	A session's writes are applied everywhere in the order issued	Your "step 2" write is applied to a replica before your "step 1" write, corrupting the result
Writes-follow-reads	A write is ordered, everywhere, after the writes whose values the session had read	Your reply propagates to a replica before the message you replied to

Each is a small, cheap promise, and each maps to a specific support ticket. Read-your-writes and monotonic reads are read-side guarantees violated by load-balancing reads across unequally-lagged replicas; monotonic writes and writes-follow-reads are write-ordering guarantees violated by multi-leader and leaderless propagation. Note the family resemblance: writes-follow-reads is precisely the read-then-write edge of happens-before, scoped to one session. This is not coincidence — it is a known result that the four guarantees, enforced for all sessions simultaneously, are essentially equivalent to causal consistency. Session guarantees are causal consistency sold by the slice.

```
sequenceDiagram
    participant U as Client session
    participant LB as Load balancer
    participant L as Leader
    participant F as Lagging follower

    U->>LB: write profile = new photo
    LB->>L: route write
    L-->>U: OK – committed
    L-->>F: async replication, delayed
    U->>LB: read profile – user refreshes
    LB->>F: routed to follower
    F-->>U: OLD photo – read-your-writes violated
    Note over U,F: Fix – session token carries
    last-written position.
    Follower serves only if caught up
    to that position, else waits
    or the read is routed to the leader.
```

The implementations are exactly the "mitigation toolkit" Volume 5, Chapter 8 presented for replication-lag symptoms, and it is worth recognizing that those mitigations *were* session guarantees in the wild, assembled ad hoc:

- **Sticky sessions:** pin each session's reads to one replica. That replica's prefix only grows, giving monotonic reads; pin to the leader (or to a replica the session writes through) and you get read-your-writes too. Fragile across failover and rebalancing.
- **Session tokens:** the durable version. Each write returns a position in the replication log (an LSN, an opaque causal token); the client presents its high-water mark on every read, and a replica serves the read only once it has applied past that mark. This is how MongoDB's causally consistent sessions and Cosmos DB's session consistency level work, and it is the pattern to reach for when you build read-your-writes over your own read replicas.

Scope discipline matters here: a "session" is a client-side object — a token holder. Log out, open an incognito window, or switch devices, and you are a new session with no guarantees relative to the old one. Users experience this as "it showed up on my phone but not my laptop," and no session guarantee claims otherwise.

Eventual consistency: a liveness property, honestly stated

The weakest interesting promise: **if no new writes are issued, all replicas eventually converge to the same value.** Read it critically. "Eventually" carries no bound — convergence is promised given quiescence, and a busy system is never quiescent. And nothing whatsoever is promised about *ordering*: reads may go backwards, sessions may not see their own writes, causally related writes may appear in any order. Strictly speaking, eventual consistency is not a consistency model in the sense this chapter defined — it does not constrain which histories are admissible while the system runs, only where the state ends up. **It is a liveness property, not a safety property**, and treating it as a model is a category error that the phrase "we use eventual consistency" invites. The honest reading of that phrase is: *the system promises convergence and nothing else; every ordering guarantee my application needs, my application must construct.*

What the application must then handle:

- **Every anomaly in this chapter:** stale reads, non-monotonic reads, lost visibility of your own writes, causal inversions like comment-before-post.
- **Conflict resolution**, because concurrent writes to the same datum *will* happen and the replicas must converge to one answer. The options: **last-writer-wins**, which picks the write with the highest timestamp and silently discards the others — and, per Chapter 2, "highest timestamp" from unsynchronized clocks means LWW is a machine for losing acknowledged writes arbitrarily; **semantic merge**, where the application supplies domain logic (merge both shopping carts, union the tag sets) — Dynamo's original design, correct but pushed onto every reader; and **CRDTs**, data types whose merge is commutative, associative, and idempotent by construction, so replicas converge to the same value in any delivery order — Chapter 11 is devoted to them, and they are the principled endpoint of this line of thinking.

Eventual consistency is the right contract for real workloads — caches, feeds, presence, metrics — where the periphery section below places it deliberately. What it is never acceptable as is a default adopted without naming the anomalies being accepted.

The lattice assembled

The models order by strength — every history admitted by a stronger model is admitted by a weaker one — into a lattice, with one crucial subtlety: **transaction isolation is an orthogonal axis**, not a rung on the same ladder. Isolation levels (Volume 5, Chapter 6) constrain how multi-object transactions interleave; the models of this chapter constrain single-operation recency and order. Strict serializability is the point where the axes meet.

```

flowchart TB
    SS["Strict serializability  
transactions plus real time  
Spanner external consistency"]
    SER["Serializability  
transaction axis – some serial order,  
real time not respected  
Volume 5 Chapter 6"]
    LIN["Linearizability  
single ops, real time  
etcd, consensus registers"]
    SC["Sequential consistency  
one total order, no real-time bound  
ZooKeeper reads"]
    CP["Causal plus  
causal with convergent conflicts  
COPS"]
    CA["Causal consistency  
happens-before respected everywhere"]
    SG["Session guarantees  
RYW, monotonic reads,  
monotonic writes, writes-follow-reads  
Terry et al. 1994"]
    EV["Eventual consistency  
convergence only – a liveness property"]

    SS --> SER
    SS --> LIN
    LIN --> SC
    SC --> CP
    CP --> CA
    CA --> SG
    SG --> EV

    AV1["needs coordination –  
unavailable under partition"]
    AV2["available under partition –  
sticky or totally available"]
    AV1 -.- LIN
    AV2 -.- CA

```

The partition line drawn on the right is the lattice's most consequential feature: everything at sequential consistency and above requires coordination and must sacrifice availability under partition (Chapter 4); causal and below can remain available. That line is *why* the lattice has its shape — each model below the line exists because someone needed to keep serving requests during a partition and asked what was the most that could still be promised.

The standard visualization of this space is the **Jepsen consistency map** (jepsen.io/consistency), and it is worth learning to read because it has become the industry's shared reference. It draws the models as a directed graph — an edge from stronger to weaker — with strict serializability at the apex branching into the transactional chain (serializable → snapshot isolation → repeatable read → read committed → read uncommitted) on one side and the single-object chain (linearizable → sequential → causal → the PRAM/session-guarantee cluster) on the other, and it colors each model by availability class: unavailable under partition (the top), *sticky available* (achievable if clients stay pinned to a replica — causal and the session guarantees), and *totally available* (servable by any replica at any time). The map's two-branch structure is exactly the two-axis point made above, drawn as one picture.

Reading the contract: real systems audited

Vendors write "strong consistency" in headlines and define it in footnotes. The discipline of this section: **find the precise claim, identify its scope, and test it**. Scope means asking, for each guarantee: single key or multi-key? Single region or global? Single session or all clients? All claims below are stated as of this book's writing (2025-era knowledge); consistency contracts are exactly the kind of thing that changes between versions, so verify against current documentation before betting an invariant on them.

DynamoDB. Reads default to *eventually consistent*; a per-request flag requests a *strongly consistent read*, which reflects all writes acknowledged before the read — but the scope is one table in one region, and strongly consistent reads are not available on global secondary indexes. **Global tables** replicate across regions asynchronously with last-writer-wins conflict resolution — cross-region, the contract has historically been eventual consistency with LWW's data-loss characteristics under concurrent cross-region writes. (AWS announced multi-region strong consistency for global tables in late 2024; treat its availability and exact semantics as something to verify against current docs.) The audit lesson: "DynamoDB strong consistency" is a per-request, per-region, base-table property, not a system property.

S3. Before December 2020, S3 offered read-after-write consistency only for new-object PUTs under conditions, and eventual consistency for overwrites and deletes — a decade of cache-busting workarounds

encrusted around this. Since December 2020, S3 provides **strong read-after-write consistency** for all operations: PUTs and DELETes of new and existing objects, and LIST reflects all completed writes. The scope caveat: this is per-object recency; S3 offers no multi-object transactions, and concurrent writers to the same key race with last-write semantics.

MongoDB. The contract is assembled from knobs on both ends: `writeConcern` (how many nodes acknowledge — `w:1` versus `w:"majority"`) and `readConcern` (`local`, `majority`, `linearizable`). Linearizable behavior for a single document requires `readConcern: "linearizable"` on reads *and* `writeConcern: "majority"` on writes, and such reads are served only by the primary, which confirms it is still primary before returning. Causally consistent sessions (3.6+) provide the session-guarantee cluster via session tokens. The honest history: defaults were weak for years (`w:1` acknowledged writes could be rolled back after failover), and successive Jepsen analyses found real violations at settings users believed were safe; the defaults have since strengthened (`w:"majority"` default from 5.0). The audit lesson: the guarantee is the *pair* of concerns, and any claim that omits one is incomplete.

Cassandra. Consistency is a per-query knob — `ONE`, `QUORUM`, `LOCAL_QUORUM`, `ALL` — and the contract is arithmetic: $R + W > N$ gives overlap, hence reads that observe the latest acknowledged write *in the absence of concurrency and failures*. Per the ABD discussion above, overlap without synchronous write-back is not linearizability, and Cassandra quorum operations are demonstrably not linearizable under concurrent writes; the linearizable path is lightweight transactions (`SERIAL`), which run Paxos at several times the latency. Chapter 7 treats the whole design. The audit lesson: `QUORUM` is a recency heuristic, not a linearizability guarantee, and the vendor documentation, read carefully, agrees.

ZooKeeper. As discussed: linearizable writes (totally ordered by ZAB), sequentially consistent reads (a client may read a stale prefix from its follower), `sync()` to upgrade a read to current. The contract is precisely documented and precisely scoped — ZooKeeper's docs are a model of how to state a consistency contract — and Chapter 8 builds on it.

The final step of the audit discipline is **test it**: consistency contracts are falsifiable claims about histories, and Jepsen exists to falsify them — generate concurrent operations, inject partitions and clock skew, record the full history, and check it against the claimed model with a checker

like Knossos or Elle. Chapter 12 covers the method; the point here is cultural: a consistency claim you have not tested, or seen tested, is marketing.

Choosing: invariants first, then the lattice

The wrong way to choose a consistency model is by temperament ("we're a strong-consistency shop"). The right way is **invariant-driven**: enumerate the application's invariants, and for each, ask what the weakest model is that can protect it.

Some invariants are *coordination invariants*: at most one of something, never negative, globally unique. A username registered once; an account balance that never goes below zero under concurrent withdrawal; exactly one holder of a lease. **These require linearizability at the decision point** — the check and the commitment must be one atomic operation against one up-to-date authority, because any staleness window is a window in which two clients both observe "available" and both proceed. No amount of cleverness at a weaker model recovers this; weaker models can only detect the violation after the fact and trigger compensation.

Most invariants are not like that. A social feed may show posts late or briefly out of (non-causal) order; a view counter may lag; a product page may show a stale price for a second if the checkout path revalidates. For these, causal or session-level guarantees remove the anomalies humans notice, and eventual consistency suffices for the rest.

The architecture that falls out of this analysis is the one mature systems converge on: a **linearizable spine with an eventual periphery**. A small consensus-backed core — often just etcd/ZooKeeper for coordination plus the primary OLTP database for transactional invariants — handles the decision points: uniqueness, balances, inventory decrements, leadership. Everything derived radiates outward through asynchronous replication into caches, search indexes, feeds, and analytics stores, each serving reads at local latency under weak guarantees, with session tokens layered where users would otherwise notice.


```

flowchart TB
    subgraph SPINE["Linearizable spine – small, coordinated, per-op  
cost paid"]
        CONS["Consensus service  
etcd or ZooKeeper  
leases, locks, config"]
        OLTP["Primary OLTP database  
serializable transactions  
balances, uniqueness, orders"]
    end
    subgraph PERI["Eventual periphery – large, available, cheap reads"]
        CACHE["Caches  
eventual, TTL-bounded"]
        SEARCH["Search index  
async projection"]
        FEED["Feeds and timelines  
causal or session guarantees"]
        OLAP["Analytics  
eventual, batch"]
    end
    CLIENT["Clients"]
    CLIENT -->|"decision-point writes  
uniqueness, money, leases"| SPINE
    CLIENT -->|"reads, with session tokens  
where staleness would show"| PERI
    OLTP -->|"async replication  
CDC, log shipping"| CACHE
    OLTP -->|"async"| SEARCH
    OLTP -->|"async"| FEED
    OLTP -->|"async"| OLAP
    CONS -->|"fencing tokens guard  
periphery writers – Chapter 2"| PERI

```

Two disciplines make this architecture work rather than merely exist. First, **per-operation mixing is the mature posture**: consistency is chosen per operation, not per system. The same inventory service does a linearizable decrement at checkout and an eventually consistent read on the product page; DynamoDB's per-request flag, Cassandra's per-query level, and MongoDB's per-operation concerns all exist precisely to support this. Second, **the boundary must be explicit**: every arrow from spine to periphery in that diagram is a place where staleness enters, and it should be documented as such — which brings us to pricing.

The distributed-systems lens

This chapter's lens is reflexive — the chapter *is* the distributed-systems story — so the lens does two jobs instead: it closes the arc opened in Volume 4, Chapter 3, and it restates the chapter economically, which is how these decisions are actually made.

The arc from shared memory, closed. Every correspondence that chapter's table promised has now been delivered. The **store buffer is replica lag**: a write acknowledged to its issuer but invisible to other observers, and "I wrote it, read it back, and it wasn't there" is the same complaint whether the window is 40 nanoseconds of store-buffer drain or 400 milliseconds of cross-region replication — read-your-writes is the distributed name for what program order gives a thread for free. **Barriers are quorum waits**: an x86 `MFENCE` drains the store buffer before proceeding exactly as a `w:"majority"` write waits for replication before acknowledging, and both buy ordering with stalls. **Sequential consistency is the same theorem transplanted** — Lamport wrote the definition once, in 1979, and both fields inherited it. **Happens-before is the same relation** — Lamport 1978 — tracked by vector clocks across machines and by volatile/lock edges within one. And **DRF-SC returns as the spine architecture**: both are the bargain *confine your coordination to declared points, and reason simply everywhere else*. The synchronized keyword and the consensus-backed decision point are the same design move at different scales. The one place the analogy breaks remains instructive: memory has no partial failure — no core is partitioned from the L3 for a minute and then reappears with divergent state — which is why this volume needs consensus, failure detection, and Chapters 4 through 12, and Volume 4 did not.

The economics. Each step down the lattice is a purchase: you receive latency (local reads instead of quorum round-trips), availability (serving through partitions), and throughput (no serialization bottleneck), and you pay in anomalies. The engineering discipline is to **price the anomaly against the invariant it endangers**, in expectation: a stale product-page price costs a support ticket; a double-spent balance costs money and trust; a comment-before-post costs a confused user for one refresh. Cheap anomalies justify deep discounts — run the feed eventually consistent. Expensive anomalies justify the full linearizable price — run the ledger on the spine. The failure mode in real organizations is not choosing wrongly once; it is never pricing at all, inheriting whatever the

datastore's default was, and discovering the actual contract during an incident.

Consistency SLOs. The operational maturity step is to treat staleness as a **measurable, budgeted quantity** rather than a vibe. Replica lag is observable (Volume 5, Chapter 8's lag metrics); probes can write a timestamped canary through the spine and measure when each periphery system serves it; and the result is a staleness distribution you can put an SLO on: "the search index reflects catalog writes within 5 seconds at p99." Some products make the bound part of the contract itself — Azure Cosmos DB's *bounded staleness* level guarantees reads lag writes by at most a configured number of versions or time interval, sitting between strong and session in its five-level menu — but you do not need a product feature to adopt the practice. A periphery with measured, alarmed staleness bounds is an engineering artifact; a periphery that is "eventually consistent, probably fast" is a hope.

Key takeaways

- **A consistency model is a predicate on operation histories** — a contract, independent of mechanism. Volume 5, Chapter 8 built the machines; this chapter named what they promise.
- **Linearizability**: every operation takes effect atomically at a point within its own invocation-response interval, consistent with real time — once a write returns, every later read anywhere sees it. Its **locality** (linearizable objects compose) is why it is the gold standard; consensus groups, leader-reads, and ABD-style write-back quorums provide it; quorum overlap *without* read write-back does not.
- **Its price is coordination on every operation**: quorum RTT latency, leader throughput bounds, and unavailability on the minority side of a partition (Chapter 4).
- **Linearizability $\neq$ serializability**: real-time single-operation recency versus any-serial-order multi-operation isolation. **Strict serializability = both**, and it is Spanner's external consistency (Volume 5, Chapters 6 and 12).
- **Sequential consistency** is Lamport's 1979 model transplanted intact: one agreed total order, program order respected, no real-time bound — stale but never contradictory. ZooKeeper reads are the canonical instance; `sync()` upgrades on demand.

- **Causal consistency** enforces happens-before everywhere and nothing more; it forbids comment-before-post while permitting concurrent writes to differ in order, and it is **provably the strongest model compatible with availability under partition** (Mahajan et al.; COPS). Vector clocks and dependency tracking (Chapter 2) implement it.
- **Session guarantees** (Terry et al. 1994) — read-your-writes, monotonic reads, monotonic writes, writes-follow-reads — are causal consistency sold by the slice, each preventing one nameable anomaly, implemented with sticky routing or session tokens. Volume 5, Chapter 8's lag mitigations were these guarantees, unnamed.
- **Eventual consistency is a liveness property, not an ordering contract**: convergence given quiescence, nothing about what reads return meanwhile. Applications inherit every anomaly plus conflict resolution — LWW silently loses acknowledged writes (Chapter 2); CRDTs (Chapter 11) are the principled alternative.
- **The lattice**: strict serializability → linearizability → sequential → causal+ → causal → session guarantees → eventual, with transaction isolation as an orthogonal axis; the Jepsen consistency map is the standard drawing, with availability classes marked.
- **Audit real products**: find the precise claim, scope it (key? region? session? index?), and test it (Chapter 12). DynamoDB's strong reads are per-request and per-region; S3 became strongly read-after-write consistent in December 2020; MongoDB's guarantee is the read-concern + write-concern pair; Cassandra `QUORUM` is not linearizable — `SERIAL` is; ZooKeeper is SC-reads + linearizable-writes by design.
- **Choose invariant-first**: coordination invariants (uniqueness, balances, leases) need linearizability at the decision point; almost everything else tolerates less. The mature shape is a **linearizable spine with an eventual periphery**, mixed per operation, with staleness measured and budgeted as an SLO.

Further reading

- Herlihy, M. and Wing, J., "Linearizability: A Correctness Condition for Concurrent Objects," *ACM TOPLAS* 12(3), 1990 — the definition, and the locality theorem. <https://dl.acm.org/doi/10.1145/78969.78972>

- Lamport, L., "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs," *IEEE Trans. Computers* C-28(9), 1979 — sequential consistency, the shared original of Volume 4, Chapter 3 and this chapter.
- Lamport, L., "Time, Clocks, and the Ordering of Events in a Distributed System," *CACM* 21(7), 1978 — happens-before, the backbone of causal consistency. <https://dl.acm.org/doi/10.1145/359545.359563>
- Terry, D. et al., "Session Guarantees for Weakly Consistent Replicated Data," *PDIS*, 1994 — read-your-writes, monotonic reads, monotonic writes, writes-follow-reads, from the Bayou project.
- Lloyd, W., Freedman, M., Kaminsky, M., and Andersen, D., "Don't Settle for Eventual: Scalable Causal Consistency for Wide-Area Storage with COPS," *SOSP*, 2011 — causal+ and its implementation.
- Mahajan, P., Alvisi, L., and Dahlin, M., "Consistency, Availability, and Convergence," UT Austin TR-11-22, 2011 — real-time causal as the strongest always-available convergent model.
- Attiya, H., Bar-Noy, A., and Dolev, D., "Sharing Memory Robustly in Message-Passing Systems," *JACM* 42(1), 1995 — the ABD register, and why linearizable reads must write back.
- Viotti, P. and Vukolić, M., "Consistency in Non-Transactional Distributed Storage Systems," *ACM Computing Surveys* 49(1), 2016 — the exhaustive survey and taxonomy of the models in this chapter.
- Jepsen, *Consistency Models* — <https://jepsen.io/consistency> — the standard map of the model hierarchy with availability classes, plus the per-system analyses referenced throughout.
- Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly, 2017 — Chapter 5 (replication and its anomalies) and Chapter 9 (linearizability and ordering); the best book-length informal treatment.
- Volume 4, Chapter 3 — Memory Models and Happens-Before — the shared-memory ancestor of this chapter.
- Volume 5, Chapter 6 — Isolation and MVCC; Chapter 8 — Replication — the orthogonal transaction axis, and the mechanics beneath these contracts.
- This volume: Chapter 2 (clocks and vector clocks), Chapter 4 (CAP and PACELC), Chapters 5-7 (consensus and quorums — the

machinery of the strong models), Chapter 11 (CRDTs), Chapter 12 (testing the contracts with Jepsen).

Chapter 4 — CAP, PACELC, and the Real Trade-Offs

What this chapter covers. CAP is probably the most-cited and most-misstated result in distributed systems. It appears on architecture slides as a triangle with "pick two," it is invoked to justify decisions it says nothing about, and its three letters are routinely expanded into definitions the theorem does not use. This chapter does three jobs. First, it states the theorem *as the theorem it actually is* — the Gilbert & Lynch 2002 formalization of Brewer's conjecture — with its exact model and exact definitions, because the precise statement is both narrower and more useful than the folklore version. Second, it clears away the folklore: the triangle, the "CP system vs AP system" labels, the conflation of CAP's C with every other C in the field. Third, and most importantly, it replaces the folklore with the trade-off structure that actually governs design: PACELC, which adds the latency-versus-consistency trade you pay on every request even when the network is healthy; harvest and yield, the forgotten vocabulary for graceful degradation; and a decision discipline for making the consistency-availability choice per operation, pricing it honestly, and testing the partition behavior you claim to have. The mechanics are made concrete on this volume's replicated key-value running example, and the chapter closes by turning the lens back on Volume 4: the same trade exists inside a single machine, and the same conclusion — coordination is the cost center — holds at every scale, now with a theorem attached.

Learning goals — after this chapter you should be able to:

- State the CAP theorem precisely: model, the three guarantees as Gilbert & Lynch define them, and the one-paragraph proof — and identify which parts of the popular version are not in it.
- Explain why "partition tolerance" is not a menu option, and therefore why the real content of CAP is the C-versus-A choice *during* a partition.

- Explain why "CP system" and "AP system" are category errors at system granularity, using per-operation consistency levels as the counterexample.
- Describe what network partitions actually look like in production — partial, asymmetric, flapping — and summarize the empirical evidence that they occur at meaningful rates.
- State PACELC, explain why the ELSE branch dominates everyday design, and classify real systems in PACELC terms with appropriate hedging.
- Define harvest and yield, and use them to design degradation modes that are chosen rather than accidental.
- Apply a per-operation decision discipline: identify the invariant, price the latency of protecting it, write the partition playbook, and find the coordination hidden in dependencies.

The theorem, stated precisely

Eric Brewer presented CAP as a conjecture in his keynote at PODC 2000. Seth Gilbert and Nancy Lynch proved a formalization of it in 2002, and their paper — four pages, entirely readable — is the thing people cite when they say "the CAP theorem." It is worth knowing exactly what it proves, because the proof's power and its limits both come from the same source: the definitions are strict and the model is specific.

The model and the three guarantees

The setting is the **asynchronous network model**: nodes communicate only by passing messages, messages may be delayed arbitrarily or lost, and there are no clocks — no node can distinguish "the reply is lost" from "the reply is slow." This is the same adversarial model Chapter 1 introduced and Chapter 2 wrestled with; it matters here because the impossibility argument leans on exactly that indistinguishability. The object under discussion is deliberately minimal: a single read/write register, replicated across nodes. Not a database, not a transaction system — one cell of shared memory.

The three guarantees, as the paper defines them:

- **Consistency** means the register is **linearizable** — "atomic consistency" in the paper's vocabulary, which is Herlihy and Wing's

linearizability applied to a single register. Chapter 3 gave the precise definition, and CAP inherits it unchanged: there must exist a total order over all operations such that each appears to take effect instantaneously at some point between its invocation and its response, and that order must respect real time — if write W completes before read R begins, R must observe W (or something later). This is the strongest single-object consistency model on Chapter 3's map. Hold onto that; it is the pivot of a debunking below.

- **Availability** means that **every request received by a non-failing node must eventually result in a response**. Note the quantifier and the absence of a deadline: *every* request, to *any* live node, must eventually be answered — in the asynchronous model there is no time bound to appeal to, so "eventually" is all that can be demanded. A node that answers "try another replica" with an error is, for the theorem's purposes, responding to precisely nothing; an algorithm that lets even one live node refuse even one request has forfeited availability as defined.
- **Partition tolerance** means the guarantees above must hold even when **the network loses arbitrarily many messages between nodes**. A partition, formally, is just that: a pattern of message loss that separates the nodes into groups that cannot hear each other. Partition tolerance is not a feature a node implements; it is a clause about the adversary the other two guarantees must survive.

The theorem: in the asynchronous network model, it is impossible for a read/write register to guarantee both availability and linearizability in all executions, including those in which messages are lost.

The proof, in one paragraph

Split the nodes into two non-empty groups, G_1 and G_2 , and let the adversary drop every message between them — a total partition, which partition tolerance obliges the algorithm to survive. A client writes value v_1 to the register at a node in G_1 . By availability, that write must eventually complete and return success; it cannot block waiting for G_2 , because it would wait forever. Later, a client reads the register at a node in G_2 . By availability, that read must also return. But no information about the write can have crossed the partition, so the read returns the old value — and a read that begins after a write has completed, yet returns the pre-write value, is a straightforward linearizability violation.

Both guarantees cannot survive the same execution; one of them yields. Gilbert and Lynch add a corollary with a sting in it: because asynchronous nodes cannot distinguish lost messages from slow ones, an algorithm that guarantees availability ends up violating consistency even in some executions where *no* message is actually lost — it merely had to be prepared for the possibility. They also prove a partially-synchronous variant (with timeouts, the sting softens: you can regain consistency within a bounded time after the partition heals), but the headline result is the asynchronous one.

That is the entire theorem. A single register, an asynchronous network, three precisely-defined properties, a two-group counterexample. Everything else that travels under the name "CAP" is commentary — some of it useful, much of it wrong. The next section sorts which is which.

What CAP does not say

The debunkings below are not pedantry. Each corresponds to a class of real design errors — architectures justified by a theorem that does not apply, or guarantees claimed that the theorem does not permit. And none of this is revisionism against Brewer: his own 2012 retrospective, *CAP Twelve Years Later: How the "Rules" Have Changed*, makes several of these corrections explicitly, and I will cite it as we go.

"Pick two of three" — but P was never on the menu

The popular rendering is a triangle: consistency, availability, partition tolerance — choose any two. The rendering fails because the three properties are not the same kind of thing. C and A are guarantees your system provides. P is a statement about the *network*, and on a real network, message loss is not something you opt out of. Switches fail, links flap, kernels drop packets under memory pressure, a misconfigured ACL blackholes a subnet (Volume 3, Chapter 11 catalogs the mechanisms). A "CA system" — one that chooses consistency and availability by declining partition tolerance — is a system that simply has no defined behavior when the network misbehaves, which is to say it has chosen to fail in an unspecified way. Brewer's retrospective says it plainly: the "2 of 3" formulation was always misleading, because partitions are rare, and when the system is not partitioned there is no reason to sacrifice either C or A.

That sentence contains the theorem's real content, so it bears restating:

CAP is not a three-way choice. It is a two-way choice — consistency versus availability — that only has to be made while a partition is in progress. Outside partitions, a well-engineered system provides both. The theorem tells you that you must decide, in advance, which one you will give up during the minutes (or hours) when the network is broken.

This reframing does real work. It converts CAP from a branding exercise ("we are an AP database") into an operational question with a concrete deliverable: *what does your system do during a partition, and did you decide that on purpose?* Brewer's retrospective proposes exactly that discipline — detect the partition, enter an explicit partition mode in which some operations are limited, and run a recovery step when connectivity returns to restore consistency and compensate for any mistakes made in the meantime. We build that playbook in a later section.

"CP system" and "AP system" are category errors

The second folklore artifact is the label: Cassandra is AP, HBase is CP, and so on, as if the choice were a property of the binary you deploy. It is not, for two reasons.

First, real systems make the choice **per operation and per configuration**. Cassandra is the clean counterexample (Volume 5, Chapter 11 covers its architecture): every read and every write carries its own consistency level. A write at `QUORUM` into a cluster split away from a majority of replicas will fail — that operation, at that moment, chose consistency over availability. A write at `ONE` into the same split will succeed against whatever replica is reachable and reconcile later — that operation chose availability and accepted divergence. Same binary, same cluster, same partition, opposite CAP behavior, selected by a parameter on the request. A system whose requests can sit at different points of the trade-off does not *have* a CAP class; its operations do.

Second, even for systems with less configurability, the honest label depends on which guarantee you are asking about — reads or writes, within a region or across regions, with which durability settings. The vocabulary "CP vs AP" survives because it is convenient shorthand, and I will use it below as shorthand for *a choice*, but attaching it to *a system* is a category error that conceals exactly the decisions this chapter exists to surface.

CAP's C is linearizability — not ACID-C, and not "strong-ish"

The theorem's consistency is the linearizable register, full stop. Two conflations do damage here.

The first is with the C in ACID. ACID consistency (Volume 5, Chapter 5) means transactions preserve application-level invariants — balances non-negative, foreign keys valid. It is a property about *integrity constraints*, largely the application's responsibility, and it has essentially nothing to do with the ordering-of-operations property CAP is about. A sentence like "we relaxed CAP consistency but we're still ACID" is not wrong so much as unparseable; the two Cs are different axes.

The second conflation is the quiet downgrade: reading CAP's C as "reasonably strong consistency" and then concluding that any system with quorums or synchronous replication "has C." Chapter 3 built a whole hierarchy below linearizability — sequential consistency, causal consistency, read-your-writes, bounded staleness — and the theorem's impossibility applies only to the top of it. This cuts in both directions, and the permissive direction is the interesting one: **weaker consistency models than linearizability can be available under partition**. Causal consistency, notably, can be maintained on both sides of a partition simultaneously (each side keeps extending its own causal history and merges later); it is close to the strongest model for which that is known to hold. So "CAP says we can't have consistency during partitions" is false as stated. CAP says you cannot have *linearizability* with full availability during partitions. If your invariant is protected by something weaker — and many are — the theorem has no objection.

CAP's A is a liveness guarantee that almost nobody actually claims

The theorem's availability is *every request to every non-failed node eventually gets a response*. Measure real systems against that literal standard and something clarifying happens: most systems that everyone calls "highly available" are not CAP-available, and do not want to be. A system behind a load balancer that health-checks nodes out of rotation is routing around nodes that would fail requests — fine engineering, but the theorem's A is about what those nodes would do if asked. A leader-based store that fails minority-side requests during a partition is not CAP-available by construction — that is exactly the sacrifice it chose. Even proudly available systems return errors under overload, which the

theorem's A does not permit. The label on the marketing page ("five nines") is a *measured* property — a fraction of requests served successfully over a window — and is related to CAP's A roughly the way uptime is related to a liveness proof, which is to say loosely. When you read "highly available" in a datasheet, the CAP theorem is not the claim being made, and violating CAP is not the rebuttal.

CAP says nothing about latency — and latency is the everyday cost

The theorem's availability has no deadline: a response in ten minutes satisfies it. That is faithful to the asynchronous model and useless to an engineer with a 100 ms budget. And the omission hides the trade-off that dominates real design, because the mechanisms that buy consistency — synchronous replication, quorum round trips, consensus — charge their toll in latency on *every operation, all the time*, not just during partitions. A partition is an event; latency is a lifestyle. The theorem that governs the event became famous, while the trade-off you pay every millisecond of every day went nameless for a decade — until Abadi named it, and we will get there right after we look at what the event actually looks like.

What partitions actually look like

The proof's partition is clean: two groups, total silence between them. Production partitions are rarely so tidy, and the messier shapes are the ones that break systems designed against the tidy mental model. Volume 3 (Chapters 1, 11, and 12) covers the underlying mechanisms; here is the summary that matters for this chapter.

Partitions are frequently partial and asymmetric. A failing switch linecard, an asymmetric routing change, or a unidirectional link fault can produce topologies where A can reach B, B can reach C, but A cannot reach C — every node is "up," every node has *some* connectivity, and yet there is no consistent global view of who can talk to whom. Failure detectors built on pairwise heartbeats (Chapter 10) disagree with each other in exactly this situation, and protocols that implicitly assume "reachability is transitive" or "either the network works or it doesn't" misbehave in ways their designers never enumerated. A classic pathology: a leader that can reach a majority of its followers but not the coordination service, or vice versa, leading to two components each holding half of the evidence needed to act.

```

flowchart LR
    subgraph CLEAN["Textbook partition – total split"]
        A1["Node A"] --- B1["Node B"]
        C1["Node C"] --- D1["Node D"]
    end
    subgraph PARTIAL["Partial partition – reachability not transitive"]
        A2["Node A"] --- B2["Node B"]
        B2 --- C2["Node C"]
        A2 -. "link down" .-> C2
    end
    subgraph ASYM["Asymmetric partition – one-way loss"]
        A3["Node A"] -->|"A to B delivers"| B3["Node B"]
        B3 -. "B to A drops" .-> A3
    end

```

Partitions flap. Links that fail cleanly and stay failed are the kind operators like; links that oscillate — seconds of loss, seconds of health, repeat — are the kind that turn a leader-election protocol into a leadership carousel, each flap triggering an election whose messages are themselves lost in the next flap. Timeout-based failure detection (Chapter 1's slow-versus-dead problem, Chapter 10's machinery) is at its worst here, and systems can spend more time reconfiguring than serving.

Partitions happen at rates that matter. The honest evidence base is Peter Bailis and Kyle Kingsbury's 2014 survey *The Network is Reliable* — the title is ironic — which compiles published measurement studies and practitioner postmortems. The picture it assembles, stated at the level of certainty the sources support: a measurement study of Microsoft's datacenters (Gill et al., SIGCOMM 2011) found network element failures to be a daily occurrence at fleet scale, with dozens of link failures per day across the fleet, most masked by redundancy but a meaningful minority causing real loss; Google engineers have described a typical first year of a new cluster as including multiple rack-wide outages and several router or switch failures; and the postmortem record is thick with partitions caused not by hardware at all but by configuration — a bad ACL push, a routing-protocol misadventure, a firmware bug — which redundant links do nothing to prevent, because the redundant links receive the same bad config. WAN and cross-region links, which multi-region architectures (Volume 7, Chapter 10) depend on, fail more often still. The survey's conclusion is the right calibration: partitions are not a daily event for any single small cluster, but they are far too frequent, at the scale and lifetime of a real fleet, to leave their handling unspecified. "It basically

never happens" is not a partition strategy, and the data says it is not even true.

PACELC: naming the trade you pay every day

Daniel Abadi's observation — a 2010 blog post, formalized in a 2012 IEEE *Computer* article — is that CAP describes only the exceptional branch of a two-branch trade-off, and that omitting the other branch had visibly distorted how systems were being compared. His formulation:

PACELC: if there is a **P**artition, trade **A**vailability against **C**onsistency;
Else, trade **L**atency against **C**onsistency.

The ELSE branch is where a system spends almost all of its life, and the trade there is mechanical, not theoretical. To make writes consistent across replicas, a write must not be acknowledged until enough replicas have seen it; that is one or more network round trips *in the critical path of every write*. Volume 5, Chapter 8 quantified this for replication: synchronous replication to a peer in the same building adds a millisecond or less; to another region, it adds the speed of light — tens of milliseconds that no engineering removes. Volume 5, Chapter 12 showed the same floor under NewSQL: a consensus round (Chapters 5 and 6 of this volume) costs at minimum one round trip from leader to a quorum of followers, so a Spanner- or CockroachDB-style commit across regions has a physics-imposed latency floor regardless of implementation quality. Choosing to relax consistency — asynchronous replication, local reads, weaker quorums — is choosing to delete those round trips from the critical path. That is the EL choice, and it is made millions of times per second in fleets where a partition has not happened for months.

This is why PACELC, not CAP, is the framework that should appear on the architecture slide. CAP answers "what happens in the bad minutes"; PACELC also answers "what does every good millisecond cost," and the second question is usually the one the business feels.

```

flowchart TD
    START["Operation arrives"] --> Q1{"Is a partition  
in progress?"}
    Q1 -->|"yes – the P branch"| PA{"Protect the invariant  
or serve the request?"}
    PA -->|"consistency"| CP["Refuse or block on the  
minority side  
PC: unavailability for some"]
    PA -->|"availability"| AP["Accept on both sides  
PA: divergence to reconcile  
see Ch7, Ch11"]
    Q1 -->|"no – the ELSE branch"| EL{"Pay round trips  
on the critical path?"}
    EL -->|"consistency"| EC["Synchronous quorum or consensus  
EC: latency floor on every write"]
    EL -->|"latency"| ELX["Async replication, local reads,  
weak quorums  
EL: staleness windows"]

```

Classifying real systems, with the required hedging

Abadi's article classifies systems into the four corners, and the exercise is illuminating as long as two hedges stay attached. First, per the category-error section above, these are labels for *defaults and design centers*, not for every operation the system can perform. Second, systems move: several of Abadi's 2012 classifications describe defaults that have since changed, so the table below describes design lineages, stated as of what those designs canonically are.

PACELC class	Design center	Canonical examples	The bargain
PA/EL	Give up consistency in both branches	Dynamo lineage: Cassandra and Riak at weak consistency levels, DynamoDB in its eventually-consistent mode	Always writable, local-latency reads; the cost is divergence and staleness, managed by the machinery of Chapter 7 (sloppy quorums, read repair) and Chapter 11 (CRDTs)

PACELC class	Design center	Canonical examples	The bargain
PC/EC	Pay for consistency in both branches	Spanner, CockroachDB, ZooKeeper, etcd, VoltDB	Linearizable operations always; the cost is minority-side unavailability during partitions and a consensus round trip on every write
PC/EL	Consistent under partition, fast otherwise	Yahoo's PNUTS is Abadi's example: timeline consistency per record, with operations that degrade rather than diverge under partition	Everyday latency of a relaxed model, without accepting divergent writes when the network breaks
PA/EC	Available under partition, consistent otherwise	Abadi's example was MongoDB's then-default configuration; systems that run synchronously in the steady state but keep accepting writes when replication is impossible	The awkward corner: consistent until it matters most; usually a description of behavior discovered later rather than a design goal

The mixed corners deserve the attention they rarely get. PC/EL says something subtle: "we will not invent data during a partition, but we also refuse to pay quorum latency on the everyday path" — a defensible position for read-heavy, record-oriented workloads. PA/EC describes systems that pay for consistency precisely when it is cheap and abandon it precisely when it is hard; when that is the outcome of an explicit failover policy with bounded loss it can be reasonable, but when it is emergent — discovered in a postmortem rather than chosen in a design review — it is the signature of a system that never wrote down its partition behavior. Kyle Kingsbury's Jepsen work (Chapter 12) has repeatedly caught systems in exactly that corner, advertising EC behavior while testing revealed PA-with-data-loss behavior under partition.

The choice made concrete: our replicated KV under partition

Return to the volume's running example: a key-value store replicated across five nodes, N1-N5, currently led by N1 (leadership and log machinery per Chapters 5 and 6; quorum arithmetic per Chapter 7). A partition separates {N1, N2} from {N3, N4, N5}. Note the awkward detail chosen deliberately: the leader is on the *minority* side. Clients are attached to both sides. What happens next is not determined by the partition — it is determined by which branch of the P choice we configured.

The CP choice: only a majority may write

Under the consistency choice, the rule is simple and brutal: a write commits only when a majority of replicas (three of five) have accepted it. N1, marooned with one follower, can no longer assemble three acknowledgments. Its writes hang and then fail; if leases are in play (below), N1 stops serving even reads once its lease expires. On the majority side, N3-N5 elect a new leader — say N3 — which can assemble a quorum and proceeds normally. The system as a whole remains linearizable and remains *mostly* available: clients who can reach the majority side notice nothing, and clients attached to the minority side get errors until the partition heals or they re-route. That asymmetric outcome — availability for some clients, refusal for others, invariants intact for all — is what "CP" actually means operationally.

```
sequenceDiagram
    participant CA as "Client at minority side"
    participant N1 as "N1 old leader, minority"
    participant N3 as "N3 new leader, majority"
    participant CB as "Client at majority side"
    Note over N1,N3: Partition separates N1,N2 from N3,N4,N5
    CA->>N1: PUT balance=90
    N1--xN3: replicate – dropped by partition
    Note over N1: Only 2 of 5 reachable – no quorum
    N1-->>CA: ERROR not enough replicas
    Note over N3: Majority side elects N3 as leader
    CB->>N3: PUT balance=70
    N3->>N3: replicate to N4, N5 – quorum of 3
    N3-->>CB: OK committed
    Note over CA: Minority clients are unavailable – the price of C
```

The AP choice: both sides accept, and we go into debt

Under the availability choice, every reachable replica keeps accepting writes — there is no majority requirement, and perhaps no leader at all (the Dynamo-style leaderless design of Chapter 7). A client on the minority side writes `cart = [book]` through N1; a client on the majority side writes `cart = [lamp]` through N4. Both are acknowledged. The register has now **diverged**: two histories exist, each internally consistent, with no ordering between them. Nothing is wrong yet in the sense the clients can observe — that is the point — but the system has taken on a debt that comes due when the partition heals: the replicas must detect the conflict (version vectors, Chapter 7), and something must resolve it. The resolution options are the subject of Chapters 7 and 11 — last-writer-wins (which silently discards one write; acceptable for a presence flag, unacceptable for money), semantic merge (union the carts), pushing the conflict to the application, or choosing data structures whose merges are automatic and principled (CRDTs). The one option that does not exist is not deciding: an AP configuration without a designed conflict-resolution story has simply chosen "corrupt quietly."

```
sequenceDiagram
    participant CA as "Client A"
    participant N1 as "N1 minority side"
    participant N4 as "N4 majority side"
    participant CB as "Client B"
    Note over N1,N4: Partition in progress – both sides accept writes
    CA->>N1: PUT cart adds book
    N1-->>CA: OK version v-N1-7
    CB->>N4: PUT cart adds lamp
    N4-->>CB: OK version v-N4-3
    Note over N1,N4: Partition heals – anti-entropy exchange
    N1->>N4: sync – versions conflict, neither dominates
    N4->>N4: merge carts – book and lamp both kept
    N4->>N1: merged value, new version
    Note over N1,N4: Divergence resolved – debt paid at merge time
```

Leases and fencing: making the minority actually stop

The CP story above contains a step that deserves suspicion: "N1 stops serving." Why would it? From N1's own point of view, the world has merely gone quiet — and Chapter 1 taught that a node cannot distinguish a partition from everyone else being slow. If N1 keeps serving reads from its local state while N3's side commits new writes, clients at N1 read stale data from a node that sincerely believes it is the leader — a

linearizability violation delivered with full confidence. The standard fix is the **lease**: leadership is held for a bounded term and must be renewed through a quorum; a leader that cannot renew must stop serving when the term expires, *even though nothing seems wrong locally*. This converts the unanswerable question "am I partitioned?" into the answerable one "has my lease expired?" — at the cost of a clock assumption (bounded clock drift; Chapter 2's territory) and of a built-in unavailability window equal to the lease length. And because a deposed leader may still hold unexpired side effects in flight, leases pair with **fencing tokens**: every leadership term carries a monotonically increasing number, downstream resources reject writes bearing a stale token, and the zombie leader's late-arriving writes bounce off. Volume 4, Chapter 2 closed on exactly this construction for distributed locks; it is the same mechanism, because it is the same problem — an actor that does not know it has lost its authority.

Reads during partitions: staleness as the middle ground

The C-versus-A choice is starkest for writes, but reads offer a genuine middle ground that the binary framing hides. A minority-side node cannot serve a *linearizable* read — but it can serve a read labeled honestly: **bounded-staleness** reads ("this data is at most 5 seconds old, by local clock"), or snapshot reads at a known-committed timestamp. Volume 5, Chapter 12 covered the mechanism in its NewSQL form — follower reads / stale reads in Spanner and CockroachDB, where a replica serves a read at a timestamp it can prove is fully replicated, trading recency for locality and partition-side availability. During a partition this becomes a designed degradation: writes on the minority side fail (CP preserved where it counts), while reads continue with an explicit staleness bound (availability preserved where the invariant permits). Many "CP" systems' real partition behavior is exactly this hybrid, which is one more reason the one-word labels mislead.

The knobs that select all of the above are, in Dynamo-lineage systems, per-request:

```
# Per-operation consistency selection, Cassandra-style (N = 5 replicas)
checkout_payment:
  write_consistency: QUORUM      # 3 of 5 must ack: fails on minority
side (CP-ish)
  read_consistency: QUORUM
# R + W > N: read sees latest committed write
shopping_cart:
  write_consistency: ONE          # any reachable replica: always
writable (AP-ish)
  read_consistency: ONE          # fast, possibly stale; conflicts
merged via CRDT cart
product_view_counter:
  write_consistency: ANY          # even a hint suffices; loss is
tolerable
  read_consistency: ONE
```

Three operations, one cluster, three different points on the trade-off — the per-operation discipline made literal in configuration.

Harvest and yield: the forgotten refinement

Three years before Gilbert and Lynch's proof, Armando Fox and Eric Brewer published a short HotOS paper — *Harvest, Yield, and Scalable Tolerant Systems* (1999) — that supplied a vocabulary the CAP debate then mostly forgot. It deserves rescue, because it describes the degradation choices you actually make, at a finer grain than C-versus-A.

- **Yield** is the probability that a request completes — successful requests over total requests. It is close to what practitioners mean by availability, and more useful than uptime: a minute of downtime at peak and a minute at 4 a.m. are identical uptime and wildly different yield.
- **Harvest** is the fraction of the relevant data reflected in a completed response. A search that consulted 9 of its 10 shards, because the tenth was on the wrong side of a partition, returns with harvest 0.9.

The insight is that a failure's cost can be spent from either account. When a shard is unreachable, a search service can fail every query touching it — full harvest on the queries that succeed, reduced yield — or answer every query from the shards it has — full yield, reduced harvest. For search, reduced harvest is obviously right: a 90%-complete result is nearly as good as a complete one, and infinitely better than an error. For an account-balance read, harvest degradation is obviously wrong: 90% of

your transactions is not your balance. The choice is per-operation, again, and it is a *design* choice: systems degrade harvest gracefully only if responses can represent partiality and callers are built to tolerate it.

Degradation mode	What degrades	Good fit	Bad fit
Fail requests touching lost data	Yield	Invariant-bearing reads: balances, inventory, auth	Broad queries where partial answers retain most value
Answer from available data, flag partial	Harvest	Search, feeds, recommendations, analytics dashboards	Anything summed or counted for correctness
Serve stale-but-complete snapshot	Freshness — a harvest variant along the time axis	Catalogs, configuration, follower reads	Data whose staleness breaks the invariant

Volume 4, Chapter 9's scatter-gather pattern already implemented harvest degradation in miniature — fan out to workers, deadline arrives, return what came back and note what did not. Volume 7, Chapter 11 builds organization-scale graceful degradation on the same two dials. The contribution of the vocabulary is that it makes the dials *visible*: "under partition, this endpoint degrades harvest, floor 80%, response marked partial; that endpoint degrades yield" is a sentence that can appear in a design document and be tested against — which is more than can be said for "we're AP."

Decision discipline

Everything above compresses into a working method — five obligations that belong in any design review for a stateful service.

1. Identify the invariant, and its blast radius. The C-versus-A choice is not made for "the system"; it is made for an invariant. Name it: no double-spend; inventory never oversold; at most one holder of this lock; likes eventually counted roughly right. Then price its violation. A diverged like-counter merges with nobody harmed. A diverged account balance is money invented or destroyed, discovered at reconciliation,

escalated to a regulator. The blast radius, not the data model, is what justifies paying for coordination — "money vs likes" is the crude but effective first cut.

2. Choose per operation, not per system. The unit of choice is the operation against the invariant. The checkout takes the quorum write and eats minority-side failures; the cart takes `ONE` and a CRDT merge; the view counter takes fire-and-forget. Any framing of the decision as "should we use a CP or an AP database" has already lost the resolution at which the real decision exists — though it may still matter for picking defaults, since teams inherit their store's path of least resistance.

3. Price the ELSE branch, in milliseconds, against a budget. Consistency's everyday cost is latency, so treat it as a line item in the latency budget (Volume 11 develops budgets properly). A cross-region synchronous commit at ~60 ms RTT inside a 100 ms p99 budget is not a philosophical tension — it is an arithmetic failure, visible in the design review if anyone does the arithmetic. The honest comparison is between invariant-violation cost (step 1) and round-trips $\times$ RTT $\times$ requests-per-day. Sometimes the answer is genuinely "pay it" — Spanner exists because Google concluded, in their words, that it is better for engineers to deal with performance problems than with the semantic problems of weak consistency. Sometimes the answer is a cheaper consistency model that still protects the invariant. It should never be "we didn't price it."

4. Write the partition playbook — and test it. Brewer's partition-mode framing becomes a concrete artifact: for each operation class, what *blocks*, what *degrades* (harvest? freshness? by how much, marked how?), and what *reconciles* afterward (merge function, compensation, operator escalation). Then treat the playbook as untested code, because that is what it is. Partitions are rare enough that the partition path is the least-executed code in the system and the most likely to be wrong; Chapter 12's toolkit — Jepsen-style partition injection, fault schedules in CI, game days — exists precisely to run this code before the network does.

5. Find the coordination hidden in your dependencies. A service's PACELC position is the composition of its dependencies' positions, and the strictest dependency wins. An "AP" API tier that synchronously calls a CP database inherits minority-side unavailability, whatever its own design says; a "low-latency" service that acquires a lock from a coordination service on every request has a consensus round trip in its critical path, whether or not anyone budgeted it (Chapter 8 returns to this —

coordination services make coordination easy to consume, which is their virtue and their hazard). Draw the dependency graph, mark every edge that blocks on quorum or lock or leader, and you have the true PACELC diagram of your system — frequently a surprise to its own architects.

The distributed-systems lens: the same trade, all the way down

This volume's lens usually points outward from a single-machine topic to its distributed twin. This chapter's topic *is* distributed, so point the lens the other way — back into Volume 4 — and notice that CAP-shaped trade-offs were there all along.

A mutex is a tiny CP choice. A thread that must have the invariant blocks until it holds the lock: consistency purchased with availability, at the scale of nanoseconds and threads rather than minutes and datacenters. The pathologies rhyme, too — a thread that dies holding a lock is the in-process version of a partitioned leader, and Volume 4, Chapter 5's deadlock is coordination's liveness bill arriving all at once. Volume 4, Chapter 4's lock-free structures make the opposite choice: some thread always makes progress — availability as a progress guarantee — and the cost is paid in weaker, subtler intermediate states and algorithms that are hard to get right. That menu — block for the invariant, or progress with weaker guarantees — is the P branch of PACELC, miniaturized.

The ELSE branch was in Volume 4 as well, wearing a different name. The Universal Scalability Law's coherency term β (Volume 4, Chapter 1) is the cost of keeping N workers' views of shared state consistent, paid continuously, partition or no partition — cache-line ping-pong is synchronous replication between cores, billed in cycles instead of milliseconds. When Chapter 1 concluded that driving β toward zero is the only way to keep the scaling curve rising, it was stating the EL preference; when a team accepts eventual consistency to delete a cross-region round trip, it is attacking the same coefficient at a larger scale.

So the moral of Volume 4 and the moral of this chapter are one moral, and it is worth stating as the through-line for the rest of this volume: **coordination is the cost center.** Agreement — between threads about a memory location, between replicas about a register — is the thing that costs latency in the good times and availability in the bad times, at every scale from a cache line to a planet. The entire engineering discipline of

distributed systems can be read as a catalog of ways to buy less of it: partitioning so writes don't contend (Chapter 7 and Volume 5, Chapter 9), weakening models to the minimum the invariant needs (Chapter 3), making merges free so agreement can wait (Chapter 11), batching agreement so its round trips amortize (Chapters 5 and 6). Volume 4 reached this conclusion empirically, from profilers and scaling curves. This chapter's contribution is that the conclusion is not contingent: there is a theorem at the bottom of it. You cannot have agreement, progress, and an unreliable medium all at once — so spend agreement only where the invariant demands it, and know, in advance and in writing, what you will do when the medium misbehaves.

Key takeaways

- **CAP, precisely:** in an asynchronous network, a replicated read/write register cannot guarantee both linearizability and availability (every request to a non-failed node eventually answered) in executions with arbitrary message loss. The proof is a two-group partition: an available write on one side and an available read on the other cannot both respect real-time order.
- **P is not a choice.** Networks partition — partially, asymmetrically, and at fleet-relevant rates (Bailis & Kingsbury). The theorem's real content is the C-versus-A choice *during* a partition; outside partitions you can and should have both.
- **"CP system" and "AP system" are category errors.** Real systems choose per operation and per configuration — a Cassandra `QUORUM` write and a `ONE` write sit at different points of the trade-off in the same cluster on the same day.
- **CAP's C is linearizability specifically** — not ACID's C, not "strong-ish." Weaker models, causal consistency notably, remain achievable under partition; if your invariant needs less than linearizability, the theorem does not forbid you availability.
- **CAP's A is a literal liveness guarantee that few "highly available" systems claim**, and CAP says nothing about latency — which is why PACELC exists: if Partitioned, A-versus-C; Else, Latency-versus-C. The ELSE branch is the everyday trade, paid in round trips on every consistent write, with consensus RTTs as its floor.

- **PACELC corners:** PA/EL (Dynamo lineage), PC/EC (Spanner, CockroachDB, ZooKeeper), and the instructive mixed cases — PC/EL (PNUTS) as a deliberate bargain, PA/EC as too often an accidental one.
- **Mechanically,** CP means majority-side-only writes with leases and fencing to make the minority genuinely stop; AP means both sides accept and you owe a designed merge (Ch7, Ch11). Bounded-staleness and follower reads are the legitimate middle ground for reads.
- **Harvest and yield** (Fox & Brewer) name the degradation dials: yield is the probability of answering, harvest the completeness of the answer. Choosing which to degrade, per operation, is what graceful degradation means.
- **Discipline:** name the invariant and its blast radius; choose per operation; price the ELSE branch against a latency budget; write and *test* the partition playbook (Ch12); and audit dependencies for inherited coordination.
- **Coordination is the cost center** — the same conclusion Volume 4 reached inside one machine, now with a theorem attached. Buy agreement only where the invariant demands it.

Further reading

- Gilbert, S. and Lynch, N., "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News* 33(2), 2002 — the proof itself; four pages, read the definitions closely. <https://dl.acm.org/doi/10.1145/564585.564601>
- Brewer, E., "CAP Twelve Years Later: How the 'Rules' Have Changed," *IEEE Computer* 45(2), February 2012 — the author's own corrections: "2 of 3" is misleading, and partition mode / recovery is the real design problem. <https://ieeexplore.ieee.org/document/6133253>
- Brewer, E., "Towards Robust Distributed Systems," PODC keynote, 2000 — where the conjecture was posed.
- Abadi, D., "Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story," *IEEE Computer* 45(2), February 2012 — PACELC and the system classifications discussed above. <https://ieeexplore.ieee.org/document/6127847>

- Fox, A. and Brewer, E., "Harvest, Yield, and Scalable Tolerant Systems," *HotOS-VII*, 1999 — the two-dial vocabulary for graceful degradation.
- Bailis, P. and Kingsbury, K., "The Network is Reliable," *ACM Queue* 12(7), 2014 — the evidence survey on real-world partitions. <https://queue.acm.org/detail.cfm?id=2655736>
- Kleppmann, M., "A Critique of the CAP Theorem," 2015, arXiv:1509.05393 — a careful examination of the theorem's definitions and their mismatch with practice; proposes sharper vocabulary.
- Gill, P., Jain, N., and Nagappan, N., "Understanding Network Failures in Data Centers," *SIGCOMM*, 2011 — the Microsoft datacenter measurement study cited via Bailis & Kingsbury.
- Brewer, E., "Spanner, TrueTime and the CAP Theorem," Google whitepaper, 2017 — how a PC/EC system reasons about its own (small) availability sacrifice.
- Volume 6, Chapter 3 — Replication and Consistency Models — the precise definition of linearizability that CAP's C means, and the hierarchy beneath it.
- Volume 5, Chapter 8 — Replication — synchronous versus asynchronous replication, where the ELSE branch's latency price is quantified.
- Volume 5, Chapter 11 — NoSQL — the Dynamo lineage and per-operation consistency levels; and Chapter 12 — NewSQL — consensus RTT floors and follower reads.
- Volume 3, Chapter 11 — Network Reliability — what partitions are made of.

Chapter 5 — Consensus I: Paxos

What this chapter covers. Chapters 1 through 4 dismantled the comfortable assumptions: messages are delayed or lost, clocks disagree, FLP says no deterministic algorithm can guarantee agreement in a fully asynchronous system, and CAP forces a choice when the network partitions. This chapter builds the machine that lives inside those constraints. **Consensus** — getting a set of machines that can crash and a

network that can delay anything to agree on a single value, and to never, under any schedule of failures, disagree — is the foundational primitive of fault-tolerant systems. Every replicated database, every lock service, every configuration store, every leader election you have ever depended on either solves consensus or delegates to something that does.

The canonical solution is Paxos, published by Leslie Lamport and famous for two things: being correct, and being incomprehensible. The second reputation is only half-deserved. The core protocol — the single-decree Synod — is small: two phases, one non-obvious rule, and a safety argument that fits on a page. What is genuinely hard is everything wrapped around it to make a production system: the replicated log, leadership, log gaps, snapshots, membership change, and recovery. We do the Synod in full rigor, walk its safety argument carefully, extend it to Multi-Paxos and state-machine replication, survey the Paxos family honestly, and finish by reconciling consensus with two-phase commit — the promise made in Volume 5, Chapter 10. Raft, which repackages these same ideas for understandability, is Chapter 6; we point at it but leave its specifics there.

Learning goals — after this chapter you should be able to:

- State the consensus problem precisely — agreement, validity, termination — and explain why Paxos guarantees safety unconditionally but liveness only under partial synchrony.
- Explain why consensus is *the* primitive: its equivalence to atomic broadcast and state-machine replication, and its identity with CAS one layer down (Volume 4, Chapter 4).
- Execute single-decree Paxos by hand: both phases, the acceptor's state machine, and the phase-2 value-adoption rule, stated exactly.
- Reconstruct the safety argument: why quorum intersection makes a chosen value impossible to un-choose.
- Explain the livelock of dueling proposers and why every practical system elects a distinguished proposer.
- Describe Multi-Paxos: the log of instances, the stable-leader optimization, gap filling, and reconfiguration.
- Place Flexible Paxos and EPaxos in the design space, in one paragraph each.
- Reconcile 2PC and consensus precisely, including Spanner's layering of 2PC over Paxos groups.

The consensus problem, precisely

Consensus is easy to state and worth stating exactly, because every word carries weight. A set of n processes each proposes a value. The protocol must satisfy:

- **Agreement.** No two correct processes decide different values.
- **Validity.** The decided value was proposed by some process. (This rules out the trivial "always decide 42" solution, which satisfies agreement vacuously and is useless.)
- **Termination.** Every correct process eventually decides.

Agreement and validity are **safety** properties: they say nothing bad ever happens, and a violation is observable at a specific instant in a specific execution. Termination is a **liveness** property: something good eventually happens. The distinction is not academic — it is the exact seam along which Paxos is built.

Chapter 1 introduced the FLP result (Fischer, Lynch, and Paterson, 1985): **in a fully asynchronous system, no deterministic protocol can solve consensus with even one process that may crash.** The intuition is that in pure asynchrony a crashed process is indistinguishable from a slow one, so any protocol can be driven by an adversarial scheduler into postponing its decision forever — always one message away from deciding, never deciding. FLP does not say consensus is impossible in practice; it says *termination cannot be guaranteed deterministically* under the weakest timing assumptions.

Paxos responds by refusing to trade away the properties that matter. It guarantees **safety always** — under any message delays, any losses, any reorderings, any number of crashes and recoveries — and **liveness only under partial synchrony**: when the network behaves reasonably for long enough (message delays bounded, some stable leader emerging), decisions happen. When the network misbehaves, Paxos does not decide wrongly; it simply does not decide yet. This is the correct engineering posture, and every serious consensus system since has adopted it. Internalize the split now: **safety is unconditional, liveness is conditional**, and when we later find a liveness hole in Paxos we will not panic, because the hole can never produce a wrong answer.

One more scoping note. Paxos tolerates **crash faults** — processes stop, possibly restart with their stable storage intact — and requires a majority

of the n acceptors to be alive and reachable to make progress: $n = 2f + 1$ nodes tolerate f failures. It does not tolerate Byzantine faults (lying processes); that is a different and more expensive problem family, which we gesture at in Chapter 12's discussion of what testing can and cannot establish.

Why consensus is *the* primitive

Consensus looks narrow — agree on one value, once — but it is the universal ingredient, because of a chain of equivalences worth knowing by name.

Consensus $\Rightarrow$ atomic broadcast $\Rightarrow$ state-machine replication.

Atomic broadcast (also called total-order broadcast) delivers messages to all processes in the same order. Given consensus, you build it by running one consensus instance per sequence-number slot: everyone agrees on what message occupies slot 1, then slot 2, and so on, and delivers in slot order. Given atomic broadcast, consensus is trivial: broadcast your proposal and decide the first value delivered. The two problems are equivalent — each implements the other. And atomic broadcast plus a **deterministic state machine** gives you *state-machine replication* (SMR): if every replica starts in the same state and applies the same commands in the same order, every replica passes through the same sequence of states. Any service you can express as a deterministic state machine — a key-value store, a lock table, a metadata catalog — becomes fault-tolerant by feeding it an agreed log. This is Lamport's 1978 state-machine approach, and it is *the* architecture of replicated systems; Multi-Paxos, below, is exactly this construction.

Consensus is the distributed CAS. Volume 4, Chapter 4 presented Herlihy's consensus hierarchy: atomic registers have consensus number 1, and compare-and-swap has consensus number ∞ — with CAS you can build wait-free consensus, and hence any concurrent object, for any number of threads. The same statement holds one layer up with the roles reversed: a consensus protocol is a compare-and-swap whose "memory cell" is replicated across failure domains. Single-decree Paxos is precisely "CAS from $\perp$ to v , exactly once, surviving the crash of any minority of the machines implementing the cell." Cassandra makes this literal — its lightweight transactions expose an **IF** clause that is CAS implemented by running Paxos, as we will see. The shared-memory world got its

universal primitive from the hardware vendor; the distributed world has to build it out of unreliable parts, and this chapter is the construction.

The practical corollary: when a design says "we just need the nodes to agree on which one is primary" or "we just need at-most-once processing of this record," it needs consensus, whether the designer knows it or not. Systems that need consensus and improvise it instead — heartbeat-based failover with no quorum, "the node with the lowest IP wins" — are the origin of a large fraction of split-brain incidents. We return to this in the lens section.

Single-decree Paxos: the Synod

The single-decree protocol — Lamport's *Synod* — chooses exactly one value, once, forever. It is the unit of everything that follows, so we do it exactly.

Roles and proposal numbers

Three roles, usually co-located on the same physical nodes but logically distinct:

- **Proposers** propose values and drive the protocol.
- **Acceptors** are the memory. A value is **chosen** when a majority of acceptors have accepted it. Acceptors are the only role whose state matters for safety.
- **Learners** find out what was chosen; they hold no protocol state.

Every proposal carries a **proposal number** n . Numbers must be totally ordered and unique across proposers — the standard construction is a monotonic counter concatenated with the proposer's node ID, so proposer 2's numbers are (1,2), (2,2), ... and can never collide with proposer 3's. A proposer that learns of a higher number in use resumes with a number higher still.

The two phases

Phase 1a — prepare. A proposer picks a new proposal number n and sends `prepare(n)` to at least a majority of acceptors (in practice, all of them).

Phase 1b — promise. An acceptor receiving `prepare(n)` compares n against the highest proposal number it has ever promised. If n is higher,

it replies with a **promise**: "*I will never accept any proposal numbered less than n* " — and crucially, the reply also carries **the highest-numbered proposal (number and value) this acceptor has ever accepted, if any**. If n is not higher than an existing promise, the acceptor rejects (or ignores) the prepare; a rejection carrying the higher number it has seen is a useful optimization, letting the proposer retry intelligently rather than blindly.

Phase 2a — accept. If the proposer collects promises from a majority of acceptors, it sends `accept( $n$ ,  $v$ )` to the acceptors, where the choice of v is governed by the one rule in Paxos that everything else exists to serve:

The value rule. If any promise in the phase-1 majority reported a previously accepted proposal, the proposer **must** set v to the value of the **highest-numbered** proposal among all those reported. Only if *no* acceptor in the majority reported any accepted proposal is the proposer free to use its own value.

This is not an optimization and not a heuristic. It is the entire safety mechanism, and the next section is devoted to why. Note what it implies about proposers: a proposer is not an advocate for its own value; it is a volunteer executor that will happily drive *someone else's* value to completion if the protocol demands it.

Phase 2b — accepted. An acceptor receiving `accept( $n$ ,  $v$ )` accepts it — records (n , v) as its accepted proposal — **unless** it has meanwhile promised a number greater than n (a later proposer's phase 1 got there first). On accepting, it notifies the learners (or a distinguished learner that fans out, saving messages). When any learner observes that a majority of acceptors accepted the same proposal, the value is **chosen**, and no different value can ever be chosen.

Two engineering details that are part of the protocol, not afterthoughts. First, **acceptor state is persistent**: the promised number and the accepted (n , v) must be written to stable storage *before* replying. An acceptor that forgets a promise after a crash-restart can break safety — this is the `fsync-before-ack` rule of Volume 5, Chapter 7, appearing in its distributed form. Second, majorities matter because **any two majorities of the same acceptor set intersect** in at least one acceptor. Every safety property below is bought with that intersection.

The acceptor, in full — this is genuinely all of it:

```

class Acceptor:
    def __init__(self, stable):
        self.stable = stable                # crash-persistent
storage
        self.promised_n = stable.load("promised_n", default=MINUS_INFINITY)
        self.accepted = stable.load("accepted", default=None) #
(n, v) or None

    def on_prepare(self, n):
        if n > self.promised_n:
            self.promised_n = n
            self.stable.store("promised_n", n)        # persist BEFORE
replying
            return Promise(n, self.accepted)          # ship prior
accepted (n, v), if any
        else:
            return Reject(self.promised_n)            # optimization:
tell them why

    def on_accept(self, n, v):
        if n >= self.promised_n:                # >= : a promise
permits its own n
            self.promised_n = n
            self.accepted = (n, v)
            self.stable.store_all(promised_n=n, accepted=(n, v)) #
persist BEFORE replying
            return Accepted(n, v)                  # also sent to
learners
        else:
            return Reject(self.promised_n)

```

The happy path, with one proposer and three acceptors:


```

sequenceDiagram
    participant P as Proposer P1
    participant A1 as Acceptor A1
    participant A2 as Acceptor A2
    participant A3 as Acceptor A3
    participant L as Learner

    Note over P: Phase 1 with n = 5
    P->>A1: prepare n=5
    P->>A2: prepare n=5
    P->>A3: prepare n=5
    A1-->>P: promise n=5, no prior accepted value
    A2-->>P: promise n=5, no prior accepted value
    Note over A3: slow or down – majority suffices
    Note over P: majority promised, none reported a value
    P is FREE to pick its own value v = X
    P->>A1: accept n=5, v=X
    P->>A2: accept n=5, v=X
    A1-->>P: accepted n=5, v=X
    A2-->>P: accepted n=5, v=X
    A1->>L: accepted n=5, v=X
    A2->>L: accepted n=5, v=X
    Note over L: majority accepted the same proposal
    X is CHOSEN – irrevocably

```

Two round trips, majority participation in each. Now the interesting part.

Why it is safe

The claim to prove: **once a value is chosen, no different value can ever be chosen**. Not by a later proposer, not after crashes, not under any message schedule. We argue it informally but carefully; the structure is an induction on proposal numbers, and the induction step is where the value rule earns its keep.

Suppose value v is chosen with proposal number n : some majority Q of acceptors accepted (n, v) . Consider any proposal numbered $n' > n$ that ever reaches phase 2 — we show its value must also be v , taking n' to be the *smallest* number greater than n that reaches phase 2 (induction then extends the argument to all higher numbers).

The proposer of n' first completed phase 1 with promises from some majority Q' . **Q' and Q intersect** — any two majorities do — so at least one acceptor a is in both. Two cases for the order of events at a :

1. a accepted (n, v) *before* promising n' . Then a 's promise reported (n, v) — or some other accepted proposal with a number between n and

n' . But by the induction hypothesis every proposal in that range that reached phase 2 carries value v , so whatever a reported, its value is v . Moreover the highest-numbered proposal reported by *anyone* in Q' is at least n and less than n' , so it too carries v . The value rule then **forces** the proposer of n' to adopt v . It cannot propose anything else.

2. a promised n' *before* accepting (n, v) . Then when `accept( $n, v$ )` later arrived, a held a promise for $n' > n$ and **rejected it** — so a never accepted (n, v) at all, contradicting $a \in Q$. This case cannot occur.

Either the later proposer is forced to carry the chosen value forward, or the choosing majority was never assembled in the first place. There is no third path, because every phase-2 acceptance at an acceptor is either before or after its phase-1 promise, and acceptor state is persistent — crashes do not create a third ordering.

Notice what the machinery is *for*. The promise ("I will reject anything below n ") freezes the past: after phase 1, no proposal older than n' can sneak into the intersection acceptors and change what the proposer learned. The reported accepted values transmit the past: anything already chosen — or possibly chosen, since the proposer cannot distinguish "chosen" from "accepted by some acceptors" — is surfaced in phase 1 and adopted. Paxos never needs to *know* whether a value was chosen; it maintains the stronger invariant that any value that *might* have been chosen is preserved by every later proposal. Uncertainty is handled by conservatism, not by detection.

Here is the argument as an execution — a duel where the second proposer is forced to adopt the first one's value:

```

sequenceDiagram
    participant P1 as Proposer P1
    participant A1 as Acceptor A1
    participant A2 as Acceptor A2
    participant A3 as Acceptor A3
    participant P2 as Proposer P2

    Note over P1,A3: P1 runs phase 1 with n=5, gets promises from A1,
A2
    P1->>A1: accept n=5, v=X
    A1-->>P1: accepted n=5, v=X
    Note over A2: P1's accept to A2 is delayed in the network
    P2->>A2: prepare n=7
    P2->>A3: prepare n=7
    A2-->>P2: promise n=7, no accepted value yet
    A3-->>P2: promise n=7, no accepted value yet
    P1->>A2: accept n=5, v=X arrives late
    A2-->>P1: REJECT – promised n=7
    Note over P1: P1 cannot assemble a majority for n=5
    X was accepted by A1 only – NOT chosen
    P2->>A1: prepare n=7 retransmitted or later round
    A1-->>P2: promise n=7, reports accepted n=5 v=X
    Note over P2: value rule: A1 reported n=5 v=X, highest seen
    P2 MUST propose v=X, abandoning its own value Y
    P2->>A1: accept n=7, v=X
    P2->>A2: accept n=7, v=X
    A1-->>P2: accepted n=7, v=X
    A2-->>P2: accepted n=7, v=X
    Note over A1,A2: majority accepted n=7 v=X – X chosen
    Safety held: only X could ever be chosen here

```

Note the subtlety the diagram surfaces: X had been accepted by only *one* acceptor — it was not chosen, and P2 was in principle free to choose Y. But P2 cannot tell whether A1's acceptance is a lone straggler or the visible edge of a majority, so the rule makes it adopt X regardless. Paxos sometimes completes a value that had not been chosen and did not need to survive. That is the price of conservatism, and it is the right price: validity still holds (X was proposed by someone), and agreement is never at risk.

Liveness: dueling proposers and the livelock

Now the hole FLP promised us. Two proposers, alternating:

P1 completes phase 1 with $n = 5$. Before P1's accepts land, P2 completes phase 1 with $n = 7$ — acceptors promise 7, and now reject P1's `accept(5, ...)`. P1, seeing rejections, retries with $n = 9$, and its prepares cause the

acceptors to reject P2's `accept(7, ...)`. P2 retries with 11. Each proposer's phase 1 invalidates the other's phase 2, forever. No value is ever chosen.

This is a **livelock** — everyone is busy, nobody progresses — and it is structurally the same failure as two threads endlessly retrying and mutually aborting a CAS loop, or the politeness-livelock of Volume 4, Chapter 5. Note carefully what has *not* happened: no acceptor has accepted conflicting values; safety is intact. The protocol is not wrong, it is stuck. FLP guarantees some such execution exists for any protocol; Paxos's virtue is that its stuck executions are merely stuck.

The fix is the one every practical system converges on: a **distinguished proposer**. Elect a single leader — typically by timeout: if you have not heard from the current leader in T milliseconds, attempt to take over with a higher proposal number, with randomized timeouts to break symmetry — and route all proposals through it. With one active proposer, there is no duel, and consensus completes in the two round trips of the happy path. Under partial synchrony the election stabilizes and liveness follows; during instability there may transiently be two self-believed leaders, and then *safety does not depend on the election being correct* — two "leaders" are just two dueling proposers, which we have already shown cannot violate agreement. The election only needs to be right *eventually*, for liveness. This layering — sloppy, fast leader election on top of an unconditionally safe core — is the signature move of the entire consensus literature, and you will see it again with different clothes in Raft's terms and elections (Chapter 6).

Multi-Paxos: from one decision to a replicated log

Real systems do not need one decision; they need an unbounded ordered sequence of them — a log. The construction is direct: run an independent single-decree instance per **slot**. Instance i chooses the command for log position i . Any majority can be down for one instance and up for another; the instances share nothing but the acceptor nodes.

Run naively, that costs two round trips (four message delays) per command, with phase 1 executed per slot. The **stable-leader optimization** removes most of it, and this is what "Multi-Paxos" means in practice: when a proposer becomes leader, it runs phase 1 *once* for all slots at and above its current position — a single `prepare(n)` covering

the infinite suffix of the log. Acceptors promise n for every instance $\geq$ the leader's start position and report any accepted proposals they hold in that range. From then on, while its leadership is unchallenged, the leader executes **only phase 2** per command: assign the next free slot, send `accept( $n$ ,  $slot$ ,  $v$ )`, and the command commits on a majority of acceptances — **one round trip per command**, and commands for different slots pipeline. This is the shape of essentially every production consensus system: steady-state replication is a single leader streaming accepts to a majority; the two-phase machinery surfaces only at leadership change.

```
sequenceDiagram
    participant C as Client
    participant L as Leader stable, n=8
    participant F1 as Acceptor F1
    participant F2 as Acceptor F2

    Note over L,F2: Phase 1 for n=8 ran ONCE at election,
    covering all slots from 17 onward
    C->>L: command W1
    L->>F1: accept n=8, slot=17, v=W1
    L->>F2: accept n=8, slot=17, v=W1
    C->>L: command W2
    L->>F1: accept n=8, slot=18, v=W2
    L->>F2: accept n=8, slot=18, v=W2
    F1-->>L: accepted slot 17
    F2-->>L: accepted slot 17
    Note over L: slot 17 chosen – one round trip
    L-->>C: W1 committed
    F1-->>L: accepted slot 18
    Note over L: slot 18 chosen with L plus F1 –
    F2 slow, majority suffices, pipelined
    L-->>C: W2 committed
    L->>F2: commit notice for 17, 18 piggybacked on later traffic
```

Gaps, recovery, and reconfiguration

The pieces the neat picture omits are exactly the pieces that make consensus engineering hard.

Log gaps. A new leader's blanket phase 1 returns, for each slot, what the acceptors have accepted — and the picture is ragged: slots 1–40 chosen, 42 accepted by one acceptor, 41 and 43 empty, because the old leader died mid-stream or a client request was lost. The new leader must bring every unresolved slot below its write point to a decision before the state

machine can execute past it: for slots with a reported accepted value, it re-proposes that value under its own number (the value rule, per instance); for genuinely empty slots it proposes an explicit **no-op** command, closing the gap. Execution order is strict: a replica may apply slot i only when every slot $\leq i$ is chosen and applied, so an unresolved slot stalls execution of everything after it — a head-of-line blocking property worth remembering when you see p99 latency spikes at leader failover.

Reconfiguration. Changing the acceptor set — replacing a dead node, growing from 3 to 5 — is itself a question every replica must agree on, because "majority" is defined relative to the membership: if half the cluster thinks membership is {A,B,C} and half thinks it is {A,B,C,D,E}, two disjoint "majorities" can choose different values for the same slot, and safety is gone. Lamport's answer is elegantly recursive: **the configuration is state-machine state**, changed by committing a reconfiguration command through the log itself, taking effect a bounded number of slots later (α slots ahead, bounding the pipeline). Correct, subtle in the details, and a significant fraction of real-world consensus bugs live here; Raft's joint-consensus and single-server-change approaches (Chapter 6) are responses to precisely this subtlety.

State-machine replication, assembled

Now the full stack: a deterministic state machine on each replica, fed by the agreed log. Each replica applies chosen commands in slot order; identical initial state plus identical command sequence yields identical state everywhere. This is the log-centric worldview of Volume 5, Chapter 7 — the WAL *is* the database, replay *is* recovery — with one addition: the log itself is now replicated and fault-tolerant. Snapshots complete the picture exactly as they do for a WAL: a replica checkpoints its state-machine state at some slot and discards the log prefix, and a lagging or fresh replica is caught up by snapshot transfer plus log suffix rather than replay from slot zero.

Writes go through the log, unavoidably: one leader round trip to a majority. **Reads** are where systems differentiate, and honesty is required:

- **Read through the log.** Submit the read as a command; it executes in slot order. Linearizable, trivially correct, and costs a full consensus round trip per read.
- **Leader-local reads with leases.** The leader serves reads from its local state machine, which is safe only if it is *still* the leader — a

deposed leader serving reads is serving stale data, and linearizability is gone. The standard fix is a **leader lease**: the majority promises not to elect a new leader for an interval, and the leader serves local reads while its lease is provably unexpired. Fast — no network round trip per read — but now correctness rests on a *timing* assumption: bounded clock drift across nodes. This should make you flinch after Chapter 2, and the flinch is correct; leases are a measured re-admission of synchrony assumptions for performance, and their safety margin (the drift bound) is a config parameter someone has to get right. Spanner's leases work this way; so do etcd's lease-based reads.

- **Read index / quorum check.** Middle ground: the leader confirms leadership with one round of heartbeats to a majority (no log write), then serves the read locally at or after the confirmed commit index. One cheap round trip, no clock assumptions. Raft's ReadIndex — details in Chapter 6.

The Paxos family

Fifteen years of variants exist; four are worth your attention, three of them briefly.

Cheap Paxos (Lamport & Massa, 2004) uses $f + 1$ active acceptors plus f idle backups that participate only during failures — trading steady-state hardware for a more delicate failure-handling path. **Fast Paxos** (Lamport, 2006) lets clients send proposals directly to acceptors, skipping the leader and saving one message delay in the conflict-free case, at the cost of larger quorums (typically $\lceil 2n/3 \rceil$) and a collision-recovery protocol when concurrent proposals conflict. Both are primarily interesting as points in the design space; neither is common in production.

Flexible Paxos (Howard, Malkhi, and Spiegelman, 2016) is different: it is a *theorem about classic Paxos* that practitioners should know. Re-examine the safety argument above: the only intersection it ever used was between a phase-1 quorum and a phase-2 quorum. Nothing requires two phase-2 quorums to intersect each other, or two phase-1 quorums to intersect each other. So the majority-everywhere rule is stronger than necessary: **any quorum systems Q1 and Q2 work, provided every Q1 quorum intersects every Q2 quorum** — $|Q1| + |Q2| > n$ for simple

counting quorums. This matters because the two phases run at wildly different frequencies: phase 2 runs *per command*, phase 1 only at *leader election*. Flexible Paxos lets you shrink the hot phase-2 quorum and pay for it with a larger, rarely-assembled phase-1 quorum — in a 5-node cluster, commit on 2 acceptances if elections require 4 promises. The trade is real: smaller phase-2 quorums also mean fewer node failures tolerated for *write availability* before a (now larger) election can even be held. But the insight reframes what a quorum is for — not "a majority," but "an intersection guarantee between the phase that learns the past and the phase that writes the present" — and it licensed a generation of quorum experimentation in real systems.

EPaxos (Egalitarian Paxos; Moraru, Andersen, and Kaminsky, 2013) attacks the leader itself: in a geo-replicated system, a single leader is a throughput bottleneck and a latency penalty for every client on the wrong continent. EPaxos has no leader — any replica proposes directly — and replaces the totally-ordered log with a **dependency graph**: commands that do not conflict (touch disjoint keys) commit independently in one round trip from whichever replica is nearest, and only conflicting commands pay for ordering, with replicas executing strongly-connected components of the graph in a deterministic order. Conceptually important — it shows total order per se is not the requirement, only order among conflicts — but the execution machinery is complex enough that production adoption has been thin, and subsequent analysis found subtle bugs in the original recovery protocol. Its ideas resurface in leaderless and multi-leader designs, including Cassandra's Accord.

Why Paxos is famously hard

The reputation deserves an honest accounting, because its causes are instructive.

The history did real damage. Lamport wrote "The Part-Time Parliament" around 1990, presenting the protocol through an extended allegory about the legislature of the Greek island of Paxos, complete with pseudo-Greek names for the legislators. Reviewers found the framing baffling; the paper sat unpublished for eight years, circulating as lore, and finally appeared in *TOCS* in 1998 mostly unchanged. By then the protocol was already load-bearing inside serious systems, but a generation of readers bounced off the allegory. Lamport's 2001 "Paxos Made Simple" — abstract: "The Paxos algorithm, when presented in plain

English, is very simple." — is the plain presentation, and this chapter's Synod section is essentially its content. Read it; it is short and it delivers.

The real difficulty was never the Synod. The honest witness here is Google's "Paxos Made Live" (Chandra, Griesemer, and Redstone, 2007), the experience report from building the Chubby lock service's replicated core. Their summary is the quote every engineer should carry: the gap between the algorithm in the literature and a production system is **non-trivial** — Multi-Paxos details, disk corruption handling (an acceptor whose disk lies has *forgotten its promises*, which is a safety threat, so Chubby ran checksummed disks and made a corrupted replica rejoin as a non-voting learner until it caught up), master leases, snapshots, reconfiguration, database transfer to new replicas, and a testing effort that dwarfed the protocol implementation. They found bugs in their own extensions that model-checking the core would never have caught, and noted pointedly that the fault-tolerance literature, by proving the core and hand-waving the rest, had left them to invent and debug the rest themselves. The paper is twenty pages of exactly the material Lamport's papers omit, and it is the single best corrective to the belief that understanding the Synod means you can ship consensus.

And that gap is what motivated Raft. Ongaro and Ousterhout's 2014 paper is titled "In Search of an Understandable Consensus Algorithm," and its explicit design criterion — evaluated by teaching both algorithms to students and measuring quiz scores — was understandability. Raft is not a different mathematical animal: it is leader-based log replication with quorum intersection, the same safety skeleton, restructured so that the leader is primary (elected first, log flows one way), the log is kept dense and ordered by construction, and the corner cases the Paxos literature leaves as exercises are specified in one paper. Chapter 6 gives it the full treatment; go there next.

Where Paxos actually runs

Chubby (Burrows, 2006) is Google's lock service: a five-replica cell running Multi-Paxos under a replicated database, exposing a filesystem-like namespace of small files and advisory locks. Its consensus is an implementation detail; its *interface* is locks and small files, which is why GFS, Bigtable, and half of Google use it for leader election and configuration — a handful of consensus cells servicing thousands of client systems that never touch Paxos directly. Chubby is the prototype of the

"coordination service" pattern — ZooKeeper and etcd are its descendants — covered in Chapter 8.

Spanner (Corbett et al., 2012) runs Paxos at planetary scale: every tablet of data belongs to a **Paxos group** of replicas spanning datacenters, with a long-lived leader holding a lease, replicating a write log — thousands of independent Multi-Paxos instances as the storage layer's replication fabric (Volume 5, Chapter 12 covers Spanner whole, including TrueTime). Its use of 2PC *on top of* Paxos groups is the subject of the next section.

Cassandra's lightweight transactions are the CAS framing made product. `INSERT ... IF NOT EXISTS` and `UPDATE ... IF value = expected` run single-decree Paxos among the replicas of the affected partition — linearizable compare-and-set on top of an otherwise eventually-consistent store, exactly Volume 4, Chapter 4's primitive rebuilt from quorums. The cost profile deserves honesty: classic LWTs take **four round trips** (prepare/promise, a read of the current value, propose/accept, commit) versus one for a normal quorum write, contending proposers can livelock under heavy contention on hot keys — the dueling-proposers failure, live in production, because per-key Paxos has no stable leader — and the latency multiple is real. Cassandra 4.1's Paxos v2 cut round trips and contention costs, and the Accord protocol (EPaxos lineage) is its successor for general transactions. Use LWTs for the rare correctness-critical CAS, not as a default write path.

2PC and consensus, reconciled

Volume 5, Chapter 10 ended with a promise: 2PC and consensus look confusingly similar — rounds, votes, a coordinator — and the difference would be made precise here. It is a difference of *problem*, not of protocol quality.

Two-phase commit solves atomic commitment: n participants each hold a local yes/no vote (can I commit my part of this transaction?), and the outcome must be commit **only if every participant voted yes**. Unanimity is the spec — participant votes are facts about local state (constraints checked, locks held, redo logged), and no majority can outvote a participant whose disk cannot commit. **Consensus solves agreement:** any proposed value may win as long as everyone agrees; a majority suffices, and a minority's silence is tolerable. From this one difference the operational contrast follows mechanically. 2PC requires

unanimity, so it **cannot tolerate even one relevant failure**: a coordinator crash after prepare leaves participants in-doubt, holding locks, blocked — they cannot commit (maybe someone voted no) and cannot abort (maybe the coordinator decided commit and told a participant that is also down). Consensus requires a majority, so it **tolerates any minority failure, including the leader**: a new leader runs phase 1, learns anything possibly decided, and continues. 2PC is availability-fragile by specification; consensus is availability-robust by specification.

The synthesis — Spanner's move, and by now the industry-standard one — is to notice that 2PC's blocking has a single root cause: **the coordinator's decision lives in one place**. So put it in more than one place: **run the coordinator itself as a state machine replicated by Paxos**. The decision (commit/abort) is committed to the coordinator group's Paxos log before it is announced; if the coordinator's leader dies, the group elects a new leader that reads the decision — or its absence — from the replicated log and resumes the protocol. Each *participant* is likewise a Paxos group, so participant votes survive individual node deaths too. The layering is exact: **2PC provides atomicity across groups** (unanimity where unanimity is the spec), **Paxos provides fault tolerance within each group** (so no single machine's death blocks anything). 2PC over Paxos is still 2PC — a partitioned-away participant *group* still blocks the transaction, per the spec — but the classic in-doubt window from a single machine crash is engineered away.

```

flowchart TB
    subgraph CG["Coordinator Paxos group – transaction T"]
        CL["Leader"] --- CF1["Replica"] --- CF2["Replica"]
    end
    end
    subgraph PG1["Participant Paxos group 1 – data shard A"]
        L1["Leader"] --- F1a["Replica"] --- F1b["Replica"]
    end
    end
    subgraph PG2["Participant Paxos group 2 – data shard B"]
        L2["Leader"] --- F2a["Replica"] --- F2b["Replica"]
    end
    end
    CL -->|"2PC prepare"| L1
    CL -->|"2PC prepare"| L2
    L1 -->|"vote yes, prepare record replicated via Paxos in group 1"| CL
    L2 -->|"vote yes, prepare record replicated via Paxos in group 2"| CL
    CL -->|"decision COMMIT written to coordinator group Paxos log FIRST"| CF1
    CL -->|"then announce commit"| L1
    CL -->|"then announce commit"| L2

```

Read the layering as a slogan: *2PC decides what all must do; Paxos makes sure nobody who decided can forget*. When someone tells you "2PC is obsolete, consensus replaced it," they have the relationship wrong — the problems are different, and the modern answer composes them.

The distributed-systems lens

This volume's lens usually connects a topic outward to large-scale engineering. Consensus is large-scale engineering's foundation, so the lens turns reflexive: connect it back down to Volume 4, where every idea in this chapter has a shared-memory twin.

Consensus is mutual exclusion writ large. A Multi-Paxos leader is exactly a lock holder: one agent at a time is licensed to mutate the shared resource (the log), everyone else queues behind it, and the phase-1/ election machinery is the lock acquisition path. The analogy carries the pathologies too. Leader churn is a **lock convoy** (Volume 4, Chapters 2 and 5): every leadership change stalls all writers behind an expensive handoff — blanket phase 1, gap resolution — so a flapping leader (aggressive election timeouts plus a jittery network) produces the same sawtooth throughput as a convoying mutex, and the tuning is the same in spirit: make handoffs rare and the critical section fast. And a leader lease

is precisely a lock with a timeout, which drags in the timeout-lock's classic hazard — the holder that does not know it has been deposed — answered here by fencing (Chapter 2) and by clock-drift safety margins.

Quorum intersection is a memory barrier. Volume 4, Chapter 3's acquire/release pair creates a visibility point: everything before the release is visible to everything after the acquire. Paxos's phase 1 is the same construction across machines. The phase-2 write to a majority is the release — the value is now lodged where no future quorum can miss it; the new proposer's phase 1 against an intersecting quorum is the acquire — it is *forced* to observe and respect everything released before it. The store buffer even has its twin: a value accepted by a lone acceptor is a buffered store, written but not yet globally visible, and the value-adoption rule is the coherence machinery that drains it into the global order rather than letting it be silently overwritten. Same design space, one layer up — the claim from Volume 4, Chapter 3's lens table, now witnessed from the other side.

And the same engineering moral. Volume 4, Chapter 4 closed with: do not write lock-free data structures; use the ones experts shipped. The distributed edition is stronger, because the failure modes are worse and the testing is harder (Chapter 12): **do not hand-roll consensus. Use etcd, ZooKeeper, or your platform's equivalent** (Chapter 8 covers them as systems). The gravitational pull to violate this is real — consensus hides inside innocuous-sounding features like "automatic failover," "exactly-once delivery," and "distributed scheduler," and teams implement heartbeat-plus-takeover without noticing they have reinvented leader election minus the safety argument. The test: if two nodes could simultaneously believe they hold a role, and that belief can corrupt data, you need real consensus underneath — a lease from etcd plus a fencing token, not a heartbeat and a prayer. "Paxos Made Live" is the measure of what doing it properly cost a team of strong engineers at Google with a correct algorithm in hand; your feature team will not budget for it, and should not have to.

Key takeaways

- **Consensus = agreement + validity + termination.** Agreement and validity are safety; termination is liveness. FLP forbids guaranteeing all three deterministically under pure asynchrony, so

Paxos guarantees **safety unconditionally and liveness under partial synchrony** — stuck is possible, wrong is not.

- **Consensus is the universal primitive:** equivalent to atomic broadcast, which plus a deterministic state machine gives state-machine replication. It is CAS with consensus number ∞ (Volume 4, Chapter 4) rebuilt across failure domains.
- **The Synod is two phases and one rule.** Phase 1: `prepare(n)` collects promises (reject everything below n) and any previously accepted proposals. Phase 2: `accept(n, v)` where **v must be the value of the highest-numbered accepted proposal reported in phase 1** — the proposer's own value only if none was reported. Acceptor state is fsync-persistent.
- **Safety = quorum intersection + the value rule.** Any later proposer's phase-1 majority intersects any choosing majority, so it either learns the chosen value and is forced to adopt it, or its promises prevented the choice from completing. Chosen values cannot be un-chosen.
- **Liveness fails via dueling proposers** — a livelock, the distributed twin of Volume 4, Chapter 5's — fixed by a timeout-elected distinguished proposer. The election may be sloppy: safety never depends on it, only liveness does.
- **Multi-Paxos = one instance per log slot + stable leader.** Phase 1 runs once per leadership over all slots; steady state is one round trip per command. New leaders must resolve log gaps (re-propose or no-op) before executing past them; reconfiguration flows through the log itself.
- **Reads are not free:** read-through-log (slow, clean), leader leases (fast, reintroduces clock assumptions), or quorum read-index (the middle). Know which one your system uses.
- **Flexible Paxos:** phase-1 and phase-2 quorums need only intersect *each other* — shrink the per-command quorum, enlarge the per-election one. **EPaxos:** no leader, order only conflicts via dependency graphs. Cheap and Fast Paxos exist; know the names.
- **The Synod is simple; the system is not.** "Paxos Made Live" documents the gap — gaps, snapshots, disk corruption, reconfiguration, testing — and that gap, not the math, is why Raft (Chapter 6) was designed for understandability and why Chubby, Spanner, and Cassandra LWT ship Paxos behind interfaces.

- **2PC $\neq$ consensus, precisely:** 2PC is atomic commitment (unanimity; blocks on coordinator failure), consensus is majority agreement (tolerates minority failure, leader included). Spanner composes them — 2PC across Paxos groups, the coordinator's own state on Paxos — so the decision can never be lost with a single machine.
- **Never hand-roll consensus.** If split-brain can corrupt your data, put etcd or ZooKeeper under it (Chapter 8). Same moral as "don't write lock-free structures," with higher stakes.

Further reading

- Lamport, L., "The Part-Time Parliament," *ACM Transactions on Computer Systems* 16(2), 1998 — the original, allegory and all; read it second, not first. <https://dl.acm.org/doi/10.1145/279227.279229>
- Lamport, L., "Paxos Made Simple," *ACM SIGACT News* 32(4), December 2001 — the plain-English Synod; the single best short read on this chapter's core. <https://lamport.azurewebsites.net/pubs/paxos-simple.pdf>
- Fischer, M., Lynch, N., and Paterson, M., "Impossibility of Distributed Consensus with One Faulty Process," *JACM* 32(2), 1985 — FLP. <https://dl.acm.org/doi/10.1145/3149.214121>
- Chandra, T., Griesemer, R., and Redstone, J., "Paxos Made Live — An Engineering Perspective," *PODC*, 2007 — the honest account of the paper-to-production gap; mandatory before implementing anything. <https://dl.acm.org/doi/10.1145/1281100.1281103>
- Burrows, M., "The Chubby Lock Service for Loosely-Coupled Distributed Systems," *OSDI*, 2006 — consensus packaged as a lock service; the ancestor of ZooKeeper and etcd. <https://research.google/pubs/pub27897/>
- Howard, H., Malkhi, D., and Spiegelman, A., "Flexible Paxos: Quorum Intersection Revisited," *OPODIS*, 2016 — the generalization of Paxos's quorum requirement. <https://arxiv.org/abs/1608.06696>
- Moraru, I., Andersen, D., and Kaminsky, M., "There Is More Consensus in Egalitarian Parliaments," *SOSP*, 2013 — EPaxos. <https://dl.acm.org/doi/10.1145/2517349.2517350>
- Ongaro, D. and Ousterhout, J., "In Search of an Understandable Consensus Algorithm," *USENIX ATC*, 2014 — Raft; the forward

pointer to Chapter 6. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>

- Corbett, J. et al., "Spanner: Google's Globally-Distributed Database," *OSDI*, 2012 — Paxos groups and 2PC-over-Paxos at scale; treated fully in Volume 5, Chapter 12. <https://research.google/pubs/pub39966/>
- Van Renesse, R. and Altinbuken, D., "Paxos Made Moderately Complex," *ACM Computing Surveys* 47(3), 2015 — Multi-Paxos with all the roles and pseudo-code spelled out; the bridge between "Paxos Made Simple" and an implementation.
- Volume 4, Chapters 4 and 5 — CAS and the consensus hierarchy; livelock — the shared-memory twins of this chapter's primitive and its failure mode.
- Volume 6, Chapters 6 and 8 — Raft, and the coordination services that let you never implement any of this yourself.

Chapter 6 — Consensus II: Raft

What this chapter covers. Chapter 5 established what consensus is, why it requires majorities, and why it cannot be both safe and live in a fully asynchronous system — and it did so through Paxos, which solves the problem with a minimum of mechanism and a maximum of reader suffering. This chapter presents Raft, which solves the *same* problem with the same majority mechanics and the same partial-synchrony liveness assumptions, but with a different engineering objective stated explicitly in its title: understandability. We take Raft apart along the seams its designers built into it — leader election, log replication, and safety — and we are precise about the two rules everything hangs on: the election restriction (a candidate must have a log at least as up-to-date as each voter's) and the commitment rule (a leader counts replicas only for entries of its own current term). The second rule is subtle enough that the paper devotes its Figure 8 to it, and we walk that figure step by step, because it is the heart of Raft and the place where casual understanding fails. We then cover the parts that turn the algorithm into a system: membership changes, snapshots, client sessions, linearizable reads, PreVote, and leader transfer. Finally we look at Raft in production — etcd, CockroachDB and TiKV's multi-raft, Consul, Kafka's KRaft — and at the

deployment arithmetic of 3 versus 5 nodes, geo-placement, and the fsync floor.

Learning goals — after this chapter you should be able to:

- Explain Raft's decomposition and state-space constraints, and why they exist.
- State the full server-state transition rules, the voting rules, and the log up-to-date check exactly, and explain why that check alone makes elections safe.
- State the commitment rule exactly — current-term entries commit by counting; prior-term entries commit only indirectly — and reproduce the Figure 8 counterexample that motivates it.
- Trace the AppendEntries consistency check through a divergent follower log, including nextIndex backoff and suffix truncation.
- Enumerate the five safety properties and sketch the Leader Completeness argument.
- Explain why naive membership changes are unsafe, and what joint consensus and single-server changes each do about it.
- Choose among ReadIndex, leader leases, and reading through the log for linearizable reads, and say what each assumes.
- Do the quorum arithmetic for cluster sizing and geo-placement, and identify the stale-leader read as the classic Raft deployment bug.

Understandability as a design constraint

The Raft paper (Ongaro & Ousterhout, "In Search of an Understandable Consensus Algorithm," USENIX ATC 2014) opens with an unusual claim for a systems paper: the primary design goal was not performance, not generality, but *understandability*. The authors' argument, developed at length in Ongaro's 2014 Stanford thesis, is that Paxos — for all its elegance as a minimal solution — had by then a two-decade track record of being misimplemented. Single-decree Paxos is subtle; Multi-Paxos, the version anyone actually deploys, was never fully specified in a published algorithm, so every production system (Chubby, ZooKeeper's ZAB, Spanner's Paxos) filled the gaps differently, and the filled-in gaps are where the bugs live. Chapter 5 made the same observation from the other side: the distance between the Synod protocol and a running replicated state machine is long, and Lamport left most of it as an exercise.

Raft's response is two deliberate engineering moves.

Decomposition. Consensus is split into three separable concerns: *leader election* (choose one server to be in charge), *log replication* (the leader accepts commands and copies its log to followers), and *safety* (constraints ensuring that state machines apply the same commands in the same order). Each can be understood, specified, and tested with limited reference to the others. Contrast Multi-Paxos, where leadership, log agreement, and recovery are entangled in one protocol whose phases are reused for different purposes.

State-space reduction. Raft deliberately forbids states that Paxos permits, trading generality for a smaller set of situations an implementer must reason about:

- **Logs never have holes.** A Raft log is a contiguous prefix of entries; a follower accepts entry i only if it already holds entries $1..i-1$ that match the leader's. Multi-Paxos allows slots to be chosen out of order and filled in later; Raft does not.
- **Leaders never overwrite or delete their own entries** (Leader Append-Only). A leader's log only grows during its term.
- **Data flows in one direction only, leader to follower.** Followers never send log entries to the leader, and a newly elected leader never reconstructs its log from followers — the election rule (below) guarantees the leader already has everything that matters. In Multi-Paxos, a new leader must actively query acceptors and adopt the highest-numbered values it finds; Raft arranges the election so that this recovery phase does not exist.

The paper backs the understandability claim with a user study: 43 students taught both algorithms scored significantly higher on Raft questions, and by wide margins reported it easier to implement and explain. Treat that as the paper's claim, measured on students under study conditions — but also note the corroborating fact that within a few years of publication there were dozens of independent Raft implementations in every mainstream language, several of production quality, which had never happened for Multi-Paxos in twenty.

The fundamentals: terms, states, and RPCs

Terms are logical epochs

Raft divides time into **terms**, numbered with consecutive integers. Each term begins with an election; at most one leader can be elected per term (Election Safety, proved below); a term with a split vote simply ends with no leader and a new term begins. A term is exactly the kind of logical epoch Chapter 2 developed: it is a Lamport-style counter, advanced by events rather than by any physical clock, and it totally orders leadership eras without requiring any two servers to agree on what time it is.

Every server persists `currentTerm` and follows two absolute rules. If any RPC — request or response — carries a term *greater* than `currentTerm`, the server updates `currentTerm` and immediately reverts to follower. If an RPC *request* carries a term less than `currentTerm`, the server rejects it. These two rules do most of the work of neutralizing stale leaders: a deposed leader's RPCs carry its old term and are refused, and the refusal carries the new term, which demotes the sender. Hold that thought for the distributed-systems lens — it is a fencing token.

The three states

Every server is in exactly one of three states, and the transition rules are short enough to state completely:

- A **follower** is passive: it responds to RPCs from leaders and candidates and issues none of its own. If an *election timeout* elapses without hearing a valid `AppendEntries` from the current leader or granting a vote, it assumes the leader is dead and becomes a candidate.
- A **candidate** increments `currentTerm`, votes for itself, resets its election timer, and sends `RequestVote` to all peers. Three exits: it receives votes from a majority and becomes leader; it receives an `AppendEntries` from a server claiming to be leader with term $\geq$ its own and steps back to follower; or the election timer expires without either (a split vote) and it starts a new election in a new term.
- A **leader** sends periodic `AppendEntries` heartbeats to suppress elections, accepts client commands, and replicates. It serves until it observes a higher term, at which point it steps down to follower. A

leader never steps down merely because of silence — an important asymmetry we will revisit under asymmetric partitions.

```
stateDiagram-v2
    [*] --> Follower: start or restart
    Follower --> Candidate: election timeout with no
    heartbeat and no vote granted
    Candidate --> Candidate: split vote
    timeout, new term, retry
    Candidate --> Leader: votes from a majority
    Candidate --> Follower: AppendEntries from leader
    with term at least its own
    Leader --> Follower: sees higher term
    in any RPC or response
    Candidate --> Follower: sees higher term
```

The two RPCs

Raft needs only two RPC types (plus `InstallSnapshot`, added later for compaction):

- **`RequestVote(term, candidateId, lastLogIndex, lastLogTerm) → (term, voteGranted)`** — issued by candidates.
- **`AppendEntries(term, leaderId, prevLogIndex, prevLogTerm, entries[], leaderCommit) → (term, success)`** — issued by leaders, both to replicate entries and, with `entries` empty, as the heartbeat. Making the heartbeat the same message as replication is itself a state-space reduction: there is no separate liveness protocol to get wrong, and every heartbeat also carries the consistency check and the commit index.

Three pieces of state must be persisted to stable storage — fsynced — before responding to any RPC: `currentTerm`, `votedFor`, and the log itself. Lose any of these across a crash and the safety proofs collapse: a server that forgets its vote can vote twice in a term and elect two leaders; a server that forgets log entries can break the majority-intersection argument that commitment relies on. This is the load-bearing durability requirement, and it is where Raft's latency floor comes from (see deployment realities below).

Leader election

Randomized timeouts: the livelock fix

Chapter 5 left Paxos with a liveness hole: two proposers can duel indefinitely, each new proposal number invalidating the other's prepare phase. FLP guarantees some such hole must exist; the engineering question is how cheaply it can be papered over in the partially synchronous case. Raft's answer is disarmingly simple: **each server draws its election timeout uniformly at random from an interval** — 150–300 ms in the paper's configuration. After a leader failure, one server usually times out well before the others, wins the election before any peer even becomes a candidate, and its heartbeats reset everyone else's timers. If two do collide and split the vote, they re-draw random timeouts for the retry, so repeated collision is exponentially unlikely.

This is the same desynchronization trick as jitter on retry backoff (Volume 4, Chapter 5, and every well-behaved client library): when identical deterministic timers produce a thundering herd, randomize the timers. The paper measures it: with a 150–300 ms range, leader failure is typically resolved in well under a second, and even adversarial schedules converge. The rule of thumb that falls out is **broadcast RTT $\ll$ election timeout $\ll$ MTBF** — the timeout must be an order of magnitude above network round-trip (so heartbeats reliably suppress elections) and far below the mean time between failures (so leaderless windows are rare).

The voting rules and the up-to-date check

A voter grants its vote if and only if all of the following hold:

1. The candidate's term is at least the voter's `currentTerm`.
2. The voter has not already voted for a *different* candidate in this term (`votedFor` is null or equals this candidate). One vote per term, first come first served, persisted before replying.
3. **The candidate's log is at least as up-to-date as the voter's**, where up-to-date is defined by comparing the last entries: the log whose last entry has the **higher term** is more up-to-date; if the last terms are **equal**, the **longer log** (higher last index) is more up-to-date.

Rules 1 and 2, plus majority intersection, give Election Safety: two leaders in one term would require two disjoint majorities of voters, and majorities intersect.

Rule 3 — the **election restriction** — is the single rule that makes elections *safe with respect to committed data*, and it is worth staring at. A committed entry is, by definition, on a majority of servers. A candidate needs votes from a majority. The two majorities share at least one server, and that server will refuse the candidate unless the candidate's log is at least as up-to-date as its own — which, by the (term, index) comparison, means the candidate's log contains everything that server's log contains from committed prefixes. Therefore no candidate missing a committed entry can assemble a majority. This is how Raft eliminates Multi-Paxos's leader-recovery phase: instead of electing anyone and having the new leader pull missing decisions from the quorum, Raft refuses to elect anyone who would need to pull. The comparison must be (term, index) in that order, not log length alone — a longer log full of uncommitted entries from a stale term loses to a shorter log whose last entry has a higher term, and Figure 8 below shows exactly why length alone would be catastrophic.

```
sequenceDiagram
    participant S3 as S3 candidate
    last entry term 2 index 7
    participant S1 as S1 voter
    last entry term 3 index 5
    participant S2 as S2 voter
    last entry term 3 index 5
    participant S5 as S5 candidate
    last entry term 3 index 5
```

```
Note over S3: was partitioned, log is long
but tops out at term 2
S3->>S1: RequestVote term 4, last 2 and 7
S1-->>S3: reject – my last term 3 beats your 2
despite your longer log
S3->>S2: RequestVote term 4, last 2 and 7
S2-->>S3: reject – same check
Note over S5: times out later, stands in term 5
S5->>S1: RequestVote term 5, last 3 and 5
S1-->>S5: grant – equal term, equal index
S5->>S2: RequestVote term 5, last 3 and 5
S2-->>S5: grant
Note over S5: majority with self-vote – leader of term 5
its log contains every committed entry
```

Split votes and re-election

When two candidates split the vote, neither reaches a majority, both time out, and both retry in a higher term with fresh random timeouts. Nothing needs to be undone: the failed term simply never had a leader, and any votes cast in it are irrelevant to later terms. The cost of a split vote is one extra timeout interval of unavailability for writes — reads may continue under a lease, per below. Note what happens to *reads and writes in flight* during any election: clients talking to the old leader get failures or timeouts and must retry against the new one, which is why the client session machinery later in this chapter is not optional.

Log replication

The happy path

The leader appends the client's command to its own log at the next index, tagged with `currentTerm`, then sends `AppendEntries` to all followers in parallel. Each follower appends the entries (after the consistency check, next section), fsyncs, and acknowledges. When the leader learns that an entry is stored on a **majority of servers, itself included**, and the entry's term is the leader's current term, it marks the entry **committed**: it advances `commitIndex`, applies the entry to its state machine, and responds to the client. Followers learn the commit index piggybacked on subsequent `AppendEntries` (`leaderCommit`) and apply in log order. Commitment of an entry commits the entire prefix before it, always.

Two properties jointly called **Log Matching** hold at all times: if entries in two logs have the same index and term, (a) they store the same command, and (b) the two logs are identical in all preceding entries. Property (a) holds because a leader creates at most one entry per index in its term and never moves it (Leader Append-Only). Property (b) is maintained inductively by the consistency check.

The consistency check, divergence, and repair

Every `AppendEntries` — heartbeats included — carries `prevLogIndex` and `prevLogTerm`: the index and term of the entry immediately preceding the new ones. The follower's handler is the induction step that keeps Log Matching true:

```

func (f *Follower) handleAppendEntries(req AppendEntriesReq)
AppendEntriesResp {
    if req.Term < f.currentTerm {
        return AppendEntriesResp{Term: f.currentTerm, Success: false} /
/ stale leader: fenced
    }
    f.observeTerm(req.Term)      // adopt higher term if any; either
way, revert to follower
    f.resetElectionTimer()       // a valid leader exists

    // Consistency check: do I hold the entry the leader thinks
precedes this batch?
    if req.PrevLogIndex > f.log.LastIndex() ||
        f.log.TermAt(req.PrevLogIndex) != req.PrevLogTerm {
        return AppendEntriesResp{Term: f.currentTerm, Success: false} /
/ leader will back off
    }

    for i, e := range req.Entries {
        idx := req.PrevLogIndex + uint64(i) + 1
        if idx <= f.log.LastIndex() {
            if f.log.TermAt(idx) == e.Term {
                continue // duplicate delivery; idempotent
            }
            f.log.TruncateFrom(idx) // conflict: delete this entry AND
ALL THAT FOLLOW
        }
        f.log.Append(e)
    }
    f.log.Sync() // fsync before acknowledging – non-negotiable

    if req.LeaderCommit > f.commitIndex {
        f.commitIndex = min(req.LeaderCommit, f.log.LastIndex())
    }
    return AppendEntriesResp{Term: f.currentTerm, Success: true}
}

```

If the check fails, the follower's log has diverged — it is missing entries, or it holds uncommitted leftovers from some deposed leader's term. The leader maintains a `nextIndex` per follower, initialized optimistically to its own last index + 1. On each rejection it decrements `nextIndex` and retries with an earlier `prevLogIndex`, probing backward until the check passes — the first point of agreement. From there, the follower truncates its conflicting suffix and adopts the leader's entries. Decrementing by one RPC per entry is painfully slow for a long divergence, so real implementations use the standard backoff optimization: the rejection

carries the term of the conflicting entry and the first index the follower holds for that term, letting the leader skip a whole term per round trip.

```
sequenceDiagram
    participant L as Leader term 5
    log terms 1 1 4 4 5
    participant F as Follower
    log terms 1 1 2 2 2 2

    Note over F: indexes 3 to 6 are leftovers from
    deposited term-2 leader – never committed
    L->>F: AppendEntries prev idx 4 term 4, entry idx 5
    F-->>L: reject – my idx 4 has term 2, not 4
    Note over L: nextIndex for F drops to 3
    conflict-term hint skips term 2 in one step
    L->>F: AppendEntries prev idx 2 term 1, entries idx 3 to 5
    Note over F: prev matches – truncate idx 3 to 6,
    append leader entries, fsync
    F-->>L: success – logs now identical
    Note over L,F: only uncommitted entries were destroyed –
    a committed entry is on a majority and, by the
    election restriction, in the leader log, so repair
    can only ever overwrite non-committed suffixes
```

Truncation looks alarming — followers deleting log entries — until you observe what can be truncated: only entries the current leader does not have, and by Leader Completeness (below) every committed entry is in the current leader's log, so truncated entries were never committed and no client was ever told they succeeded. This is the payoff of the one-direction data flow: repair is purely mechanical overwriting of followers with the leader's log, no negotiation.

The commitment rule, exactly — and Figure 8

Here is the rule, stated precisely because paraphrases of it are where implementations go wrong:

A leader advances `commitIndex` to `N` only if `N` is stored on a majority of servers **and** `log[N].term == currentTerm`. Entries from earlier terms are never committed by counting replicas; they become committed only *indirectly*, when a later entry from the leader's current term commits, because commitment covers the whole prefix (Log Matching guarantees the prefix matches on every server holding the later entry).

Why the restriction? Because "present on a majority" is **not durable** for an old-term entry — a server can be elected leader without it and overwrite it. The paper's Figure 8 exhibits the counterexample, and every Raft implementer should be able to reproduce it from memory. Five servers, S1-S5; all hold a term-1 entry at index 1.

flowchart TD

A["a. S1 leads term 2. Appends index 2 term 2, replicates it to S2 only, then crashes.

S1: t1 t2 – S2: t1 t2 – S3: t1 – S4: t1 – S5: t1"]

B["b. S5 leads term 3 with votes from S3 S4 S5 – its last entry t1 idx1 ties theirs. It appends index 2 term 3 locally, then crashes.

S5: t1 t3 – others unchanged"]

C["c. S1 restarts, leads term 4 with votes from S2 S3 S4. It appends index 3 term 4 locally, and repairs S3 with the old term-2 entry at index 2.

Index 2 term 2 is now on a MAJORITY: S1 S2 S3"]

D["d. UNSAFE BRANCH – suppose S1 commits index 2 now, by majority count alone, then crashes. S5 can win term 5: its last term 3 beats last term 2 on S2 S3 and t1 on S4. S5 replicates index 2 term 3 everywhere – OVERWRITING a supposedly committed entry"]

E["e. SAFE BRANCH – S1 instead replicates index 3 term 4 to a majority before crashing. Index 3 commits by the rule – current term – and index 2 commits with it, as prefix. Now S5 cannot win: last term 3 idx 2 loses to last term 4 idx 3 held by a majority"]

A --> B --> C

C --> D

C --> E

Walk branch (d) slowly. At step (c) the term-2 entry at index 2 sits on S1, S2, S3 — a majority. If majority presence meant commitment, S1 would apply it and acknowledge the client. But S5's log ends in term 3, which is *newer than term 2*, so the up-to-date check lets S5 collect votes from S2, S3, and S4 (their last terms are 2, 2, 1) and become leader of term 5 — perfectly legally — whereupon log repair overwrites index 2 on every server with S5's term-3 entry. An acknowledged write has vanished: State Machine Safety is gone. The problem is that the entry's *term* (2) no longer certifies anything about election outcomes once higher terms exist; replication count alone cannot distinguish (c) from a world where the entry is genuinely safe.

Branch (e) shows what actually makes it safe: a **current-term** entry on a majority. Once index 3 (term 4) is on a majority, any future candidate

must beat term 4 at the up-to-date check on at least one member of that majority, and any log that does so necessarily contains index 3 — and, by Log Matching, index 2 beneath it. The current-term requirement is exactly what re-couples "replicated on a majority" to "wins all future elections."

One practical corollary: a freshly elected leader may hold committed entries from prior terms that it cannot yet *prove* committed (it cannot count replicas for them), so it cannot advance `commitIndex` — or safely serve `ReadIndex` reads — until it commits something in its own term. Real implementations therefore have every new leader immediately append a **no-op entry** in its new term; committing it commits the entire inherited prefix and establishes a known commit frontier.

The five safety properties

The paper's Figure 3 states the guarantees; all have now appeared:

Property	Statement
Election Safety	At most one leader can be elected in a given term.
Leader Append-Only	A leader never overwrites or deletes entries in its own log; it only appends.
Log Matching	If two logs contain an entry with the same index and term, the logs are identical through that entry.
Leader Completeness	If an entry is committed in term T , it is present in the logs of the leaders of all terms $> T$.
State Machine Safety	If a server has applied the entry at a given index, no server ever applies a different entry at that index.

The keystone is **Leader Completeness**, and the argument sketch is the one made twice above, run as an induction over terms: an entry committed in term T is on a majority with term $\leq T$ certificates — either directly (current-term commit, so on a majority with term T as their last relevant term) or via a current-term successor entry. Any leader of term $U > T$ was elected by a majority that intersects the storing majority; the intersection voter's up-to-date check forces the winner's log to dominate its own by (term, index); a case analysis on whether the last terms were equal or the winner's was higher shows the winner's log must contain the committed entry (the higher-term case leans on the induction hypothesis:

whoever created that higher-term entry was a leader whose log, by induction, contained the entry). Ongaro's thesis gives the full proof; the protocol was subsequently machine-verified in TLA+ and, in the Verdi project, in Coq. State Machine Safety follows: apply order is log order below `commitIndex`, and Leader Completeness plus Log Matching pin the committed prefix on every server forever.

Practical Raft

Everything so far replicates a log among a fixed set of servers with an ever-growing log. Real systems need to change the membership, bound the log, and talk to clients correctly.

Membership changes

You cannot switch atomically from configuration `C_old` to `C_new` by fiat, because servers adopt the new configuration at different times, and during the transition **a majority of `C_old` and a majority of `C_new` can be disjoint**. Concretely: growing from 3 servers to 5, two old servers can form a `C_old` majority (2 of 3) and elect one leader while three new servers form a `C_new` majority (3 of 5) and elect another — two leaders in the same term, using entirely legal votes. Any membership scheme must make such disjoint quorums impossible at every instant.

The paper's original answer is **joint consensus**: the leader first replicates a configuration entry `C_old,new` under which *every* decision — elections and commitment alike — requires separate majorities from both `C_old` and `C_new`. Once `C_old,new` commits, the leader replicates `C_new`, and once that commits the old servers can be shut down. At no point can two disjoint quorums exist, because any quorum during the transition includes a `C_old` majority. A distinctive wrinkle: configuration entries take effect on each server **as soon as they are appended**, not when committed — a server always uses the latest configuration in its log.

Ongaro's thesis proposes the simpler mechanism most implementations actually use: **single-server changes**. Add or remove one server at a time, and any majority of the old configuration overlaps any majority of the new one arithmetically, so no joint phase is needed. Honesty requires the footnote: in 2015 Ongaro himself reported a safety bug in the single-server algorithm as described in the thesis — with concurrent changes across leader turnover, a specific interleaving can commit a configuration

entry that a later leader removes, reintroducing disjoint quorums. The fix he published is that a leader must not append a configuration change until it has committed an entry in its current term (the no-op above suffices) and must not start one while a previous change is uncommitted. etcd's raft library implements single-server changes with these guards (and grew optional joint consensus later, for atomic multi-server swaps). The moral is squarely on-theme: even the understandable consensus algorithm has corners subtle enough to catch its own author, and they are precisely the corners a library has already debugged for you.

Two operational details. New servers should join as **non-voting learners** first, catching up on the log without counting toward (or endangering) quorum, and be promoted only when nearly current — otherwise adding a far-behind server can stall commitment. And a server *removed* from the configuration no longer receives heartbeats, times out, and starts elections that disrupt the cluster with ever-higher terms; PreVote (below) and leadership-transfer discipline mitigate this.

Log compaction and snapshots

The log cannot grow forever. Raft's answer is the same one Volume 5, Chapter 7 gave for the WAL: **checkpoint and truncate**. Each server independently snapshots its state machine at some applied index, records `lastIncludedIndex` and `lastIncludedTerm` (needed so the consistency check still works at the truncation boundary), persists the snapshot, and discards the log prefix through that index. Snapshotting is local and requires no coordination, because everything snapshotted is committed and immutable.

The wrinkle is a follower so far behind — or freshly added — that the entries it needs have been compacted away. For this the leader sends **InstallSnapshot**: the snapshot itself, chunked, after which the follower discards its conflicting log and resumes normal `AppendEntries` from `lastIncludedIndex + 1`. Operationally, snapshot transfer is the expensive path — for a large state machine it is a bulk data copy that can saturate links and stall the follower — which is why implementations tune log retention to make it rare, and why CockroachDB and TiKV engineering blogs spend real ink on snapshot rate-limiting and scheduling.

Client sessions and exactly-once at the state machine

Raft gives you a linearizable *log*; it does not by itself give clients exactly-once semantics. A client whose leader crashed after committing its command but before replying will retry — against the new leader — and without protection the command executes twice. The at-least-once retry plus deduplication pattern of Chapter 9 applies, implemented *inside the state machine*: each client opens a **session** (itself a committed log entry), tags every command with a monotonic **sequence number**, and the state machine keeps, per session, the highest sequence applied and the cached response. A committed duplicate is not re-executed; the cached response is returned. Because the session table is part of the state machine, it is replicated and snapshotted with everything else, and deduplication survives leader changes. This is exactly-once *effect at the state machine*, not exactly-once delivery — the network still delivers at-least-once; the state machine makes re-delivery harmless. Sessions must eventually expire (unbounded response caches are a leak), and expiry must be a deterministic, log-driven decision — expire by log-time agreement, never by each replica's local clock, or replicas diverge.

Linearizable reads

The tempting bug: the leader serves reads from its local state machine with no protocol at all. Wrong, twice over. First, a deposed leader that has not yet observed the new term will happily serve values the new leader has already overwritten — a stale read that violates linearizability. Second, even a genuine leader fresh from election may not know the true commit frontier until its no-op commits. The three correct options, in decreasing cost:

Read through the log. Append the read as a log entry, commit it, apply it in order. Trivially linearizable, costs a full replication round and log space per read. Rarely the right choice alone, but it is the fallback semantics everything else must match.

ReadIndex. The leader records its current `commitIndex` as the read's index, then confirms it is *still* leader by exchanging heartbeats with a majority, then waits for its state machine to apply through the read index, then reads locally:

```
// Leader-side ReadIndex: linearizable read without a log write.
func (l *Leader) LinearizableRead(ctx context.Context, query Query) (Result, error) {
    if !l.hasCommittedEntryInCurrentTerm() {
        return nil,
        ErrLeaderNotReady // wait for the new-term no-op to commit first
    }
    readIndex := l.commitIndex // capture BEFORE the leadership check

    // One round of heartbeats; a majority of acks proves no higher
    term existed
    // when they answered, hence no other leader committed past
    readIndex.
    if ok := l.broadcastHeartbeatsAwaitMajority(ctx); !ok {
        return nil, ErrLeadershipLost // possibly deposed: retry via
        current leader
    }

    l.waitUntilApplied(readIndex) // state machine catches up to
    the frontier
    return l.stateMachine.Read(query), nil // local read, no log entry
    written
}
```

Cost: one round trip to a majority per read (batchable across many concurrent reads — one heartbeat round can validate thousands of queued ReadIndex requests), no disk write, no log growth. Assumptions: none beyond Raft's own. This is etcd's default for linearizable reads.

Leader leases. Skip even the heartbeat round: after a successful quorum round at time t , the leader assumes leadership is safe until $t + \text{electionTimeout/clockDriftBound}$, and serves reads locally within the lease window. Now correctness depends on **bounded clock rates** — precisely the assumption Chapter 2 taught you to distrust. A VM migration, a GC pause between the clock read and the reply, or a clock running fast can extend a stale leader's confidence past a completed election elsewhere, and stale reads follow. Systems that take this bet (CockroachDB's leases, TiKV's lease reads) engineer around it with conservative drift bounds and by tying leases to Raft events; systems that refuse it (etcd's default) pay the ReadIndex round trip. This is the same lease-caveat family as Volume 4, Chapter 2's fencing discussion: a lease is a bet on clocks, and you either bound the clocks or verify at the resource.

Follower reads are stale reads unless extra machinery is added — a follower may lag arbitrarily. Honest designs either forward a ReadIndex

request through the leader and wait to apply that far (linearizable, still one leader round trip, but the *read work* moves to the follower), or embrace boundedly stale reads at an explicitly chosen timestamp, which is what CockroachDB's closed-timestamp follower reads do (Volume 5, Chapter 12). Staleness chosen on purpose and labeled is a feature; staleness by accident is the classic bug.

PreVote and leader transfer

Two refinements from Section 9.6 of the thesis and production practice, briefly:

PreVote. A partitioned server times out repeatedly, incrementing `currentTerm` each time. When the partition heals it carries an inflated term that — by the higher-term rule — instantly deposes a perfectly healthy leader, causing a needless election (and in pathological flapping, repeated ones). PreVote adds a preliminary round: a would-be candidate asks peers whether they *would* vote for it — same up-to-date and timeout checks — **without anyone incrementing terms**, and only proceeds to a real election on a majority of yeses. A node partitioned from a live leader never gets them (peers with a fresh leader refuse), so its term never inflates. etcd, TiKV, and hashicorp/raft all ship PreVote (etcd off-by-default historically, on in current practice).

Leader transfer. For planned maintenance or load balancing, the leader stops accepting new proposals, brings the target follower fully up to date, and sends it a TimeoutNow instruction; the target starts an election immediately — bypassing its randomized timeout — and wins, since its log is current. Downtime shrinks from an election-timeout detection window to about one round trip.

Raft in the wild

etcd is the lineage's reference point: its Go Raft package (now the standalone `etcd-io/raft` library) is the most battle-tested implementation in existence and deliberately ships as a *library* — the consensus core is a pure state machine; storage, transport, and threading are the embedder's problem. Kubernetes stores every object in etcd, which makes this particular Raft implementation load-bearing for a substantial fraction of the industry; Chapter 8 examines etcd as a coordination service in its own right.

CockroachDB and TiKV run Raft at a different scale: not one group, but **one Raft group per range/region** of the keyspace — tens of thousands of groups per node (Volume 5, Chapter 12). This multi-raft regime has failure modes single-group deployments never see. Naively, every group heartbeats independently: 10,000 ranges × heartbeats at 10 Hz is a heartbeat storm that melts the network with liveness traffic. Both systems coalesce heartbeats per node-pair — one physical message carries liveness for every group sharing that pair — and both suppress idle groups entirely: CockroachDB *quiesces* ranges with no traffic (no heartbeats at all; node-level failure detection wakes them), and TiKV *hibernates* regions similarly. TiKV's `raft-rs` is a Rust port of etcd's library, so the two ecosystems share a lineage and, occasionally, bugs and fixes.

Consul (and Nomad and Vault) run on `hashicorp/raft`, an independent Go implementation, for service-catalog and KV state. **Kafka's KRaft** (KIP-500 and its successors) replaced the ZooKeeper dependency with a Raft-based metadata quorum: the controller quorum replicates the cluster metadata log using a Raft variant that is *pull-based* — followers fetch from the leader, reusing Kafka's replica-fetch machinery — rather than the paper's push-based `AppendEntries`; same safety argument, inverted transport (Volume 10, Chapter 3). The breadth of this list is the understandability thesis's real evidence: independent, interoperating-in-spirit, production-grade implementations in Go, Rust, Java, and C++ within a decade.

Raft versus Multi-Paxos, honestly

Strip the presentation away and the two are siblings. Same problem; same $2f+1$ majority quorums and the same intersection argument; same partial-synchrony liveness (both need a stable leader/distinguished proposer to make progress, both are safe without one); comparable steady-state message complexity (one round from leader to majority per command). Raft's terms are Paxos's proposal numbers; Raft's election is Paxos's phase 1 amortized over a term; Raft's `AppendEntries` is phase 2 over a contiguous batch of slots.

The real differences are the constraints Raft adds:

- **Strong leader, always.** Multi-Paxos permits any replica to propose into any slot (at a cost); Raft forbids it. This simplifies reasoning and

repair but makes the leader a throughput and latency bottleneck by construction — every byte flows through it, and a leader in the wrong region taxes every write (see below).

- **Log contiguity.** Multi-Paxos acceptors can accept slot 100 before slot 99 exists; a Raft follower cannot. Out-of-order acceptance lets Multi-Paxos variants pipeline aggressively across slots and recover holes lazily; Raft trades that flexibility for the trivial repair story and the hole-free invariant. Protocols in the Egalitarian Paxos family exploit exactly the freedom Raft renounces (leaderless, per-command quorums) and pay for it in complexity — the trade is real, and Raft sits deliberately at the simple end.
- **Election-time recovery versus post-election recovery.** Raft's up-to-date check moves log recovery *into* the election; Multi-Paxos elects, then recovers. Net messages are similar; the difference is where the subtlety lives, and Raft chose to put it in one comparison of two integers.

The understandability payoff is not that Raft is *smarter* — as an algorithm it is arguably less general — but that its state space is small enough that implementations tend toward correctness, and that claim has the empirical support Multi-Paxos never accumulated: the paper's user study (taken as the paper's claim) plus the observed proliferation of independent implementations that pass Jepsen-grade testing.

Deployment realities

3 versus 5, and why 4 is worse than both. A cluster of $2f+1$ voters tolerates f failures: 3 nodes $\rightarrow$ quorum 2 $\rightarrow f=1$; 5 nodes $\rightarrow$ quorum 3 $\rightarrow f=2$. Even counts buy nothing: 4 nodes $\rightarrow$ quorum $\lfloor 4/2 \rfloor + 1 = 3 \rightarrow$ still $f=1$, with a fourth machine's cost, a larger replication fan-out, and one more node that can fail. Run odd voter counts, full stop. Three is the default; five when you need to survive a node failure *during* maintenance on another, at the price of a bigger quorum on every commit. Beyond five, quorum latency and heartbeat overhead climb for rarely-needed fault tolerance — scale out with more Raft *groups* (multi-raft), not more members per group. **Non-voting learners** serve catch-up and read scaling without quorum cost; **witness** members — voters that store log metadata but no state-machine data — appear in some systems as a cheap tiebreaker to place in a third site.

Geo-placement. With one replica per region across three regions, every commit needs the leader plus one remote ack: commit latency is the RTT to the *nearest* remote region — equivalently, the second-fastest of the three region RTTs from the leader. Sort your candidate regions by RTT before choosing them, and put the leader (or leaseholder) in the region nearest the write traffic; a mis-placed leader adds a full cross-region RTT to every write. The tension is blast radius versus latency (Volume 7, Chapter 10): three replicas in one region commit in microseconds and die together; three regions survive a regional failure and pay tens of milliseconds per write, forever, on every write. There is no protocol fix — it is physics plus quorum arithmetic — only placement choices and, in multi-raft systems, per-range placement so each range pays only for the durability it needs.

The fsync floor. Every commit requires the entry durable on a quorum *before* acknowledgment — leader and followers fsync the Raft log ahead of their acks. Raft's write latency floor is therefore $\max(\text{quorum network RTT, slowest-quorum-member fsync})$, and on same-AZ clusters the fsync usually dominates (Volume 5, Chapter 7's arithmetic, now paid on a quorum). Group commit applies exactly as it did for the WAL: batch many entries per fsync. Running the Raft log on storage that lies about flushes — or disabling fsync for benchmarks that quietly become production — converts a crash from a liveness event into silent log divergence; Jepsen's filesystem- and fsync-fault testing has caught real systems here.

Asymmetric partitions and the stale leader. A leader cut off from a quorum does not step down by protocol — it keeps retrying `AppendEntries` into the void, cannot commit, and, if it serves reads without `ReadIndex` or lease discipline, serves *stale* reads while a new leader on the majority side accepts writes. This is the classic Raft deployment bug, found by Jepsen in multiple systems' default read paths. The standard mitigation is **CheckQuorum**: a leader that fails to reach a majority within an election timeout steps down voluntarily. CheckQuorum plus PreVote is the production pairing — one demotes stale leaders, the other stops rejoining nodes from deposing healthy ones — and even then, reads are only linearizable via `ReadIndex` or a correctly-bounded lease, never by leadership optimism alone.

The distributed-systems lens

Reflexively, this time — Raft is itself the infrastructure the rest of this volume leans on, so the lens points at the connections you should now see in both directions.

Raft's log is the WAL, generalized. Volume 5, Chapter 7 built durability from an append-only log, recovery from replay, and bounded logs from checkpoint-plus-truncate. Raft is the same machinery with the log *replicated*: recovery-by-replay is how every follower and every restarted server reconstructs state; snapshots are checkpoints; `InstallSnapshot` is shipping a checkpoint to a replica that fell off the log's tail. A single-node database recovers its own past; a Raft group recovers any member from the quorum's shared past. Once you see this, CockroachDB and TiDB (Volume 5, Chapter 12) stop looking exotic: a distributed database is a WAL that achieved quorum.

The leader is a lock with a lease, and terms are fencing tokens. Volume 4, Chapter 2 ended with the fencing-token argument: a leaseholder that stalls can act after its lease expired, so the *resource* must reject stale tokens. Map it across: leadership is the lock; the election timeout is the lease; a partitioned or paused leader is exactly Kleppmann's stalled lock-holder. Raft's fencing token is the **term**, and the resource that enforces it is the quorum itself — every `AppendEntries` carries the term, and followers with a higher `currentTerm` reject the stale leader's writes mechanically. This is why Raft is safe under arbitrary pauses while timeout-based locking is not: the fence is checked on every single operation at the point of acceptance, not assumed from a clock. When Chapter 8 builds locks and leader election *on top of* etcd, the same discipline recurses one level up: etcd hands out lease revisions as fencing tokens, and your storage must check them, because your application's stale leader is not fenced by Raft's terms — only etcd's own state is.

Consume Raft; do not reimplement it. The engineering moral this volume keeps repeating applies with maximum force here. The protocol is machine-checked; your weekend implementation of it is not, and the bug will be in single-server membership changes, or `ReadIndex` during leader turnover, or `fsync` ordering — the corners this chapter flagged. Use etcd or Consul when you need a coordination service; embed `etcd-io/raft`, `hashicorp/raft`, or `raft-rs` when you need the log in-process. And then test *your* assumptions anyway: Chapter 12's central finding is that

Jepsen's catalogue of consensus failures — stale default reads, membership-change data loss, retry-amplified double-applies — is overwhelmingly a catalogue of *implementation and integration* bugs in systems built on Raft, not counterexamples to Raft. The protocol being correct is necessary. It has never once been sufficient.

Key takeaways

- Raft is Multi-Paxos's problem solved under three deliberate constraints — no log holes, leaders never overwrite their own entries, data flows only leader→follower — chosen to shrink the state space an implementer must reason about. Understandability was the explicit design goal, and the proliferation of correct independent implementations is its real evidence.
- **Terms are logical epochs**: at most one leader per term, higher term always wins, stale terms are rejected on every RPC. Terms are fencing tokens enforced by the quorum on every operation — this, not timeouts, is why Raft tolerates arbitrary pauses.
- Elections are made live by **randomized timeouts** (the jitter fix for dueling candidates) and made safe by one rule: a voter refuses any candidate whose last entry loses the **(term, index)** comparison against its own. Majority intersection then guarantees every elected leader already holds every committed entry — no recovery phase.
- The **commitment rule, exactly**: count replicas only for current-term entries; prior-term entries commit only as the prefix of a current-term commit. Figure 8 is the counterexample to every simpler rule — an old-term entry on a majority can still be overwritten. New leaders commit a no-op to establish the frontier.
- The **AppendEntries consistency check** (prevLogIndex/prevLogTerm) inductively maintains Log Matching; divergent followers are repaired by nextIndex backoff and suffix truncation, which only ever destroys uncommitted entries.
- Naive membership switches allow **disjoint majorities**; use joint consensus or guarded single-server changes (whose original description had a real bug — use a library). Snapshots are checkpoint-plus-truncate; InstallSnapshot ships the checkpoint.
- Linearizable reads require discipline: **ReadIndex** (quorum heartbeat round, no clock assumptions), **leader leases** (cheaper, bets on

bounded clock drift), or reading through the log. Leader-local reads with none of these are the classic stale-read bug. Deploy **PreVote + CheckQuorum** together.

- Sizing is quorum arithmetic: 3 nodes $\rightarrow f=1$, 5 $\rightarrow f=2$, even counts add cost without tolerance. Geo commit latency is the RTT to the nearest remote quorum member; the fsync on the Raft log is the latency floor underneath everything.
- Raft's log is Volume 5's WAL replicated; the leader is Volume 4's lock-with-a-lease done right. Consume Raft via etcd/Consul or a maintained library, and point Jepsen-style testing at your integration — that is where the bugs actually are.

Further reading

- Ongaro, D. and Ousterhout, J., "In Search of an Understandable Consensus Algorithm," *USENIX ATC*, 2014 — the Raft paper; the extended version contains the full Figure 8 discussion and the user study. <https://raft.github.io/raft.pdf>
- Ongaro, D., *Consensus: Bridging Theory and Practice*, PhD thesis, Stanford, 2014 — membership changes, log compaction, client sessions, ReadIndex/leases, PreVote, and the safety proof; the companion TLA+ specification is on the Raft site.
- Ongaro, D., "bug in single-server membership changes," raft-dev mailing list, July 2015 — the honest footnote to the thesis's simpler membership algorithm, with the fix.
- The Raft site — interactive visualization, the TLA+ spec, and the long list of implementations. <https://raft.github.io/>
- etcd documentation and the `etcd-io/raft` library — the reference implementation lineage, including ReadIndex, learners, and joint-consensus support. <https://etcd.io/docs/> and <https://github.com/etcd-io/raft>
- TiKV blog, "The Design and Implementation of Multi-raft" and the `raft-rs` introduction — heartbeat coalescing, hibernating regions, and Raft-per-region at scale. <https://tikv.org/blog/>
- Cockroach Labs blog, "Scaling Raft" and the CockroachDB architecture docs on Raft, leases, and follower reads — quiescing ranges, leaseholders, closed timestamps. <https://www.cockroachlabs.com/blog/>

- Jepsen analyses — etcd (2014 and the 3.4.3 analysis, 2020), Consul (2015), RethinkDB (2016, a Raft membership-change bug in the wild), and others: implementation and integration failures around correct cores. <https://jepsen.io/analyses>
- KIP-500, "Replace ZooKeeper with a Self-Managed Metadata Quorum," and KIP-595, "A Raft Protocol for the Metadata Quorum" — KRaft's pull-based Raft variant. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-500>
- Howard, H. and Mortier, R., "Paxos vs Raft: Have we reached consensus on distributed consensus?," *PaPoC*, 2020 — a careful side-by-side that formalizes how close the two algorithms really are.
- Volume 5, Chapter 7 — Write-Ahead Logging — the single-node ancestor of the Raft log.
- Volume 4, Chapter 2 — Threads, Mutual Exclusion, and Locks — leases and fencing tokens, whose distributed resolution is this chapter.
- Chapter 8 — Coordination Services — building locks, leases, and discovery on top of etcd.

Chapter 7 — Quorum Systems and Dynamo-Style Replication

What this chapter covers. Chapters 5 and 6 built the consensus machine: a leader, a log, and a majority that agrees on a total order of operations. This chapter follows the other lineage of replicated storage — the one that deliberately refuses to build that machine. A quorum system keeps N copies of each value, writes to W of them, reads from R of them, and relies on nothing more than arithmetic: if $R + W > N$, every read set overlaps every write set, so a read must touch at least one replica that saw the latest write. No leader to elect, no failover pause, no log to agree on. The price is that overlap is a much weaker property than agreement, and most of this chapter is about being precise on exactly how much weaker.

The archetype is Amazon's Dynamo, described in a 2007 SOSP paper that is arguably the most influential storage-systems paper of its decade — not because its techniques were individually novel (most were not, and the

paper says so), but because it assembled them into a coherent design with an explicit priority: **an always-writable shopping cart matters more than a consistent one**. We walk the design mechanically — the consistent-hashing ring, sloppy quorums, hinted handoff, read repair, Merkle-tree anti-entropy, version vectors, gossip membership — and then follow its descendants (Cassandra, Riak, Voldemort, and DynamoDB-the-service, which is not what its name suggests) to see which parts survived contact with operations and which were quietly traded away. Throughout, we keep consensus in view: what a quorum system cannot do that Paxos can, why Cassandra eventually bolted Paxos on anyway, and why Raft — itself a quorum system — gets so much more from its quorums.

Learning goals — after this chapter you should be able to:

- State the $R + W > N$ intersection guarantee and prove it with the pigeonhole argument, and state what $W + W > N$ adds and why.
- Tune N , R , and W per operation, predict the latency and availability consequences, and read a Cassandra consistency level as a point on that dial.
- Explain precisely why $R + W > N$ does not give linearizability: partial writes, non-monotonic reads, and clock-ordered conflict resolution — and why sloppy quorums void even the overlap.
- Walk the Dynamo write and read paths end to end, including hinted handoff, read repair, and Merkle-tree anti-entropy, explaining the mechanism of each.
- Execute a version-vector merge by hand, explain when siblings arise, and argue why last-write- wins is a data-loss policy wearing a convenience costume.
- Distinguish Dynamo-the-paper from DynamoDB-the-service without hand-waving.
- Sketch ABD's read-then-write-back and say why the write-back phase is what buys linearizable reads.
- Compare leaderless and leader-based replication honestly, and say where each belongs in 2026.
- State precisely why a quorum read/write system does not solve consensus, and what Raft adds to its quorums that Dynamo does not.

Quorums from first principles

Keep N copies of every value — N is the **replication factor**, typically 3. A write is sent to all N replicas but the client is acknowledged after W of them confirm. A read queries replicas and returns after R of them respond, taking the newest version among the responses. N , R , and W are the whole configuration language of a quorum system, and one inequality does the work.

The intersection argument

If $R + W > N$, then every read quorum intersects every write quorum: any set of R replicas shares at least one member with any set of W replicas.

The proof is pigeonhole and fits in two sentences. Suppose some read set of size R and some write set of size W were disjoint. Then the system would contain at least $R + W$ distinct replicas; but it contains only N , and $R + W > N$ — contradiction. So a read that gathers R responses is guaranteed to have heard from at least one replica that acknowledged the most recent successful write, and if versions are comparable (a big *if* we return to shortly), the read returns that write's value or something newer.

```
flowchart LR
    subgraph RS["N = 5 replicas of one key"]
        A["Node A"]
        B["Node B"]
        C["Node C"]
        D["Node D"]
        E["Node E"]
    end
    WQ["Write quorum  
W = 3"] --> A
    WQ --> B
    WQ --> C
    RQ["Read quorum  
R = 3"] --> C
    RQ --> D
    RQ --> E
    C ==>| "intersection is non-empty:  
R + W = 6 exceeds N = 5  
C saw the write and answers the read" | OUT["Read observes latest  
write"]
```

Notice what the argument does and does not use. It uses only counting — no synchrony assumption, no leader, no agreement protocol. It does not care *which* W replicas acknowledged the write or *which* R answered the read; any-W and any-R overlap. That generality is the source of the availability win: as long as any W replicas are reachable you can write, and any R reachable you can read, with no failover, no election, no view change. Chapter 5 used exactly the same counting — two majorities of the same set must intersect — as the foundation of Paxos safety. The inequality is shared. Everything else is not, and the gap between "the read touched a replica that has the write" and "the system behaves like one copy" is the subject of the next section.

Preventing write splits: $W + W > N$

A second inequality is usually wanted: $W > N/2$, equivalently $W + W > N$. This forces any two write quorums to intersect, so two concurrent writes cannot both complete against disjoint replica sets, each believing itself sole and current. With $W \leq N/2$, a partition can split the replicas into two halves that each accept a full write quorum for the same key — two "successful" writes, neither aware of the other, and nothing in the arithmetic to say which wins. Note carefully what $W + W > N$ buys and what it does not: the two writes' replica sets intersect, so the overlapping replica *sees* both — but seeing both is not ordering them. Whether the system can then say which write is newer depends entirely on the versioning scheme, which is where quorum systems diverge from consensus and where Dynamo's version vectors and Cassandra's timestamps enter the story.

The tunable-consistency dial

Because R and W are just parameters, they can be chosen per table, per operation, even per individual request — this is **tunable consistency**, and Cassandra's consistency levels (Volume 5, Chapter 11 covers the data-model surface) are its most widely deployed vocabulary. The common points on the dial, for $N = 3$:

N	W	R	R+W>N	Write survives	Read survives	Character
3	1	1	no	2 nodes down	2 nodes down	Fastest, most available; reads routinely stale; lost-write window on failure
3	2	2	yes	1 node down	1 node down	The canonical QUORUM/QUORUM; balanced
3	3	1	yes	0 nodes down	2 nodes down	Read-optimized; a single dead node blocks all writes
3	1	3	yes	2 nodes down	0 nodes down	Write-optimized; a single dead node blocks all reads
5	3	3	yes	2 nodes down	2 nodes down	Larger cluster, same balance, more failure headroom
5	2	2	no	3 nodes down	3 nodes down	Deliberately weak: high availability, probabilistic freshness

Read the table as a budget. Latency: a request completes at the speed of the W-th (or R-th) fastest replica, so raising W or R hands your tail latency to slower nodes — with W = ALL, the slowest replica in the set prices every write. Availability: an operation needs its quorum reachable, so raising W or R shrinks the set of failures you tolerate; the "survives" columns are just $N - W$ and $N - R$. Freshness: only the rows where the inequality holds give the intersection guarantee. There is no free row. This dial is Chapter 4's PACELC made concrete, and we return to that in the lens section.

Cassandra exposes the dial per operation. In `cqlsh`:

```
CONSISTENCY QUORUM;
INSERT INTO orders (order_id, cart) VALUES (8843, {'sku-1129', 'sku-0077'});

CONSISTENCY ONE;      -- subsequent reads: cheap, possibly stale
SELECT cart FROM orders WHERE order_id = 8843;
```

and per statement in a driver:

```

SimpleStatement write = SimpleStatement.builder(
    "UPDATE orders SET cart = cart + {'sku-0912'} WHERE order_id
= ?")
    .addPositionalValue(8843L)
    .setConsistencyLevel(ConsistencyLevel.QUORUM)        // W: majority
of replicas
    .build();

SimpleStatement audit = SimpleStatement.builder(
    "SELECT cart FROM orders WHERE order_id = ?")
    .addPositionalValue(8843L)
    .setConsistencyLevel(ConsistencyLevel.LOCAL_ONE)    // R: any local
replica, fast
    .build();

```

Per-operation tuning is genuinely useful — write user actions at QUORUM, serve a recommendation sidebar at ONE — but it means the consistency of a *read* depends on the level used by the *write* it observes, a coupling that spans teams and codebases. Hold that thought for the operational-realities section.

What quorum overlap does not give you

It is tempting to read $R + W > N$ as "consistent" — the Cassandra level is even named QUORUM, and folklore says quorum reads see quorum writes, so all is well. The precise statement is much narrower: **overlap guarantees the read set contains the latest acknowledged value; it does not guarantee the system is linearizable, and in practice such systems are not.** Overlap is necessary for a linearizable register built this way, and it is not sufficient. Three mechanisms break it.

Partial writes have no rollback

A write that reaches fewer than W replicas — the coordinator crashed mid-flight, or some replicas timed out — reports failure to the client, but the replicas it did reach keep the value. There is no rollback, no two-phase abort, nothing to undo it; the "failed" write is simply *present on some replicas and absent on others*, and later reads observe it or not depending on which R replicas answer. A failed write that becomes visible, then invisible, then visible again as read sets vary is already outside linearizable behavior. Consensus systems do not have this problem because an unchosen value is never applied; quorum systems have it structurally.

Non-monotonic reads during replication lag

Even a fully successful quorum write is applied to its W replicas at different moments. During that window, consider two readers. Reader 1's quorum happens to include a replica that already has the new value; the read returns it. Moments later — in real time, strictly after — Reader 2's quorum happens to include only replicas that have not yet applied it; the read returns the old value. The new value appeared and then disappeared: a violation of linearizability's real-time ordering even though every operation obeyed $R + W > N$. This anomaly, and the partial-write one above, are fixable — but only by adding protocol: the reader must **write back** the newest value it observed to a quorum before returning it, so that any later read's quorum intersects the write-back. That read-side write phase is exactly ABD's contribution, treated below, and it is what production Dynamo-style stores mostly do not do in the strict form (their read repair is asynchronous and best-effort). This is also the consistent general finding of the Jepsen analyses of Dynamo-style stores: even so-called strict quorums — $R + W > N$ faithfully enforced — permit lost updates and stale reads unless the read and write protocols are carefully constructed around them, and the defaults are not.

Conflict resolution by clocks

Overlap gives the read a *set* of versions; something must decide which is newest. If that something is a wall-clock timestamp — as in Cassandra — then ordering between concurrent writes is decided by clocks that Chapter 2 taught you not to trust. Two writes to the same key through different coordinators get timestamps from different machines; with skew, the write that happened *later* can carry the *earlier* timestamp and silently lose, at any consistency level including ALL, because the anomaly is in the ordering rule rather than the replica counts. We take this up properly with last-write-wins below.

Finally, everything in this section assumed the quorums are **strict**: drawn from the same fixed set of N replicas. Dynamo's sloppy quorums abandon even that, and with it the intersection guarantee entirely — which is the right place to begin the Dynamo walk.

Dynamo, walked mechanically

The 2007 paper (DeCandia et al., SOSP) opens with the requirement that explains every design decision: Amazon's shopping cart must accept writes even during network partitions and server failures, because a rejected add-to-cart is directly lost revenue, and Amazon's measurements said availability, not consistency, was where the money was. Dynamo is therefore an **AP** design in Chapter 4's terms: always writable, eventually consistent, with conflict resolution pushed to read time and, ultimately, to the application. It is a flat key-value store — `get(key)` and `put(key, context, value)` — with no schema, no secondary indexes, and no cross-key operations. Each of its mechanisms answers one question.

Placement: the ring, virtual nodes, preference lists

Which N nodes hold a given key? Dynamo hashes each key onto a fixed circular space — a **consistent-hashing ring** — and assigns each storage node many positions on that ring, called **virtual nodes**. A key is stored on the first N *distinct physical* nodes encountered walking clockwise from the key's hash position; that ordered list, extended past N with further nodes to be used as fallbacks, is the key's **preference list**. Virtual nodes exist for load smoothing: a physical node's departure scatters its many ring segments across many successors instead of dumping them all on one neighbor, and heterogeneous hardware gets proportionally many virtual nodes. Placement is also made failure-domain-aware by constructing preference lists to span distinct racks or datacenters, so one domain failure cannot take out all N replicas. That is all this chapter needs about the ring; the algorithmic depth — hash-space partitioning strategies, balance bounds, jump/rendezvous/ring variants — belongs to Volume 14, Chapter 5, and stays there.

Sloppy quorums and hinted handoff

What happens to writes when preference-list nodes are down? A strict quorum system answers: if fewer than W of the N home replicas are reachable, the write fails. Dynamo refuses that answer — availability was the whole point — and substitutes a **sloppy quorum**: the coordinator walks further along the ring and writes to the first N *healthy* nodes it finds, counting their acknowledgments toward W even though some of them are not home replicas for the key. A substitute node stores the value with a **hint** — metadata naming the home replica it is standing in for — in

a local table, and periodically attempts delivery: when the home node recovers, the substitute replays the hinted writes to it and then deletes them. This is **hinted handoff**.

```
sequenceDiagram
    participant CL as Client
    participant CO as Coordinator
    participant A as Node A home replica
    participant B as Node B home replica DOWN
    participant C as Node C home replica
    participant D as Node D next on ring

    CL->>CO: put key k with value v, W = 2
    CO->>A: write v
    CO--xB: write v - timeout, B unreachable
    CO->>C: write v
    CO->>D: write v with hint intended for B
    A-->>CO: ack
    D-->>CO: ack - W reached, sloppy
    CO-->>CL: success
    Note over D: hint stored locally:
    replay this to B when it returns
    Note over B: B recovers
    D->>B: hinted handoff: deliver v
    B-->>D: ack
    Note over D: hint deleted
```

Understand precisely what has been traded. Write availability is now superb: a write succeeds as long as *any* W nodes anywhere in the (reachable side of the) cluster are up. But the intersection proof from the start of the chapter quietly assumed both quorums were drawn from the same fixed N -member set. Under sloppy quorums they are not: the write above landed on $\{A, C, D\}$, and a read a moment later — with B back, or with a different set of nodes visible to a different coordinator — may assemble its R responses from $\{A?, B, C\}$... or, in a partition where none of the home replicas are reachable, from substitutes that never saw the value at all. **$R + W > N$ no longer guarantees intersection, because " N " no longer names a fixed set.** During failures — that is, exactly when you might hope the guarantee earns its keep — a Dynamo "quorum" read can miss a Dynamo "quorum" write entirely, until hints are replayed and anti-entropy converges. The guarantee has been downgraded from "reads see writes" to "reads eventually see writes, with high probability, sooner if failures are short." For a shopping cart, with a merge function that can absorb late-arriving values, that is a defensible

trade. It must be made knowingly: sloppy quorums are an availability feature purchased with the consistency guarantee itself, and systems that expose the option (Riak's `sloppy_quorum`, Cassandra's hinted handoff writes at low consistency levels) should be read accordingly.

Read repair: convergence on the read path

How do stale replicas catch up? The first mechanism rides on ordinary reads. A read at $R > 1$ already collects versions from several replicas; when the coordinator sees that some responded with older versions (or none), it sends them the newest version after answering the client. Keys that are read often are thus repaired continuously, for free, with the freshest data exactly where the traffic is. Cassandra implements the same idea with an optimization: ask one replica for full data and the others for a hash digest, and only fetch and reconcile full copies when digests mismatch — the blocking repair then happens before the client sees the result, within the replicas the read touched. Note the limits: read repair fixes only the replicas the read consulted, only for keys that get read, and (in Dynamo's asynchronous form) only *after* the client response — which is why it does not rescue linearizability, as the non-monotonic-read anomaly showed. Cold keys are never repaired by reads at all. Something must sweep them.

Anti-entropy with Merkle trees

How do replicas converge on keys nobody reads? Periodically, each pair of replicas that share a key range runs **anti-entropy**: compare what we hold, transfer what differs. The naive comparison — exchange every key and version — costs bandwidth proportional to the data held, which is absurd when replicas differ in a handful of keys out of millions. The **Merkle tree** makes the comparison cost proportional to the *difference* instead.

Each replica builds, per key range, a tree of hashes: leaves are hashes over small sub-ranges of keys (hashing the keys and versions they contain), and each interior node is the hash of its children's hashes. Two replicas then compare top-down. If the roots match, the ranges are identical with overwhelming probability — one hash exchanged, comparison over. If the roots differ, some descendant differs; recurse into the children, pruning every subtree whose hashes match, until the mismatching *leaves* are isolated. Those leaves name the small sub-ranges

that actually diverge, and only their keys are exchanged and reconciled. A single divergent key costs one root comparison plus one path of length $\log(\text{leaves})$ — a few hashes — instead of a full-range scan.

```
flowchart TB
    subgraph R1["Replica 1 – Merkle tree over range"]
        A0["root hash 7f21"] --> A1["left half 9c04"]
        A0 --> A2["right half 5e77"]
        A1 --> A3["leaf keys 0-499 b1aa"]
        A1 --> A4["leaf keys 500-999 e29c"]
    end
    subgraph R2["Replica 2 – same range"]
        B0["root hash 3d90"] --> B1["left half 9c04"]
        B0 --> B2["right half 84f2"]
        B1 --> B3["leaf keys 0-499 b1aa"]
        B1 --> B4["leaf keys 500-999 e29c"]
    end
    A0 -.->| "step 1: roots differ – descend" | B0
    A1 -.->| "step 2: left halves match – prune subtree" | B1
    A2 -.->| "step 3: right halves differ – recurse" | B2
    A2 ==>| "step 4: mismatching leaves found  
exchange only those keys" | B2
```

The cost that the paper is honest about, and that operators rediscover: the trees must be built, and rebuilt as data changes. Cassandra constructs them on demand during repair by scanning and hashing the data (a "validation compaction"), which is why full repairs are I/O-heavy events to be scheduled, not background noise — more on this under operational realities.

Version vectors and siblings

When two writes race, who wins? Dynamo's answer is: nobody, yet. Each stored value carries a **version vector** — the paper says "vector clock," but the structure counts *updates to this key* per coordinating node rather than all events per process, so version vector is the precise term (Chapter 2 drew this distinction). Each `put` goes through a coordinator node, which increments its own counter in the vector; the client supplies the vector it last read (the opaque `context` in the API) so the new version *descends from* what the client saw. Comparison is componentwise: vector V1 descends from V2 if every counter in V1 is $\geq$ its counterpart in V2. If neither descends from the other, the writes were **concurrent** — neither writer saw the other's update — and the store keeps *both* values as **siblings** rather than discarding either. A subsequent read returns all

siblings plus a merged context; the *client* reconciles them and writes the merged result back, which descends from both and collapses the siblings.

Walk the paper's own example, with coordinators *Sx*, *Sy*, *Sz*:

1. A client writes D1 through *Sx*: version `[Sx:1]` .
2. The same client updates it through *Sx*: D2, version `[Sx:2]` . D2 descends from D1; D1 is garbage-collectable.
3. A client that read D2 writes through *Sy*: D3, version `[Sx:2, Sy:1]` .
4. Meanwhile another client that also read D2 writes through *Sz*: D4, version `[Sx:2, Sz:1]` .
5. Compare D3 and D4: `Sy:1 > Sy:0` but `Sz:0 < Sz:1` — neither descends from the other. Concurrent. A node that receives both keeps both as siblings.
6. A read now returns {D3, D4} with merged context `[Sx:2, Sy:1, Sz:1]` . The client merges the values and writes back through *Sx*: D5, version `[Sx:3, Sy:1, Sz:1]` , which descends from both siblings. Convergence.

```
flowchart TB
    D1["D1 via Sx  
vv Sx:1"] --> D2["D2 via Sx  
vv Sx:2"]
    D2 --> D3["D3 via Sy  
vv Sx:2, Sy:1"]
    D2 --> D4["D4 via Sz  
vv Sx:2, Sz:1"]
    D3 --> D5["D3 -->|\"concurrent with D4:  
kept as siblings\"| D5[\"client reads both, merges,  
writes D5 via Sx  
vv Sx:3, Sy:1, Sz:1\"]"]
    D4 --> D5
```

What should "merge" mean? That is an application question, and the shopping cart is the canonical answer because it has a natural one: **set union of the cart items**. Two concurrent add-to-cart operations merge into a cart containing both items; no add is ever lost. The paper admits the asymmetry frankly — a concurrent *deletion* can resurrect: union cannot distinguish "item absent because never added" from "item absent because removed," so a removed item present in the other sibling comes back. Amazon judged a rare resurrected item strictly better than a lost one. Making deletion merge correctly requires keeping per-element

metadata — tombstones with their own causal tags — which is precisely the road that leads to CRDTs, where Chapter 11 picks it up as the OR-set.

The alternative to all this bookkeeping is **last-write-wins**: stamp each write with a timestamp, and on conflict keep the highest stamp and discard the rest. It is worth stating plainly what LWW is: *a policy of silently deleting one of two acknowledged writes*. Both clients were told "success"; one client's data is gone; nothing recorded that it ever existed. And the choice of which write dies is made by wall clocks — Chapter 2's unreliable narrators — so under skew the discarded write can even be the later one. LWW is the correct choice in the narrow case where values are full overwrites and the newest-by-intent one is genuinely the only one that matters (a sensor's latest reading, a cache). Chosen as a default for general data, it is a data-loss policy that will execute during precisely the concurrent-update scenarios that quorum replication exists to survive. Cassandra chose it as the default anyway; the next section examines that trade.

Membership by gossip

How does every node know the ring? Dynamo has no configuration master. Membership changes are introduced at any node by an operator and spread by a **gossip protocol**: each node exchanges its membership view with a random peer every second, and views converge cluster-wide in logarithmic rounds. Each node thus knows the full ring and can coordinate any request — symmetric, no single point of failure, at the cost of transient disagreement about membership while gossip converges (one more reason quorum membership is fuzzy at the edges). Failure detection is likewise local and probabilistic: a node treats a peer as down when it stops answering, and retries later — sufficient because hinted handoff and anti-entropy make the consequences of a wrong guess self-healing. Gossip and failure detection get their full treatment in Chapter 10; here it is enough that Dynamo's membership layer is exactly as eventual as its data layer, by design.

ABD: quorums can give you a real register

Before crowning consensus as the only way to strong consistency, one theory landmark deserves a paragraph. Attiya, Bar-Noy, and Dolev showed (1995, from a 1990 conference result) that a **linearizable read/write register** — not consensus, just a single register with atomic reads

and writes — *can* be built from plain majority quorums in a fully asynchronous system, crashes and all, with no leader. The trick is in the read: a reader queries a majority, takes the value with the highest timestamp, and then — before returning — **writes that value back to a majority**. The write-back is the whole theorem. Without it, the non-monotonic anomaly from earlier is live: a read can observe a value that a later read fails to observe. With it, any subsequent read's majority intersects the write-back's majority, so once a value has been *returned* it can never un-happen. (Writers, in the multi-writer version, do a read phase first to pick a timestamp higher than any they see.) ABD is why "quorum systems are only eventually consistent" is false as a theoretical claim: reads cost two round trips instead of one, writes two phases, and you get a linearizable register — but *only* a register. Reads and writes, no read-modify-write: ABD cannot give you compare-and-swap, because that is consensus (Chapter 5, via Herlihy's hierarchy in Volume 4, Chapter 4). Production Dynamo-style stores skip the synchronous write-back for latency, which is a legitimate choice — but it is the reason their quorum reads are weaker than ABD's, not an inevitability of leaderlessness.

The descendants

Cassandra: Dynamo's ring, Bigtable's data model, LWW's risks

Cassandra (open-sourced by Facebook in 2008; Lakshman & Malik's paper, 2010) took Dynamo's distribution layer — ring, replication, gossip, hinted handoff, read repair, anti-entropy — and replaced the flat key-value API with a Bigtable-style wide-column model queried through CQL (Volume 5, Chapter 11). The consequential divergence is conflict resolution: **no version vectors, no siblings**. Every cell carries a timestamp (microseconds, supplied by client or coordinator), and the highest timestamp wins — LWW at cell granularity, always, at every consistency level. The upside is real: values need no causal metadata, reads return exactly one answer, merges are commutative timestamp comparisons, and the whole sibling-handling burden on application code disappears. The cost is the one already stated: concurrent updates to the same cell are resolved by clock order, so an acknowledged write can be silently discarded — and with skewed clocks, even a QUORUM or ALL write sequence can lose the causally-later update. The Jepsen analyses of Cassandra demonstrated exactly this class of lost updates; the general Jepsen finding across Dynamo-style stores bears repeating — strict

quorum settings do not by themselves prevent anomalies; the resolution and read-back protocols decide.

Cassandra's own answer for the cases that cannot tolerate LWW is telling: **lightweight transactions** — `INSERT ... IF NOT EXISTS`, `UPDATE ... IF value = expected` — implemented with a Paxos round per operation among the key's replicas. When you need compare-and-swap, quorum overlap is not enough, and Cassandra ships a consensus protocol (Chapter 5) in its belly to provide it, at several round trips per operation and with its own SERIAL consistency levels. That a flagship leaderless store embeds Paxos for its conditional writes is the cleanest possible admission of the boundary this chapter keeps drawing.

Operationally, two Cassandra realities matter here. Deletes are **tombstones** — markers written over the data, reconciled like any write, and purged only by compaction after `gc_grace_seconds` (default ten days); Volume 5, Chapter 2's LSM mechanics explain why deletion must work this way in an append-oriented store. And repair is entangled with them: a replica that misses a delete and is not repaired within the grace window can re-spread the deleted data after the tombstone is purged — resurrection again, this time as an operational failure mode rather than a merge semantic. Hence the iron rule: full anti-entropy repair on every node, on a schedule shorter than `gc_grace_seconds`, forever.

Riak: the faithful heir

Riak was the most faithful open-source Dynamo: the ring, sloppy quorums with per-request N/R/W, hinted handoff, and — crucially — real version vectors with siblings surfaced to the client (`allow_mult`). Its history refined the causality mechanism itself: plain per-node version vectors can spuriously multiply siblings under certain client interleavings, and Riak's **dotted version vectors** fixed the bookkeeping by tagging each sibling with the exact event ("dot") that created it. Riak's second contribution was admitting that shipping siblings to application code was a usability tax most teams paid badly, and answering with **Riak Data Types** (2.0): counters, sets, and maps whose merge functions are built in and mathematically guaranteed to converge — CRDTs, the systematic completion of the shopping-cart idea, and Chapter 11's subject.

DynamoDB-the-service is not Dynamo-the-paper

The naming has confused a decade of engineers, so be precise. **Amazon DynamoDB** (the AWS service, 2012) shares with the 2007 paper a lineage, an availability obsession, and part of a name — and almost none of the replication design. As described in Amazon's own USENIX ATC 2022 paper, DynamoDB partitions each table by key; **each partition is a replication group with a leader**, elected and maintained by Multi-Paxos across three replicas in three availability zones. Writes go to the leader and commit on a quorum of the group; an *eventually consistent* read (the cheap default) may be served by any replica, while a *strongly consistent* read is served by the leader. There are no sloppy quorums, no siblings, no client-side merges, no tunable W. In this chapter's vocabulary, DynamoDB is a **leader-based, consensus-replicated store with an optional stale-read fast path** — architecturally a cousin of Chapter 6's Raft-based systems, not of the leaderless design that shares its name. The paper's ideas survive in the service's spirit (predictable latency, incremental scaling, availability as a product requirement) far more than in its mechanisms.

Voldemort, briefly

LinkedIn's Project Voldemort (2009) was a near-direct open-source Dynamo implementation — consistent hashing, per-store N/R/W, version vectors with client-resolved conflicts — used for years to serve high-volume read-write workloads and, in a separate read-only mode, to serve datasets bulk-built in Hadoop. It mattered as proof that the paper's recipe could be reproduced outside Amazon; LinkedIn later retired it in favor of successor systems, and its historical role is as the design's replication study.

Leaderless versus leader-based replication

Volume 5, Chapter 8 covered single-leader replication within one database; Chapters 5 and 6 here built consensus-backed leader replication. Set leaderless quorum replication beside them honestly:

Property	Leaderless quorum (Dynamo-style)	Leader-based consensus (Raft/Multi-Paxos)
Write availability	Any W reachable replicas suffice; with sloppy quorums, nearly always writable	Requires a live leader plus quorum; writes stall during election (seconds)
Failover	None — no leader to fail over	Detection + election pause; the classic availability dip
Latency profile	One round to W replicas; no single mandatory hop	All writes traverse the leader; leader can be remote from client
Ordering	None global; per-key causality at best (version vectors) or clock order (LWW)	Total order of the log, per group
Read-modify-write / CAS	Not natively; requires embedded consensus (Cassandra LWT)	Native — propose to the log
Transactions	No	Foundation for them (per-group; cross-group via 2PC over groups)
Conflicts	First-class: siblings or LWW loss	Cannot occur; the leader serializes
Hot-spot behavior	Load spreads across replicas and coordinators	Leader is the throughput ceiling per group

The pattern is clean: leaderless buys *write availability and smooth failure behavior* by giving up *ordering*, and everything downstream of ordering — CAS, transactions, invariants spanning operations — goes with it. Where does each belong? Leaderless fits high-write-rate key-value workloads whose values merge naturally or tolerate LWW, that span regions and must accept writes in all of them, and that cannot tolerate election pauses: carts, sessions, time-series ingest, presence, caches with durability. Leader-based fits everything that needs an invariant to hold: metadata, configuration, coordination, financial state, anything with uniqueness or ordering requirements. The modern convergence is visible in what gets built: new infrastructure over the last decade — etcd, CockroachDB, TiKV, YugabyteDB, Kafka's KRaft, DynamoDB's own

internals — overwhelmingly chooses per-shard Raft/Paxos, taking consensus ordering and paying the election pauses, while leaderless quorum systems hold the niches where merge-friendly AP semantics are a genuine fit and Cassandra-scale write ingest is the requirement. The two lineages also hybridize, as DynamoDB shows: quorum durability underneath, a leader for order on top.

Operational realities

Repair is a scheduled discipline

An unrepaired Dynamo-style cluster does not stay converged; it drifts. Every dropped hint (hint windows are finite — Cassandra discards hints older than a few hours by default), every node replaced from a stale backup, every write that raced a topology change adds divergence, and read repair only sweeps the keys that traffic touches. Entropy accumulates in the cold data. The consequences surface late and strangely: resurrected deletes after tombstone purge, reads whose answer depends on which replicas respond, restore-from-one-replica missing data the others had. The discipline is unglamorous and non-optional: scheduled full anti-entropy repair, every node, every cycle, with the cycle length bounded by the tombstone grace period; monitored like backups are monitored, because like backups, nothing visibly breaks when you stop.

Quorum arithmetic under failure domains

Do the arithmetic before the incident does it for you. With $N = 3$, $W = R = \text{QUORUM} = 2$: one replica down and every quorum operation still works, with zero margin; a second failure — or a GC pause, or an overloaded node missing its timeout — and quorum operations on the affected ranges fail outright. Now project onto failure domains: if two of a key's three replicas share a rack, one rack switch is that second failure. This is why rack- and AZ-aware placement (Cassandra's `NetworkTopologyStrategy`) puts each replica in a distinct domain — making "one domain down" cost exactly one replica per key, which QUORUM absorbs — and why multi-DC deployments use `LOCAL_QUORUM`, scoping the quorum to the local datacenter so that a WAN partition or remote-DC outage neither blocks writes nor silently ships your latency across the ocean. Capacity planning follows the same line: $N = 5$

tolerates two arbitrary replica failures at QUORUM but pays for five copies and a slower quorum; most operators instead keep $N = 3$ and spend the effort on making failure domains genuinely independent.

Consistency-level misconfiguration as an incident class

Because the dial is per-operation and per-team, the classic incident needs no failure at all — just two settings that do not add up. Service A writes at ONE (it was fast in the load test); service B reads at ONE; $R + W = 2 \leq 3 = N$; B's read lands on a replica the write has not reached yet. The ticket is titled "*where did my write go*," the data is not lost, and by the time anyone investigates, replication has converged and nothing reproduces. Variants recur endlessly: reads at QUORUM against writes at ONE (same arithmetic, misplaced confidence); Cassandra's ANY level, which counts a mere hint as a successful write — durable nowhere a read can find it; EACH_QUORUM writes stalling on a remote DC outage; a driver default of LOCAL_ONE quietly overriding the QUORUM everyone assumed. The defenses are organizational as much as technical: treat the (write-level, read-level) *pair* as a single reviewed contract per dataset, assert it in code rather than config defaults, and when an engineer reports reads missing acknowledged writes, check consistency levels before suspecting the database.

The distributed-systems lens

Quorums versus consensus, precisely. Both rest on the same intersection arithmetic, so state the difference exactly: a quorum read/write system guarantees that information *reaches* the overlap, and nothing more — there is no proposal numbering, no acceptance protocol, no way for the overlapping replica to rule on which of two concurrent writes is *first*. It cannot solve consensus, and therefore cannot give you CAS or a total order; when Cassandra needed `IF NOT EXISTS`, it had to embed Paxos (Chapter 5). Raft (Chapter 6) is the converse demonstration: it *is* a quorum system — commit is a majority ack, election is a majority vote — but it adds a leader and a log, and the leader is what converts quorum overlap into total order: one process serializes proposals, the log names each slot, and intersection then guarantees the *order* survives failures, not merely the data. Same inequality; profoundly different contract. Flexible Paxos closes the loop from the other side:

even consensus only ever needed its quorums to intersect, not to be majorities.

R/W tuning is PACELC made concrete. Chapter 4's PACELC says: under partition, availability versus consistency; else, latency versus consistency. The N/R/W dial is that abstraction with knobs. ONE is the EL/PA corner; QUORUM with strict quorums buys overlap at quorum latency; sloppy quorums are the PA choice in its purest form — they redefine the quorum rather than refuse the write. What this chapter adds to Chapter 4 is the fine print: the "C" that QUORUM buys is overlap, not linearizability, unless you also pay for ABD-style read write-backs or a leader.

Version vectors are version-tag reasoning at fleet scale. Volume 4, Chapter 4 taught the ABA problem: a value that looks unchanged may have changed and changed back, so CAS loops carry a version tag alongside the value. A version vector is that same idea with one counter per coordinator, and Dynamo's `put(key, context, value)` is exactly a tagged CAS-shaped update: the context says which history this write extends, and the store detects — though, unlike CAS, does not reject — a concurrent interleaving, surfacing it as siblings instead. The fencing tokens of Chapter 2 are the same family: monotonic version metadata that lets a system detect stale actors. Learn the pattern once — *never compare values; compare histories* — and it serves from a lock-free stack to a planet-scale cart.

Hinted handoff is store-and-forward. A hint is a queued message with an intended recipient, durable buffering at an intermediary, retry until delivery, and at-least-once semantics — a message broker's contract (Volume 10) implemented inside a database's replication layer. The correspondence runs deep: the hint window is the queue's retention, hint replay is redelivery, and the reason hinted handoff weakens quorum guarantees is the same reason a queued message is not yet a delivered one — durability at an intermediary is not visibility at the destination.

Key takeaways

- **$R + W > N$ guarantees intersection** — every read quorum shares a replica with every write quorum, by pigeonhole — and $W + W > N$ prevents two disjoint write quorums. That is all the arithmetic guarantees: overlap, not ordering, not linearizability.

- **Strict quorums still admit anomalies:** partial writes have no rollback, replication lag produces non-monotonic reads, and conflict resolution decides everything the counting does not. ABD shows the repair — synchronous read write-back yields a linearizable register — and its cost explains why production stores skip it.
- **Dynamo is a coherent AP design:** ring plus virtual nodes for placement, sloppy quorums plus hinted handoff for write availability, read repair plus Merkle-tree anti-entropy for convergence, version vectors plus siblings for conflicts, gossip for membership. Every mechanism trades consistency for availability, deliberately.
- **Sloppy quorums void the intersection guarantee** — "N" stops naming a fixed set — precisely during failures. They are an availability feature bought with the freshness guarantee; enable them knowingly.
- **Merkle trees make anti-entropy cost proportional to the divergence,** not the dataset: compare roots, recurse only into mismatching subtrees, exchange only mismatching leaf ranges.
- **Siblings preserve concurrent writes; LWW silently deletes one of them,** with the victim chosen by wall clocks. The shopping cart merges by union; deletes need tombstone metadata; the systematic completion of merge semantics is CRDTs (Chapter 11).
- **The descendants diverged on exactly the conflict question:** Cassandra chose LWW and later embedded Paxos for CAS; Riak kept siblings and grew CRDTs; DynamoDB-the-service is a leader-based, Paxos-replicated system that shares little but a name with the paper.
- **Leaderless buys write availability and no failover pause; leaders buy order** — and with it CAS and transactions. Modern infrastructure has mostly converged on per-shard consensus, with leaderless holding the merge-friendly AP niches.
- **Operations are part of the design:** repair on a schedule bounded by tombstone grace, replicas spread across failure domains, and read/write consistency levels reviewed as a pair — because "write ONE, read ONE, where did my write go" is an incident class, not a puzzle.

Further reading

- DeCandia, G. et al., "Dynamo: Amazon's Highly Available Key-value Store," *SOSP*, 2007 — the paper this chapter walks. <https://dl.acm.org/doi/10.1145/1294261.1294281>
- Attiya, H., Bar-Noy, A., and Dolev, D., "Sharing Memory Robustly in Message-Passing Systems," *JACM* 42(1), 1995 — the ABD register: linearizability from quorums via read write-back. <https://dl.acm.org/doi/10.1145/200836.200869>
- Lakshman, A. and Malik, P., "Cassandra: A Decentralized Structured Storage System," *ACM SIGOPS Operating Systems Review* 44(2), 2010 — Dynamo's distribution under Bigtable's data model.
- Elhemali, M. et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," *USENIX ATC*, 2022 — the service's actual architecture: Multi-Paxos leaders per partition. <https://www.usenix.org/conference/atc22/presentation/elhemali>
- Kleppmann, M., *Designing Data-Intensive Applications*, O'Reilly, 2017 — Chapter 5's treatment of leaderless replication and the quorum-anomaly figures; Chapter 9 for why quorums alone are not linearizable.
- Jepsen analyses — <https://jepsen.io/analyses> and the early aphyr.com Cassandra and Riak posts — empirical demonstrations of lost updates under LWW and of strict-quorum anomalies.
- Apache Cassandra documentation — <https://cassandra.apache.org/doc/latest/> — consistency levels, hinted handoff, read repair, repair and `gc_grace_seconds` operational guidance.
- Riak documentation on causal context and dotted version vectors — <https://docs.riak.com/riak/kv/latest/learn/concepts/causal-context/> — the sibling machinery in its most faithful production form.
- Preguiça, N., Baquero, C. et al., "Dotted Version Vectors: Logical Clocks for Optimistic Replication," 2010 — the fix for sibling explosion. <https://arxiv.org/abs/1011.5808>
- Volume 6: Chapter 2 (clocks and fencing), Chapter 4 (PACELC), Chapters 5–6 (consensus), Chapter 10 (gossip and failure detection), Chapter 11 (CRDTs). Volume 5: Chapter 8 (single-system replication),

Chapter 8 — Coordination Services: ZooKeeper and etcd

What this chapter covers. Chapters 5 and 6 built consensus from first principles and left an uncomfortable conclusion: implementing Paxos or Raft correctly is a multi-year project, and the bugs live in the parts the papers leave as exercises. This chapter is the industry's answer — do not implement consensus, *rent* it. A coordination service is a small, strongly consistent, highly available kernel of shared state that everything else coordinates through: consensus productized behind an ergonomic API. We study the two that matter in mechanism-level depth: ZooKeeper's znode tree, sessions, ephemeral nodes, and one-shot watches; etcd's MVCC revision store, leases, resumable range watches, and compare-and-swap transactions. We derive the classic recipes — leader election, locks, barriers, group membership — and show where the naive versions go wrong. The moral center is the fencing imperative, paying off the promise of Volume 4, Chapter 2: a lease plus a paused client equals two nodes that both believe they hold a lock, and only a monotonic token *enforced by the protected resource* restores safety. We close with Kubernetes as the largest deployed case study, the operational disciplines that keep a coordination service from becoming a fleet-wide failure amplifier, and the cases where you should not use one at all.

Learning goals — after this chapter you should be able to:

- Explain what a coordination service is for, why centralizing coordination beats N ad-hoc implementations, and what blast radius you accept in exchange.
- Describe ZooKeeper's data model, session semantics (disconnected versus expired), and watch semantics precisely, and state its consistency guarantees without overclaiming.
- Write the correct ZooKeeper lock/election recipe, explain why watching the predecessor avoids the herd effect, and why a fired watch means "re-read", never "you have the lock".

- Describe etcd's revision-based MVCC model, leases, watches, and transactions, and express a lock or election as lease + compare-and-swap + watch.
- Walk through the paused-client scenario and implement fencing with a ZooKeeper zxid or etcd revision as the token, enforced at the resource.
- Summarize Chubby's design lessons: coarse-grained locks, advisory semantics, sequencers, and why Google built a lock *service* rather than a library.
- Explain Kubernetes' use of etcd: resourceVersion as revision, list+watch informers, and level-triggered reconciliation as the correct consumption pattern for lossy watches.
- Size and operate an ensemble: quorum arithmetic, session-timeout tuning against GC pauses, compaction and defragmentation, and the metrics that predict trouble.

Consensus, productized

Chapter 5 ended with the observation that Multi-Paxos was never fully specified, and Chapter 6 showed how much machinery sits between "Raft, the algorithm" and "Raft, a system you would bet a database on": snapshots, membership changes, client sessions, read paths, PreVote. A team that embeds this machinery in its application takes on all of it, forever, times the number of applications that do it. The alternative is to run consensus *once*, in one hardened implementation, and expose its replicated state machine to everyone through a small API. That is a coordination service. ZooKeeper embeds ZAB, an atomic broadcast protocol from the Paxos family; etcd embeds Raft — literally the reference implementation whose library half the industry reuses. Applications never see the protocol. They see a tiny filesystem-like or key-value store with unusual guarantees: writes are agreed by a majority, survive minority failure, and are observed in one order by everybody.

What makes such a store a *coordination* service rather than merely a small database is a second ingredient: **liveness-coupled state**. Both systems tie data to the aliveness of the client that created it — ZooKeeper through sessions and ephemeral znodes, etcd through leases — so a client's death (or apparent death) is automatically converted into a visible state change that other clients can watch for. Locks that free themselves,

membership that updates itself, leaders whose disappearance triggers re-election: every recipe in this chapter is that one mechanism plus atomic conditional updates.

The trade is stated honestly by everyone who has run one: the coordination service becomes your single most critical dependency. Every leader election, lock, and piece of service discovery shares its fate; when it is down, nothing that needs coordination can make a coordination *decision*. That is why the second half of this chapter is about blast radius, capacity discipline, and consumers that degrade gracefully rather than stopping the world. One hardened implementation beats N ad-hoc ones, but it is one implementation whose outage is everyone's outage.

ZooKeeper

ZooKeeper (Hunt, Konar, Junqueira, and Reed, USENIX ATC 2010) came out of Yahoo! and is the direct intellectual descendant of Google's Chubby, with one deliberate philosophical difference: where Chubby exposes locks, ZooKeeper exposes *wait-free* primitives — small data objects with notifications — from which clients build locks, elections, and barriers themselves. The paper calls this a coordination *kernel*: mechanism in the service, policy in the client.

The data model: znodes

The namespace is a tree of **znodes**, addressed by slash-separated paths like `/services/checkout/members/host-17`. Every znode can hold a small blob of data (the server-side limit, `jute.maxbuffer`, defaults to about 1 MB; well-behaved uses stay in the tens of bytes to kilobytes) *and* have children — it is simultaneously file and directory. Three properties of znodes carry all the weight:

- **Ephemeral znodes** exist only as long as the session that created them. When the session ends — cleanly via `close`, or by expiry — the server deletes them. Ephemerals cannot have children. This is the liveness coupling: an ephemeral znode is a machine-verified claim that "the client that created me is still alive, as far as the ensemble can tell".
- **Sequential znodes** are created with a suffix the *parent* assigns from a monotonic counter, zero-padded to ten digits: ask for `/locks/db/lock-` and you get `/locks/db/lock-0000000042`. Creation is atomic

with numbering, so concurrent creators receive distinct, totally ordered names. Sequence plus ephemerality is the workhorse combination for locks and elections.

- **Versions.** Every znode carries a data version (bumped on each `setData()`), a child version, and an ACL version. Conditional operations — `setData(path, data, expectedVersion)` and `delete(path, expectedVersion)` — fail if the version does not match. This is compare-and-swap (Volume 4, Chapter 4) transplanted into a replicated store, and it is what makes read-modify-write safe against concurrent clients.

Znodes also expose **zxids** — the ZAB transaction IDs of the operations that touched them: `czxid` (creation), `mzxid` (last modification), `pxxid` (last child change). A `zxid` is a 64-bit number, epoch in the high bits and a counter in the low bits, and it totally orders all writes the ensemble has ever committed. File that away: `zxids` are natural fencing tokens.

Sessions, precisely

A client connects to any server in the ensemble and establishes a **session** with a negotiated timeout — the client requests one, and the server clamps it into a window derived from the ensemble's `tickTime` (by default between 2 and 20 ticks; with the default 2-second tick, 4 to 40 seconds). The client library keeps the session alive with heartbeats, sending a ping whenever the connection has been idle for roughly a third of the timeout, and transparently reconnects to another server if its current one fails — the session, its ephemerals, and its watches all survive a server change, because sessions are replicated state, not per-server state.

The session **expires** when the ensemble has not heard from the client for the timeout. Expiry is declared by the ensemble, not the client, and this asymmetry is the single most important thing to understand about ZooKeeper client programming. From the client's point of view there are two distinct bad states:

- **Disconnected.** The client library has lost its TCP connection and is trying other servers. The session may still be perfectly alive — the ensemble's timer is still running but has not fired. Crucially, *the client cannot know*. It is on the wrong side of a partition and has no authority over the session's fate.

- **Expired.** The ensemble declared the session dead, deleted its ephemerals, and notified watchers. The client learns this only when it manages to reconnect and is told the session is gone. `Expired` is terminal: the client must construct a new session and re-create its state.

Correct clients handle both, differently. On `Disconnected`, a client holding a leadership role or a lock must assume the worst *for actions* — stop doing things that require the lock, or make every action individually safe via fencing — while hoping for the best *for state*, since the session usually survives a brief blip. On `Expired`, all bets are off: ephemerals are gone, another client may hold your lock, and you rejoin as a newcomer. Treating `Disconnected` as `Expired` causes needless failovers on every network hiccup; treating a long disconnection as harmless causes split brain. The window between the two is the uncertainty interval that Chapter 10 formalizes in failure detectors; no client-side cleverness eliminates it — only fencing at the resource makes it harmless.

Watches

`Reads (getData, exists, getChildren)` can set a **watch**: a request to be notified once when the watched thing changes — data watches for data changes and deletion, child watches for child list changes. Three properties define the model:

1. **One-shot.** A watch fires at most once. If you want continued notification you must re-register by issuing another read.
2. **Ordered.** A client sees its watch events, its read results, and its write acknowledgements in a single global order consistent with the write order. In particular, a client never reads the new state of a znode before receiving the watch event for the change that produced it.
3. **Notification, not delivery.** The event tells you *that* something changed, not *what*. It carries the path and event type, not the data.

One-shot watches have an inherent race: between the event firing and your re-registration, further changes are invisible. The discipline that makes this safe: **on watch fire, re-read the state and re-register the watch in one operation** (every read that sets a watch does both atomically), and derive your action from the state you read, never from the event. Multiple changes may collapse into one notification; your code

must be correct if it only ever observes the latest state. This is the condition-variable `while` loop from Volume 4, Chapter 2, reborn: a watch event is a wakeup, possibly stale, possibly coalesced, and the predicate must be re-tested before acting. (ZooKeeper 3.6 added persistent and recursive watches via `addWatch`, which remove the re-registration step but not the need to read state.)

Consistency guarantees, stated exactly

ZooKeeper's guarantees are frequently overquoted, so let us be precise. **Writes are linearizable:** every write goes through the leader, is committed by ZAB on a majority, and all writes form a single total order (the `zxid` order) respecting real time. **Reads are not linearizable by default.** Any server answers reads from its own replica without consulting the leader — that is where ZooKeeper's read scalability comes from — so a read may return stale data if the serving replica lags. What you get is strong *per-client* ordering, packaged in the paper as two guarantees: all requests from one client execute in FIFO order (so a client sees its own writes), and a client's view never moves backward in `zxid` time, even across reconnection — the library refuses to attach to a server behind the last `zxid` the client has seen. In Chapter 3's vocabulary: sequential consistency with read-your-writes and monotonic reads per session, atop linearizable writes.

When a client genuinely needs to observe all writes committed before some point — "the config was updated and a notification sent out of band; now read the *new* config" — ZooKeeper offers `sync()`: an asynchronous barrier that causes the client's server to catch up with the leader before subsequent reads are served. `sync` followed by a read gives an effectively fresh read without paying quorum-read cost on every read. If you find yourself calling `sync` before every read, you wanted etcd's model or quorum reads (Chapter 7), and should say so rather than fighting the grain of the system.

ZAB, and how it relates to Raft

Underneath, ZooKeeper replicates via **ZAB** (ZooKeeper Atomic Broadcast; Junqueira, Reed, and Serafini, DSN 2011), a leader-based atomic broadcast protocol for primary-backup replication. A leader is elected, brings a quorum of followers to an identical log prefix in an explicit *synchronization* phase (sending a diff, a truncation, or a full

snapshot as needed), then broadcasts state changes as idempotent transactions identified by `zxid`; commits require majority acknowledgment. The mapping to Chapter 6 is almost mechanical: `epoch` $\approx$ `term`, `zxid` $\approx$ (`term`, `index`), and ZAB's election favors the server with the most up-to-date history much as Raft's election restriction does. The structural difference: ZAB separates recovery into its own phase — a new leader first makes a quorum identical to itself, then resumes broadcast — whereas Raft repairs followers lazily during normal `AppendEntries` traffic; and ZAB is framed as *primary order* atomic broadcast rather than a general replicated state machine. For this chapter's purposes the systems are peers: both yield a majority-committed, totally ordered log, the raw material every guarantee above is made of.

The recipes

The ZooKeeper distribution and paper describe standard client-side constructions — "recipes" — and the details are where correctness lives. (In practice you use a vetted library, Apache Curator, for exactly the reasons Vol 4, Ch 2 gave for not hand-rolling mutexes; but you must understand the recipe to operate it.)

The naive lock, and the herd. The obvious recipe: every contender tries to `create("/locks/db/lock", EPHEMERAL)`; the winner holds the lock; losers watch the `znode` and retry when it fires. It is *correct*, but on every release the ensemble notifies every waiter, all `N` stampede to create, one wins, and `N-1` re-watch: the **herd effect**. Each handoff costs $O(N)$ messages, a full lock cycle $O(N^2)$, and the ensemble absorbs the spikes. With hundreds of waiters this is a self-inflicted DoS of your most critical dependency.

The correct recipe — ephemeral sequential + watch the predecessor.

```

lock(path):
    me = create(path + "/lock-", data, EPHMERAL | SEQUENTIAL)  # e.g.
lock-0000000042
    loop:
        children = getChildren(path, watch=false)
        if me has the lowest sequence number in children:
            return ACQUIRED                                     # holders in sequence
order
    pred = child with the largest sequence number below mine
    if exists(pred, watch=true):                               # watch ONLY my
predecessor
        wait for the watch event
        # predecessor gone – either released or its session expired;
        # loop and re-evaluate: do NOT assume the lock is mine

unlock():
    delete(me)         # or: session close / expiry deletes it for us

```

Why this shape:

- **Total order without agreement.** The sequence numbers assigned at create time *are* the queue; clients merely observe their position.
- **Each waiter watches exactly one znode — its predecessor.** A release or crash notifies one client, not N; handoff is O(1) messages at any queue depth. This is the entire fix for the herd effect, and the same trick MCS queue locks use in shared memory: wait on your predecessor, not on the shared lock word.
- **The loop is mandatory** (again). Your predecessor's znode can vanish because its *session expired while it was still waiting* — in which case some earlier znode may still exist and you are not at the head. The watch event means "re-evaluate", never "acquired".
- **Crash safety is free.** Every queue entry is ephemeral, so a crashed or expired contender — waiting or holding — is removed by the ensemble and the queue heals itself.

Leader election is the same recipe with a different reading: the client holding the lowest sequence *is* the leader; everyone else is a hot standby watching its predecessor, and succession is automatic and ordered. Patroni's PostgreSQL failover (Volume 5, Chapter 8) is exactly this pattern over ZooKeeper or etcd, with the leader key doubling as the advertisement of who to follow.

```
sequenceDiagram
    participant A as Client A
    participant B as Client B
    participant C as Client C
    participant ZK as ZooKeeper ensemble

    A->>ZK: create /locks/db/lock- ephemeral sequential
    ZK-->>A: lock-0000000001
    B->>ZK: create /locks/db/lock- ephemeral sequential
    ZK-->>B: lock-0000000002
    C->>ZK: create /locks/db/lock- ephemeral sequential
    ZK-->>C: lock-0000000003
    A->>ZK: getChildren – I am lowest
    Note over A: A holds the lock
    B->>ZK: exists lock-0000000001 watch=true
    C->>ZK: exists lock-0000000002 watch=true
    Note over B,C: each waiter watches ONLY its predecessor
    A->>ZK: delete lock-0000000001
    ZK-->>B: watch event – predecessor gone
    Note over C: C is not notified – no herd
    B->>ZK: getChildren – re-evaluate, now lowest
    Note over B: B holds the lock
```

Barriers. The *double barrier* enters and leaves a computation in lockstep: to enter, each participant creates an ephemeral child under the barrier node and watches until the child count reaches N, then proceeds; to leave, each deletes its child and waits until the children are gone. The correctness discussion is the by-now-familiar one: watch fires are hints; the predicate is the child count, re-read every time.

Group membership. Each member creates an ephemeral znode (often ephemeral-sequential, for a stable join order) under a group parent, carrying its address as data; observers `getChildren` with a watch. Membership is then *definitionally* live: a member that dies or partitions away is removed by session expiry, within one session timeout. This is a failure detector (Chapter 10) with its output materialized as data — so Chapter 10's timeout-tuning arguments apply verbatim, and we return to them under operations.

etcd

etcd (from the CoreOS lineage, now a CNCF project) is the same idea a decade later, revised in light of ZooKeeper experience: a flat key-value store instead of a tree, Raft (Chapter 6) instead of ZAB, gRPC instead of a

custom protocol, and — the deepest change — an **MVCC store addressed by revision**, which turns watches from fragile one-shot callbacks into a resumable change feed.

Revisions: the MVCC core

Keys are opaque byte strings, and "directories" are only a convention of key prefixes plus range queries (`Range` over `[key, range_end)` ; a prefix query is a range). The store keeps multiple versions: every mutating transaction increments a single global, monotonically increasing **revision**, and each key version is stored indexed by the revision that wrote it — the same MVCC construction as a versioned database engine (Volume 5, Chapter 6), applied to a coordination store. Each key carries `create_revision` (revision of its creation), `mod_revision` (revision of its last write), and `version` (a per-key update counter). Consequences:

- Any read can be executed *at* a revision, giving consistent point-in-time snapshots across keys.
- A watch can start *from* a revision, replaying history forward — no gap between "read the state" and "watch for changes", the gap that ZooKeeper's discipline exists to bridge.
- The global revision is a total order over all writes, ready-made as a fencing token.

By default etcd reads are **linearizable** — served through Raft read-index confirmation so a minority partition or stale member cannot answer with old data — with `serializable` reads available as an explicit opt-in for cheap, possibly stale local reads. Note the inversion of ZooKeeper's default: etcd makes the strong read the default and the fast-stale read the option.

Old versions accumulate, so the history must be **compacted**: compaction at revision R discards versions older than R (the operational side is covered below). Compaction is what bounds the "watch from an old revision" capability — you can resume from any revision the store still remembers, and no further.

Leases

A **lease** is a first-class TTL object: `LeaseGrant(ttl)` returns a lease ID; puts may attach keys to it; a `KeepAlive` gRPC stream refreshes it; when it expires or is revoked, *all attached keys are deleted atomically*. This is the

ZooKeeper ephemeral mechanism with the coupling made explicit and many-to-one: one lease, refreshed by one heartbeat stream, can cover a client's entire footprint — its membership key, its lock keys, its leader claim — and `LeaseRevoke` gives a clean, immediate release path that does not require tearing down a connection. `LeaseTimeToLive` lets any client inspect a lease's remaining TTL, which ZooKeeper sessions never exposed.

```
$ etcdctl lease grant 15
lease 694d77aabccd0f1e granted with TTL(15s)
$ etcdctl put --lease=694d77aabccd0f1e /members/checkout/host-17 '10.4.
7.17:8443'
OK
$ etcdctl lease keep-alive 694d77aabccd0f1e      # blocks, refreshing
until killed
```

Kill the keep-alive, wait out the TTL, and `/members/checkout/host-17` disappears — observed by every watcher, at a specific revision. The semantics warnings from ZooKeeper sessions carry over unchanged: the *server* decides expiry; a client that cannot reach the cluster does not know whether its lease lives; and lease expiry deleting your lock key is precisely the situation fencing exists for.

Watches: resumable, not one-shot

`Watch` is a gRPC stream: watch a key or range/prefix, optionally `--rev=N` to start from a past revision, and receive every subsequent event (PUT and DELETE, each stamped with its revision) until you cancel. Contrast each ZooKeeper pain point: not one-shot, so no re-register race; revision-addressed, so a disconnected client resumes from `last_seen_revision + 1` and misses nothing; range-scoped, so one watch covers a whole subsystem's prefix. Events arrive in revision order, without gaps — with one carve-out: if the requested resume revision has been **compacted**, the watch fails with a compaction error and the client must fall back to ZooKeeper-style behavior — read current state, then watch from the read's revision. Every serious etcd consumer implements this list-then-watch recovery path; Kubernetes' informers are the canonical example below.

Transactions: If / Then / Else

etcd's write-side primitive is the mini-transaction: a guarded batch, executed atomically at one revision.

```
Txn:
  If      - a list of comparisons: value, version, create_revision,
           mod_revision, or lease of given keys vs. given constants
  Then    - operations (Put / Range / DeleteRange) if all comparisons
hold
  Else    - operations if any comparison fails
```

This is compare-and-swap (Volume 4, Chapter 4) generalized: multi-key, comparing on MVCC metadata rather than only values, with both branches returning results and the response carrying the revision at which it executed. `create_revision = 0` means "the key does not exist", which makes `acquire-if-absent` a one-liner. The real `etcdctl` syntax, acquiring a lock key (compares, then success ops, then failure ops):

```
$ etcdctl txn -i
compares:
create("/locks/orders") = "0"

success requests (get, put, del):
put /locks/orders "holder=api-7f9c"

failure requests (get, put, del):
get /locks/orders

FAILURE

get /locks/orders
/locks/orders
holder=api-3b21
```

The transaction failed — some other holder's key exists — and the Else branch atomically tells us who. In real use the Put attaches the client's lease (via the client API) so a dead holder's claim self-deletes, and the Then branch's response revision is retained as the fencing token. Everything etcd offers as higher-level sugar is this primitive composed: locks, elections, `etcdctl put --prev-kv`, optimistic read-modify-write loops comparing `mod_revision`.

Election and locks: lease + CAS + watch

etcd's Go client ships a `concurrency` package (and the server exposes equivalent Lock/Election RPCs) whose construction is worth internalizing because it is the etcd translation of the ZooKeeper recipe:

1. A **Session** wraps a lease and its keep-alive loop — the analogue of the ZooKeeper session.
2. To campaign, a client writes a key under the election prefix — keyed by its lease ID, attached to its lease, guarded by a `create_revision = 0` transaction so it never overwrites itself.
3. Ordering comes from `create_revision`: the contender whose key has the **lowest creation revision** is the leader/holder — creation revisions play the role of ZooKeeper's sequence numbers, assigned atomically by the store's write order.
4. Everyone else **watches for deletion of the keys with lower creation revisions than its own** — the watch-the-predecessor structure again, so a handoff wakes one waiter, not the herd.
5. Release is deleting your key, or your lease expiring and deleting it for you.

flowchart TD

```
S["Create session:  
lease grant TTL, start keep-alive"] --> C["Txn If create_revision of my  
key = 0  
Then put key with lease"]  
C --> Q{"Is my create_revision the  
lowest under the prefix?"}  
Q -->|"yes"| L["Leader / lock held  
token = my create_revision"]  
Q -->|"no"| W["Watch keys with lower  
create_revision for DELETE"]  
W -->|"deletion event"| Q  
L --> R["Resign: delete key  
or lease expiry deletes it"]  
R --> N["Next-lowest waiter's watch fires  
one client wakes, no herd"]
```

Nothing here is protocol magic; it is lease (liveness), CAS transaction (atomic claim), and watch (notification), which is why the same construction ports to any store with those three primitives.

The API surface

The full gRPC surface is small enough to enumerate, which is itself a design statement: **KV** (Range, Put, DeleteRange, Txn, Compact), **Watch** (one bidirectional stream method), **Lease** (Grant, Revoke, KeepAlive, TimeToLive, Leases), plus Auth, Cluster (membership), and Maintenance (status, defragment, snapshot, alarms). A JSON/HTTP gateway fronts the same services. Compare Chapter 6's inventory of what a production Raft system needs — this API is that inventory with the consensus hidden behind it.

The fencing imperative

Volume 4, Chapter 2 ended its distributed-lock discussion with a promissory note: leases trade a liveness failure for a safety hazard, and this chapter would treat the cure properly. Here is the debt, paid. Everything above — sessions, ephemerals, leases, elections — is unsafe without this section whenever the lock protects an external resource.

Two believers

Walk through it concretely. Client A holds a lock: an etcd key on a 15-second lease (the ZooKeeper version with a session and an ephemeral is identical in every step that matters).

1. $t = 0\text{ s}$ — A acquires the lock, begins a read-modify-write against shared storage.
2. $t = 5\text{ s}$ — A's process enters a stop-the-world GC pause. (Or: the container is descheduled by the CPU quota; the VM is live-migrated; a page-fault storm; SIGSTOP; a network partition. The cause is irrelevant — what matters is that A stops running *and cannot know for how long*.)
3. $t = 15+ \text{ s}$ — No keep-alives have arrived. etcd expires the lease and deletes the lock key. From the cluster's perspective this is correct behavior — it is exactly what leases are for; the alternative is a lock held forever by a corpse.
4. $t = 16\text{ s}$ — Client B's watch fires; B's claim transaction succeeds; B legitimately holds the lock and starts writing.
5. $t = 25\text{ s}$ — A's GC pause ends. **A resumes exactly where it stopped, mid-critical-section, with no indication anything**

happened. Its next line of code is a write to shared storage. The expiry notice is sitting unread in a socket buffer; checking the lease *before* the write only shrinks the window, since a pause can strike between check and write.

Two clients now believe they hold mutual exclusion, and the invariant the lock protected is gone. Note what this is *not*: not an etcd or ZooKeeper bug, not a mistuned timeout, not fixable by client-side checks. It is the impossibility from Chapter 1 — a paused process is indistinguishable from a dead one — colliding with the necessity of leases. The service's view and the holder's view *cannot* be kept synchronized through an unbounded pause.

```
sequenceDiagram
    participant A as Client A
    participant E as etcd – lease 15s
    participant B as Client B
    participant S as Storage – enforces tokens

    A->>E: acquire lock
    E-->>A: held, token 33 – create_revision
    A->>S: write with token 33
    S-->>A: ok, high water mark 33
    Note over A: GC pause begins – A frozen
    Note over E: no keep-alives for 15s
    lease expires, key deleted
    B->>E: acquire lock
    E-->>B: held, token 34
    B->>S: write with token 34
    S-->>B: ok, high water mark 34
    Note over A: pause ends – A resumes
    mid-critical-section, unaware
        A->>S: write with token 33
        S-->>A: REJECTED – 33 below 34
        Note over A,S: stale holder is harmless:
    safety enforced at the resource
```

Fencing tokens

The fix accepts that the stale holder will act, and makes its actions harmless. A **fencing token** is a number that (a) the lock service issues with each grant, (b) is strictly greater for every later grant, and (c) the *protected resource* checks, rejecting any operation bearing a token lower than the highest it has accepted. Both systems already mint suitable tokens as a side effect of their write ordering: in ZooKeeper, the lock

znode's `czxid` (or a counter znode's version); in etcd, the lock key's `create_revision` — the `concurrency` package hands it to you as `Mutex.Header().Revision`. Chapter 6's log index, wearing an API.

The resource-side guard is a one-line idea. Against a database:

```
-- schema: resource(id, payload, fence bigint not null default 0)
UPDATE resource
  SET payload = $2, fence = $1
  WHERE id = 'orders-checkpoint'
    AND fence < $1;          -- reject stale tokens
-- driver: if rows_affected == 0, we are a stale holder: abort, do not
retry
```

Against a filesystem or object store, the same shape via conditional put (ETag / generation preconditions) with the token embedded in the object. And when the protected state lives in etcd itself, fencing collapses into a transaction — guard every write on the lock still being *your* grant:

```
resp, err := cli.Txn(ctx).
  If(clientv3.Compare(clientv3.CreateRevision(lockKey), "=", myToken)
).
  Then(clientv3.OpPut("/state/orders-checkpoint", payload)).
  Commit()
// err == nil && !resp.Succeeded => lock lost; stop.
```

Three points engineers get wrong. First, **the resource must enforce the check** — the lock service cannot, because the whole problem is that the stale holder acts without consulting it; if the resource cannot check a token, no lock service, however consistent, gives you mutual exclusion over it (Kleppmann's 2016 argument, summarized in Vol 4, Ch 2's Redlock discussion). Second, the token must come from the lock grant, not a clock — wall-clock timestamps violate strict monotonicity under skew (Chapter 2). Third, fencing changes the goal: the lock becomes an *optimization* that makes contention rare, while safety comes from the resource's monotonicity check — a healthier way to think about distributed locks in general.

Chubby's lessons

Google's Chubby (Burrows, "The Chubby lock service for loosely-coupled distributed systems", OSDI 2006) predates and shaped both systems, and

the paper is a catalogue of lessons learned from running coordination as a service, several of which the industry keeps relearning:

- **A lock service, not a library.** Burrows argues the choice deliberately: a service lets a system of two clients use locks without itself running consensus replicas; it centralizes availability engineering in one specialist team; and — subtle but decisive — electing a leader usually requires *advertising the result*, so the service doubles as a small consistent store for the winner's identity. ZooKeeper and etcd inherited all three arguments.
- **Coarse-grained locks.** Chubby is designed for locks held for hours or days — electing a primary — not milliseconds. Coarse grain keeps load independent of application transaction rate and makes brief outages survivable by holders. Fine-grained locking, Burrows advises, belongs in the application, optionally bootstrapped from a coarse Chubby lock.
- **Advisory, not mandatory.** Chubby locks only exclude other Chubby lock attempts; they do not protect the data. Every recipe in this chapter is advisory in the same sense — which is why fencing at the resource is the load-bearing safety mechanism, and Chubby supplies it as **sequencers**: an opaque string naming the lock, its mode, and a generation counter, which servers receiving requests from lock holders validate. Fencing tokens, 2006.
- **Sessions and KeepAlives**, with a client-side *grace period* during which a client that has lost its master blocks operations rather than failing them — the disconnected-versus-expired distinction, designed in from the start.
- **Clients will surprise you.** In practice Chubby became Google's internal name service — most load being reads and caching, not locking — and the paper is frank that developers neither plan for its unavailability nor resist storing inappropriate data in it, forcing quotas and review. Every ZooKeeper and etcd operator since has rediscovered both facts.

Case study: Kubernetes

Kubernetes is etcd's biggest consumer and the best public demonstration of how to build *around* a coordination service correctly; Volume 12,

Chapter 2 covers the controller machinery in depth, so here we take only the coordination view.

All cluster state — every object you `kubectl get` — lives in etcd under a prefix (conventionally `/registry/...`), written and read exclusively by the API server; nothing else talks to etcd directly, which concentrates the capacity discipline in one client. Every object carries a `resourceVersion`, which is (opaquely) the etcd revision of its last modification, and the API's list and watch operations are etcd's Range and Watch re-exposed: a client lists at some `resourceVersion`, then watches from it, receiving every subsequent change without gaps. When a watch is too stale to resume — the underlying revision compacted — the API returns `410 Gone` and the client relists and re-watches: etcd's compaction recovery recipe, institutionalized in the client libraries as the informer's list+watch loop.

The consumption pattern on top is the important lesson. Controllers are **level-triggered reconciliation loops**: a watch event carries no instruction, it merely nudges the controller to run `reconcile(object)`, which reads the *current* desired and actual state and computes the difference. Missed, coalesced, or replayed events, a relist after `410` — all harmless, because correctness depends only on eventually observing the latest state, not on observing every transition. This is the ZooKeeper watch discipline ("the event means re-read") elevated from a coding rule to a system architecture, and it is why a Kubernetes control plane recovers from an etcd outage by *converging* rather than replaying: workloads keep running on their nodes throughout, and when etcd returns, reconciliation closes whatever gap accumulated. Edge-triggered designs — act on the event's content — are brittle under exactly the losses that distributed watches make routine.

```
flowchart LR
    E["etcd – revisions"] --> AS["API server  
watch cache"]
    AS -->| "list at resourceVersion  
then watch from it" | INF["Informer cache"]
    INF -->| "event nudges key onto queue" | WQ["Work queue  
coalesces duplicates"]
    WQ --> REC["reconcile:  
read desired and actual state  
compute diff, act"]
    REC -->| "writes via API server" | AS
    AS -->| "410 Gone after compaction" | INF
    INF -->| "relist and rewatch" | AS
```

Controller replicas coordinate leadership through the API itself: a `Lease` object (`coordination.k8s.io/v1`) holding `holderIdentity` and a duration; the leader renews it, and rivals take over when renewal lapses (controller-manager defaults: 15 s lease, 10 s renew deadline, 2 s retry). Note what this is: a *timing-based advisory* election — the client-go leadelection package's own documentation warns it does not guarantee only one client acts as leader. Kubernetes tolerates that because reconciliation is idempotent and convergent, so a brief two-leaders episode wastes work rather than corrupting state. The general rule falls out: an unfenced election is fine when the actions are idempotent; anything else needs the previous section.

Operational realities

Quorum sizing. The arithmetic is Chapter 6's, unchanged: 3 members tolerate 1 failure, 5 tolerate 2; even sizes are strictly worse (4 members still tolerate only 1, with a larger quorum and one more machine to fail). Run 3 for most workloads, 5 to survive a failure *during* maintenance or a zone loss; almost never more, since every write pays the majority round-trip and the fsync floor. Spread members across failure domains, remembering from Chapter 6 that geo-spreading a quorum moves the WAN into your write latency.

The failure amplifier. Because everything coordinates through it, the service's outage is a fleet-wide coordination outage: no lock handoffs, no failovers, no membership changes, control planes read-only or down. Two disciplines follow. *Consumers* must degrade to last-known state and keep serving — the level-triggered pattern above; a service that halts because it cannot renew a lease has chosen the wrong failure mode for most purposes. *Operators* must protect the service's headroom, which mostly means saying no: coordination stores are for coordination-scale data — identities, endpoints, leases, small configs — at low write rates. The moment application data or high-frequency state (per-request counters, queue payloads, metrics) lands there, you have coupled your most critical dependency's capacity to your least disciplined workload. etcd enforces some of this mechanically: ~1.5 MiB default max request size and a backend quota (2 GiB default, 8 GiB the advised ceiling) beyond which the cluster raises a `NOSPACE` alarm and refuses writes until compacted, defragmented, and disarmed.

Watch fan-out. Watches invert write cost: one write to a key watched by 10,000 clients is 10,000 notifications, and a popular prefix can turn a modest write rate into an outbound-bandwidth and CPU problem. Worse is the *recovery herd*: a mass reconnect (network blip, rolling restart) triggers simultaneous relists from thousands of clients — the expensive path. Mitigations: aggregate watchers behind a caching tier (the Kubernetes API server's watch cache exists precisely to absorb this for etcd), watch narrow prefixes, jitter reconnects, and keep watched values small since events carry them.

Session and lease timeout tuning. The timeout is a failure-detector parameter (Chapter 10 treats the dilemma formally): too tight, and an ordinary stop-the-world GC pause or CPU-throttled container misses its heartbeats, the ensemble declares a healthy client dead, ephemerals vanish, and leadership flaps — false failover being genuinely dangerous, since it manufactures the two-believers scenario at higher frequency; too loose, and real failures take the whole timeout to detect, which is your failover time. Practical notes:

Session / lease timeout tuning

- Floor: > worst observed GC pause + heartbeat interval + one network RTT,
with margin. Measure pauses (GC logs) before choosing; do not guess.
- Ceiling: your failover-time budget. An ephemeral-based election cannot fail over faster than the session timeout.
- ZooKeeper: timeout negotiated into $[2, 20] \times \text{tickTime}$ (default tick 2 s
=> 4 - 40 s). The 30-40 s range is common for leader locks; sub-10 s demands a well-tuned client runtime.
- etcd: lease TTL per lease; keep-alive interval well under TTL/3. Kubernetes' 15 s / 10 s / 2 s defaults are a sane reference point.
- Remember the coordination service has its OWN failure detector: etcd heartbeat-interval 100 ms / election-timeout 1000 ms defaults assume a LAN; widen them for WAN or noisy disks, or the cluster itself flaps.
- No timeout eliminates the pause problem. Timeouts bound detection time;
fencing provides safety. Tune the former, never omit the latter.

Compaction and defragmentation (etcd). MVCC history grows without bound until compacted; run auto-compaction (`--auto-compaction-retention`, periodic or revision mode) sized to how far back your watchers may need to resume — compacting too aggressively converts

routine reconnects into relist storms. Compaction frees logical but not file space; periodic `etcdctl defrag` (member by member — it briefly blocks that member) reclaims it and keeps you clear of the quota. Take scheduled `etcdctl snapshot save` backups; a store that everything depends on and nobody can restore is an outage with a delay timer. ZooKeeper's equivalent hygiene is `snapshot/txn-log` purging via `autopurge.*`.

Observability. The metrics that predict trouble, roughly in order: **leader changes** (`etcd_server_leader_changes_seen_total`; ZooKeeper leader elections) — a healthy cluster elects almost never, and flapping means overloaded disks or network; **fsync and commit latency** (`etcd_disk_wal_fsync_duration_seconds`, `etcd_disk_backend_commit_duration_seconds`) — consensus sits on the fsync floor (Chapter 6), so p99s here are the write-latency early warning; **proposal health** (`etcd_server_proposals_pending`, `proposals_failed_total`); **watcher and lease counts** for fan-out exposure; DB size versus quota; on the ZooKeeper side, outstanding requests and average latency from `mntr`. Alert on leader-change rate and fsync p99 first; they degrade first.

When not to use a coordination service

The failure amplifier argument cuts both ways: every use you *avoid* shrinks the blast radius. Decline the coordination service when:

- **The data path is high-throughput.** A majority-fsync per write and a few-GiB working set is the wrong engine for request-rate traffic. Coordination stores hold the *pointers* — who is primary, where the shards live — while the bytes flow elsewhere. If your coordination cluster's write rate scales with user traffic, the design is wrong.
- **You are building a queue.** Sequential znodes look temptingly like one; the result inherits `getChildren` scans over unbounded children, watch herds, and the 1 MiB payload ceiling. Use a queue (Volume 10); coordinate the queue's *consumers* here if needed.
- **Stale reads are acceptable.** Config that can lag by seconds, discovery that briefly tolerates a dead endpoint — DNS with TTLs, or any replicated cache, does this with none of the consensus tax. Paying linearizability costs for data you then cache client-side anyway is the most common oversubscription.

- **The lock only protects efficiency.** Vol 4, Ch 2's lock-avoidance ladder applies with more force here: if the worst case of two workers duplicating work is wasted compute, you want no lock, or an unfenced advisory one at most. Reserve the fenced apparatus for locks whose violation corrupts state.

The rule of thumb: a coordination service should hold data that is small, slow-changing, and worth a consensus round-trip *because it is load-bearing for correctness*. Everything else has a cheaper home.

The distributed-systems lens

Step back and the chapter compresses into three ideas.

These services are consensus with an ergonomic API — and the ergonomics are the product. ZAB and Raft supply the same commodity: a majority-committed total order of writes. ZooKeeper and etcd add a *vocabulary* for spending that order — trees or keyspaces, versions and revisions, conditional writes, notifications — so application programmers manipulate state instead of protocols. The reason to centralize was never that consensus cannot be embedded; etcd's own Raft library disproves that. It is that one hardened implementation, one operational practice, and one place to enforce capacity discipline beat N independent ones — at the price that the one becomes the most critical dependency you run.

Sessions and leases convert liveness into state transitions. A failure detector (Chapter 10) ordinarily outputs a *suspicion* — a boolean with error bars, consumed by whoever polls it. Sessions and leases materialize that suspicion as a *data change*: the ephemeral vanishes, the lease's keys are deleted, watchers are notified in the same total order as every other write. This is the deep trick of the genre — the timing-and-suspicion half of distributed systems becomes consumable through the same watch-and-read API as everything else. But materializing a suspicion does not make it true: the detector still cannot distinguish dead from slow, which is why every liveness-coupled grant needs a fencing token, enforced at the resource rather than the service.

The recipes are the distributed twins of Volume 4's primitives, with honest downgrades. The lock recipe is the mutex; the watch is the condition variable; a lease-bounded set of permit keys is the semaphore; the double barrier is the barrier. Each twin is weaker in the same three

ways: *advisory* (nothing forces access through the lock — Chubby's frank admission), *timeout-coupled* (ownership is a lease on someone else's clock, not a fact), and *fenced* (safety ultimately lives in the resource's monotonicity check). The condition-variable analogy carries its discipline with it: a watch event is a spurious-wakeup-prone `notify`, the re-read is the `while` loop, and level-triggered reconciliation is that `while` loop promoted to an architecture — the real Kubernetes design lesson. Under loss, coalescing, and reconnection, edge-triggered handling of change events is unmaintainable; systems that reconcile observed state against desired state are the ones that survive their coordination service's bad days.

Key takeaways

- A coordination service is **consensus productized**: ZAB inside ZooKeeper, Raft inside etcd, so applications get majority-committed, totally ordered state without implementing a protocol. The price: it becomes your most critical dependency — treat its blast radius as a design input.
- ZooKeeper = **znode tree + sessions + one-shot watches**. Ephemerals tie state to session liveness; sequentials give atomic ordering; versions give CAS. Writes are linearizable; default reads are locally served and can be stale — per-session ordering plus `sync()` for freshness.
- **Disconnected is not expired**. Only the ensemble can expire a session; a disconnected client must stop trusting its locks without assuming they are gone. That gap is the failure detector's uncertainty window, and no client code can close it.
- The correct lock/election recipe is **ephemeral sequential + watch your predecessor**: total order from sequence numbers, $O(1)$ handoff instead of a herd, self-healing via ephemerality — and a mandatory re-check loop, because a fired watch means "re-read", never "acquired".
- etcd = **MVCC revisions + leases + resumable range watches + If/Then/Else transactions**. Revisions give point-in-time reads, gapless watch resumption (up to compaction), and ready-made fencing tokens; a lock is lease + CAS on `create_revision` + watch, wearing an API.

- **Fencing is not optional** when a lock protects external state: lease expiry plus a paused client yields two believers, unavoidably. Use `czxid /version` (ZooKeeper) or `create_revision` (etcd) as a monotonic token, and have the **resource** reject lower tokens than it has seen.
- Chubby's lessons hold: **coarse-grained** locks, **advisory** semantics with sequencers, and a lock *service* because elections need their results advertised — plus the warning that clients will misuse the service unless constrained.
- Kubernetes shows the right consumption pattern: **list+watch from a revision plus level-triggered reconciliation**, so missed or coalesced events are harmless and etcd outages cause convergence delay, not corruption.
- Operate deliberately: **3 or 5 members** (even counts add cost, not tolerance), timeouts above worst-case GC pause but within the failover budget, compaction and defrag on a schedule, small values and low write rates, alerts on leader changes and fsync latency first.
- Don't use one for data paths, queues, stale-tolerant config, or efficiency-only locks — every avoided use shrinks the blast radius of the one dependency everything else shares.

Further reading

- Hunt, P., Konar, M., Junqueira, F., and Reed, B., "ZooKeeper: Wait-free coordination for Internet-scale systems," *USENIX ATC 2010* — the system paper: data model, guarantees, recipes, and the coordination-kernel philosophy.
- Burrows, M., "The Chubby lock service for loosely-coupled distributed systems," *OSDI 2006* — the design rationale for a lock service, coarse-grained locks, sequencers, and a candid account of how clients actually behave.
- Junqueira, F., Reed, B., and Serafini, M., "Zab: High-performance broadcast for primary-backup systems," *DSN 2011* — the ZAB protocol paper.
- Kleppmann, M., "How to do distributed locking" (2016) — the fencing-token argument this chapter's central section builds on. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>

- etcd documentation — API guarantees, KV/Watch/Lease APIs, and the operations guide (compaction, defragmentation, quotas, metrics). <https://etcd.io/docs/>
- Apache ZooKeeper documentation — the programmer's guide (sessions, watches, consistency guarantees) and the recipes page. <https://zookeeper.apache.org/doc/current/>
- Kubernetes documentation — "Operating etcd clusters for Kubernetes" and the API concepts page covering resourceVersion, watch semantics, and the list+watch pattern. <https://kubernetes.io/docs/tasks/administer-cluster/configure-upgrade-etcd/>
- Ongaro, D. and Ousterhout, J., "In Search of an Understandable Consensus Algorithm," *USENIX ATC 2014* — the protocol under etcd; read alongside Chapter 6.
- Volume 4, Chapter 2 — Threads, Mutual Exclusion, and Locks — the in-process primitives whose distributed twins this chapter derives, and the original statement of the fencing problem.
- Chapter 10 — Failure Detection — the formal treatment of the timeout dilemma that session and lease tuning instantiates.

Chapter 9 — Idempotency, Deduplication, and Exactly-Once

What this chapter covers. Every reliability mechanism in a distributed system — TCP retransmission, HTTP client retries, message-broker redelivery, workflow re-execution — is a retry, and every retry can create a duplicate. This chapter explains why that is not an implementation weakness but a mathematical consequence of asynchronous networks with failures, and then builds the complete engineering response to it. The central claims are worth stating up front, because much vendor marketing depends on blurring them: **exactly-once delivery is impossible** in the presence of failures, while **exactly-once processing is achievable** — but only as an end-to-end application property, assembled from at-least-once delivery plus idempotent effects. Nothing in the middle of the stack can give it to you; every hop that touches state must supply its own discipline.

We start from the root cause — the sender's irreducible ambiguity when an acknowledgment fails to arrive, the same Two Generals problem from Chapter 1 — and derive the delivery- semantics taxonomy precisely. We then define idempotency formally and practically, and develop the single most important design move in this space: converting non-idempotent operations into idempotent ones by *naming the effect*. From there the chapter is concrete: the idempotency-key protocol end to end (client discipline, the server-side atomic check-and-record algorithm, storage choices, Stripe's public design), deduplication at each layer of a real stack (TCP, Kafka, SQS, and the consumer-side table that is the reliable backstop), a full payment-flow walkthrough showing that the chain is only as idempotent as its weakest link, and the retry hygiene and state-machine patterns that make the whole thing safe to operate. We close by connecting idempotency back to Volume 4's compare-and-swap and to the consensus machinery of Chapter 6.

Learning goals — after this chapter you should be able to:

- State precisely why a sender that receives no acknowledgment cannot distinguish a lost request from a lost ack from a slow response, and why this forces at-least-once semantics on any system that retries.
- Define at-most-once, at-least-once, and exactly-once processing rigorously, state what each costs, and choose correctly for a given workload.
- Explain the end-to-end argument and why exactly-once processing cannot be delegated to a transport or broker.
- Convert a non-idempotent operation into an idempotent one by naming the effect, and explain why "apply payment P123" differs fundamentally from "add \$50".
- Implement the idempotency-key pattern correctly: the atomicity requirement, the broken check-then-act version and its race, response replay, and in-progress duplicate handling.
- Describe what Kafka's idempotent producer and SQS FIFO deduplication actually guarantee — and, more importantly, what they do not.
- Build the consumer-side dedupe table pattern with correct offset-commit ordering.

- Apply retry hygiene: HTTP method semantics, idempotency keys on unsafe methods, backoff with jitter, and retry budgets.
- Explain how Raft-based systems achieve exactly-once command application via client sessions, and why the dedupe table is a distributed compare-and-swap on operation identity.

The root cause: silence is ambiguous

Consider the smallest possible distributed interaction: a client sends a request to a server and waits for a response. The request is "charge this card \$50." The client waits, and the response does not come. What happened?

Exactly one of three things, and the client cannot tell which:

1. **The request was lost.** The server never saw it. The charge did not happen.
2. **The response was lost.** The server processed the charge, debited the card, and sent an acknowledgment that died in the network. The charge *did* happen.
3. **Everything is merely slow.** The request is queued, or the server is garbage-collecting, or the response is in flight. The charge may happen at any moment, or may already have.

These three situations are **observationally identical** from the client's side: silence. No amount of waiting resolves the ambiguity, because case 3 has no upper bound in an asynchronous network — a response could always arrive one second after any deadline you pick. This is the Two Generals problem from Chapter 1 wearing work clothes: two parties communicating over a lossy channel cannot reach common knowledge that a message was delivered, no matter how many acknowledgments and acknowledgments-of-acknowledgments they exchange. The last message sent is always unacknowledged, so someone is always uncertain.

```
sequenceDiagram
    participant C as Client
    participant N as Network
    participant S as Server

    C->>N: request: charge card, attempt 1
    N->>S: delivered
    S->>S: charge applied to ledger
    S->>N: response: success
    Note over N: response LOST
    Note over C: timeout fires.
    Lost request? Lost response?
    Just slow? Indistinguishable.
    C->>N: request: charge card, attempt 2 – RETRY
    N->>S: delivered
    Note over S: without dedup: card
    charged a SECOND time
```

Faced with this ambiguity, the client has exactly two options, and they define the two implementable delivery semantics:

- **Give up.** Report failure or uncertainty upward and never resend. If the request was lost, the operation never happens. This is **at-most-once**: zero or one executions.
- **Retry.** Resend the request until an acknowledgment arrives. If the original was actually processed (case 2), the server now receives it twice. This is **at-least-once**: one or more executions.

There is no third option. "Exactly-once delivery" would require the client to retry *only when the request was truly lost* — which requires distinguishing case 1 from case 2, which is precisely what cannot be done. Any protocol claiming exactly-once delivery over a lossy network is either quietly at-most-once (it can lose messages), quietly at-least-once (it can duplicate them, and something downstream deduplicates), or quietly assuming a failure-free network. This is not a theoretical nicety. It is the reason your payment service double-charged a customer during last quarter's network blip, and it is the reason every serious messaging system documents itself as at-least-once and pushes the remaining work to you.

The inescapable conclusion, and the thesis of this chapter: **reliability requires retries, retries create duplicates, and therefore the only end-to-end contract you can actually build is at-least-once delivery**

plus deduplication. Everything that follows is technique for making that contract cheap and safe.

Delivery semantics, precisely

The three terms get used loosely; here are the rigorous definitions. For a given logical operation sent once by the application:

Semantics	Executions	Mechanism	What it costs	Where it is right
At-most-once	0 or 1	Send, never retry. Fire-and-forget.	Silent loss under failure. No duplicate risk, no state.	Metrics, logs, telemetry, cache invalidation hints — anything where a gap is cheaper than the machinery.
At-least-once	1 or more	Retry until acknowledged. Durable send buffers, broker redelivery.	Duplicates under failure. Receiver must tolerate or deduplicate.	The default for anything that matters. The transport layer of every serious system.
Exactly-once processing	Effects of exactly 1	At-least-once delivery plus idempotent or deduplicated processing at every state-touching hop.	Dedup state, atomicity discipline at each hop, bounded windows to manage.	Money, orders, inventory, anything where duplicate effects are incidents.

Note the deliberate wording in the third row: **exactly-once processing**, sometimes called *effectively-once*. Messages may arrive twice; the *effect* happens once. The distinction between delivery and processing is the load-bearing wall of this whole topic:

Delivery is an event at the transport: bytes arriving at a process. **Processing** is a state change in the application: a row written, a balance debited, an email sent. Exactly-once *delivery* is impossible. Exactly-once *processing* is an application property you can build, because the application — unlike the transport — can recognize an operation it has already performed and decline to perform it again.

This is a direct instance of the **end-to-end argument** of Saltzer, Reed, and Clark (1984): a function that requires knowledge available only at the endpoints cannot be completely implemented in the middle of the stack. The transport does not know what an "operation" is — it moves bytes. Only the application knows that two byte-sequences arriving five seconds apart are the same logical payment, and only the application can consult its own state to see whether that payment has already been applied. Lampson makes the same point in "Hints for Computer System Design": lower layers should provide performance, not attempted perfection, because the ends must implement end-to-end correctness anyway. A broker that worked heroically to suppress duplicates would still not save you — the producer's application-level retry (a user clicking twice, a crashed service replaying its outbox) enters the broker as a *new message* the broker has no way to recognize. Dedup in the middle is an optimization; dedup at the ends is the guarantee.

One consequence worth internalizing: the phrase "exactly-once" on a product datasheet is always a claim about a *bounded scope*. Kafka's "exactly-once semantics" is exactly-once within a Kafka-to-Kafka pipeline under its transaction protocol. It is a real and useful guarantee — and the moment your consumer writes to Postgres or calls Stripe, you have left its scope and are back to building effectively-once yourself. We return to this in the layer-by-layer section.

Idempotency: the property that makes duplicates harmless

Deduplication is one response to duplicates: detect and drop them. **Idempotency** is the other, and the more elegant: make the operation such that performing it twice has the same effect as performing it once, and duplicates become harmless without being detected.

Formally, an operation `f` is idempotent when `f(f(x)) = f(x)` — applying it to its own result changes nothing. For our purposes the useful reading is over system state: an operation is idempotent if executing it $N \geq 1$ times leaves the system in the same state (and, ideally, returns the same answer) as executing it once.

Some operations have this property naturally:

- **Absolute assignment.** `SET balance = 100`, `PUT /users/42` with a full representation,
`UPDATE orders SET status = 'shipped' WHERE id = 7`. Setting a value to `v` twice yields `v`.
- **Deletion by identity.** `DELETE FROM sessions WHERE id = 'abc'`. The second execution deletes zero rows; the state is the same. (The *response* may differ — 404 versus 200 — which matters for the caller but not for the state. Design the response to be stable too where you can: "the thing is gone" is true both times.)
- **Transitions into absorbing states.** A state machine where `cancelled` has no outgoing edges: cancelling a cancelled order is a no-op by construction. Absorbing states are idempotency you get from your domain model, and a reason to design status fields as explicit state machines rather than free-form strings.
- **Set insertion.** Adding element `e` to a set that already contains `e` — the algebraic heart of the CRDTs in Chapter 11.

And some operations are naturally *not* idempotent — precisely the ones businesses care most about:

- **Relative updates.** `balance = balance + 50`, counters, `INSERT` of a new row per call, appending to a log.
- **Effects in the outside world.** Charge a card, send an email, ship a parcel. Doing it twice does it twice.

The essential design move — the one idea to take from this chapter if you take only one — is that **any non-idempotent operation can be made idempotent by naming the effect**. "Add \$50 to the balance" is not idempotent because the instruction carries no identity: two copies are indistinguishable from two intentional deposits. But "apply *payment P123*, a \$50 deposit" is idempotent, because P123 either has been applied or has not, and the system can check. The imperative *increment* has become a declarative *fact*, and facts can be recorded at most once:

```
-- Non-idempotent: two deliveries, two increments, corrupted balance.
UPDATE accounts SET balance = balance + 50 WHERE id = 42;

-- Idempotent: the effect is NAMED. The ledger row is the fact;
-- the unique constraint makes the fact insertable at most once.
INSERT INTO ledger (payment_id, account_id, amount)
VALUES ('P123', 42, 50)
ON CONFLICT (payment_id) DO NOTHING;
-- balance is derived from the ledger, or updated in the same
-- transaction ONLY when the insert actually inserted a row.
```

Pat Helland calls this pattern out in "Idempotence Is Not a Medical Condition": in a world of retried messages, the receiver must treat incoming requests as *statements about an operation with an identity*, not as anonymous commands. The table of named effects — a ledger keyed by payment ID, a dedupe table keyed by message ID, an idempotency-key table keyed by client-chosen key — is the same structure at every layer, and the rest of this chapter is that structure in different costumes. Everything downstream of this point is a variation on: *give every logical operation a unique name, record the name atomically with the effect, and refuse to repeat a name you have already recorded*.

Idempotency keys: the end-to-end protocol

The idempotency-key pattern is "name the effect" industrialized for request/response APIs. It has a client half and a server half, and both halves carry discipline that is easy to get subtly wrong.

The client's half: one key per logical operation, reused across retries

The client generates a unique key — a UUIDv4 is standard — for each **logical operation**, attaches it to the request, and, critically, **reuses the same key for every retry of that operation**. This reuse is the entire mechanism. A fresh key per attempt is indistinguishable from a fresh operation per attempt, and the server will dutifully execute each one; you have built elaborate machinery that dedupes nothing.

The subtle part is defining "logical operation," because it decides where the key is generated. If the user clicks *Pay* and your frontend calls your API, which calls the payment service: is the operation the click, the API call, or the internal call? The key must be minted at the level whose

duplicates you need to suppress. A key generated per-HTTP-call by an SDK's retry wrapper protects against network retries but not against the user double-clicking, because the second click gets a second key. Robust systems mint the key as early as the logical intent exists — often in the frontend when the payment form is rendered — and thread it through every hop. Keys should also be **scoped**: to an API key or account (so tenants cannot collide with or probe each other's keys) and to an endpoint or operation type (so a key used on `POST /charges` says nothing about `POST /refunds`).

Keys carry a **TTL** decision. The server must remember keys long enough to cover the longest plausible retry horizon — client-side retry loops, queued jobs, a mobile app coming back from a tunnel — but not forever, or the table grows without bound. Stripe retains keys for 24 hours and states so publicly; after expiry, a reused key is treated as new. Whatever window you pick, the crucial honesty is to *have* a stated window and to know what falls outside it. Every dedup window in this chapter is finite; we will keep meeting this caveat.

The server's half: check and record must be one atomic action

The server's algorithm sounds trivial: if you have seen this key, return the stored result; otherwise execute the operation and remember key plus result. Here is the version every team writes first, and it is broken:

```
# BROKEN – check-then-act race. Do not deploy.
def handle(request):
    existing = db.query("SELECT response FROM idempotency_keys WHERE
key = %s",
                        request.idempotency_key)           # (1) check
    if existing:
        return existing.response
    result = perform_charge(request)                       # (2) effect
    db.execute("INSERT INTO idempotency_keys ...")         # (3) record
    return result
```

Two copies of the same request — a timeout-triggered retry racing the slow original, or a double-click fanned out across two app servers — both execute (1) before either executes (3). Both find nothing. Both perform the charge. This is exactly the check-then-act race of Volume 4, Chapter 1, relocated from two threads sharing memory to two requests sharing a database, and the fix is the same shape: the check and the act must be

one atomic step, here a single database transaction with a uniqueness guarantee doing the work a CAS did in shared memory.

The correct structure claims the key *first*, inside the same transaction that will perform the effect, and lets the database's unique constraint arbitrate the race:

```
CREATE TABLE idempotency_keys (
  key          text          NOT NULL,
  account_id   bigint        NOT NULL,
  endpoint      text          NOT NULL,          -- key is scoped
  request_hash text          NOT NULL,          -- detect key reuse
  with different body
  status        text          NOT NULL DEFAULT 'in_progress',
                                   -- 'in_progress' | 'succeeded' |
  'failed'
  response_code int,
  response_body jsonb,
  created_at    timestampz NOT NULL DEFAULT now(),
  PRIMARY KEY (account_id, endpoint, key)
);
```

```

def handle(request):
    with db.transaction() as tx:
        # (1) Atomically claim the key. Exactly one concurrent request
        wins.
        claimed = tx.execute("""
            INSERT INTO idempotency_keys (key, account_id, endpoint,
            request_hash)
            VALUES (%s, %s, %s, %s)
            ON CONFLICT (account_id, endpoint, key) DO NOTHING
            RETURNING key
            """, request.key, request.account, request.endpoint, hash(request.body))

        if not claimed:
            row = tx.query("SELECT * FROM idempotency_keys WHERE ...")
            if row.request_hash != hash(request.body):
                return error(422, "idempotency key reused with
different request")
            if row.status == 'in_progress':
                return error(409, "original request still processing;
retry later")
            return (row.response_code, row.response_body) # (2)
            replay stored result

            # (3) Effect and key-completion in the SAME transaction.
            result = perform_charge(tx, request) # writes
            ledger rows via tx
            tx.execute("""
                UPDATE idempotency_keys
                SET status = 'succeeded', response_code =
%s, response_body = %s
                WHERE ...
            """, result.code, result.body)
            return result

```

Three properties of this structure deserve explicit attention.

The effect and the key-record commit together or not at all.

Because `perform_charge` writes through the same transaction, a crash after the charge's rows are written but "before" the key is marked cannot occur — they are one commit. If the process dies mid-transaction, everything rolls back: no charge, no completed key, and the client's retry starts clean. Recording the key in a *different* store than the effect reopens a window: crash between the two writes and you have either a charge with no key (retry double-charges) or a key with no charge (retry is wrongly suppressed and the operation is silently lost — arguably worse). This is why **storing idempotency state in the same database**

as the effect is the default choice: atomicity is free. It is the same insight that motivates the transactional outbox of Volume 5, Chapter 10 — there, "publish this event" is made atomic with the business write by making the event a row in the same database; here, "remember this key" is made atomic the same way. Outbox and idempotency table are siblings: both smuggle a second concern into the one commit you can actually make atomic.

Duplicates get the stored response, not a re-execution. The table stores the response code and body, and a duplicate — arriving seconds or hours later — receives a byte-replay of the original outcome, including original failures. This is what makes retries *safe from the client's perspective*: retrying cannot make the world different, only reveal what already happened. Replaying stored failures is a design decision with nuance: Stripe, for example, stores and replays outcomes, but a validation error that consumed the key would trap the client, so implementations commonly release or overwrite keys for request-invalid errors while pinning them for anything that might have had effects.

In-progress duplicates need an answer. The awkward case is a duplicate arriving while the original is still executing. There is no stored response to replay and re-executing is forbidden. Three honest options: **block** the duplicate until the original commits (simple; ties up a connection; in Postgres you get this by making the losing `INSERT` wait on the winner's row lock — though the idiomatic `ON CONFLICT DO NOTHING` shown above returns immediately instead); **reject** with `409 Conflict` and let the client retry after a delay (Stripe's documented behavior); or **return 202 with a polling location**. Rejecting with 409 is the most common and composes best with standard retry loops.

Redis as the key store: honest caveats

Teams reach for Redis to keep idempotency state out of the primary database: `SET key value NX PX <ttl>` is an atomic claim, fast, with built-in expiry. Two caveats must be stated plainly. First, **you have reintroduced the two-store atomicity gap**: the Redis claim and the database effect are separate operations, and a crash between them leaves a claimed key with no effect (lost operation) — so you need reconciliation, an in-progress state with a short TTL, or acceptance of that failure mode. Second, **Redis durability is configurable and by default is not a database's**: with asynchronous replication and default

persistence, a failover can lose recent writes, i.e., forget recently-claimed keys, i.e., let a duplicate through. For rate-limiter-grade dedup — where a rare duplicate is tolerable — Redis is a fine, cheap choice. For money, put the keys next to the money.

Stripe's design, as the canonical public example

Stripe's API is the reference implementation most engineers have touched. At the concept level, accurately: clients send an `Idempotency-Key` header on `POST` requests, with a recommended random UUID per logical operation, reused on retry. Stripe records the key with the *result* of the first request — status code and body — and replays that stored response for any subsequent request with the same key, whether the first attempt succeeded or failed. Keys are scoped to the account and expire after 24 hours. Reusing a key with a different request body is an error, which is why the `request_hash` column appears in the DDL above — a client bug that reuses keys across different operations should be loudly rejected, not quietly given someone else's cached response. A duplicate arriving while the original is in flight receives an error rather than blocking. The wire-level exchange:

```
POST /v1/charges HTTP/1.1
Host: api.stripe.com
Authorization: Bearer sk_live_...
Idempotency-Key: 8b2e4d9c-1f5a-4c47-9b3e-2a6f8e0d7c11
Content-Type: application/x-www-form-urlencoded

amount=5000&currency=usd&source=tok_visa

HTTP/1.1 200 OK
Idempotency-Key: 8b2e4d9c-1f5a-4c47-9b3e-2a6f8e0d7c11

{ "id": "ch_3Nx...", "amount": 5000, "status": "succeeded", ... }
```

The client times out, retries with the **same header**, and receives the same `200` and the same `ch_3Nx...` body — replayed from the key store, no second charge. The whole protocol, drawn once:

```

flowchart TB
    A["Request arrives with  
Idempotency-Key K"] --> B["BEGIN transaction"]
    B --> C["INSERT key K  
unique constraint –  
did we win?"]
    C --> D["yes: first time"]
    C --> E["no: key exists"]
    D --> F["Perform effect:  
write ledger rows  
in SAME transaction"]
    E --> G["Store response on key row  
status = succeeded"]
    F --> H["COMMIT – effect and key  
are one atomic unit"]
    G --> I["Return response"]
    H --> J["no: key exists"]
    H --> K["succeeded or failed"]
    J --> L["stored status?"]
    K --> M["Replay stored  
response verbatim"]
    L --> N["in_progress"]
    M --> O["409 Conflict –  
original still running,  
client retries later"]
    N --> P["different request_hash"]
    O --> Q["422 – key reused  
for a different request"]

```

Deduplication layer by layer: what each hop actually gives you

A real request crosses many hops, and several of them advertise deduplication. The honest inventory matters, because the recurring failure mode is trusting a layer's dedup beyond its actual scope.

TCP deduplicates and orders segments *within a single connection* via sequence numbers — a retransmitted segment is recognized and dropped (Volume 3). The guarantee evaporates at connection boundaries, which is exactly where application retries live: your HTTP client timed out, *closed the connection*, and resent on a new one. TCP sees two unrelated byte streams. Transport dedup is real and it is not the dedup you need — the end-to-end argument's canonical illustration.

Kafka's idempotent producer (on by default since Kafka 3.0) assigns each producer a producer ID and a per-partition monotonically increasing sequence number to each batch; the broker tracks the highest sequences per producer-partition and drops duplicates that arise from the *producer's internal retries* — a batch acked but whose ack was lost, then resent. Read the scope precisely: it dedupes **broker-side retry**

duplicates from a single producer session to a single partition. It does nothing about application-level duplicates: if your service crashes after `send()` succeeds but before recording that fact, restarts, and sends the "same" event again, that is a new message from (typically) a new producer session, and the broker cannot know otherwise. Kafka's **transactional producer** extends this: with a stable `transactional.id`, a producer can atomically commit a batch of writes to multiple partitions *together with its consumer offsets*, giving genuine exactly-once for **consume-transform-produce pipelines whose input and output are both Kafka** — a consumer under `read_committed` never sees uncommitted or aborted results, and reprocessing after a crash cannot double-emit. This is a real achievement (KIP-98). Its boundary is Kafka itself: the moment the transform writes to an external database or calls an external API, that side effect is outside the transaction and can happen twice. The mechanics — epochs, zombie fencing, the transaction coordinator — belong to Volume 10, Chapter 3; here you need only the scope.

SQS FIFO queues deduplicate on a `MessageDeduplicationId` (explicit, or a SHA-256 of the body with content-based dedup enabled) — but only within a **five-minute window**. A producer that retries within five minutes is deduped; an outbox relay that crashes and replays a send six minutes later produces a delivered duplicate. This limitation generalizes and deserves promotion to a principle: **every dedup window is finite**. TCP's window is the connection. Kafka's is the producer session and the broker's per-producer sequence state. SQS's is five minutes. Your idempotency-key table's is its TTL. Each layer forgets; plan explicitly for the duplicate that arrives after the layer in front of you has forgotten — which is why the *last* hop before the effect must dedupe against durable state whose window is as long as the effect matters.

Consumer-side deduplication is that last hop, and it is the reliable backstop of the whole architecture. The pattern: a `processed_messages` table keyed by the message's logical operation ID (the *named effect* again — an ID minted at the source and carried in the payload, not a broker offset, so it survives re-partitioning and replays), written **in the same transaction as the effect**:

```

def on_message(msg):
    op_id = msg.value["operation_id"]          # named effect, minted at
    the source
    with db.transaction() as tx:
        inserted = tx.execute("""
            INSERT INTO processed_messages (operation_id)
            VALUES (%s) ON CONFLICT DO NOTHING RETURNING operation_id
            """, op_id)
        if inserted:
            apply_effect(tx, msg.value)        # same transaction: dedup
+ effect commit together
            # duplicate: transaction commits having changed nothing
            consumer.commit_offsets(msg)       # (4) AFTER the
transaction commits

```

The offset commit at (4) is deliberately last, and the ordering is the whole point. Committing offsets *before* the effect commits is at-most-once: crash between them and the message is consumed-but-unprocessed, gone forever. Committing *after* is at-least-once: crash between the database commit and the offset commit and the broker redelivers a message you already processed — which is precisely the redelivery window the dedupe table exists to absorb. The insert hits the conflict, the transaction is a no-op, the offset gets committed on the second pass. At-least-once delivery below, idempotent processing above: effectively-once, built by hand, from parts you can see.

Choosing semantics: what does a duplicate cost, what does a loss cost?

The taxonomy becomes a decision by pricing two failure modes. **What does losing one message cost? What does applying one message twice cost?** At-most-once is right when loss is cheap and machinery is not: a dropped metrics datapoint costs nothing measurable, and nobody builds dedupe tables for gauge samples. At-least-once alone (no dedup) is right when duplicates are naturally harmless — the effect is a SET, a delete, an absorbing-state transition. Full effectively-once — at-least-once plus keys and dedupe tables — is mandatory when duplicates are incidents: charges, orders, inventory movements, anything a finance team reconciles. Metrics can drop; money cannot; and the common estimating error is assessing the *message* rather than the *effect*: an innocuous-looking "user signed up" event that triggers a welcome credit is a money message.

```

flowchart TB
    A{"Is losing an occasional  
message acceptable?"} -->|"yes"| B["AT-MOST-ONCE  
fire and forget  
metrics, telemetry, hints"]
    A -->|"no"| C["You need retries:  
AT-LEAST-ONCE delivery  
duplicates WILL occur"]
    C --> D{"Is the effect naturally  
idempotent – SET, delete,  
absorbing state?"}
    D -->|"yes"| E["At-least-once is enough.  
Verify idempotency with  
duplicate-injection tests"]
    D -->|"no"| F{"Can you name the effect  
with a unique operation ID?"}
    F -->|"yes"| G["EFFECTIVELY-ONCE:  
idempotency keys plus  
transactional dedupe table  
at every state-touching hop"]
    F -->|"no"| H["Redesign until you can.  
An unnameable effect  
cannot be made safe to retry"]

```

The last box is not a joke. If you cannot attach an identity to an operation, you cannot distinguish a duplicate from a legitimate repeat, even in principle — no infrastructure can rescue a protocol in which "add \$50" twice-on-purpose and "add \$50" twice-by-retry are the same bytes. The fix is always upstream, in the protocol: make the client say which \$50 this is.

The full chain: a payment, end to end

Assemble the pieces into the flow you will actually build. A user pays for an order; the API records it and returns quickly; a consumer fulfills the charge against a payment provider asynchronously.

```

flowchart LR
    U["Browser  
mints key K at  
form render;  
reuses K on retry"]
    A["API service  
idempotency table:  
claim K + write order  
+ outbox row in  
ONE transaction"]
    O["Outbox relay  
publishes event  
op_id = K;  
at-least-once,  
may re-send"]
    B["Broker  
at-least-once;  
producer dedup covers  
broker retries only"]
    C["Consumer  
processed_messages  
insert + effect in  
ONE transaction;  
offset commit after"]
    P["Payment provider  
provider idempotency  
key derived from K:  
charge applied once"]

    U -->|"POST with  
Idempotency-Key K"| A
    A -->|"outbox poll"| O
    O -->|"publish"| B
    B -->|"deliver, possibly  
more than once"| C
    C -->|"charge with  
Idempotency-Key"| P

```

Walk the hops and name each one's discipline. The **browser** mints key **K** when the payment form renders, so a double-click and a timeout-retry carry the same key. The **API** claims **K** in its idempotency table, writes the order, and writes an outbox row — one transaction, so the order and the promise-to-publish are atomic (Volume 5, Chapter 10). The **outbox relay** is itself a retrier — it publishes and only then marks the row sent, so a crash in between re-publishes: at-least-once, by design, and the event carries **K** as its operation ID for exactly this reason. The **broker** delivers at-least-once; its producer-side dedup absorbs its own retry duplicates and nothing more. The **consumer** dedupes on the operation

ID transactionally with its effect and commits offsets last. The **provider call** — the one effect that lives outside all of your transactions — is protected the only way an external effect can be: by the provider's own idempotency key, derived deterministically from `K`, so a consumer crash straddling the call re-issues it with the same key and Stripe returns the stored charge instead of making a new one.

Six hops, six disciplines — and the sobering observation that motivates the diagram: **the chain is exactly as idempotent as its weakest link**. Skip the discipline at any single hop and duplicates flow through it: a fresh key per browser retry, an outbox write outside the order's transaction, a consumer that commits offsets first, a provider call without a key — any one of these reintroduces the double-charge that every other hop was built to prevent. There is no hop whose neighbors can cover for it; that, once more, is the end-to-end argument, now with a price tag.

Retry hygiene: the sender's half of the contract

Idempotency is the receiver's discipline; it has a sender's counterpart, because retries are also a *load* phenomenon, and because retrying the wrong thing is how duplicates are minted in the first place.

Never automatically retry a non-idempotent operation without an idempotency key. HTTP's method semantics, per RFC 9110 §9.2.2, encode which operations are safe to retry blind: GET, HEAD, OPTIONS, and TRACE are *safe* (read-only); PUT and DELETE are *idempotent* (multiple identical requests have the same effect as one — which is why PUT carries a full representation rather than a delta); POST and PATCH carry **no** idempotency guarantee. Proxies, load balancers, and HTTP client libraries take these semantics seriously — many will transparently retry an idempotent method on connection failure and refuse to retry POST — and so must your retry middleware: a config flag that retries POSTs "for resilience," without keys, is a duplicate generator with good intentions. The `Idempotency-Key` header is the escape hatch that makes POST retryable; it is Stripe/PayPal/Square-style industry practice, and the subject of an IETF Internet-Draft in the HTTPAPI working group (`draft-ietf-httpapi-idempotency-key-header`) — which, to state its status honestly, remains a draft and not a published RFC as of this writing, so the header's semantics are convention, standardized per-API rather than by the protocol. API-design implications — key requirements per endpoint, documenting replay behavior — are Volume 8's territory.

Back off, jitter, and budget. A retry is deferred load: when a service browns out, every client's timeout fires together, and synchronized retries arrive as a wave that ensures the service never recovers — the retry storm, the distributed cousin of the livelock of Volume 4, Chapter 5, treated operationally in Volume 11. The standard discipline: exponential backoff (double the delay per attempt, capped), full jitter (randomize each delay over its whole range, decorrelating the herd), and a **retry budget** — cap retries as a fraction of total traffic per client (3 attempts is a policy; so is "retries may not exceed 10% of requests") so that a hard-down dependency degrades your traffic rather than multiplying it. Retries also multiply *across layers*: three attempts at the HTTP client inside three attempts at the job runner inside a browser user hammering refresh is twenty-seven-plus executions of one intent. Retry at one well-chosen layer; the layers above should surface failure, not silently multiply.

Hedging is deliberate duplication. Tail-latency hedging — send to a second replica when the first is slow, take the first answer — differs from timeout-retry only in not waiting. Both create in-flight duplicates on purpose; hedge only reads or keyed operations, and treat "the loser's response arrives anyway" as normal, not exceptional. Timeout-then-retry has one more trap worth naming: the timeout does not *cancel* the original request. The server may still be executing it; your retry races your original. This is why the in-progress arm of the idempotency algorithm exists, and why "just make the timeout longer" is sometimes the correct dedup fix.

State machines and re-entrancy: idempotency for multi-step work

Sagas and workflow engines (Volume 5, Chapter 10) add a wrinkle: the unit that retries is not a single request but a *step* in a long-running process, and the executor guarantees each step runs at-least-once — after a crash, the step re-runs from the top. Steps must therefore be **re-entrant**: written to be entered again after partial execution.

The core pattern is **check current state, then act** — where, unlike the broken check-then-act of the idempotency section, the check-and-transition is made atomic with a guard on the transition itself:


```
-- Step: move order to 'charged' and record the charge.
-- Guarded transition: only fires from the expected prior state.
UPDATE orders
SET status = 'charged', charge_id = 'ch_3Nx...'
WHERE id = 42 AND status = 'payment_pending';
-- 0 rows updated => step already ran (or state moved on): do nothing,
succeed.
```

The `WHERE status = 'payment_pending'` clause is a conditional write — a compare-and-swap on the state column, in exactly the sense of Volume 4, Chapter 4 — and it makes the transition idempotent: the second execution finds the precondition false and changes nothing. Explicit state machines with guarded transitions turn "this step may re-run" from a hazard into a non-event, and absorbing terminal states (`completed`, `cancelled`) make whole tails of the workflow trivially re-entrant. Where steps run on multiple workers with possible overlap — a presumed-dead worker limps back and resumes its step — guards should also carry a **version or fencing token** (Volume 4, Chapter 8; Chapter 8 of this volume): `WHERE version = 17` or `WHERE fencing_token <= 42`, so a stale executor's late write hits a false precondition instead of clobbering newer state. Guarded transition, named effect, claimed key: by now you can see these are one idea — *make the write conditional on the world still being in the state your decision assumed*.

Testing: duplicates are an input, not an anomaly

If at-least-once is the contract, duplicate delivery is a *normal input* to your system, and untested code paths handle it the way untested code paths handle everything. The property to assert is crisp — **for every logical operation, N deliveries ($N \geq 1$) produce exactly the effect of one** — and it is mechanically checkable: wrap your test harness's delivery step so every message is delivered twice (or a random 1–3 times, at random delays, interleaved with other traffic), and assert the same final state and ledger contents as the single-delivery run. Teams that flip this switch on an existing test suite usually learn something within the hour. Add the crash-shaped variants deliberately: redeliver *after* the effect committed but *before* the offset/ack was recorded (the redelivery window), and race two copies of the same request concurrently (the check-then-act window). Both are two-line fault injections in a harness and both find real bugs. Deterministic-simulation and jepsen-style

approaches that make such schedules reproducible are Chapter 12's subject; the habit that belongs in this chapter is smaller: no consumer or handler is done until its tests deliver everything twice.

The distributed-systems lens: recovering atomicity without shared memory

Step back and place this chapter in the arc of the suite, because its patterns are not new inventions — they are old friends from Volume 4, rebuilt for a world with partial failure.

The dedupe table is a distributed compare-and-swap on operation identity. In Volume 4, Chapter 4, atomicity came from hardware: CAS succeeds for exactly one contending thread, and the losers learn they lost. Across machines there is no shared cache line to CAS — but `INSERT ... ON CONFLICT DO NOTHING RETURNING` on a unique key is *precisely* a CAS: one winner, informed losers, arbitrated by the database's own internal concurrency control rather than a coherence protocol. Where shared-memory CAS compares a *named version* of a memory word, the idempotency insert compares a *named effect* against the set of effects already applied — versioned values there, named operations here, the same move. And the guarded transition of the previous section (`WHERE status = ...`, `WHERE version = ...`) is literally conditional-write CAS, unchanged. Volume 4 taught that atomic claim-then-act is how you make progress safe under concurrency; distributed systems lose the hardware primitive and re-derive it from uniqueness constraints and transactions. Idempotency is not a new discipline; it is *the* discipline, ported.

Consensus does not exempt you — Raft clients need this chapter too. It is tempting to think a replicated log (Chapter 6) solves duplication: entries are totally ordered and applied once per replica. But the *client* of a Raft-based system faces the same silence ambiguity as every other client — it proposed a command, the leader crashed after commit but before responding, and the retry (to the new leader) appends the *same command as a second entry*, which every replica will faithfully apply twice. Consensus guarantees each log entry is applied once; it does not guarantee your operation appears in the log once. The standard remedy, described in Ongaro's Raft dissertation, is exactly this chapter's pattern embedded in the state machine: clients attach a session ID and a

per-command serial number — a named effect — and the state machine keeps, *as part of its replicated state*, the latest serial and cached response per session, discarding duplicates and replaying stored answers. The dedupe table rides inside the state machine so that snapshots and leader changes preserve it. At-least-once proposal plus dedup-in-the- state-machine equals exactly-once application: the formula does not change even at the bottom of the stack, on top of the strongest primitive we have.

Reliability is an end-to-end property assembled from unreliable parts. This is the recurring moral of the volume, met first in Chapter 1: TCP builds ordered streams from duplicating datagrams; quorums build durable reads from failing nodes (Chapter 7); and effectively-once processing builds "it happened exactly once" from a network that cannot even promise "it happened." No layer hands the guarantee down; each layer contributes a mechanism, and the ends compose them. The engineer's version of the moral: when a vendor says exactly-once, ask *within what scope, over what window, atomic with which of my writes* — and when the answer runs out, that is where your dedupe table goes.

Key takeaways

- **Silence is ambiguous.** A sender with no acknowledgment cannot distinguish lost request, lost response, and slow response — the Two Generals frame. The only choices are give up (at-most-once) or retry (at-least-once). Exactly-once *delivery* is impossible.
- **Exactly-once processing is real but end-to-end:** at-least-once delivery plus idempotent or deduplicated effects at every state-touching hop. No transport or broker can supply it for you — the end-to-end argument, applied.
- **Name the effect.** "Apply payment P123," not "add \$50." A named operation can be recorded at most once; an anonymous imperative cannot even define what a duplicate is. Naturally idempotent shapes — SET, delete-by-id, absorbing states — need no further machinery; everything else gets an ID.
- **Idempotency keys work only with discipline on both ends:** the client reuses one key per logical operation across all retries; the server claims the key and performs the effect in **one transaction**,

stores the response, and replays it to duplicates. Check-then-act across two steps is a race — the claim must be the atomic first act.

- **Same store as the effect = free atomicity** — the outbox's sibling insight. Redis keys are fine for tolerable-duplicate dedup; for money, the keys live next to the money.
- **Every dedup window is finite:** TCP's connection, Kafka's producer session, SQS FIFO's five minutes, your key TTL. Kafka's idempotent producer dedupes only its own broker-side retries; its transactions are exactly-once only Kafka-to-Kafka. The durable consumer-side dedupe table, transactional with the effect, offsets committed after, is the reliable backstop.
- **Price both failure modes:** loss versus duplicate. Metrics can drop; money cannot. Judge the *effect* triggered, not the message's apparent importance.
- **The chain is as idempotent as its weakest link.** Client key, API table, outbox, broker, consumer table, provider key: every hop needs its own discipline, and no hop can compensate for a neighbor.
- **Retry hygiene is the sender's half:** never auto-retry POST without a key (RFC 9110 defines GET/PUT/DELETE idempotent; POST is not; the Idempotency-Key header is still an IETF draft), backoff with full jitter, retry budgets, retry at one layer, and remember a timeout does not cancel the original.
- **Steps re-run; write them re-entrant:** guarded transitions (`WHERE status = ...` , version/fencing predicates) are conditional-write CAS and make re-execution a no-op.
- **Test with duplicates as normal input:** the property is $N \text{ deliveries} \Rightarrow 1 \text{ effect}$; deliver everything twice in tests and inject the crash between effect-commit and ack.
- **The dedupe table is distributed CAS on operation identity**, and even Raft needs one: client sessions with serial numbers inside the replicated state machine are this chapter's pattern at the bottom of the stack.

Further reading

- Saltzer, J., Reed, D., and Clark, D., "End-to-End Arguments in System Design," *ACM TOCS* 2(4), 1984 — the argument this whole chapter

instantiates. <https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf>

- Lampson, B., "Hints for Computer System Design," *SOSP*, 1983 — including the case for end-to-end reliability with lower layers as optimization.
- Helland, P., "Idempotence Is Not a Medical Condition," *ACM Queue* 10(4), 2012 — the definitive treatment of messaging idempotency and naming effects. <https://queue.acm.org/detail.cfm?id=2187821>
- Leach, B., "Designing robust and predictable APIs with idempotency," Stripe blog, 2017 — and the Stripe API idempotency documentation. <https://stripe.com/blog/idempotency>, https://docs.stripe.com/api/idempotent_requests
- RFC 9110, *HTTP Semantics*, §9.2 — the definitions of safe and idempotent methods. <https://www.rfc-editor.org/rfc/rfc9110#section-9.2.2>
- *The Idempotency-Key HTTP Header Field*, IETF HTTPAPI working group Internet-Draft (draft-ietf-httpapi-idempotency-key-header) — note its status: a draft, not a published RFC. <https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>
- KIP-98, "Exactly Once Delivery and Transactional Messaging," and the Apache Kafka documentation on the idempotent producer (`enable.idempotence`) and transactions. <https://kafka.apache.org/documentation/>
- AWS documentation, *Exactly-once processing* and *Using the message deduplication ID*, Amazon SQS FIFO queues — the five-minute deduplication interval, stated plainly.
- Ongaro, D., *Consensus: Bridging Theory and Practice*, Stanford PhD dissertation, 2014, §6.3 — client sessions and duplicate command detection in Raft. <https://web.stanford.edu/~ouster/cgi-bin/papers/OngaroPhD.pdf>
- Treat, T., "You Cannot Have Exactly-Once Delivery," *Brave New Geek*, 2015 — a concise informal statement of the delivery/processing distinction. <https://bravenewgeek.com/you-cannot-have-exactly-once-delivery/>
- Gray, J., "Notes on Data Base Operating Systems," 1978 — the classic early treatment of message retries, duplicates, and transaction atomicity.

- Volume 5, Chapter 10 — sagas and the transactional outbox, the write-side sibling of the idempotency table. Volume 10, Chapters 2, 3, and 6 — delivery semantics, Kafka mechanics, and the outbox relay in depth. Volume 8 — idempotency keys as API-design surface. Volume 4, Chapters 1, 4, and 8 — check-then-act, CAS, and fencing, the shared-memory ancestors of everything here.

Chapter 10 — Failure Detection and Membership: Gossip and SWIM

What this chapter covers. Chapter 1 established the ambiguity that defines this volume: in an asynchronous system, a crashed node and a slow node are indistinguishable. Chapters 5 through 8 then built protocols that all quietly depend on answering exactly the question that ambiguity says cannot be answered: *who is alive right now?* This chapter confronts that dependency. We start from the impossibility and its consequence — every practical failure detector is a timeout plus a policy, trading false positives against detection latency — and the theory that makes the trade-off precise: Chandra and Toueg's unreliable failure detectors, and why "eventually accurate" is enough for consensus to make progress. We then work through the mechanics — heartbeats versus probes, the fragility of fixed timeouts, the phi accrual detector's continuous suspicion level — and the monitoring topologies that carry them, from all-to-all to gossip. Epidemic protocols get a full treatment: infection-style spread in $O(\log N)$ rounds, push versus pull, anti-entropy versus rumor-mongering, and the eventually-consistent membership views they produce. The centerpiece is SWIM, the protocol under HashiCorp's Serf, Consul, and Nomad: randomized probing, indirect probes that distinguish "my path is broken" from "the target is dead," suspicion with refutation by incarnation number, and dissemination piggybacked on the probes themselves. We close with the operational reality — gray failures, flapping, GC-pause false positives, correlated rack failures — and the synthesis: the detector's output is an input to every stateful protocol above it, so its error rate multiplies through the stack.

Learning goals — after this chapter you should be able to:

- Explain why perfect failure detection is impossible in an asynchronous system, connect that to FLP, and state what every practical detector therefore is: a timeout plus a policy.
- Define completeness and accuracy for unreliable failure detectors, place $\diamond P$ as the realistic target, and explain in one paragraph why eventual accuracy suffices for consensus liveness.
- Describe the phi accrual detector mechanically — the inter-arrival window, the suspicion level ϕ , and what a threshold of 8 actually means — and tune it with intent.
- Compare heartbeating topologies (all-to-all, centralized, hierarchical, gossip) by message load, detection latency, and failure modes of the detector itself.
- Walk the arithmetic of epidemic dissemination: why infection reaches N nodes in $O(\log N)$ rounds, and when push, pull, or push-pull is the right mode.
- Specify SWIM precisely: randomized probing, ping-req through k proxies, the suspect $\rightarrow$ refute-or-confirm state machine, incarnation numbers, and piggybacked dissemination — plus Lifeguard's local-health refinements.
- Distinguish membership *change* from failure *detection*, and gray failure from crash failure — and argue for application-level health signals with honest awareness of their cascade risks.
- Trace a false positive through the stack: from one late heartbeat to elections, lease churn, rebalancing storms, and orchestrator evictions.

The impossibility, and what a detector actually is

Recall the shape of the problem from Chapter 1. Node A messages node B and hears nothing back. At least four worlds are consistent with that observation: B crashed before receiving; B received and crashed before replying; B is alive but slow — a GC pause, a saturated run queue, a full accept backlog; or B is fine and the network delayed or dropped a message in either direction. In an asynchronous system — no bound on message delay or relative processing speed — these worlds are *observationally identical*. No finite wait resolves them, because any finite wait is consistent with "the reply is still coming."

This is not a corollary of FLP; it is closer to being its engine. The Fischer-Lynch-Paterson result (1985; Chapters 1 and 5) says no deterministic consensus protocol can guarantee termination in an asynchronous system where even one process may crash, and the heart of the proof is exactly this ambiguity: the adversary keeps the protocol forever uncertain whether a decisive process is dead or merely delayed, so it can neither safely wait (perhaps forever, for a corpse) nor safely proceed (the "corpse" may wake and contradict the decision). Reliable failure detection is precisely the capability FLP proves unimplementable in the pure asynchronous model — with a perfect detector, consensus would be easy — which is why Chapter 5's protocols are safe always but live only when timing cooperates.

Since perfection is off the table, every practical failure detector — every one, from a load balancer's health check to Raft's election timer to SWIM — reduces to the same two components:

1. **A timeout.** Some expectation of when evidence of life should arrive, and a clock that notices when it has not. (Per Chapter 2's rule, a *monotonic* clock: wall-clock adjustments must not manufacture or suppress timeouts.)
2. **A policy.** What to do with the observation "evidence is late": how late is suspicious, how suspicious is convicting, who gets told, and what they are allowed to do about it.

The policy is where the engineering lives, because the timeout embodies an unavoidable trade between the only two ways a detector can be wrong:

- **False positives** — declaring a live node dead. Costs: unnecessary failovers and elections, lease churn, split-brain *risk* when the "dead" node keeps serving (Chapter 8's fencing problem; Volume 5, Chapter 8 documents database failovers triggered by exactly this), and rebalancing work that must be redone when the node turns out to be fine.
- **Detection latency** — the window during which a genuinely dead node is still in the membership. Costs: lost capacity that the load balancer keeps routing to, requests misdirected at a corpse and burned on timeouts, and — for protocols that need the dead node's role filled — unavailability until detection completes.

Shorten the timeout and you buy detection latency at the price of false positives; lengthen it and the exchange runs the other way. There is no third option, only better and worse curves — and the rest of this chapter is about buying a better curve: adaptive timeouts flatten it, indirect probing removes one whole class of false positives, and suspicion periods give victims a chance to object before the verdict lands.

Unreliable failure detectors: the theory that made timeouts respectable

Chandra and Toueg's 1996 paper "Unreliable Failure Detectors for Reliable Distributed Systems" did something quietly profound: instead of pretending detectors could be perfect, it axiomatized their imperfection and asked how much imperfection consensus can tolerate. A detector is modeled as an oracle at each process outputting a list of suspects, characterized on two independent axes:

- **Completeness** — does it eventually suspect processes that really crashed? *Strong completeness*: every crashed process is eventually suspected by every correct process.
- **Accuracy** — does it avoid suspecting processes that are alive? *Strong accuracy*: no correct process is ever suspected. *Eventual accuracy*: it may make mistakes for an arbitrary finite period, but eventually stops suspecting correct processes.

Completeness is cheap: suspect everyone you have not heard from lately, and every crashed process is eventually suspected, since the crashed genuinely stop sending. Accuracy is the expensive axis — accuracy is exactly the crashed-versus-slow problem. The interesting classes combine strong completeness with weakening grades of accuracy: **P** (the perfect detector — strong accuracy, unimplementable in asynchrony) and $\diamond\mathbf{P}$ ("eventually perfect" — strong completeness plus *eventual* strong accuracy: after some unknown time, no correct process is suspected). Chandra and Toueg showed consensus is solvable with detectors weaker still ($\diamond\mathbf{S}$; in the companion paper with Hadzilacos, the leader oracle Ω is the weakest sufficient one), given a majority of correct processes. But $\diamond\mathbf{P}$ is the class worth memorizing, because it is what a timeout-based detector under *partial synchrony* (Chapter 1's realistic model) actually gives you: while the network misbehaves, your timeouts fire wrongly;

once it stabilizes and your timeout has adapted above the true delay bound, mistakes stop.

Here is the one paragraph that makes "eventually accurate" respectable rather than a cop-out. Well-designed protocols never let the failure detector touch *safety* — agreement in Paxos and Raft is enforced by quorums and epochs/terms regardless of what any detector believes, which is why a false suspicion can trigger a useless election but never two contradictory commits. The detector is consulted only for *liveness*: deciding when to stop waiting for someone and elect a replacement. A detector that lies for a while therefore costs only time — wasted elections, stalled progress — and one that eventually stops lying eventually stops costing time, letting some leader survive long enough to drive the protocol to termination. Eventual accuracy is precisely what makes "we will terminate eventually" true, and nothing stronger is needed because safety was never resting on the detector. That division of labor — safety from quorums, liveness from the detector — is the most important design pattern in this volume, already seen without the name in Chapters 5, 6, and 8.

Detector mechanics: heartbeats, probes, and adaptive timeouts

Push versus probe

There are two elementary ways to gather evidence of life. **Heartbeating** (push): the monitored node periodically announces "I am alive"; silence past a deadline raises suspicion. **Probing** (ping): the monitor asks and expects an answer within a timeout. The difference is not cosmetic. A heartbeat's absence conflates *the node* with *the path from the node to you* — and with the node's ability to schedule its heartbeat thread, which is exactly what a GC pause or CPU throttling takes away. A probe establishes the round-trip freshly and lets the prober control the schedule, but a single prober still cannot distinguish "target dead" from "my link to the target is degraded." SWIM's central insight, below, is that *multiple vantage points* can.

Fixed timeouts and their fragility

The naive detector — heartbeat every T , declare dead after $k \cdot T$ of silence — has one knob, and the knob has no right setting. The timeout must

exceed heartbeat interval plus network delay plus processing jitter, with margin for the tail; but delay and jitter are not constants. They differ between a rack-local link and a cross-AZ link, between 2 a.m. and peak, and they spike during the very overload and partial-partition scenarios in which you most need the detector to be right. A fixed timeout tuned for the median is a false-positive machine at the tail (Volume 3's RTT distributions apply directly); one tuned for the p999 is somnolent the rest of the time. And one global constant is shared across heterogeneous pairs: the timeout generous enough for the flaky WAN pair is absurdly slow for two nodes on the same switch.

The fix is the one TCP reached decades ago for retransmission (Volume 3): stop guessing a constant and *measure the distribution*.

The phi accrual detector

The phi accrual failure detector (Hayashibara, Défago, Yared, and Katayama, SRDS 2004) makes two moves. First, it adapts: each monitor keeps a sliding window (commonly the last few hundred samples) of inter-arrival times of heartbeats from each peer, and maintains estimates of their distribution — mean and variance in the paper's normal-distribution formulation. Second, and more interestingly, it changes the *output type*. Instead of a boolean "up/down," it outputs a continuous **suspicion level**:

$$\phi(t_{\text{now}}) = -\log_{10}(P_{\text{later}}(t_{\text{now}} - t_{\text{last}}))$$

where `t_last` is the arrival time of the most recent heartbeat and `P_later(Δt)` is the probability, under the estimated distribution, that a heartbeat would arrive *later* than Δt — the probability that the current silence is innocent. The negative log-base-10 gives φ a direct reading: **φ is the number of nines of confidence that the node is down**. Convict at φ = 1 and, in the long run, about 1 in 10 convictions is a false positive (the heartbeat was merely late); at φ = 2, 1 in 100; at φ = 3, 1 in 1,000. The arithmetic intuition: suppose a peer's heartbeats arrive every 1,000 ms with standard deviation 100 ms. At 1,100 ms of silence, lateness is ordinary — `P_later` is large, φ well under 1. At 1,500 ms (five standard deviations), `P_later` has collapsed and φ has climbed past 6. The *same* 1,500 ms silence from a noisy peer — σ of 400 ms — yields φ near 1: nothing to see. The detector automatically demands more silence before

suspecting jittery peers and less for metronomic ones — exactly the per-link, per-load adaptivity the fixed timeout lacked.

The threshold is now honest, tunable *policy*, separated from measurement. Cassandra convicts at `phi_convict_threshold` (default 8; its documentation recommends 10-12 on cloud networks with fat latency tails), approximating the distribution as exponential rather than normal — cheaper, more pessimistic in the tail, same shape of behavior. Akka Cluster implements the paper's design with a configurable threshold (historically 8) plus a floor on the standard deviation so an eerily regular heartbeat stream does not become hair-trigger sensitive. Two practical subtleties: the window must be primed before ϕ is meaningful (implementations seed it conservatively), and a node that *pauses* — GC, CPU throttle — poisons its own statistics on resumption until enough fresh samples arrive.

```
flowchart TD
    HB["Heartbeat from peer arrives"] --> REC["Record inter-arrival time  
in sliding window for that peer"]
    REC --> EST["Update distribution estimate  
mean and variance"]
    EST --> ASK["Any moment: how suspicious is peer?"]
    ASK --> ELAPSED["Compute  
silence so far:  
delta = now - last arrival"]
    ELAPSED --> PLATER["P_later of delta:  
probability a heartbeat  
is this late innocently"]
    PLATER --> PHI["phi = -log10 of P_later"]
    PHI --> POLICY["phi vs threshold"]
    POLICY --> TRUST["Keep trusting;  
phi keeps rising as silence grows"]
    POLICY --> CONVICT["phi at or above threshold  
e.g. 8 means 10^-8 innocent"]
    CONVICT --> POLICY
```

One more virtue of the continuous output: different consumers can apply different thresholds to the *same* detector — a load balancer stops routing at $\phi = 3$ (cheap to be wrong, expensive to be slow) while the rebalancer that migrates terabytes waits for $\phi = 10$. A boolean detector forces one policy on everyone; an accrual detector lets each consumer buy its own point on the curve.

Heartbeating topologies: who watches whom

A detector between one pair of nodes is only half the design; a cluster needs to decide *who monitors whom*, and the topology determines message load, detection latency, and — often forgotten — the failure modes of the detection machinery itself.

All-to-all. Every node heartbeats every other. Detection is direct and fast, with no privileged component to lose — but the load is $O(N^2)$ messages per interval (at $N = 1,000$ and one heartbeat per second, a million messages per second cluster-wide just for liveness), and every node maintains timers and windows for $N - 1$ peers. Fine for small, fixed clusters — essentially what a 3-5 node Raft group does; disqualifying at fleet scale.

Centralized. Every node heartbeats one monitor. Load is $O(N)$ and the view conveniently global — but the monitor is a single point of failure *and* a single point of ambiguity. Partition the monitor from a rack and it cannot distinguish "rack died" from "my link to the rack died"; it will confidently mark a healthy rack dead — the detector's own partition becomes everyone's false positive. Practical centralized systems replicate the monitor and its store — Kubernetes is exactly this shape, kubelets renewing per-node Lease objects against the apiserver/etcd with a node controller applying grace periods (Volume 12) — and must apply the mass-failure safeguards discussed later, because the central vantage point sees correlated ambiguity at full cluster width.

Hierarchical. Monitors per rack or zone, aggregated upward. Load and latency scale well, and the topology can mirror failure domains, which helps attribute correlated failures. The cost is machinery: each layer needs its own detector for the layer below plus a monitor-failover story — you have recursed on the problem rather than solved it.

Gossip-based. No distinguished monitors at all: liveness information itself is replicated epidemically, so every node converges on a full membership view with per-node load that is $O(1)$ messages per round and cluster load $O(N)$. Detection latency and dissemination take $O(\log N)$ rounds. This is the design that scales, and it needs a section of its own.

Gossip: epidemic dissemination and eventually-consistent membership

The mechanism and the math

The template, from Demers et al.'s 1987 epidemic algorithms paper (built for replica synchronization in Xerox's Clearinghouse directory): time is divided into rounds (say, one per second); each round, each node picks k random peers (the *fanout*, typically 1-3) and exchanges state with them. That is the whole protocol. Its power is the arithmetic of infection. Call a node "infected" once it holds a given piece of news; one node starts infected. In push mode, each infected node pushes to k random peers per round, so while the infected are a small minority almost every push lands on a susceptible peer and the infected population multiplies by roughly $(1 + k)$ per round — exponential growth, hence essentially all N nodes reached in $O(\log N)$ rounds. At $N = 10,000$ with $k = 1$ and one round per second, saturation arrives in on the order of 15-20 seconds; double the cluster and you add about one round. The randomness also buys robustness: there is no spanning tree to break, and losing half the nodes mid-epidemic delays saturation by a round or two.

The mode matters at the margins. **Push** is efficient early (every transmission likely informs someone new) but wasteful late — when 99% are infected, 99% of pushes are redundant, and the last stragglers wait for a lucky dart. **Pull** inverts this: a susceptible node asking random peers "what's new?" almost always succeeds late in the epidemic, closing the tail fast but wasting queries early. **Push-pull** — both directions in one contact — gets both regimes' strengths and is the standard choice; Cassandra's gossip and SWIM-family implementations use it in their synchronization paths.

Demers et al. also drew a distinction worth keeping crisp. **Anti-entropy** periodically reconciles *entire* states with a random peer, guaranteeing any difference is eventually repaired, at a per-contact cost proportional to state size (tamed with digests or Merkle trees — Chapter 7 showed Dynamo doing exactly this for replica data). **Rumor-mongering** spreads only recent *updates*, hot and cheap, but because it loses interest in old rumors (stop forwarding after m redundant contacts, or with probability $1/m$), a rumor can die out before reaching everyone. Production systems run both: rumors for speed, a slow anti-entropy sweep as the backstop.

```

flowchart LR
    subgraph R0["Round 0"]
        A0["1 node infected"]
    end
    subgraph R1["Round 1"]
        A1["about 2 infected  
each pushed to k=1 peer"]
    end
    subgraph R2["Round 2"]
        A2["about 4 infected"]
    end
    subgraph R3["Round 3"]
        A3["about 8 infected"]
    end
    growth["growth about 1+k per round"]
    subgraph RL["Round about log N"]
        AL["nearly all N infected  
pull or anti-entropy  
catches stragglers"]
    end
    R0 --> R1 --> R2 --> R3 --> RL

```

Gossiping membership itself

To turn gossip into a failure detector, make the payload the membership list: a map from node identity to (address, status, **version**). In the classic heartbeat-counter design, each node increments its own counter every round; entries merge by keeping, per node, the version-wise newest record — per-entry last-writer-wins where "time" is a logical version, not a wall clock, exactly Chapter 2's discipline. (Cassandra versions entries with a *generation* — bumped on restart — plus a monotonic version within it, so a rebooted node's fresh state always supersedes its pre-crash state.) Each node runs a local detector — Cassandra runs phi accrual — over the freshness of each peer's entry as seen through gossip, and each node's *conclusions* ("I mark X down") spread the same way.

Two properties of the resulting view are permanent, not bugs. First, **staleness is graded by distance**: a node's information about itself is fresh; about others, delayed by however many gossip hops the news has traveled — detection through gossiped heartbeats is inherently a few rounds slower than direct probing. Second, and most often mishandled: **the view is eventually consistent**. During the $O(\log N)$ spread window, different nodes *simultaneously hold different membership views*, all locally justified. Consumers must tolerate this: two Cassandra

coordinators may briefly disagree about whether a replica is up and route differently, which is acceptable precisely because Chapter 7's quorum reads and writes, hinted handoff, and read repair absorb it. Dynamo made the same bet explicitly — gossiped, eventually-consistent membership and ring state, with the data path engineered to survive disagreement. If a consumer *cannot* tolerate divergent views — if two nodes acting on different membership must never both act — gossip membership alone is not enough, and that decision needs Chapter 5-6 consensus or Chapter 8 leases layered on top. Knowing which decisions need which is precisely the skill.

SWIM: separating detection from dissemination

SWIM — Scalable Weakly-consistent Infection-style process group Membership, from Das, Gupta, and Motivala (DSN 2002) — begins from a criticism of heartbeat-gossip designs: they entangle *failure detection* (noticing) with *dissemination* (telling everyone), forcing both to scale together. SWIM splits them, and fixes the false-positive problem while it is at it.

The probe cycle

Each node runs a protocol period T (think 1 second). Each period, it picks **one** member and pings it directly — in practice from a randomized round-robin shuffle of the membership list rather than independent draws, which bounds worst-case time-to-first-probe: every member is probed within one pass through the list. An ack within the probe timeout (a small fraction of T , sized near the RTT ceiling — Volume 3) ends the period happily. Per node that is $O(1)$ detection load regardless of cluster size; cluster-wide, $O(N)$ — and because every node probes independently, a dead node's silence is noticed by *someone* within about one period in expectation (the paper puts expected time-to-first-detection near $T/(1 - e^{-1}) \approx 1.6 T$, independent of N).

Indirect probes: the ping-req insight

Here is the move that earns SWIM its place in this book. When A's direct ping to B times out, A concludes **nothing**. A silent timeout is one vantage point's evidence, and A's vantage point includes A's own NIC, switch, uplink, and scheduler — any of which could be the culprit. So A selects k other members ($k = 3$ is typical) and sends each a **ping-req**: "please ping

B and relay the result." The proxies probe B over *their* network paths. If any proxy reaches B, A learns B is alive — and implicitly that the failure was A's path, not B. Only if the direct probe *and* all k indirect probes fail within the period does A move against B, and now the accusation is backed by $k + 1$ independent path failures rather than one. This is the structural remedy for single-observer ambiguity: it cannot tell A anything about a node that is truly down, but it slashes the false positives caused by local congestion, a flapping link, or a one-way partition between A and B — which, in production, outnumber true crashes handily.

Suspicion, refutation, and incarnation numbers

Even $k + 1$ failed paths do not prove death — B may be paused, or partitioned from all $k + 1$ observers but not from the cluster's majority. So SWIM's verdict is not "dead" but **suspect**: a state, disseminated like any other update, that starts a timer proportional to $T \cdot \log N$ (sized so the news can plausibly reach B before it expires). A suspected member is still a member — still probed, still routable — and the suspicion message doubles as a summons: on learning it is suspected, B **refutes** by asserting *alive* with a higher **incarnation number**. Incarnation numbers are the protocol's versioning device (Chapter 2 again): only B may increment B's own incarnation, and messages about B are totally ordered by (incarnation, status severity) — *alive* at incarnation $i + 1$ overrides *suspect* at i , while at equal incarnation *suspect* overrides *alive* and *confirm-dead* overrides both. Every false accusation thus gets a bounded window in which the victim can override it with a strictly newer fact about itself, and the merge function gets the conflict-resolution rule plain heartbeat counters lacked. If the timer expires unrefuted, the suspector promotes B to **confirmed dead** and disseminates that; confirmation is terminal for that incarnation — a partitioned node that returns must rejoin as a fresh member.

The suspicion window is the false-positive/latency trade-off made explicit and *bought at a discount*: it adds latency only to the suspicion-to-confirm tail, in exchange for catching nearly all false accusations before they become membership changes.

```

sequenceDiagram
    participant A as Node A prober
    participant P as Proxy P one of k
    participant B as Node B target
    participant C as Cluster via piggyback
    A->>B: ping
    Note over A,B: probe timeout expires - no ack seen by A
    A->>P: ping-req target B
    P->>B: ping on P's own network path
    alt B reachable from P
        B-->>P: ack
        P-->>A: ack relayed - B alive
        Note over A: fault was A's path not B - no accusation
    else no proxy reaches B
        A->>C: disseminate suspect B incarnation i
        Note over B: suspicion reaches B via gossip
        B->>C: alive incarnation i+1 refutes suspicion
        Note over A,C: if timer of about T log N expires unrefuted
        A->>C: disseminate confirm B dead
    end
end

```

Infection-style dissemination, piggybacked

SWIM's second separation: there is no broadcast channel and no separate gossip subsystem. Membership updates — joins, suspicions, refutations, confirmations — ride as piggybacked payload on the ping, ping-req, and ack messages the detector is *already sending*. Each node keeps a small queue of updates, attaches the freshest few to every outgoing detector message, and retransmits each update $O(\log N)$ times before retiring it — rumor-mongering with a bounded retransmit budget, achieving epidemic $O(\log N)$ -round spread with zero additional messages. Detection load per node stays constant as the cluster grows; dissemination latency grows only logarithmically. That scaling signature is what lets SWIM-family clusters run at thousands of nodes.

Lifeguard: the accused might be the problem

HashiCorp's production experience with SWIM (in memberlist, the library under Serf, Consul, and Nomad) surfaced a systematic bias: many false accusations are made by unhealthy nodes. A node deep in GC or CPU starvation fails to process acks in time and begins suspecting healthy peers — the detector's own degradation reads as everyone else's death. Lifeguard (Dadgar, Phillips, and Currey, 2018) adds three local refinements: **Local Health Aware Probe** — each node maintains a local-

health multiplier (a counter of self-evidencing failures: missed acks, failing to refute in time) and scales *its own* probe timeouts and protocol period up when the counter is high — "when I show signs of being slow, I trust my own timeouts less"; **Local Health Aware Suspicion** — suspicion timeouts start long and shrink as *independent* confirmations arrive from other members, so a lone accuser cannot fast-track a conviction but corroborated suspicion converges quickly; and the **Buddy System** — a suspected node is prioritized for direct notification so it can refute sooner. HashiCorp reported these cut false positives dramatically (roughly an order of magnitude in their published experiments) for single-digit-percent increases in detection latency — buying a better point on the curve rather than sliding along it.

The knobs, with arithmetic

The parameters of a SWIM deployment and how they compose (values are memberlist's LAN defaults; its WAN and local profiles scale the same knobs):

Parameter	memberlist LAN default	Role in the arithmetic
Probe interval (protocol period T)	1 s	Expected time-to-first-detection $\approx 1.6 T$; every knob below is in units of it
Probe timeout	500 ms	Direct-ping deadline; should sit above the RTT tail, far below T
Indirect probes k	3	False accusation now requires $k + 1 = 4$ independent path failures
Suspicion multiplier	4	Suspicion timeout = $\text{mult} \times \log_{10}(N + 1) \times T \rightarrow$ at $N = 100$, $4 \times 2 \times 1 \text{ s} \approx 8 \text{ s}$ to refute
Max suspicion multiplier (Lifeguard)	6	Ceiling before independent confirmations shrink the window
Gossip interval / fanout	200 ms / 3 nodes	Piggyback dissemination pace; $O(\log N)$ rounds to saturate

Parameter	memberlist LAN default	Role in the arithmetic
Retransmit multiplier	4	Each update retransmitted $\approx \text{mult} \times \log_{10}(N + 1)$ times

End-to-end, with LAN defaults at $N = 100$: a crashed node is first missed within ~ 1.6 s, survives as a suspect for ~ 8 s awaiting refutation, and the confirmation saturates the cluster in a few 200 ms gossip rounds — call it 10–12 s from crash to cluster-wide confirmed-dead, with the suspicion window dominating. That window is not waste; it is the insurance premium against convicting the living.

```
// hashicorp/memberlist – the SWIM knobs, annotated with what each one
// buys.
// These are the LAN defaults; DefaultWANConfig and DefaultLocalConfig
// scale
// the same knobs for higher-latency and same-host deployments
// respectively.
conf := memberlist.DefaultLANConfig()
conf.ProbeInterval = 1 *
time.Second           // protocol period T; lower = faster
                        // detection, more traffic,
tighter
                        // deadlines under load
conf.ProbeTimeout = 500 * time.Millisecond // direct-ping deadline;
must clear
                        // your RTT p99 or you
fabricate
                        // suspicion under
congestion
conf.IndirectChecks = 3 // k proxies for ping-req;
each adds
                        // an independent vantage
point
conf.SuspicionMult = 4 // suspicion window = mult
* log10(N+1) * T;
                        // the victim's time to
refute
conf.SuspicionMaxTimeoutMult = 6 // Lifeguard ceiling,
shrunk by
                        // independent
confirmations
conf.GossipInterval = 200 * time.Millisecond
conf.GossipNodes = 3 // fanout per gossip tick
conf.RetransmitMult = 4 // each update sent
~mult*log10(N+1) times
```

Consul surfaces these same knobs as the `gossip_lan / gossip_wan` stanzas in agent configuration, and its documentation is admirably blunt that the defaults are the right answer for almost everyone — the honest annotation for the snippet above is that you should change `ProbeTimeout` if your RTT tail demands it, consider the WAN profile across regions, and otherwise leave the arithmetic alone.

Membership change is not failure detection

A membership protocol answers two questions worth separating: *who is in the group* (administrative intent) and *who is currently alive* (observation). Conflating them causes real damage. In SWIM-family systems, membership is a small versioned state machine per node — `alive → suspect → dead`, plus `left` — whose transitions carry (incarnation, status) versions and merge by the ordering given earlier. **Join** is explicit: the newcomer contacts any member, push-pull syncs the full membership state, and an *alive* record at a fresh incarnation spreads epidemically. **Graceful leave** is likewise explicit: the departing node announces *left* and the record spreads *before* the silence begins — which is the point. A graceful leaver triggers no suspicion machinery and no refutation window, and lets consumers distinguish "drained on purpose" from "died mid-write": Serf keeps the distinction (`leave` versus `failed`) so operators and reconcilers can react differently, and Consul reaps *failed* nodes only after a long deadline (72 hours by default) while *left* nodes exit promptly. The rule follows: **deployment tooling should always leave, never just kill** — every avoidable suspicion event is an avoidable dice-roll on the false-positive machinery. Note the asymmetry: *dead* is a third party's claim at some incarnation; *left* is self-asserted and needs no refutation window.

Where each design lives

These designs are fits to different contexts, not alternatives on a menu — worth a tour, because you operate all of them already.

Raft embeds its detector in the protocol (Chapter 6). `AppendEntries` heartbeats from the leader are the liveness evidence; a follower's randomized election timeout (canonically 150-300 ms against a much smaller heartbeat interval) is the timeout-plus-policy, and the policy is "start an election." All-to-all monitoring is irrelevant at $N = 3-5$; what

matters is that false positives are cheap by design — a spurious election costs milliseconds and a term number, never safety — the Chandra-Toueg division of labor implemented in 300 lines.

Kubernetes is the replicated-centralized topology (Volume 12). Kubelets renew per-node Lease objects against the apiserver every ~10 s; the node controller applies a grace period (40 s by default) before marking a node NotReady, then waits further (the 5-minute default toleration) before evicting pods. Pod-level liveness/readiness/startup probes run *locally* on the kubelet — a deliberate two-layer design: node-level detection through the central store, application-level health at the edge, each with its own timeout and policy, and with the eviction safeguards below as the policy's adult supervision.

Cassandra and Akka Cluster run phi accrual over gossip — adaptive per-peer suspicion atop epidemic dissemination, conviction thresholds as configuration. **Consul, Serf, and Nomad run SWIM with Lifeguard** via memberlist — Consul for agent liveness and its catalog's health, Nomad for client-node liveness. **Load balancers and service meshes** run the humblest and most ubiquitous detector: active health checks (periodic HTTP/TCP probes with consecutive-success and consecutive-failure thresholds — hysteresis by construction) and passive checks (outlier detection à la Envoy: eject a backend on consecutive 5xx or latency deviation, for a bounded time, with a cap on the ejectable fraction of the pool — a mass-failure safeguard we will meet again below). Volume 7, Chapter 4 covers the data-plane mechanics; Volume 11 the SRE practice.

Operational wisdom

Timeout tuning is SLO engineering

The timeout is not a config value; it is a business decision wearing a config value's clothes. Detection latency bounds failover time, which bounds the availability you can promise; false-positive rate drives failover *frequency*, which drives both risk (every failover is a chance to discover a new bug) and toil. So tune like an SLO engineer: decide what detection latency the availability target requires, measure the false-positive rate you are paying for it (refuted suspicions in SWIM, nodes returning NotReady→Ready without a restart, LB ejections that self-heal — countable events; count them), and move along the curve deliberately.

The classic false-positive sources are not network at all: **GC pauses and CPU throttling** stop the heartbeat thread while the node is, by any external definition, alive. This is precisely Chapter 8's session-timeout dilemma — the ZooKeeper client that misses its session deadline during a stop-the-world pause and finds its ephemeral nodes gone, leadership revoked by its own garbage collector — and it is what Lifeguard's local-health multiplier attacks from the other side: let the slow node distrust itself before others must. If your convict thresholds are tighter than your GC p99.9 pause plus scheduling jitter, you have configured a self-DoS; fix it by raising the threshold, fixing the pauses (Volume 2), or moving heartbeat emission out of the pause domain.

Gray failure: alive is not the same as working

Huang et al.'s HotOS 2017 paper named the pattern liveness detection is structurally blind to: **gray failure**, where a component is degraded in a way some observers see and the detector does not. The canonical case is the node whose ping handler — a trivial code path — answers promptly while its request path fails: disk errors on the data volume, a wedged worker pool, an exhausted dependency connection pool. The paper's sharper point is *differential observability*: the failure is fully visible to clients and invisible to the detector; systems fall over not because failures are undetectable but because the detector and the victims are watching different things. The remedy is to make health mean "doing its job": application-level signals — error rate on real requests, queue depth, end-to-end canary probes exercising the actual serving path — which is what readiness probes and LB passive checks (observing *real* request outcomes) approximate. But deep health checks carry their own trap, and honesty requires stating it: a check that transitively interrogates dependencies converts one dependency outage into an "unhealthy" verdict for *every* caller — the whole fleet fails its checks at once, the LB ejects everyone, and a partial outage becomes a total one. The discipline (developed in Volume 11): a health check should report on *that node's* own ability to do its work, degrade to partial-health rather than binary death when a dependency is impaired, and never let "all backends unhealthy" produce an empty pool — fail open, or eject up to a capped fraction, exactly as Envoy's outlier detection does.

Flapping, hysteresis, and dampening

A node oscillating across the conviction threshold — marginal link, load-correlated pauses — generates a membership *event stream*, and every event costs consumers real work (rebalancing, elections, route updates). Detectors therefore need **hysteresis** — separate, stickier thresholds for leaving the healthy state than for re-entering it (the LB's consecutive-success count to readmit versus consecutive-failure count to eject) — and **dampening/quarantine** for repeat offenders: a node that fails and recovers n times in a window is readmitted on a longer probation each time, the design BGP route-flap dampening standardized decades ago. SWIM has a mild structural advantage — a refutation cycle costs an incarnation bump and some gossip, not a topology change — but a flapper still spams the update stream, and memberlist does not itself quarantine chronic flappers; that policy belongs to the layer above.

Correlated failure and mass-suspicion safeguards

Chapter 1's failure models were mostly independent; racks and AZs are not. A ToR switch dies or a zone partitions, and the detector — working exactly as designed — produces a **mass suspicion event**: dozens or hundreds of simultaneous convictions. If every consumer reacts as it would to one death, the reaction is the outage: re-replicating every "lost" partition saturates the network (a rebalancing stampede, which slows heartbeats and manufactures *more* suspicion — a metastable feedback loop, Volume 11), and when the partition heals, all that work reverses. Mature systems therefore rate-limit the *reaction* and treat correlated suspicion as grounds for less confidence, not more: Kubernetes's node controller caps eviction rate (0.1 nodes/s by default), slows further when a zone exceeds an unhealthy fraction, and stops evicting entirely when a zone is largely unhealthy — encoding "if 60% of a zone looks dead, disbelieve the detector before the zone." Envoy caps the ejectable fraction of a cluster; Elasticsearch delays re-allocating shards from a departed node (`delayed_timeout`, 1 m by default), betting on quick returns; Cassandra marks nodes down without moving data at all, leaving re-replication to explicit operator action. The shared principle: **detection can be fast, but irreversible reaction must be damped in proportion to how correlated the evidence is** — failure-domain awareness belongs in the policy even when the detector is domain-blind.

The distributed-systems lens: the detector is everyone's input

Step back and look at where the detector's output flows. Leader election consumes it (Chapters 5–6): a false suspicion is a spurious election. Lease and session expiry consume it (Chapter 8): a false expiry revokes locks whose holders still believe. Quorum systems and rebalancers consume it (Chapter 7): a false death triggers data movement measured in terabytes. Orchestrators and autoscalers consume it (Volume 12): a false NotReady evicts real workloads. Volume 5, Chapter 8's promised payoff lands here: database failover false positives are not a database problem — they are this chapter's curve surfacing in the replication layer, the same late heartbeat promoted into a promotion.

```
flowchart TD
    FD["Failure detector  
timeout plus policy"] --> MEM["Membership view  
eventually consistent"]
    MEM --> ELECT["Leader election  
Paxos and Raft, Ch5-6"]
    MEM --> LEASE["Lease and session expiry  
coordination services, Ch8"]
    MEM --> QUORUM["Quorum routing and rebalancing  
Dynamo-style stores, Ch7"]
    MEM --> ORCH["Orchestrator and autoscaler  
eviction and rescheduling, Vol 12"]
    MEM --> LB["Load balancer pools  
ejection and readmission, Vol 7"]
    ELECT --> COST["False positive cost:  
spurious elections, lease churn,  
terabyte rebalances, eviction storms"]
    LEASE --> COST
    QUORUM --> COST
    ORCH --> COST
```

Because the detector sits under all of them, its error rate does not add through the stack — it *multiplies*. One flapping node convicted five times a day, in a cluster where conviction triggers an election, a lease sweep, and a rebalance, is not five small events; it is five cluster-wide churn cycles, each a fresh opportunity for the metastable feedback loop above. Hence the moral operators of large fleets learn exactly once: **detector storms cause outages bigger than the failures they detect**. The corollary is that detector *quality* — a lower false-positive rate at the same latency, via phi accrual's adaptivity, SWIM's indirect probes, Lifeguard's

self-awareness, suspicion windows, damped reactions — is among the highest-leverage reliability investments in the stack, because it is amortized across every consumer at once.

And at the bottom, the shape of the thing is Chapter 1's lesson made permanent. The timeout under every abstraction in this diagram is the tax the asynchronous model levies: FLP says you cannot know, partial synchrony says you can eventually guess well, Chandra-Toueg says eventual good guessing is enough for liveness if safety never depends on the guess — and Chapter 2's monotonic-clock rule says at least measure your guesses with a clock that cannot run backward. Every system in this volume is a machine for living gracefully with a detector that is guaranteed to be wrong sometimes. The ones that survive production budgeted for it.

Key takeaways

- **Perfect failure detection is impossible in asynchrony** — crashed and slow are observationally identical, the engine of FLP. Every practical detector is a **timeout plus a policy**, trading false positives (unneeded failovers, split-brain risk, rebalance churn) against detection latency (dead capacity, misrouted requests).
- **Chandra-Toueg made the imperfection precise**: completeness (suspect the crashed — cheap) versus accuracy (spare the living — expensive). $\diamond P$ is what timeouts under partial synchrony really give you, and it suffices because well-built protocols take safety from quorums and use the detector only for liveness — mistakes cost time, never correctness.
- **Fixed timeouts are fragile; measure the distribution**. Phi accrual keeps per-peer inter-arrival statistics and outputs $\phi = -\log_{10} P(\text{this silence is innocent})$ — nines of confidence — making the threshold explicit per-consumer policy (Cassandra `phi_convict_threshold 8`; Akka).
- **Topology decides scaling and the detector's own failure modes**: all-to-all is $O(N^2)$; centralized monitors are a SPOF whose own partition becomes everyone's false positive; gossip gives $O(1)$ per-node load and $O(\log N)$ spread with no privileged component.
- **Gossip membership is eventually consistent by construction** — nodes briefly, legitimately disagree. Consumers must tolerate

divergent views (as Dynamo's quorums do) or layer consensus on top for the decisions that cannot.

- **SWIM separates detection from dissemination:** one random probe per period (~1.6 periods to first detection), **ping-req through k proxies** to distinguish "my path failed" from "the target failed," a **suspect → refute → confirm** machine with **incarnation numbers** so victims can override accusations, and updates piggybacked on the probes. Lifeguard adds self-distrust — a slow node inflates its own timeouts before accusing others.
- **Leave is not death:** graceful departure is a self-asserted, versioned transition that skips the suspicion machinery. Deploy tooling should always leave, never just kill.
- **Liveness is not health:** gray-failed nodes answer pings while failing requests. "Healthy" must mean "doing its job" — but dependency-deep checks can convert one outage into everyone's conviction, so cap ejections and fail open.
- **Damp the reaction, not just the detection:** hysteresis and quarantine against flapping; rate-limited, failure-domain-aware responses to mass suspicion, because correlated convictions usually indict the network or the detector, not the nodes.
- **The detector's output feeds every stateful protocol above it** — elections, leases, quorums, orchestration — so its false-positive rate multiplies through the stack. Detector storms cause outages bigger than the failures; detector quality is amortized leverage.

Further reading

- Das, A., Gupta, I., and Motivala, A., "SWIM: Scalable Weakly-consistent Infection-style Process Group Membership Protocol," *DSN 2002* — the protocol itself, with the load and detection-latency analysis. https://www.cs.cornell.edu/projects/Quicksilver/public_pdfs/SWIM.pdf
- Hayashibara, N., Défago, X., Yared, R., and Katayama, T., "The ϕ Accrual Failure Detector," *SRDS 2004* — suspicion as a continuous, adaptively-estimated quantity.
- Chandra, T. D. and Toueg, S., "Unreliable Failure Detectors for Reliable Distributed Systems," *Journal of the ACM* 43(2), 1996 — completeness, accuracy, and the detector hierarchy; with Chandra,

Hadzilacos, and Toueg, "The Weakest Failure Detector for Solving Consensus," *JACM* 43(4), 1996.

- Dadgar, A., Phillips, J., and Currey, J., "Lifeguard: Local Health Awareness for More Accurate Failure Detection," *DSN 2018 industry track* — the local-health refinements to SWIM, with measured false-positive reductions. <https://arxiv.org/abs/1707.00788>
- Huang, P. et al., "Gray Failure: The Achilles' Heel of Cloud-Scale Systems," *HotOS 2017* — differential observability and why liveness detection misses what clients see. <https://www.microsoft.com/en-us/research/publication/gray-failure-achilles-heel-cloud-scale-systems/>
- Demers, A. et al., "Epidemic Algorithms for Replicated Database Maintenance," *PODC 1987* — anti-entropy and rumor-mongering, and the analysis behind $O(\log N)$ spread.
- Fischer, M., Lynch, N., and Paterson, M., "Impossibility of Distributed Consensus with One Faulty Process," *JACM* 32(2), 1985 — the impossibility this chapter's machinery works around.
- HashiCorp `memberlist` — the production SWIM + Lifeguard implementation and its configuration surface. <https://github.com/hashicorp/memberlist> — and the Serf gossip documentation, which walks the protocol's convergence behavior. <https://developer.hashicorp.com/serf/docs/internals/gossip>
- DeCandia, G. et al., "Dynamo: Amazon's Highly Available Key-value Store," *SOSP 2007* — gossiped, eventually-consistent membership consumed by a quorum data path (Chapter 7's subject).
- Chapter 1 — Foundations — the system models and the crashed-versus-slow ambiguity; Chapter 2 — the logical-clock and monotonic-clock discipline this chapter's versioning leans on; Chapter 8 — the session-timeout dilemma, the same problem wearing coordination-service clothes.

Chapter 11 — CRDTs and Eventual Consistency

What this chapter covers. This volume has offered two answers to the same question — what do you do when concurrent updates land at different replicas? Chapter 7 gave the Dynamo answer: accept both, keep

siblings, and push the merge problem up to the application, where it is solved badly or not at all. Chapters 5 and 6 gave the consensus answer: prevent the conflict from existing by serializing every write through a quorum, and pay for it in latency on every operation, partition or no partition. This chapter covers the third way: **conflict-free replicated data types**, data types whose merge function is built into the type itself and is mathematically guaranteed to converge, so that replicas can accept writes independently, sync in any order, over any topology, with duplicates and delays — and still end up in the same state. The guarantee comes from a small piece of order theory, the join-semilattice, and we develop it properly because the three algebraic properties involved are not decoration: they are precisely what makes anti-entropy free, deduplication unnecessary, and convergence a theorem rather than a hope. We walk the standard type catalog with mechanics — counters, sets, registers, sequences — including the observed-remove set in enough depth to see why naive replicated sets are broken. We then draw the honest boundary: the invariants CRDTs cannot protect, the metadata they accumulate, and the escrow patterns that hybridize them with coordination. The chapter closes a promise made in Volume 4, Chapter 4 — Atomics, CAS, and Lock-Free Data Structures — whose `LongAdder` was introduced as "the in-process form of a CRDT counter." Here is the other half of that sentence.

Learning goals — after this chapter you should be able to:

- Define a join-semilattice and explain why a commutative, associative, idempotent merge makes replica divergence impossible to sustain and at-least-once sync safe.
- State Strong Eventual Consistency precisely and explain how it strengthens plain eventual consistency.
- Distinguish state-based and operation-based CRDTs by their delivery requirements, and explain what delta-CRDTs fix.
- Walk the mechanics of G-Counter, PN-Counter, G-Set, 2P-Set, OR-Set, LWW-Register, and MV-Register, including the concurrent add/remove case that breaks naive sets.
- Explain why sequence CRDTs make collaborative text editing work, at the level of position identifiers, without the algorithmic details.
- Identify invariants that no CRDT can protect, explain why via invariant confluence, and apply escrow/reservation patterns as the hybrid.

- Choose merge semantics deliberately (add-wins vs remove-wins, LWW's loss acceptance) as a business decision, and property-test convergence.

The problem: concurrent updates with no coordinator

Recall where Chapter 7 left us. A Dynamo-style store accepts a write at any replica, and when two clients write the same key concurrently on opposite sides of a partition — or merely through different coordinators in the same healthy datacenter — the system ends up holding two causally concurrent versions. Version vectors detect this honestly: neither write descends from the other. But detection is not resolution. The store hands both siblings to the application, and the application must merge them. Amazon's shopping-cart example made this look tractable — union the carts — but the folklore result was the resurrected deletion: union cannot distinguish "item removed on one side" from "item added on the other," so deleted items reappear. The general lesson from Chapter 7 stands: **app-side merge is a correctness obligation that most applications silently fail**, usually by configuring last-write-wins and losing data quietly.

The consensus chapters solved the problem from the other end: if every write flows through Raft's leader or a Paxos quorum, concurrent conflicting updates never both commit, so there is nothing to merge. The price is written into Chapter 4's PACELC vocabulary: a round trip to a quorum on the write path — else-latency paid on every operation — and unavailability on the minority side of every partition. For a global counter of page views, that price is absurd. For a collaborative document edited from three continents, it is fatal: nobody will accept 150 ms of consensus per keystroke.

Multi-primary database replication, which Volume 5, Chapter 8 covered, sits in the worst spot of all: it accepts concurrent writes like Dynamo but resolves them with ad hoc rules — last-writer- wins, priority of one site over another, or "log it and call a human" — none of which come with a convergence argument, and some of which (LWW on skewed clocks, per Chapter 2) discard committed data. That chapter promised a principled treatment of conflict resolution. This is it.

The CRDT move is to change *where the merge lives*. Instead of a store that replicates opaque blobs and an application that merges on read, the

replicated object is a **data type** — a counter, a set, a map, a sequence — whose interface includes a merge function, and whose merge function is constrained by algebra so that convergence is guaranteed no matter when, how often, or in what order replicas exchange state. Conflicts are not detected and resolved; they are *defined away*, because the merge is total: every pair of states has a well-defined, agreed combination. The question shifts from "how do we resolve conflicts?" to "which types have such a merge, and what semantics can they express?" — and the answer to the second part is the honest boundary of this chapter.

The mathematics: join-semilattices

The theory is small and worth owning, because every design decision in the rest of the chapter traces back to it.

A **partial order** on a set of states is a relation $\sqsubseteq$ that is reflexive, antisymmetric, and transitive — " $s \sqsubseteq t$ " reads as "t knows at least as much as s." Partial means exactly what it says: two states can be incomparable, and incomparable states are precisely the concurrent ones, each holding information the other lacks. A **join-semilattice** is a partially ordered set in which every pair of states has a **least upper bound** (a *join*, written $s \sqcup t$): the smallest state that is $\sqsupseteq$ both. "Upper bound" means the join loses nothing from either input; "least" means it adds nothing that neither input contained. A CRDT's merge function is the join.

Any join operation automatically has three properties, and each one neutralizes a specific distributed-systems pathology:

Property	Algebra	Pathology it kills
Commutative	$s \sqcup t = t \sqcup s$	Message <i>reordering</i> : replicas may receive states in any order
Associative	$(s \sqcup t) \sqcup u = s \sqcup (t \sqcup u)$	Arbitrary <i>grouping and topology</i> : gossip pairwise, via relays, in batches — the result is the same
Idempotent	$s \sqcup s = s$	<i>Duplication</i> : re-delivering a state is a no-op, so at-least-once sync is safe with zero dedup machinery

Read that third row against Chapter 9, because it is this chapter's quiet superpower. Chapter 9 spent its length building idempotency keys, dedup tables, and exactly-once illusions on top of at-least-once delivery, and concluded that the cheapest deduplication is a *naturally idempotent operation* that needs none. A CRDT merge is exactly that. Anti-entropy between CRDT replicas needs no sequence numbers, no acknowledgment tracking, no retransmission bookkeeping beyond "try again later": push your state as often as you like, to whomever you like, and nothing can be double-counted. The entire apparatus of reliable delivery collapses into "eventually, every replica's state reaches every other replica along some path" — which is what a gossip protocol from Chapter 10 provides almost by accident.

Convergence then follows in two lines. Merging only moves states *upward* in the order ($s \sqsubseteq s \sqcup t$ always), and updates are designed to be **inflations** — an increment or an add produces a state strictly above the old one, never off to the side or below. So every replica's state climbs a single shared lattice, and two replicas that have absorbed the same set of updates sit at the join of those updates — the *same point*, regardless of the path taken to it. Divergence cannot persist because there is nowhere for it to live: any two states, however they came about, have a defined meeting point above them, and anti-entropy pushes both there.

flowchart TB

```

    S["Common state S  
both replicas agree"] --> A["Replica A applies update a  
state Sa – an inflation, Sa is above S"]
    S --> B["Replica B applies update b  
state Sb – concurrent with Sa,  
the two are incomparable"]
    A --> J["merge = join = least upper bound  
Sa joined with Sb"]
    B --> J
    J --> C["Same state at both replicas.  
Loses nothing: upper bound of both.  
Invents nothing: LEAST upper bound.  
Any merge order or grouping lands here."]
  
```

Strong Eventual Consistency, precisely

Chapter 3 defined eventual consistency and was candid about its weakness: "if updates stop, replicas eventually agree" says nothing about *what* they agree on, *when*, or what happens while updates continue.

Shapiro, Preguiça, Baquero, and Zawirski (2011) defined the guarantee CRDTs actually provide, and named it **Strong Eventual Consistency (SEC)**:

Eventual delivery — an update delivered at one correct replica is eventually delivered at all correct replicas; **convergence** — any two correct replicas that have delivered the *same set* of updates are in *equivalent states*, immediately; **termination** — all method executions terminate.

The word doing the work is *immediately*. Vanilla eventual consistency promises agreement in some unspecified future after a hypothetical quiescence that production systems never reach. SEC promises that state is a *function of the set of delivered updates* — not of their arrival order, not of the sync topology, not of how many duplicates were seen. Two replicas with the same update set do not "eventually reconcile"; there is nothing to reconcile, because there is no execution in which they differ. Conflict *resolution* has no step in the protocol at which it could occur, because merge is total and deterministic. SEC is, in a precise sense, the strongest guarantee available without coordination: it is achievable with zero synchronous communication, while anything stronger (per Chapter 3's hierarchy — linearizability, and even some session guarantees under failures) provably requires waiting for other replicas. Place it on Chapter 3's spectrum as the principled *far end*: not a shrug, but a fixed point.

State-based versus operation-based

The lattice story above is the **state-based** (CvRDT, convergent) formulation: replicas apply updates locally and periodically ship their *state*; the receiver joins it in. Its delivery requirements are as weak as requirements get — messages may be lost (retry later), duplicated (idempotence), reordered (commutativity), and routed arbitrarily (associativity). If states occasionally reach a replica through three different gossip paths, nothing whatsoever goes wrong. The cost is bandwidth: the state of a large set is large, and shipping the whole thing per sync round is absurd at scale.

Delta-CRDTs (Almeida, Shoker, Baquero) are the standard fix: instead of the full state, a replica ships a *delta* — a small state, itself a lattice element, representing just the recent inflations (the entries touched since the last exchange with that peer). Deltas are joined into the receiver's

state exactly like full states, so all the delivery tolerance survives; full-state exchange remains available as the fallback that repairs anything a lost delta missed. Riak and Akka Distributed Data both ship this. The mental model: full state sync is anti-entropy, delta sync is the fast path, and because both are joins, mixing them freely is safe.

Operation-based CRDTs (CmRDT, commutative) invert the trade. Replicas broadcast *operations* — "add('milk')", "increment by 3" — which are small, and each replica applies every operation once. The algebra moves from the merge to the operations: **concurrent operations must commute**, so that replicas applying them in different orders agree. But the delivery substrate must now do real work: each operation must be delivered to each replica **exactly once** (or be made idempotent by tagging — Chapter 9 again), and for most types delivery must be in **causal order** — an operation must not arrive before operations it causally depends on, in Chapter 2's sense. Removing an element before its add has arrived is the canonical disaster. Causal broadcast is buildable (vector clocks plus buffering, per Chapter 2) and libraries provide it, but it is genuine infrastructure with genuine failure modes, and it is the tax op-based designs pay for their small messages and low latency of propagation.

```
flowchart TB
    subgraph SB["State-based CvRDT"]
        U1["update applied locally"]
        state_inflates["state inflates"] --> G1["gossip STATE or DELTA any peer, any time"]
        G1 --> M1["receiver JOINS it in"]
        M1 --> P1["tolerates loss, duplication, reordering, any topology."]
        Cost_SB["Cost: message size, or delta bookkeeping"]
    end
    subgraph OB["Operation-based CmRDT"]
        U2["update applied locally"]
        operation_generated["operation generated"] --> G2["broadcast the OPERATION"]
        G2 --> M2["receiver APPLIES it once"]
        M2 --> P2["small messages, fast propagation."]
        Cost_OB["Cost: exactly-once delivery and causal order required – real infrastructure"]
    end
```

The two formulations are a **duality**, not a rivalry: Shapiro et al. prove each can emulate the other (ship operations that describe state deltas; ship states that carry operation logs), so they are equally expressive. The choice is an engineering one about where you would rather pay — bandwidth and merge computation, or delivery infrastructure. Databases and infrastructure systems (Riak, Akka, Redis Enterprise) lean state/delta-based because gossip tolerance matches their membership churn; collaborative-editing systems lean op-based or hybrid because a keystroke is an operation and latency of propagation is the product.

The type catalog

G-Counter: the LongAdder debt, paid

A **G-Counter** (grow-only counter) is a map from replica ID to a non-negative count. A replica increments *only its own entry*. The value is the sum of all entries. The merge is the **entrywise maximum**.

```
class GCounter:
    def __init__(self, replica_id, n_replicas):
        self.i = replica_id
        self.counts = [0] * n_replicas

    def increment(self, by=1):
        self.counts[self.i] += by          # only ever your own entry

    def value(self):
        return sum(self.counts)

    def merge(self, other):
        self.counts = [max(a, b) for a, b in zip(self.counts, other.counts)]
```

Why max and not sum? Because each entry has a **single writer** and only grows, the larger of two values for entry k is simply the more recent knowledge of replica k 's count — max takes the freshest fact about each stripe. Summing would double-count: merge the same state twice (which at-least-once sync *will* do) and the counter inflates. Max is idempotent; sum is not. This is the whole design in one decision, and it is worth checking the lattice properties against it: states ordered entrywise, join = entrywise max — commutative, associative, idempotent by inspection, and increment is an inflation. Convergence follows from the theory with nothing further to prove.

```

flowchart LR
    A0["Replica A  
A:5 B:2 C:0  
value 7"] --> M["merge:  
entrywise max  
A: max 5,3 = 5  
B: max 2,4 = 4  
C: max 0,1 = 1"]
    B0["Replica B  
A:3 B:4 C:1  
value 8"] --> M
    M --> R["Merged state  
A:5 B:4 C:1  
value 10 – every increment  
counted exactly once,  
re-merging changes nothing"]

```

Now pay the Volume 4 debt in full. `LongAdder` replaced one contended `AtomicLong` with striped per-thread cells summed on read, trading an instantaneously exact value for contention-free writes. The G-Counter is *the same object* with the sharing assumption removed. In-process, the cells live in shared memory, so "merge" is trivial — `sum()` just reads them all. Across replicas nothing is shared, so the cells must travel, and the single-writer discipline that in `LongAdder` merely avoided cache-line ping-pong now *enables the max-merge*: because only replica k writes entry k , and only upward, entrywise max losslessly reconciles any two snapshots however stale. Contention → striping → aggregate-on-read was Volume 4's arc; coordination → per-replica state → join is this volume's; they are one idea at two scales, and the trade is the same at both: writes scale freely, and the read is exact only at quiescence.

Cost, honestly: state is $O(\text{number of replicas that ever incremented})$, which is fine for a cluster of storage nodes and unbounded for "every browser tab is a replica" — client-side designs interpose servers or prune IDs for this reason.

PN-Counter

Decrement breaks the grow-only trick — a decrement is not an inflation of the max-merged map. The **PN-Counter** fixes it with the standard CRDT maneuver, *two monotone things instead of one non-monotone thing*: a P G-Counter for increments, an N G-Counter for decrements, $\text{value} = P - N$, $\text{merge} = \text{merge both halves}$. The value can now move both ways while the *state* still only grows. Note what is not promised: nothing

stops the value going negative, because no replica can see concurrent decrements elsewhere. Hold that thought for the boundary section.

Sets: G-Set, 2P-Set, and why naive sets break

A **G-Set** is a set with only `add`; merge is union — the simplest lattice there is. Useful whenever removal genuinely never happens (issued certificate serials, seen-event IDs).

A **2P-Set** adds removal via a second G-Set of tombstones: present = added and not removed. It converges, but with a semantics almost nobody wants: **an element removed once can never be re-added** — the tombstone is forever, and merge resurrects it to kill any later add. It also answers the wrong question for concurrent add/remove: remove wins even against an add it never observed. The 2P-Set matters mostly as the cautionary midpoint between "union resurrects deletions" (Chapter 7's cart) and the type that gets it right.

OR-Set: the one to actually understand

The **observed-remove set** (OR-Set) is the workhorse replicated set — Riak's set, Akka's `ORSet`, the model for Redis Enterprise sets — and its design principle generalizes: **make removal act on evidence, not on names**.

Every `add(e)` attaches a globally **unique tag** (replica ID + local counter, or a UUID) to the element; the state holds (element, tag) pairs. `remove(e)` does not remove the *name* "e" — it tombstones exactly the *tags of e that this replica has observed*. An add that this remove never saw carries a tag the remove did not tombstone, so it survives the merge. Concurrent add and remove of the same element therefore resolve **add-wins**, deterministically:

```

class ORSet:
    def __init__(self, replica_id):
        self.i, self.n = replica_id, 0
        self.entries = set()          # {(elem, tag)} – live evidence of
adds
        self.tombstones = set()      # {(elem, tag)} – observed and
removed

    def add(self, e):
        self.n += 1
        self.entries.add((e, (self.i, self.n)))    # fresh unique tag

    def remove(self, e):
        seen = {(x, t) for (x, t) in self.entries if x == e}
        self.tombstones |= seen
# kill only what was OBSERVED
        self.entries -= seen

    def contains(self, e):
        return any(x == e for (x, _) in self.entries)

    def merge(self, other):
        self.tombstones |= other.tombstones
        self.entries = (self.entries | other.entries) - self.tombstones

```

The state (entries $\cup$ tombstones ordered by inclusion, with tombstones dominating) is a lattice; merge is unions with tombstone subtraction; all three properties hold. Walk the case that breaks naive sets:

```

sequenceDiagram
    participant A as Replica A
    participant B as Replica B
    Note over A,B: both hold milk with tag t1
    A->>A: remove milk – tombstones t1 only
    B->>B: add milk – NEW unique tag t2
    A->>B: sync state
    B->>A: sync state
    Note over A,B: merged entries contain milk with tag t2
    tombstones contain t1
    milk PRESENT at both – add wins
    Note over A,B: a LATER remove at either replica
    observes t2 and kills it – removes
    that saw the add still work

```

Contrast every alternative: plain union resurrects the deletion permanently; 2P-Set kills the concurrent add it never saw; LWW flips a coin weighted by clock skew. The OR-Set gives a *chosen* answer — an add

concurrent with a remove survives — and applies it deterministically everywhere. Whether add-wins is the *right* answer is a business question (a remove-wins variant exists and is sometimes correct — think access revocation, where the safe default is out); the point is that the answer is picked at type-selection time, not improvised per conflict.

Two costs need naming. Tags make the set grow with *add operations*, not with distinct elements. Tombstones grow with removes and, naively, live forever. The **optimized OR-Set** (Bieniusa et al. 2012, the "ORSWOT" — observed-remove set without tombstones — that Riak ships) replaces explicit tombstones with a version vector summarizing what each replica has seen: an entry absent from a state whose vector *covers* the entry's tag has provably been removed, so absence-plus-causality encodes the tombstone. Delta variants ship only recently changed entries. The semantics are unchanged; the metadata drops from O(operations) toward O(elements + replicas). This pattern — replace explicit per-operation evidence with a causal summary — recurs across every optimized CRDT, and its limits reappear in the boundary section.

Registers: LWW and MV

A register — one opaque value, `set` and `get` — has no internal structure to merge, so its CRDTs are really policies about concurrent writes.

The **LWW-Register** totally orders writes by (timestamp, replica-ID tiebreak); merge keeps the larger. It converges, it is tiny, and every popular "CRDT-ish" store leans on it — Cassandra's cell-level reconciliation is exactly this. Its two sins were prosecuted in Chapter 2 and Chapter 7 and both convictions stand. First, of two *concurrent* writes, one is silently discarded — "last" is a fiction imposed on writes that had no order; an update can be accepted, acknowledged, and later vaporized by a merge, with no error surfaced anywhere. Second, with wall-clock timestamps, skew decides *which* one dies — a replica with a fast clock wins arguments for seconds at a time, and the discarded write can be the causally *later* one. LWW is a legitimate choice only when overwrite-loss is genuinely acceptable (mutable profile fields, cache-like data, "any recent value is fine"). In practice its blast radius is contained by granularity: **per-field LWW** inside a CRDT map (Riak maps, Redis Enterprise hashes) loses, at worst, one concurrently-edited field rather than one of two whole documents — concurrent edits to *different* fields

both survive. That single design decision eliminates most real-world LWW grief, without fixing the concurrent-same-field case.

The **MV-Register** is the honest register: guarded by a version vector, its merge keeps *all* causally concurrent values as siblings and discards only dominated ones. It refuses to invent an order that does not exist — precisely Chapter 7's Dynamo behavior, now stated as a type. The application still faces the siblings on read, so the MV-Register is less a solution than an honest interface to the problem; its virtue is that the *state* converges deterministically and no write is silently dropped. Use it where loss is unacceptable and a human or a domain rule can adjudicate; use a real structured CRDT where one can express the merge.

Sequences: the collaborative-editing showcase

Text is the hardest case: an ordered sequence where concurrent users insert *at positions*, and positions shift under each other's edits. Sending "insert at index 12" is meaningless once a concurrent edit has moved index 12. Sequence CRDTs all share one move: **replace indices with stable position identifiers** that name a location densely — between any two identifiers, another can always be minted — so an insertion names its neighbors once and is thereafter order-independent.

RGA (Roh et al. 2011) links each character to the identifier of the character it was inserted after, breaking concurrent-sibling ties by timestamp; deletion tombstones the character. **Logoot** and its LSEQ refinement (Weiss et al. 2009) instead give each character a position from a dense ordered space — variable-length digit paths, so a fresh position always fits between neighbors — making characters totally ordered by identifier with no tombstone-free lunch either. The **Yjs/YATA** lineage and **Automerge** (Kleppmann's RGA-derived design) are the production embodiments: heavily engineered identifier encodings, run-length compression of consecutive insertions, and delta sync, bringing per-character metadata down from the naive-academic estimates that once made text CRDTs look hopeless to overheads that real editors ship. Honesty requires naming the known wart: **interleaving anomalies**. Kleppmann, Mulligan, and Gomes showed that several published sequence CRDTs can interleave two users' concurrent insertions at the same spot character-by-character — "Alice" and "Bob" typed concurrently merging as "ABlobice"-style shuffles — converged, consistent, and wrong

to any human reader. Newer designs constrain this; it is a live research edge, not a solved detail.

Collaborative editors are *the* CRDT showcase for a structural reason: the workload is almost entirely insertions, which commute beautifully once positions are identifiers; users are the replicas, offline work is the feature, and per-keystroke consensus is unthinkable. When the data type fits this well, CRDTs are not a compromise — they are simply the correct architecture.

Type	State	Merge	Concurrent-conflict semantics
G-Counter	per-replica counts	entrywise max	none possible — increments commute
PN-Counter	two G-Counters	both halves	none — but no floor invariant
G-Set	set	union	none — adds commute
2P-Set	add-set + tombstones	unions	remove wins, forever; no re-add
OR-Set	tagged entries + causal context	union minus observed tags	add-wins (remove-wins variant exists)
LWW-Register	value + timestamp	keep larger timestamp	one write silently lost; clocks decide
MV-Register	values + version vector	keep concurrent siblings	surfaced to reader — no loss, no decision
RGA / Logoot / Yjs	identified elements	positional	intention-preserving order, mostly; interleaving edge cases

What CRDTs cannot do

This section is the difference between using CRDTs and being burned by them.

Global invariants that span concurrent updates need coordination — no data type dodges this. Uniqueness ("one account per email"),

non-negativity ("balance ≥ 0 "), capacity ("at most N seats"): each of these can be violated by two operations that are *individually* legal and concurrently applied. Two replicas each check "9 seats left," each sell 5, each locally valid — merged, the invariant is dead, and no merge function can un-sell a ticket. This is not a weakness of the catalog above; it is Chapter 4's theorem wearing a different coat. Bailis et al. made it a usable test, **invariant confluence**: an invariant is preservable without coordination if and only if merging any two invariant-satisfying states yields an invariant-satisfying state. "Total ≥ 0 " fails the test (two states each at -0 distance from the floor merge below it); "this ID was generated by exactly one replica" fails; "the set only grows" passes; "each replica's entry is its own writes" passes. Run your invariant through this test *before* choosing a CRDT. If it fails, you need consensus for those operations — or the hybrid below.

Escrow and reservations: pre-partition the invariant. The classical move (O'Neil's escrow transactions, 1986; revived as *bounded counters* by Balegas et al.) is to spend coordination *ahead of time, in bulk*, so the hot path needs none: split the global allowance into per-replica allotments, and let each replica consume only its own share, coordination-free.

```
# Invariant: total sold <= 100. Coordinate ONCE to split the allowance:
rights = {"eu-west": 25, "us-east": 25, "us-west": 25, "ap-south": 25}

def sell(replica, n):
    if rights[replica] >= n:
        rights[replica] -= n          # local decision, invariant provably
    safe:                             #
        return "SOLD"                # no merge can overspend a
    partitioned right                  #
        return "TRANSFER_NEEDED"     # borrow rights from a peer –
    coordination,                     #
                                     # but rare, async-refillable, off
    the hot path
```

The invariant is safe *by construction* — a replica cannot spend rights it does not hold, and rights-transfers are the only coordinated operation, needed only near exhaustion. This is Volume 5, Chapter 9's shard-local-invariant moral, replayed at replica granularity: an invariant you can partition is an invariant you can enforce locally. The residue is a familiar trade: a replica can refuse a sale while global stock remains (its allotment is empty, a peer's is not) — you have converted potential overselling into

potential false scarcity, which for most businesses is the right direction to be wrong in.

Cross-object transactions don't compose. Each CRDT converges *individually*; nothing makes two of them converge *in step*. "Move item from cart A to cart B atomically" is not expressible as independent merges — an observer can see both or neither mid-sync. CRDT maps give you one composite object with a joint merge, which covers many cases; true multi-object atomicity is back in Volume 5's territory.

Metadata and garbage are the operational tax. Version vectors grow with the number of actors ever seen (Chapter 10's churn problem, again); OR-Set contexts grow with operations until compacted; sequence CRDTs tombstone deletions. Compaction is not free-running: discarding a tombstone is safe only when it is **causally stable** — provably delivered at *every* replica — and "every replica" is a membership question, which is why CRDT garbage collection is entangled with Chapter 10's failure detection. A replica that is partitioned-but-not-removed blocks stability forever, exactly as Volume 4, Chapter 4's stalled thread blocked epoch-based reclamation: same shape, one process wide there, one cluster wide here. Production systems bound actors (Riak's per-object actor limits), lease replica IDs, or accept operator-triggered compaction. Budget for this; it is where CRDT deployments actually hurt.

Real deployments, honestly described

Riak shipped the first serious database CRDT suite (Riak 2.0's data types: counters, sets, maps, flags, registers, built on the optimized OR-Set work) — the direct answer to its own siblings problem from Chapter 7. **Redis Enterprise's Active-Active** geo-replication implements its types as CRDTs conceptually equivalent to the catalog above — counters merge per-replica contributions, sets are observed-remove, strings offer LWW — giving multi-region writes with typed merges rather than Volume 5 Chapter 8's ad hoc rules. **Akka Distributed Data** provides `ORSet`, `PNCounter`, `LWWMap` and friends replicated by gossip with delta propagation, as cluster-internal shared state. **Phoenix Presence** is a lovely small example: "who is online in this channel" tracked per-node as an observed-remove structure and merged by gossip — no central presence store, and a node crash simply stops contributing state. **Automerger** and **Yjs** anchor the local-first ecosystem: full document

CRDTs (maps, lists, text, counters) embedded in applications, syncing peer-to-peer or through dumb relays.

The pattern in what fits: **shopping carts, counters and metrics, presence and membership, feature flags, collaborative documents** — data where the merge semantics of the catalog match the domain's own answer to "what should happen?" The anti-fit is equally clear: **ledgers and inventory** — anything whose essence is a non-confluent invariant. Sell inventory with a PN-Counter and you have chosen overselling; keep a balance in one and you have chosen overdraft. Those need consensus, or escrow, and a vendor page saying "CRDT-based" does not repeal the theorem.

Using CRDTs well

Choose semantics first, type second. The type catalog is really a menu of *answers to business questions*, and picking the type is picking the answer. Concurrent add and remove: should the item stay (cart — add-wins OR-Set) or go (access revocation — remove-wins variant)? Concurrent writes to one field: is losing one acceptable (LWW) or must both surface (MV-Register)? Write the chosen answer down where product owners can see it, because it is a product decision — "concurrent re-add beats delete" changes user-visible behavior. Teams that skip this step rediscover it in production as a bug report that is actually a semantics dispute.

Design the sync topology deliberately. State/delta CRDTs compose naturally with Chapter 10's gossip and Chapter 7's anti-entropy: periodic pairwise exchange with random peers gives propagation in $O(\log N)$ rounds with per-node cost independent of cluster size, and CRDT idempotence means the gossip layer needs no reliability features at all. Op-based types need the causal-broadcast substrate — buy it from a library (Akka, Automerge's sync protocol, Yjs providers) rather than building it; its edge cases are Chapter 2's edge cases.

Test convergence the only way that works: property-based, in random orders. Convergence bugs are order-dependent by definition; example-based tests cannot find them. Generate random operations at random replicas, sync in random pairwise orders with duplication, and assert equality — a direct check of commutativity, associativity, and idempotence in one property:

```

from hypothesis import given, strategies as st
import random

ops = st.lists(st.tuples(st.integers(0, 2),          #
                        st.sampled_from(["add", "remove"]),
                        st.sampled_from("abc")), max_size=40)

@given(ops=ops, seed=st.integers(0, 2**32))
def test_orset_strong_convergence(ops, seed):
    rs = [ORSet(i) for i in range(3)]
    for rid, op, elem in ops:
        getattr(rs[rid], op)(elem)          # ops applied at
home replica only
    rng = random.Random(seed)
    for _ in range(30):                    # random gossip,
WITH duplicates
        i, j = rng.sample(range(3), 2)
        rs[i].merge(rs[j])
    for i in range(3):                    # one final full
exchange so all
        for j in range(3):                # updates are
delivered everywhere
            rs[i].merge(rs[j])
    assert rs[0].entries == rs[1].entries == rs[2].entries # SEC:
same updates,
                                                    # same
state. Period.

```

Note what the final full exchange encodes: SEC promises identical state given the same *delivered* set, so the test forces full delivery, while the random middle section stresses every order, grouping, and duplication on the way there. Extend it with partitions (withhold merges between groups for a while) and semantic assertions ("a remove that observed an add deletes it"). This slots directly into Chapter 12's toolbox — the same move as simulation testing, applied to a data type — and it is cheap enough that there is no excuse for shipping a hand-rolled merge without it. Which is also the moment to repeat Volume 4's advice verbatim: prefer the library (Riak, Akka, Automerge, Yjs) to your own implementation; the optimized variants especially are subtle, and published ones have shipped with bugs.

The distributed-systems lens

CRDTs close the arc this volume has been drawing since Chapter 3, and the closing thought is about *when coordination is necessary at all*.

Chapter 3 laid out the consistency spectrum and priced its strong end; Chapter 4 said the real everyday trade is PACELC's ELSE branch — latency versus consistency on the healthy path. CRDTs are the EL corner made *rigorous*: not "we relaxed consistency and hope the conflicts are rare," but a precise guarantee (SEC), achieved with zero synchronous coordination, with conflict handling that is total, deterministic, and chosen in advance. The spectrum's weak end, done properly, turns out to have a theory as sharp as the strong end's.

The generalization is the **CALM theorem** (Hellerstein and Alvaro): a program has a consistent, coordination-free distributed implementation *if and only if* it is expressible in monotone logic — logic where new information can only produce new conclusions, never retract old ones. Monotone questions ("has this event been seen?", "what is the union so far?") never need to wait, because no future message can make a "yes" wrong. Non-monotone questions ("is this name unique?", "is the balance still positive?", anything with negation or an aggregate over *all* data) must wait, because answering requires knowing that nothing is missing — and knowing that is what coordination *is*. CRDTs are CALM's data-structure face: a join-semilattice is monotonicity made concrete (state only inflates), and invariant confluence is the working engineer's version of the same test. Together they turn "can we skip consensus here?" from a matter of taste into a property you can check per operation — this volume's most practical sentence.

And the full circle: Volume 4, Chapter 4 ended by promising that `LongAdder` was a CRDT counter in miniature. You can now read the whole progression as one idea crossing scales. A contended `AtomicLong` and a Raft leader are the same design: one serialization point, exact answers, throughput bounded by coordination. `LongAdder` cells and G-Counter entries are the same escape: give every participant its own monotone stripe, let stripes meet in a join, read an answer that is exact only at rest. Cache-line ping-pong and consensus round trips are the same cost; striping and SEC are the same purchase. The trajectory of the field runs the same direction: **local-first software** (Kleppmann et al.'s program: your data on your device, servers as sync accelerators rather than

owners) and edge computing both need writes accepted far from any quorum, and both are pushing CRDTs from research darling to default infrastructure. The coordination-avoidance principle underneath is this volume's parting moral: coordinate where the invariant demands it — and prove, not assume, that it demands it — because everywhere else, the lattice is free.

Key takeaways

- CRDTs are the third answer to concurrent updates: not app-side merge (Chapter 7), not serialize-everything (Chapters 5–6), but types whose **merge is built in and provably convergent**.
- The math is the **join-semilattice**: states form a partial order, merge is the least upper bound, and its three properties each kill a pathology — commutativity kills reordering, associativity kills topology sensitivity, **idempotence kills duplication and makes at-least-once anti-entropy safe with no dedup machinery** (Chapter 9's dream primitive).
- **Strong Eventual Consistency**: replicas that have delivered the same set of updates are in the same state *immediately* — convergence is a function of the update set, not of order, and there is no conflict-resolution step because merge is total.
- **State-based** CRDTs need only eventual, unreliable delivery but ship state (delta-CRDTs fix the bandwidth); **op-based** CRDTs ship small operations but require exactly-once, causally ordered delivery. The two are formally equivalent; the delivery requirements decide.
- Know the catalog by its semantics: G-Counter (single-writer entries, max-merge — LongAdder across machines), PN-Counter (two G-Counters, no floor), OR-Set (**unique tags; removes kill only observed tags; add-wins**), LWW-Register (silently loses concurrent writes and trusts clocks — Chapter 2's warning applies; per-field LWW contains the damage), MV-Register (honest siblings), sequence CRDTs (stable position identifiers; interleaving anomalies are a real edge).
- The boundary is **invariant confluence**: uniqueness, non-negativity, and capacity invariants are not confluent and require consensus — or **escrow/reservations**, which pre-partition the invariant so the hot path is coordination-free. Ledgers and inventory are the anti-fit.

- Metadata is the operational tax: causal contexts and tombstones grow, and compaction requires **causal stability**, entangling CRDT GC with membership (Chapter 10) — a lost-but-not-removed replica blocks GC like a stalled thread blocks epoch reclamation.
- Use them well: **pick merge semantics as an explicit business decision first**, gossip for state-based sync, library-provided causal broadcast for op-based, and **property-test convergence** under random orders, duplication, and partitions (Chapter 12).
- The synthesis: CRDTs are the principled far end of Chapter 3's spectrum, the rigorous EL of Chapter 4's PACELC, and the data-structure face of **CALM** — monotone logic needs no coordination. Coordinate only where a non-confluent invariant forces it.

Further reading

- Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M., "Conflict-free Replicated Data Types," *SSS 2011* — the SEC definition and the CvRDT/CmRDT framework. <https://inria.hal.science/hal-00932836>
- Shapiro, M., Preguiça, N., Baquero, C., Zawirski, M., "A Comprehensive Study of Convergent and Commutative Replicated Data Types," INRIA Research Report RR-7506, 2011 — the type catalog in full formal detail. <https://inria.hal.science/inria-00555588>
- Almeida, P. S., Shoker, A., Baquero, C., "Delta State Replicated Data Types," *JPDC* 111, 2018 — delta-CRDTs. <https://arxiv.org/abs/1603.01529>
- Bieniusa, A. et al., "An Optimized Conflict-free Replicated Set," INRIA RR-8083, 2012 — the tombstone-free OR-Set behind Riak's implementation. <https://arxiv.org/abs/1210.3368>
- Hellerstein, J. M., Alvaro, P., "Keeping CALM: When Distributed Consistency Is Easy," *CACM* 63(9), 2020 — the CALM theorem, accessibly. <https://arxiv.org/abs/1901.01930>
- Bailis, P. et al., "Coordination Avoidance in Database Systems," *VLDB* 2015 — invariant confluence. <https://arxiv.org/abs/1402.2237>
- Balesgas, V. et al., "Putting Consistency Back into Eventual Consistency," *EuroSys* 2015 — bounded counters and reservation-style invariant enforcement.
- Kleppmann, M., "CRDTs: The Hard Parts" (talk, 2020) — tombstones, metadata overhead, and sequence-CRDT subtleties from the

Automerge author. <https://martin.kleppmann.com/2020/07/06/crdt-hard-parts-hydra.html>

- Kleppmann, M., Mulligan, D. P., Gomes, V. B. F., et al., "Interleaving Anomalies in Collaborative Text Editors," *PaPoC 2019* — the interleaving problem, precisely.
- Kleppmann, M., Wiggins, A., van Hardenberg, P., McGranaghan, M., "Local-first Software: You Own Your Data, in Spite of the Cloud," *Onward! 2019* — <https://www.inkandswitch.com/local-first/>
- Automerge documentation — <https://automerge.org/docs/> — and Yjs documentation — <https://docs.yjs.dev/> — the production document-CRDT libraries.
- Riak Data Types documentation — <https://docs.riak.com/riak/kv/latest/developing/data-types/> — the first mainstream database CRDT suite.
- Chapter 2 — Time, Clocks, and Ordering — causal delivery, version vectors, and why LWW's timestamps lie.
- Chapter 7 — Quorum Systems and Dynamo-Style Replication — the siblings problem CRDTs answer.
- Chapter 12 — Testing Distributed Systems — the property-based and simulation techniques the convergence test belongs to.
- Volume 4, Chapter 4 — Atomics, CAS, and Lock-Free Data Structures — `LongAdder`, the in-process ancestor of the G-Counter.

Chapter 12 — Testing Distributed Systems: Jepsen, Chaos, and Simulation

What this chapter covers. Volume 4, Chapter 10 — Testing and Debugging Concurrent Systems closed with a promise: every technique there has a distributed sibling, and this chapter is where the sibling lives. It is also where this volume settles its accounts. Eleven chapters have made claims — that a linearizable register behaves like *this*, that Raft elects at most one leader per term, that a CRDT converges, that an idempotent consumer survives duplicate delivery — and claims that cannot be checked are marketing. This chapter is about checking them:

what the properties even are (histories, safety, liveness, consistency contracts as testable predicates); how Jepsen-style testing records concurrent histories against a real cluster under injected faults and checks them offline; fault injection as a discipline with a menu mapped to Chapter 1's failure models; deterministic simulation testing in the FoundationDB lineage, the strongest current practice; TLA+ and its relatives for the design layer; property-based testing for distributed components; and the verification that continues after deploy. The synthesis artifact is a testing pyramid for distributed systems — six layers from protocol spec to production checkers, each with stated coverage and stated blind spots — because no single layer is sufficient and every layer is cheap precisely where the layers above it are blind.

Learning goals — after this chapter you should be able to:

- Explain why the cross-node interleaving space defeats conventional testing even more thoroughly than the single-machine case, and why no compiler-level instrumentation chokepoint exists to rescue it.
- Define history-based correctness checking precisely: record invocations and responses, search for a legal sequential witness respecting real-time order; state why the search is NP-hard and how Knossos and Porcupine cope in practice.
- Describe the Jepsen method as a method — generators, clients, nemesis, checker — including the standard fault repertoire and nemesis composition patterns, and characterize the classes of findings it has produced without overclaiming specifics.
- Build a fault-injection toolkit mapped to Chapter 1's failure models, using iptables, tc netem, toxiproxy, and mesh-level injection, and manage the reproducibility problem with seeds and shrinking.
- Explain deterministic simulation testing: what the simulator must own, what the architecture costs up front, why seeded replayable universes invert the economics of rare bugs, and how the modern tools (TigerBeetle's simulator, Antithesis, madsim/turmoil) differ.
- Decide when TLA+ pays, write and read a simple invariant, and state the refinement gap honestly.
- Position production verification — invariant checkers, shadow traffic, canaries, chaos with blast radius — as the pyramid's top layer, with the practice and culture deferred to Volume 11.

Why the network makes it worse

Start from Volume 4, Chapter 10's arithmetic: n threads of k steps generate an astronomical interleaving space, a unit test samples one boringly fair schedule per run, and everything that works either extracts more evidence per run, explores schedules systematically, or shrinks the space by design. Every clause of that argument survives the move to distributed systems, and every clause gets worse.

First, the space itself grows new dimensions. On one machine the nondeterminism is scheduling: which thread runs next. Across machines, the schedule is joined by the network — every message may be delayed arbitrarily, reordered against its siblings, duplicated by a retry layer, or dropped; links may fail asymmetrically (A hears B, B does not hear A); partitions may isolate any subset from any other, including the partial and overlapping partitions of Chapter 10 — Failure Detection that no clean "split into two halves" mental model captures. Add clock skew (Chapter 2: every node has its own idea of now, and NTP merely bounds the disagreement on a good day), and add partial failure itself — any node may crash at any step and, in the crash-recovery model, return with its disk but not its memory. The single-machine interleaving space was the multinomial coefficient of thread steps. The distributed space is that, per node, *multiplied* by every subset of messages that could be delayed past every other, *multiplied* by every fault the failure model admits at every point. Nobody writes this number down; the point is that sampling it uniformly is not on the menu.

Second — and this is the deeper problem — the chokepoint is gone. A race detector works because a compiler can instrument *every* access to shared memory and a runtime can maintain happens-before over all of them: one process, one address space, total visibility. There is no analogous vantage point across machines. State is shared by messages that traverse NICs, kernels, switches, and cloud fabric you do not control; the closest thing to "instrument every access" would be instrumenting the network itself, and even then you would see packets, not intentions. This is why the entire distributed testing tradition pivots from *instrumenting the mechanism* to *checking the history*: observe the system only at its client boundary — operations invoked, responses returned — and ask whether that observable history is one the claimed consistency contract permits. The internal mechanism becomes a black box; the contract, in the sense of Chapter 3, becomes the test oracle.

Third, the failure modes live precisely where unit tests never go. A distributed bug is almost never inside one node's code path, which unit tests cover fine; it is in the *cross-node interleaving* — the leader that commits an entry while a partition is forming, the client whose lease expires during a GC pause (Chapter 8's paused-client, the single most reliable producer of split-brain war stories), the read served by a replica that missed one message. No test that runs a single process can even *express* these schedules. And production, as always, is the flakiest scheduler you own: Chapter 1's asynchronous model — no bound on message delay, no bound on processing time — is not a theoretician's abstraction but a description of a congested top-of-rack switch at 2 a.m. Production will eventually visit the interleaving your tests could not express. The techniques below are all ways of visiting it first.

What are we even checking

Before any tool, the vocabulary — because "we tested it and it worked" is meaningless until you can say what property held.

Safety, liveness, and histories

Chapter 1's partition of properties governs everything here. A **safety** property says nothing bad ever happens: no two leaders in one term, no committed write lost, no read returns a value that was never written. Safety violations are *finite* evidence — a specific history prefix that is already wrong — which makes them the natural target of testing: a checker can hold up a concrete history and say *this* is the violation. A **liveness** property says something good eventually happens: elections eventually complete, replicas eventually converge. No finite observation refutes "eventually," so liveness testing is necessarily cruder — timeouts standing in for eventually, with the attendant false alarms. Every serious distributed test suite is overwhelmingly a safety suite, with liveness checked as "made progress within a generous bound," and you should read any test report with that asymmetry in mind.

The object being checked is a **history**: a sequence of invocation and response events, recorded at the clients, each tagged with the process, the operation, and its wall-clock interval. Operations whose intervals overlap are concurrent; operations where one's response precedes the other's invocation are ordered in real time. Everything downstream —

linearizability checking, Elle's anomaly search, Jepsen's verdicts — is a predicate over histories.

Linearizability checking, precisely

Chapter 3 defined linearizability: every operation appears to take effect atomically at some instant between its invocation and its response, consistently with the object's sequential specification. The definition is directly operational. To check a recorded history, search for a **sequential witness**: a total order of all completed operations that (a) is a legal execution of the sequential object — reads return the latest preceding write — and (b) respects real time — if operation X completed before operation Y began, X precedes Y in the witness. If a witness exists, the history is linearizable; if none exists, you hold a correctness violation regardless of what the implementation did internally, which is the entire charm of black-box checking.

Walk one tiny history for a single register, initially 0, with three clients:

A:	---- write(1) ----	ok at t=30
B:	-- read --	returns 1 at t=20
C:	-- read --	returns 0 at t=50

Consider first only A and B. B's read overlaps A's write and returns 1. Witness: `write(1), read=1` — the write linearizes early in its interval, before B's read. Legal, real-time respected: the history so far is linearizable. Now add C: a read that *begins after A's write completed* ($t=40 > t=30$) and returns 0. Any witness must place C's read after `write(1)` by real-time order, and a legal register cannot return 0 after 1 was written with no intervening write. No witness exists — the history is not linearizable. This exact shape, a **stale read after a completed write**, is the most common real-world verdict: it is what a replica serving reads behind a partition looks like from the outside.

```

sequenceDiagram
    participant A as Client A
    participant R as Register history
    participant B as Client B
    A->>R: invoke write 1
    B->>R: invoke read
    R-->>B: response 1
    R-->>A: response ok
    Note over R: witness search:
write 1 then read=1
legal and respects real time – OK
    participant C as Client C
    C->>R: invoke read
    R-->>C: response 0
    Note over R: C began after write completed
no legal order puts read=0 after write 1
NO witness – violation

```

The search is expensive in principle: deciding linearizability of an arbitrary history is **NP-complete** (Gibbons and Korach, 1997), because concurrent operations can be permuted combinatorially and each choice constrains the rest. Practical checkers survive on three observations. The **Wing-Gong / WGL** approach is a backtracking search: pick a minimal operation (one no other completed operation must precede), apply it to the sequential model, recurse, undo on failure — with memoization of already-refuted configurations doing most of the saving. **Knossos**, Jepsen's checker, implements this lineage in Clojure; **Porcupine** (Go) is the implementation most engineers will actually embed in their own tests, checking histories against a user-supplied sequential model with partitioning tricks — a history over independent keys splits into per-key histories checked separately, which converts many exponential problems into many small ones. The operational consequence: keep checked histories *short and contended*. A thousand operations on one hot key is a strong test and a feasible check; a million operations sprayed over a million keys is neither.

One recording subtlety matters more than any algorithm: a client that times out does not know whether its operation executed. The checker must treat such an operation as *possibly present*, *possibly absent*, *possibly taking effect at any later point* — an indeterminacy that both bloats the search and is absolutely mandatory for soundness, because counting a timed-out write as definitely-absent will flag correct systems

and counting it as definitely-present will excuse broken ones. Chapter 9's two-generals shadow falls across the test harness too.

Checking weaker contracts

Most systems do not claim linearizability, and checking a system against a contract it never signed is noise. The weaker models of Chapter 3 have their own predicates. **Sequential consistency** drops the real-time constraint: some total order must explain all results, per-process order preserved — strictly easier to satisfy, still expensive to check. **Causal consistency** and the **session guarantees** (read-your-writes, monotonic reads) are checked as graph conditions over the history: build the order that must hold — program order, writes-into order — and look for cycles or for a read observing a state that its session's past forbids.

For transactions, the state of the art is **Elle** (Kingsbury and Alvaro, 2020), and its idea is worth understanding even if you never run it. Rather than searching for a witness, Elle searches for *anomalies* — the cycles that Adya's formalization of weak isolation (Volume 5, Chapter 6's anomaly zoo) proved must exist when isolation is violated. The trick is to make version order *inferable*: operate on data types whose history reveals ordering — appends to a list, whose successive reads expose exactly which writes happened in which order — so the test can reconstruct a write-follows-write graph without any access to database internals. Then combine the inferred version order with per-transaction read/write dependencies and search the combined graph for cycles: a write-write cycle is **G0** (dirty write territory), cycles involving reads of uncommitted or intermediate data are **G1** violations, a cycle with exactly one anti-dependency is **G-single** — the signature of snapshot isolation's write skew. Every cycle found is a *certificate*: a concrete set of transactions that no serial (or SI, or RC — depending on the claimed level) execution could produce. Cycle search in a graph is polynomial, which is how Elle checks histories of millions of transactions that would be hopeless for a witness search — the trade being that it checks for known anomaly patterns rather than proving full conformance.

Jepsen: the method, not the lore

Jepsen is famous for its findings; what deserves study is the method, because it is reusable by any team against any stateful system, and

because each stage embodies a decision you will face when you build your own harness.

```
flowchart LR
  GEN["Generator  
stream of operations  
reads, writes, CAS"]
  CLI["Clients  
real client libraries  
against real cluster nodes"]
  NEM["Nemesis  
fault injector  
partitions, kills, pauses, skew"]
  SUT["Real cluster  
5 nodes, real network,  
real disks"]
  HIST["History  
every invocation and response  
timestamped, timeouts marked"]
  CHK["Checkers offline  
Knossos, Elle, custom invariants"]
  VERDICT["Verdict plus  
counterexample history"]
  GEN --> CLI --> SUT
  NEM --> SUT
  CLI --> HIST --> CHK --> VERDICT
```

The pipeline: a **generator** emits a randomized stream of operations — for a register test, reads, writes, and compare-and-sets on a small set of hot keys; for a transactional test, Elle's list-append transactions. **Clients** — real client libraries, speaking the system's real wire protocol, because the client library's retry and failover behavior is part of the system under test — execute these against a small real cluster, typically five nodes, while recording every invocation and response into the history. Concurrently, the **nemesis** injects faults into the cluster on its own schedule. When the run ends, the harness heals all faults, waits for the cluster to settle, performs final reads, and hands the complete history to **checkers** offline. The separation is the design's power: fault injection needs no knowledge of the consistency model, and the checker needs no knowledge of the faults — the history is the only interface between them.

The nemesis repertoire is mundane Unix, which is a feature — every fault is one you can explain to an auditor. Partitions are iptables rules dropping traffic between chosen node subsets. Process crashes are `kill -9`; crash-recovery is `kill` plus `restart` with the data directory intact. Process

pauses are SIGSTOP / SIGCONT — and pause deserves emphasis, because it is the **GC-pause simulator**: a stopped process is exactly Chapter 8's paused client, the one whose session lease expires while it holds a lock it believes is still valid, and systems that survive crashes routinely fail under pause because a paused node *comes back and keeps talking* with stale beliefs, which a crashed node never does. Clock skew is injected by stepping or slewing node clocks — the direct test of every "assuming well-synchronized clocks" clause from Chapter 2, and of every last-write-wins scheme that quietly bet on one.

Partition *shape* matters as much as partition presence, and the named patterns each target a different protocol weakness. A **majority/minority split** (3-2) tests the basic quorum story of Chapters 5-7: the majority side should retain availability, the minority side should refuse or degrade per the model, and — the actual test — writes accepted by the minority before it noticed must not be silently lost on heal. A **bridge** partition puts one node in both halves — A and B see each other only through C — attacking protocols that implicitly assume connectivity is transitive. **Partial**, asymmetric partitions (A reaches B, B cannot reach A) target failure detectors (Chapter 10) and any leader-lease design where "I can hear the followers" is mistaken for "the followers can hear me." Composed nemeses interleave these with pauses and clock jumps in randomized sequences, because real incidents are compositions too.

What has a decade of Jepsen analyses actually found? Characterized as classes — the specifics live in the published reports, and the reports are the citable record: **stale and dirty reads under partition** in systems claiming strong consistency, usually a replica or a deposed leader serving reads it should have refused; **lost updates and lost writes** in systems using last-write-wins conflict resolution over skewed clocks — Chapter 2's warning and Chapter 7's Dynamo-lineage trade-off made empirical, acknowledged writes disappearing after heal; **transactional isolation weaker than advertised** — systems documenting serializable or snapshot isolation exhibiting Elle-detectable anomalies of levels below their claims; **split-brain during leader election** — two nodes simultaneously believing themselves leader across a partition or a pause, each accepting writes, violating precisely the single-leader-per-term invariant Chapter 6 proved for correct Raft; and **retry-amplified duplicate effects** where client libraries retried non-idempotent operations across failover (Chapter 9's territory). Two meta-observations matter more than any single class. First, most findings needed *composed*

faults and would not reproduce under any single clean fault — the bugs live in the corners. Second, the ecosystem effect: a large fraction of analyzed systems shipped fixes in response, and several vendors subsequently built Jepsen-style harnesses into their own CI. The method's public legacy is less "database X was broken in 2015" than the normalization of adversarial, history-checked testing as table stakes for anyone claiming a consistency model.

The limits are structural, and Jepsen's own documentation is candid about them. It is **black-box**: it observes the client boundary, so an internal invariant violation that never surfaces in client-visible results is invisible. It is **probabilistic**: a run samples the fault space, so the absence of a counterexample is evidence, not proof — a clean ten-hour run means the faults tried, in the orders tried, at the moments tried, produced no visible violation. Runs are **slow and poorly reproducible**: real clusters, real timing, so a failure seen once may take days to see again, and bisecting a fix against a probabilistic failure is miserable. And checking cost bounds history size, which bounds how much load the test can even generate. These limits are exactly the ones deterministic simulation exists to remove — at a price Jepsen does not charge, namely that Jepsen tests your *real stack*, JVM and kernel and client library included.

Fault injection as a discipline

Strip the harness away and the nemesis remains: deliberately inflicting the failure model on a running system. Done casually — "let's kill a pod and see" — it produces anecdotes. Done as a discipline, it starts from Chapter 1's failure hierarchy and asks, for each model the design claims to tolerate, *have we actually inflicted it?*

Failure model (Ch. 1)	Injection	Notes
Crash-stop	<code>kill -9</code> , VM terminate	The easy one; most systems survive it
Crash-recovery	Kill, restart with disk intact	Tests recovery code and log replay; corrupt or truncate the tail of the log for the honest version

Failure model (Ch. 1)	Injection	Notes
Omission / partition	iptables drop, netem loss, link removal	Include <i>asymmetric</i> drops; symmetric partitions flatter your failure detector
Timing	SIGSTOP / SIGCONT, clock step/slew, netem delay	The pause is the GC/VM-migration simulator; most valuable and least practiced
Byzantine	Message corruption, lying nodes	Out of scope for most shops — say so explicitly in the test plan rather than pretending; Chapter 1's argument for when 3f+1 is worth it stands

The standard Linux toolkit covers most of the menu. `tc netem` shapes a link's behavior — delay with jitter, loss, reordering, duplication — and `iptables` implements partitions:

```
# Degrade eth0: 100ms +/- 50ms delay, 1% loss, 0.3% duplication,
reordering
tc qdisc add dev eth0 root netem delay 100ms 50ms loss 1% duplicate 0.3
% reorder 25% 50%

# Partition: node 10.0.0.3 becomes unreachable in BOTH directions...
iptables -A INPUT -s 10.0.0.3 -j DROP
iptables -A OUTPUT -d 10.0.0.3 -j DROP

# ...or asymmetric: we hear them, they never hear us (drop only
outbound)
iptables -A OUTPUT -d 10.0.0.3 -j DROP

# Heal
iptables -F && tc qdisc del dev eth0 root
```

eBPF-based tools refine this with per-flow and per-syscall precision — dropping only heartbeats, only packets larger than a threshold, only the third retransmission — which is how you reach faults netem's uniform randomness will practically never compose. One layer up, **Toxiproxy** (Shopify) proxies a TCP upstream and injects "toxics" per connection, which is the right tool for application-level tests: your service's connection pool, timeouts, and retry policy exercised against a

dependency that is slow, half-duplex-broken, or resetting — without touching kernels:

```
toxiproxy-cli create --listen 0.0.0.0:26379 --upstream
redis.internal:6379 redis
toxiproxy-cli toxic add --type latency --attribute latency=800 --
attribute jitter=300 redis
toxiproxy-cli toxic add --type limit_data --attribute bytes=1024 --
downstream redis
```

Service meshes lift the same idea to HTTP/gRPC — Istio and friends can inject delay and abort faults per route by configuration, which makes fault injection deployable to a staging environment without owning any node (mechanics in Volume 12's mesh chapters). The general rule: inject at the lowest layer that produces the failure honestly. A mesh-injected HTTP 503 does not test your TCP keepalive handling; a netem-blackholed connection does.

The discipline's hard problem is **reproducibility**. A fault-injected test that fails once and never again has negative value — it consumed an engineer-week and returned a ghost. The mitigations are unglamorous and non-optional: **seed everything** — the operation generator, the fault scheduler, every random choice — and print the seed in the failure report; **log everything** — the full operation history and the full fault schedule with timestamps, so the failing run is at least forensically analyzable even if it will not rerun; and **shrink** — property-testing's minimization applied to fault schedules, replaying with faults removed one at a time to find a minimal schedule that still fails. Shrinking against a real cluster is expensive and only statistically valid (the bug is probabilistic), which is one more finger pointing at the same conclusion: if you want *exact* replay, the environment itself must be deterministic.

Deterministic simulation testing

That conclusion has a name, and its canonical implementation is **FoundationDB** — the story told properly in Will Wilson's Strange Loop talk and the FDB testing documentation, and the direct escalation of Volume 4, Chapter 10's deterministic-simulation section from one process to a cluster.

FoundationDB is written in Flow, a C++ extension compiling actor-style concurrency to a single-threaded event loop. In simulation, an *entire*

cluster — every process, plus the network, the disks, and the clocks — runs inside **one ordinary OS process**, single-threaded, driven by one seeded PRNG. The network is a simulated network delivering simulated packets with randomized delays, drops, and partitions; the disk is a simulated disk with randomized latencies, reorderable writes, and injectable corruption; time is a virtual clock that advances only when the simulator advances it, so a "60-second election timeout" costs microseconds of wall clock and a simulated week of cluster time fits in a coffee break. Crucially, the code being tested is the **real implementation** — the same Flow code that ships — compiled against simulated interfaces rather than a model of the code. On top, the simulator injects faults at rates no physical environment approaches, including *buggify* points: deliberate rare-behavior amplifiers the production code carries (buffer this write, return this error early, pick the worst timeout) that only activate in simulation.

```
flowchart TB
    SEED["Seed N  
one PRNG drives everything"]
    SIM["Single-threaded simulator  
owns scheduling, virtual time,  
simulated network and disks"]
    P1["Real node code  
process 1"]
    P2["Real node code  
process 2"]
    P3["Real node code  
process 3..n"]
    FAULTS["Injected faults  
partitions, crashes, disk corruption,  
clock skew, buggify points"]
    CHECK["Invariant checks and workloads  
run inside the simulation"]
    REPLAY["Any failure replays EXACTLY  
from its seed — bisectable,  
debuggable forever"]

    SEED --> SIM
    SIM --> P1
    SIM --> P2
    SIM --> P3
    FAULTS --> SIM
    SIM --> CHECK --> REPLAY
```

The payoff is an inversion of the flaw of averages that governs all probabilistic testing. A Jepsen run samples the fault space at wall-clock

speed and cannot rerun a hit. A simulation fleet runs *hundreds of thousands of seeded universes per night*, each one faster than real time, each one biased toward disaster — and any failure, however rare its trigger, **replays exactly** from its seed, every time, forever. The seed is the bug report. Irreproducibility — the property that makes distributed bugs expensive — is simply deleted, and with it most of the cost of the long-tail bug. The cultural consequence at FoundationDB, widely cited and consistent with the public record, is that the team found essentially all of its serious correctness bugs in simulation before users could, to the point that the engine's reliability became its reputation — and that when a rare production issue did appear, the reflex was to ask why the simulator had not found it, then extend the simulator.

The price is architectural and paid entirely up front: **all nondeterminism must flow through the simulator**. Every timestamp, every random number, every network call, every disk write, every spawn of concurrent work must go through an interface the simulator can own; one direct call to `gettimeofday` or one real thread torpedoed determinism silently. This is Volume 4, Chapter 10's inject-the-clock-and-executor discipline escalated from good practice to load-bearing constraint, and it is why retrofitting DST onto an existing codebase ranges from expensive to impossible — FoundationDB built Flow *before* building the database, and considered the simulator the more important artifact.

The modern wave divides by where determinism is enforced. **TigerBeetle** (a financial-ledger database) follows the FDB path — its VOPR simulator runs whole clusters deterministically with storage-fault injection down to misdirected and torn writes, and the project treats simulator-hours as a first-class engineering metric. In the Rust ecosystem, **madsim** and **turmoil** provide simulated runtimes — deterministic executors plus simulated networks — so that code written against the right abstractions (tokio-style, with time and I/O injected) can run thousands of seeded cluster-simulations in ordinary tests; they demand the same discipline (no raw `Instant::now()`, no raw sockets) at library rather than language level. **Antithesis** — founded by FoundationDB alumni — moves the enforcement *below* the code: a deterministic hypervisor executes an entire unmodified software stack (containers, kernels, your binaries) deterministically, injecting faults and exploring branches, with snapshot-and-rewind used to steer execution toward interesting states. Conceptually it buys FDB-style replayability *without* the architectural rewrite, at the cost of running inside a

specialized platform and of guidance: a hypervisor does not know your invariants, so you still write the workloads and the checks.

Position DST against Jepsen honestly, because you want both and they blind-spot each other:

	Jepsen-style	Deterministic simulation
Stack under test	Real binaries, real kernel, real network, real client libraries	Real logic, simulated environment
Reproducibility	Statistical; failures may never recur	Exact, from seed
Throughput of fault space	Wall-clock bound	Thousands of universes nightly, faster than real time
Blind spots	Rare interleavings; internal invariants	Everything the simulator does not model: kernel bugs, TCP corner cases, real disk firmware, GC behavior
Cost	A harness and a cluster; weeks	The architecture itself; must be designed in
Adoptable mid-life?	Yes	Rarely, absent Antithesis-style hypervisors

A simulation pass plus a Jepsen pass is strong evidence. Either alone is one eye open.

Formal methods, pragmatically

Everything so far tests an *implementation*. A different class of bug lives above implementations: the protocol itself is wrong, and every correct implementation of it will fail. Testing cannot reliably surface design bugs whose triggers need five nodes, two partitions, and a specific message order — but **model checking** can, because at the design level the state space of a small instance is exhaustively enumerable.

TLA+ is the working engineer's tool here. You specify the protocol as a state machine — variables, an initial predicate, a next-state relation — plus the invariants and temporal properties it claims, and the TLC model

checker enumerates *every* reachable state of a small instance (three nodes, two keys, a handful of messages) and checks every claim in every state, producing a minimal counterexample trace on failure. The flavor, in two lines any Raft reader can verify against Chapter 6:

```
\* Election safety: at most one leader per term
ElectionSafety ==
  \A t \in Terms :
    Cardinality({s \in Servers : state[s] = "Leader" /\
currentTerm[s] = t}) <= 1
```

An invariant like this, checked over every reachable state of a five-server model, is a different kind of evidence than any test run: within the model's bounds, it is exhaustive. PlusCal, an algorithm-shaped syntax that translates to TLA+, lowers the entry cost for engineers who want sequential-looking pseudocode rather than raw temporal logic.

The credible industrial witness is Amazon's experience report, "How Amazon Web Services Uses Formal Methods" (CACM, 2015): teams including S3 and DynamoDB used TLA+ on production-critical designs and reported finding subtle, serious bugs in *designs that had already passed careful human review* — bugs whose shortest counterexample traces ran to dozens of steps, which is precisely the depth code review and testing do not reach. Equally telling, the report's authors found the specifications valuable as *documentation*: a spec is the one precise, executable statement of what the protocol assumes and guarantees, immune to the drift that afflicts design docs. That experience — design bugs found pre-implementation, "exhaustively testable design" as a goal — is the honest sales pitch, and it comes from teams whose failure costs justify the effort.

The limits, equally honestly. **The spec is not the code.** TLC verifies the model you wrote, and the implementation can diverge from it in any way it likes — the *refinement gap*. Unless you do refinement proofs (heroic, rare in industry) or conformance-check the implementation against the spec (an active area; some shops replay implementation traces against the model), the spec's guarantee stops at the design boundary. Model checking is also bounded: TLC exhausts *small* instances, and while small-scope experience says most design bugs appear at small scale, that is an empirical regularity, not a theorem. Neighbors worth knowing: the **P** language (used in Windows device drivers and at AWS) specifies systems as communicating state machines and checks them with the pleasant

property that specs look like programs; **Alloy** does bounded relational checking, strong for data-model and configuration invariants.

When does it pay? The honest heuristic: when the protocol is *novel*, when it is *consensus-adjacent* (leases, leader election, membership, replication — anything where Chapter 5 and 6 subtleties lurk), or when it *moves money or data irrecoverably*. A week of TLA+ on a new cross-region replication protocol is cheap insurance. TLA+ on a CRUD service is procrastination with extra steps: the failure modes there are not protocol-deep, and the spec will model a framework you do not control.

Property-based and model-based component testing

Between the protocol spec and the full-cluster harness sits a layer that most teams under-invest in: the distributed *component* — the retry wrapper, the idempotent consumer, the replication buffer, the CRDT — tested exhaustively in one process. This is Volume 4, Chapter 10's Lincheck pattern with the schedule replaced by a delivery schedule: generate randomized *operation sequences and fault schedules*, run them against the component, and compare against a simple **model oracle** — the obviously-correct sequential implementation of the same contract.

The duplicate-delivery test every Chapter 9 consumer deserves, in Hypothesis:

```

from hypothesis import given, strategies as st

# Commands with unique idempotency keys
commands = st.lists(
    st.tuples(st.uuids(), st.sampled_from(["credit", "debit"]), st.integers(1, 100)),
    min_size=1, max_size=30,
)

@given(cmds=commands, chaos=st.randoms())
def test_at_least_once_delivery_is_effectively_once(cmds, chaos):
    # Build a hostile delivery schedule: duplicates and reorderings,
    # as an at-least-once broker is entitled to produce.
    schedule = list(cmds)
    for c in cmds:
        if chaos.random() < 0.5:
            schedule.insert(chaos.randrange(len(schedule)), c) #
duplicate, anywhere

    account = IdempotentAccount()                # system under test
    for cmd in schedule:
        account.apply(cmd)                        # may dedupe, may crash-
recover internally

    model = SimpleAccount()                      # oracle: each unique
command once
    for cmd in cmds:
        model.apply(cmd)

    assert account.balance() == model.balance()

```

When this fails, Hypothesis **shrinks**: it re-runs with smaller command lists and fewer duplicates until it presents a minimal counterexample — typically two commands and one duplicate in one specific position — which is the reproducibility discipline of the fault-injection section, handed to you by the framework. The same skeleton tests everything with an algebraic contract: retry wrappers (inject failures before/after the effect — the *after* case is the one that finds bugs, per Chapter 9), replication buffers (inject reorder, assert log matching), and above all **CRDTs**, whose contract from Chapter 11 is pure algebra: generate arbitrary operation sets, deliver them to N replicas in different orders with different duplications, merge pairwise in arbitrary patterns, and assert convergence — all replicas equal — plus commutativity and idempotency of merge directly. A CRDT implementation that survives a night of randomized-delivery property tests has had its actual

mathematical claim checked, not vibes-checked; this is the testing note Chapter 11 promised, discharged.

Verification in production

The pyramid does not end at deploy, because Chapter 1's asynchronous reality guarantees production will eventually produce an environment no test did. What follows is the mechanics; the practice, culture, and organizational discipline of operating these live in Volume 11, and this section deliberately stops short of them.

Continuous checkers are the workhorse: background jobs that *audit* invariants the way a test checker audits a history, but forever, against the live system. Replica-divergence checkers walk replicas of the same shard comparing merkle trees or checksums and alarm on divergence — Chapter 7's anti-entropy machinery repurposed from repair to *detection*. Orphaned-state detectors sweep for the debris of interrupted sagas and half-completed workflows: the reservation with no order, the ledger entry with no matching journal line. Reconciliation jobs recompute derived state from source-of-truth and diff. This is the "trust but verify" layer, and it embodies this chapter's epistemology: every invariant a design document asserts should have a process, somewhere, actively trying to catch it false. A checker that has run for a year without firing is a far stronger statement than a test suite that passed once.

Shadow traffic and dark launches de-risk protocol changes specifically: mirror production traffic to the new implementation, discard its responses, diff its behavior — the highest-fidelity workload generator you will ever have, at the cost of side-effect hygiene (shadows must not send the email twice; Chapter 9 again). **Canaries with automated rollback** are the last detector: a slice of real traffic on the new version, promoted or reverted on metrics. And **chaos engineering**, positioned precisely: it is this chapter's fault injection graduated to production, with blast-radius control, steady-state hypotheses, and an abort switch. The *mechanics* — what a partition is, what SIGSTOP simulates, why asymmetric faults matter — are this chapter's; the *discipline* — game days, safety culture, organizational learning — is Volume 11, Chapter 8's, and conflating the two produces teams that kill pods for sport and learn nothing.

The pyramid, assembled

The synthesis: six layers, cheap where bugs are cheap, each covering the blind spots of its neighbors.

```
flowchart TB
    L1["1 Protocol spec – TLA+, P  
exhaustive on small instances  
blind to: the implementation"]
    L2["2 Component model and property tests  
oracles, fault schedules, CRDT algebra  
blind to: cross-component composition"]
    L3["3 Deterministic simulation  
whole cluster, seeded, replayable  
blind to: everything unmodeled – kernel, real network"]
    L4["4 Fault-injected integration – Jepsen-style  
real stack, history-checked  
blind to: rare interleavings, internal invariants"]
    L5["5 Staging chaos  
real environment, composed faults  
blind to: production scale and traffic"]
    L6["6 Production verification  
checkers, shadow traffic, canaries, chaos  
blind to: nothing – but detects after the fact"]
    L1 --> L2 --> L3 --> L4 --> L5 --> L6
```

Read it economically. The bug-cost curve is the steepest in software: a protocol bug found in TLC costs a counterexample trace and an afternoon; the same bug found by a component test costs a debugging session; found in simulation, a seed and a day; found by Jepsen, a week of non-reproducible investigation; found in production, an incident, a postmortem, and — for the lost-write class — customers you cannot make whole. The layers are ordered by exactly this gradient, and the lower layers are cheap *per bug* precisely because they are exhaustive within their scope: enumeration at the bottom, sampling in the middle, detection at the top. The common budgeting error is inverting the investment — an elaborate staging chaos program atop components that were never property-tested and a protocol nobody specified — which buys expensive discovery of cheap bugs. Fund the bottom first. (Where this pyramid meets the ordinary unit/integration/e2e pyramid and the organization's overall strategy is Volume 15, Chapter 1's subject.)

The distributed-systems lens: closing the volume

Testing is where this volume stops being a survey and becomes an audit, because every chapter's claims turn out to be *checkable*, and the checking machinery is this chapter's:

- **Chapter 1** defined the failure models — and they became the fault menu: crash-stop is `kill -9`, crash-recovery is restart-with-disk, omission is the partition, timing is SIGSTOP and skew, Byzantine is the row most shops rightly leave unfunded. A system is tested *against its model*, and a test plan that names no model is not a plan.
- **Chapter 2**'s clock skepticism became an injection: step the clocks and watch every LWW resolution and lease assumption defend itself or die.
- **Chapter 3**'s consistency contracts became the checkers' oracles: linearizability as witness search, causal and session guarantees as graph conditions, isolation as Elle's cycle hunt.
- **Chapters 5 and 6** supplied the invariants — single leader per term, log matching, committed entries survive — as TLA+ invariants at the bottom of the pyramid and simulation assertions in its middle.
- **Chapter 8**'s paused client became SIGSTOP, the nemesis move that finds more real bugs than any other. **Chapter 9**'s idempotency became the duplicate-injection property test. **Chapter 10**'s failure-detection limits became the asymmetric-partition scenarios. **Chapter 11**'s convergence claim became a property checked by randomized delivery.

The volume's running key-value store ends here too: after eleven chapters of upgrades — replication, consensus, quorums, leases, idempotent APIs, CRDT-merged registers — its final upgrade is not a feature but a harness: a spec for its replication protocol, property tests for its components, seeded simulation if it was built for one, a nemesis-driven history check before every release, and a divergence checker that never stops running. That is the difference between the store the volume built and the stores the volume's cautionary tales were about.

And that is the closing claim, stated plainly: **distributed systems are not made correct by cleverness; they are made correct by adversarial verification against explicit models.** Every impossibility result in Chapter 1 says the environment is allowed to be your adversary;

every Jepsen report says shipped systems routinely lose to that adversary in ways their designers did not imagine; every FoundationDB-style success says the way to win is to be a *worse* adversary to your own system than production will be — more partitions, more pauses, more skew, more crashes, replayable at will, checked against contracts written down before the fight. A design is a conjecture. Verification is what promotes it. That discipline — not the elegance of the protocol, not the pedigree of the paper it came from — is what separates systems that work from systems that have merely worked so far.

Key takeaways

- **The network is the scheduler, and nobody can instrument it.** The distributed interleaving space adds message delay, reorder, loss, partitions, skew, and partial failure to Volume 4's arithmetic, and no compiler-level chokepoint exists across machines — so distributed testing pivots from instrumenting mechanisms to checking histories at the client boundary.
- **Say the property first.** Safety violations are finite, checkable evidence; liveness is only ever "progress within a bound." Histories of invocations and responses are the universal object; consistency contracts (Chapter 3) are the oracles.
- **Linearizability checking is witness search:** a total order that is legal for the sequential object and respects real time. NP-complete in general; Knossos and Porcupine make it practical via WGL-style backtracking and per-key partitioning. Keep checked histories short and contended, and treat timeouts as indeterminate.
- **Elle checks transactions by hunting cycles,** inferring version order from list-append histories and finding G0/G1/G-single anomalies in polynomial time — certificates, not vibes.
- **Jepsen is a method:** generate real operations, inject composed faults (iptables partitions — majority/minority, bridge, partial; SIGSTOP as the GC-pause simulator; clock skew), record the full history, check offline. Its finding classes — stale reads under partition, LWW-lost writes, under-delivered isolation, election split-brain — reshaped the field. Its limits — black-box, probabilistic, poorly reproducible — are structural.

- **Fault injection is a discipline:** map the menu to Chapter 1's failure models, inject at the lowest honest layer (netem/iptables/eBPF, toxiproxy, mesh), and seed, log, and shrink or the failures you find are ghosts.
- **Deterministic simulation is the strongest current practice:** one seeded process owning all time, network, disk, and randomness under real implementation code; thousands of hostile universes nightly; exact replay from seed. The price is architectural and up-front — all nondeterminism through the simulator — which is why FDB and TigerBeetle designed for it and why Antithesis pushes determinism into a hypervisor instead. DST and Jepsen blind-spot each other; run both.
- **TLA+ finds design bugs testing cannot,** exhaustively on small instances, as Amazon's CACM report documents from S3 and DynamoDB experience — but the spec is not the code, and the refinement gap is real. Spend it on novel, consensus-adjacent, or irrecoverable-loss protocols, not CRUD.
- **Property-based tests with model oracles** cover the middle: duplicate-and-reorder schedules against idempotent consumers, convergence algebra against CRDTs, with shrinking as built-in reproducibility.
- **Verification continues in production:** divergence checkers, orphan sweeps, reconciliation, shadow traffic, canaries — and chaos engineering is this chapter's mechanics under Volume 11's discipline.
- **Fund the pyramid bottom-up.** Exhaustive layers are cheap per bug; production detection is the most expensive place to learn anything. A conjecture becomes a system through adversarial verification against explicit models — nothing else promotes it.

Further reading

- Kingsbury, K., the Jepsen analyses and methodology docs — <https://jepsen.io/analyses> — the primary record of the findings characterized in this chapter, with per-system reports.
- Kingsbury, K. and Alvaro, P., "Elle: Inferring Isolation Anomalies from Experimental Observations," *VLDB*, 2020 — cycle-based transactional checking; <https://github.com/jepsen-io/elle>.

- Herlihy, M. and Wing, J., "Linearizability: A Correctness Condition for Concurrent Objects," *ACM TOPLAS*, 1990 — the definition every checker implements.
- Wing, J. and Gong, C., "Testing and Verifying Concurrent Objects," *JPDC*, 1993 — the WGL checking algorithm; Gibbons, P. and Korach, E., "Testing Shared Memories," *SIAM J. Computing*, 1997 — the NP-completeness result.
- Knossos — <https://github.com/jepsen-io/knossos>; Porcupine (Athalye) — <https://github.com/anishathalye/porcupine> — practical linearizability checkers.
- Wilson, W., "Testing Distributed Systems with Deterministic Simulation," Strange Loop, 2014 — the FoundationDB story; and the FDB testing documentation, <https://apple.github.io/foundationdb/testing.html>.
- TigerBeetle, "VOPR" and simulation testing docs — <https://docs.tigerbeetle.com/> — a contemporary DST implementation with storage-fault depth.
- Antithesis documentation — <https://antithesis.com/docs/> — hypervisor-level deterministic testing of unmodified stacks.
- madsim — <https://github.com/madsim-rs/madsim> — and turmoil — <https://github.com/tokio-rs/turmoil> — deterministic simulation for Rust services.
- Newcombe, C. et al., "How Amazon Web Services Uses Formal Methods," *CACM* 58(4), 2015 — the industrial TLA+ experience report cited throughout the formal-methods section.
- Lamport, L., *Specifying Systems* and the TLA+ home — <https://lamport.azurewebsites.net/tla/tla.html>; the P language — <https://p-org.github.io/P/>; Alloy — <https://alloytools.org/>.
- Shopify Toxiproxy — <https://github.com/Shopify/toxiproxy>; Linux `tc-netem` man page — the fault-injection workhorses.
- Volume 4, Chapter 10 — Testing and Debugging Concurrent Systems — the single-machine sibling; Volume 11, Chapter 8 — chaos engineering as practice and culture; Volume 15, Chapter 1 — where this pyramid meets overall testing strategy.